

# Principia Softwarica: The Plan 9 Shell **rc**

version 0.1

Yoann Padioleau  
`yoann.padioleau@gmail.com`

with code from  
Tom Duff

March 24, 2026

---

The text and figures are Copyright © 2014–2026 Yoann Padioleau.  
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.  
MIT license.

---

# Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                          | <b>8</b>  |
| 1.1      | Motivations                                  | 8         |
| 1.2      | The Plan 9 shell: <code>rc</code>            | 9         |
| 1.3      | Other shells                                 | 9         |
| 1.4      | Getting started                              | 11        |
| 1.5      | Requirements                                 | 11        |
| 1.6      | About this document                          | 12        |
| 1.7      | Copyright                                    | 12        |
| 1.8      | Acknowledgments                              | 12        |
| <b>2</b> | <b>Overview</b>                              | <b>13</b> |
| 2.1      | Essential shell features                     | 13        |
| 2.1.1    | Running commands                             | 13        |
| 2.1.2    | Redirections                                 | 14        |
| 2.1.3    | Pipes                                        | 15        |
| 2.1.4    | Storing commands in a script                 | 16        |
| 2.1.5    | A shell as a scripting language              | 16        |
| 2.2      | <code>rc</code> command-line interface       | 17        |
| 2.3      | <code>boot.rc</code>                         | 17        |
| 2.3.1    | <code>#!</code>                              | 18        |
| 2.3.2    | Initialization script                        | 18        |
| 2.3.3    | Variables                                    | 18        |
| 2.3.4    | <code>\$path</code>                          | 18        |
| 2.3.5    | Comments                                     | 19        |
| 2.3.6    | Quoting and escaping                         | 19        |
| 2.3.7    | The environment                              | 19        |
| 2.3.8    | Builtins                                     | 19        |
| 2.3.9    | Other features                               | 19        |
| 2.4      | Code organization                            | 20        |
| 2.5      | Software architecture                        | 21        |
| 2.5.1    | The command-line user interface              | 21        |
| 2.5.2    | <code>rc</code> 's components                | 21        |
| 2.5.3    | Trace of a simple command: <code>ls /</code> | 24        |
| 2.6      | Book structure                               | 25        |
| <b>3</b> | <b>Core Data Structures</b>                  | <b>26</b> |
| 3.1      | Token                                        | 26        |
| 3.2      | Abstract syntax Tree                         | 26        |
| 3.3      | ByteCode and <code>codebuf</code>            | 29        |
| 3.4      | Thread and <code>runqueue</code>             | 30        |

|          |                                                                                                                                                                            |           |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 3.5      | Words and lists of words                                                                                                                                                   | 31        |
| 3.5.1    | Words                                                                                                                                                                      | 31        |
| 3.5.2    | List of sequence of words                                                                                                                                                  | 33        |
| 3.6      | Variables                                                                                                                                                                  | 33        |
| <b>4</b> | <b>main()</b>                                                                                                                                                              | <b>36</b> |
| 4.1      | Overview                                                                                                                                                                   | 36        |
| 4.2      | Command-line arguments processing                                                                                                                                          | 36        |
| 4.2.1    | Login mode: <code>rc -l</code>                                                                                                                                             | 37        |
| 4.2.2    | Interactive mode: <code>rc -i</code>                                                                                                                                       | 37        |
| 4.3      | Initialization                                                                                                                                                             | 38        |
| 4.4      | Bootstrapping bytecodes (simplified)                                                                                                                                       | 38        |
| 4.5      | Setting <code>runq</code>                                                                                                                                                  | 38        |
| 4.6      | Setting <code>runq-&gt;argv</code>                                                                                                                                         | 39        |
| 4.7      | Bytecode interpreter loop                                                                                                                                                  | 40        |
| 4.8      | Reading commands: <code>Xrdcmds()</code>                                                                                                                                   | 40        |
| <b>5</b> | <b>Input</b>                                                                                                                                                               | <b>43</b> |
| 5.1      | Overview                                                                                                                                                                   | 43        |
| 5.2      | Reading a character: <code>getnext()</code>                                                                                                                                | 43        |
| 5.2.1    | End-of-file management                                                                                                                                                     | 44        |
| 5.2.2    | Multiple lines commands and escaped newlines                                                                                                                               | 44        |
| 5.2.3    | Displaying the prompt                                                                                                                                                      | 45        |
| 5.3      | Looking ahead: <code>nextc()</code> and <code>advance()</code>                                                                                                             | 46        |
| <b>6</b> | <b>Lexing</b>                                                                                                                                                              | <b>48</b> |
| 6.1      | <code>yylex()</code>                                                                                                                                                       | 48        |
| 6.2      | Spaces and comments ( <code>'#'</code> )                                                                                                                                   | 49        |
| 6.3      | Newlines                                                                                                                                                                   | 50        |
| 6.4      | Operators ( <code>'&amp;'</code> , <code>'&amp;&amp;'</code> , <code>' '</code> , <code>'  '</code> , <code>'&lt;'</code> , <code>'&gt;'</code> , <code>'\$'</code> , ...) | 50        |
| 6.5      | Quoted strings ( <code>'...'</code> )                                                                                                                                      | 53        |
| 6.6      | Keywords and identifiers ( <code>if</code> , <code>for</code> , <code>while</code> , <code>switch</code> , <code>fn</code> , ...)                                          | 54        |
| 6.7      | Array subscript ( <code>&lt;arr&gt;(&lt;n&gt;)</code> )                                                                                                                    | 56        |
| <b>7</b> | <b>Parsing</b>                                                                                                                                                             | <b>57</b> |
| 7.1      | Overview                                                                                                                                                                   | 57        |
| 7.2      | Simple commands ( <code>&lt;cmd&gt; &lt;arg1&gt;...&lt;argn&gt;</code> )                                                                                                   | 58        |
| 7.3      | Operators                                                                                                                                                                  | 60        |
| 7.3.1    | Sequences ( <code>';'</code> , <code>'&amp;'</code> )                                                                                                                      | 60        |
| 7.3.2    | Logical operators ( <code>'&amp;&amp;'</code> , <code>'  '</code> , <code>'!'</code> )                                                                                     | 60        |
| 7.3.3    | String matching ( <code>'~'</code> )                                                                                                                                       | 61        |
| 7.3.4    | Pipe ( <code>' '</code> )                                                                                                                                                  | 61        |
| 7.3.5    | Redirections ( <code>'&gt;'</code> , <code>'&lt;'</code> )                                                                                                                 | 61        |
| 7.4      | Control flow statements ( <code>if</code> , <code>if not</code> , <code>while</code> , <code>switch</code> , <code>for</code> )                                            | 62        |
| 7.5      | Functions ( <code>fn</code> )                                                                                                                                              | 63        |
| 7.6      | Variables ( <code>&lt;x&gt; = ...</code> )                                                                                                                                 | 63        |
| 7.7      | Lists ( <code>(...)</code> )                                                                                                                                               | 63        |

|           |                                               |            |
|-----------|-----------------------------------------------|------------|
| <b>8</b>  | <b>Bytecode Generation and Interpretation</b> | <b>64</b>  |
| 8.1       | Overview                                      | 64         |
| 8.1.1     | emitxxx()                                     | 64         |
| 8.1.2     | compile()                                     | 65         |
| 8.1.3     | outcode()                                     | 65         |
| 8.1.4     | argv management                               | 66         |
| 8.1.5     | Process status management                     | 67         |
| 8.1.6     | Subprocesses management                       | 68         |
| 8.2       | Simple commands                               | 69         |
| 8.2.1     | Bytecode generation                           | 69         |
| 8.2.2     | Xsimple()                                     | 70         |
| 8.2.3     | Fork                                          | 71         |
| 8.2.4     | Exec                                          | 72         |
| 8.2.5     | \$path management                             | 73         |
| 8.2.6     | Wait                                          | 74         |
| 8.2.7     | Fork optimization                             | 75         |
| 8.3       | Operators                                     | 75         |
| 8.3.1     | Basic sequence                                | 75         |
| 8.3.2     | Logical operators                             | 76         |
| 8.3.3     | String matching                               | 77         |
| 8.3.4     | Redirection                                   | 78         |
| 8.3.5     | Pipe                                          | 83         |
| 8.3.6     | Asynchronous execution                        | 86         |
| 8.4       | Control flow statements                       | 88         |
| 8.4.1     | if                                            | 88         |
| 8.4.2     | while                                         | 89         |
| 8.4.3     | for                                           | 90         |
| 8.4.4     | switch                                        | 90         |
| 8.4.5     | Blocks: '{...}'                               | 93         |
| 8.5       | Functions                                     | 93         |
| 8.5.1     | Function definitions (fn <foo> ... )          | 93         |
| 8.5.2     | Function uses (<foo>(...))                    | 95         |
| 8.6       | Variables                                     | 95         |
| 8.6.1     | Variable definitions (<x>=...)                | 95         |
| 8.6.2     | Variable uses (\$<x>)                         | 97         |
| 8.6.3     | Special variables                             | 101        |
| <b>9</b>  | <b>Builtins</b>                               | <b>102</b> |
| 9.1       | Overview                                      | 102        |
| 9.2       | \$ cd                                         | 103        |
| 9.3       | \$ exit                                       | 105        |
| 9.4       | \$ .                                          | 105        |
| 9.5       | \$ eval                                       | 107        |
| 9.6       | \$ wait                                       | 108        |
| <b>10</b> | <b>Environment</b>                            | <b>110</b> |
| 10.1      | Updenv()                                      | 110        |
| 10.2      | Vinit()                                       | 111        |

|                                                                                                                                                               |            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>11 Signals</b>                                                                                                                                             | <b>113</b> |
| 11.1 Overview                                                                                                                                                 | 113        |
| 11.2 Registering handlers: <code>Trapinit()</code>                                                                                                            | 114        |
| 11.3 Trap dispatch: <code>dotrap()</code>                                                                                                                     | 115        |
| <b>12 Initialization</b>                                                                                                                                      | <b>117</b> |
| 12.1 Actual bootstrapping code                                                                                                                                | 117        |
| 12.2 Initialization script and <code>rc -m /path/to/rcmain</code>                                                                                             | 118        |
| 12.3 Actual environment                                                                                                                                       | 118        |
| <b>13 Globbing</b>                                                                                                                                            | <b>119</b> |
| 13.1 Lexing globbing characters                                                                                                                               | 119        |
| 13.2 Expanding globbing characters                                                                                                                            | 120        |
| 13.3 <code>glob()</code>                                                                                                                                      | 121        |
| 13.4 <code>match()</code>                                                                                                                                     | 124        |
| <b>14 Advanced Topics</b>                                                                                                                                     | <b>128</b> |
| 14.1 Reading commands from a string: <code>rc -c</code>                                                                                                       | 128        |
| 14.2 Failing fast: <code>rc -e</code>                                                                                                                         | 128        |
| 14.3 Unicode                                                                                                                                                  | 129        |
| 14.4 Advanced constructs                                                                                                                                      | 129        |
| 14.4.1 Subshell: <code>@ &lt;cmd&gt;</code>                                                                                                                   | 129        |
| 14.4.2 Here documents: <code>&lt;&lt; &lt;HERE&gt;</code>                                                                                                     | 130        |
| 14.4.3 Read-write redirections: <code>&lt;&gt; &lt;file&gt;</code>                                                                                            | 135        |
| 14.4.4 General redirections: <code>&gt;[2] &lt;file&gt;</code>                                                                                                | 135        |
| 14.4.5 Advanced dup: <code>&gt;[&lt;fd0&gt;=&lt;fd1&gt;]</code> , <code>&lt;&gt;[&lt;fd0&gt;=&lt;fd1&gt;]</code> , <code>&lt;[&lt;fd0&gt;=&lt;fd1&gt;]</code> | 137        |
| 14.4.6 Advanced pipes: <code>  [&lt;fd&gt;]</code> , <code>  [&lt;fd0&gt;=&lt;fd1&gt;]</code>                                                                 | 138        |
| 14.4.7 Command output as a file: <code>'&lt;{&lt;cmd&gt;}'</code>                                                                                             | 138        |
| 14.4.8 Command substitution: <code>'{&lt;cmd&gt;}'</code>                                                                                                     | 139        |
| 14.4.9 Stringification of variables: <code> \$"&lt;foo&gt;</code>                                                                                             | 141        |
| 14.5 Advanced builtins                                                                                                                                        | 142        |
| 14.5.1 <code>\$ exec</code>                                                                                                                                   | 142        |
| 14.5.2 <code>\$ whatis</code>                                                                                                                                 | 142        |
| 14.5.3 <code>\$ rfork</code>                                                                                                                                  | 144        |
| 14.5.4 <code>\$ flag</code>                                                                                                                                   | 145        |
| 14.5.5 <code>\$ shift</code>                                                                                                                                  | 146        |
| 14.5.6 <code>\$ finit</code>                                                                                                                                  | 146        |
| <b>15 Conclusion</b>                                                                                                                                          | <b>148</b> |
| 15.1 Patterns and techniques                                                                                                                                  | 148        |
| 15.2 Connections to other books                                                                                                                               | 148        |
| 15.3 Missing features                                                                                                                                         | 149        |
| 15.4 Beyond the Plan 9 shell                                                                                                                                  | 149        |
| <b>A Debugging</b>                                                                                                                                            | <b>151</b> |
| A.1 AST dumper                                                                                                                                                | 151        |
| A.2 Bytecode generator trace: <code>rc -r</code>                                                                                                              | 153        |
| A.3 Printing subprocesses status: <code>rc -s</code>                                                                                                          | 155        |
| A.4 Printing commands: <code>rc -x</code>                                                                                                                     | 155        |
| A.5 Printing characters: <code>rc -v</code>                                                                                                                   | 155        |

|          |                                             |            |
|----------|---------------------------------------------|------------|
| A.6      | Printing all characters: <code>rc -V</code> | 155        |
| <b>B</b> | <b>Error Management</b>                     | <b>156</b> |
| <b>C</b> | <b>Utilities</b>                            | <b>158</b> |
| C.1      | Memory management                           | 158        |
| C.2      | Command-line arguments                      | 159        |
| C.3      | Buffered IO                                 | 163        |
| C.4      | Format                                      | 166        |
| C.5      | String conversions                          | 169        |
| C.6      | Misc                                        | 169        |
| <b>D</b> | <b>Examples of rc scripts</b>               | <b>170</b> |
| D.1      | <code>/rc/lib/rcmain</code>                 | 170        |
| D.2      | <code>/usr/glenda/lib/profile</code>        | 171        |
| <b>E</b> | <b>Shell Companion Utilities</b>            | <b>173</b> |
| E.1      | <code>echo</code>                           | 173        |
| E.2      | <code>pwd</code>                            | 174        |
| E.3      | <code>test</code>                           | 174        |
| E.4      | <code>sh</code> , the simple shell          | 181        |
| <b>F</b> | <b>Extra Code</b>                           | <b>191</b> |
| F.1      | <code>misc/</code>                          | 191        |
| F.1.1    | <code>misc/echo.c</code>                    | 191        |
| F.1.2    | <code>misc/pwd.c</code>                     | 191        |
| F.1.3    | <code>misc/test.c</code>                    | 191        |
| F.2      | <code>sh/</code>                            | 192        |
| F.2.1    | <code>sh/sh.c</code>                        | 192        |
| F.3      | <code>rc/</code>                            | 194        |
| F.3.1    | <code>rc/fns.h</code>                       | 194        |
| F.3.2    | <code>rc/getflags.h</code>                  | 196        |
| F.3.3    | <code>rc/io.h</code>                        | 196        |
| F.3.4    | <code>rc/rc.h</code>                        | 196        |
| F.3.5    | <code>rc/exec.h</code>                      | 199        |
| F.3.6    | <code>rc/globals.c</code>                   | 199        |
| F.3.7    | <code>rc/getflags.c</code>                  | 200        |
| F.3.8    | <code>rc/io.c</code>                        | 200        |
| F.3.9    | <code>rc/input.c</code>                     | 201        |
| F.3.10   | <code>rc/var.c</code>                       | 202        |
| F.3.11   | <code>rc/env.c</code>                       | 202        |
| F.3.12   | <code>rc/glob.c</code>                      | 202        |
| F.3.13   | <code>rc/executils.c</code>                 | 203        |
| F.3.14   | <code>rc/exec.c</code>                      | 203        |
| F.3.15   | <code>rc/processes.c</code>                 | 206        |
| F.3.16   | <code>rc/tree.c</code>                      | 206        |
| F.3.17   | <code>rc/lex.c</code>                       | 207        |
| F.3.18   | <code>rc/trap.c</code>                      | 207        |
| F.3.19   | <code>rc/simple.c</code>                    | 208        |
| F.3.20   | <code>rc/pcmd.c</code>                      | 208        |

|        |                            |     |
|--------|----------------------------|-----|
| F.3.21 | <code>rc/here.c</code>     | 208 |
| F.3.22 | <code>rc/code.c</code>     | 209 |
| F.3.23 | <code>rc/pfnc.c</code>     | 210 |
| F.3.24 | <code>rc/utils.c</code>    | 210 |
| F.3.25 | <code>rc/status.c</code>   | 210 |
| F.3.26 | <code>rc/builtins.c</code> | 210 |
| F.3.27 | <code>rc/path.c</code>     | 211 |
| F.3.28 | <code>rc/fmt.c</code>      | 212 |
| F.3.29 | <code>rc/words.c</code>    | 212 |
| F.3.30 | <code>rc/error.c</code>    | 212 |
| F.3.31 | <code>rc/main.c</code>     | 213 |
| F.3.32 | <code>rc/plan9.c</code>    | 213 |
| F.3.33 | <code>rc/unix.c</code>     | 214 |

|          |     |
|----------|-----|
| Glossary | 225 |
|----------|-----|

|       |     |
|-------|-----|
| Index | 226 |
|-------|-----|

|            |     |
|------------|-----|
| References | 234 |
|------------|-----|

# Chapter 1

## Introduction

The goal of this book is to explain with full details the source code of a shell.

### 1.1 Motivations

Why a shell? Because I think you are a better programmer if you fully understand how things work under the hood, and the shell is the central piece of the *command-line user interface* (CLI). The shell is a thin layer around the kernel (hence its name) allowing you to run commands in a terminal.

Most users now use a *graphical user interface* (GUI) to execute programs (e.g., macOS, Microsoft Windows, X Window), but most programmers still spend a significant portion of their time in a shell to run compilation commands, editors, debuggers, or *scripts* to automate repetitive tasks. Integrated Development Environments (IDEs) can handle some of those use cases, but the shell still reigns when a programmer needs more flexibility. The power of pipes, redirections, variables, and basic control flow constructs allows sometimes in one command-line to perform tasks that would require hundreds of lines in a regular programming language<sup>1</sup>.

In fact, the rise of AI coding assistants like Claude Code has given the command-line a second wind. These tools operate as shell programs, composing existing commands through pipes and scripts rather than through graphical menus. The shell's text-based, composable nature turns out to be a perfect interface for AI agents that can read, write, and chain commands together—something much harder to automate inside a GUI-based IDE.

There are very few books explaining how a shell works. I can cite *Advanced UNIX Programming* [Roc04], but it explains only the code of a mini-shell. This is a pity because the implementation of a real shell covers many interesting topics (e.g., programming language design, compilation, interpretation, system programming, etc.) as you will see soon in this book.

Here are a few questions I hope this book will answer:

- What is the difference between a shell and a terminal? What is the difference between a shell and a login program?
- What happens when the user types `ls` in a terminal? What is the trace of such a command through the different layers of the software stack?
- What are the main features of a shell?
- How are redirections and pipes implemented? What are the system calls involved?
- Why `ls` and `rm` are regular programs but not `cd`? Why `cd` has to be a shell builtin? What is a shell builtin?

---

<sup>1</sup>For example, [BKM86] contrasts writing a program to count words in Pascal and in a shell.

- How does C-c, which interrupts a process, work? Which process receives the signal after an interruption? The shell or the command run from the shell?

## 1.2 The Plan 9 shell: rc

I will explain in this book the code of the Plan 9 shell `rc` [Duf90]<sup>2</sup> (for Run Commands), which contains about 6700 lines of code (LOC). `rc` is written mostly in C, with its parser using also Yacc [Joh79].

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. Moreover, `rc` is arguably simpler to use and to understand than `sh` [Bou79], the UNIX shell, or any of its derivatives (e.g., `bash` [Ram94], the most popular shell under Linux). For example, by treating the content of any variables uniformly as a list of strings, `rc` does not need the extra operator `$@` used in `sh` (`$*` is enough). Moreover, the syntax of `rc`, specified formally and succinctly by a small grammar, is also easier to learn than the (unspecified) syntax of `sh`, partly because `rc` is inspired by the syntax of C with its curly braces (`sh`, instead, is using multiple keywords inspired by Algol).

`rc` itself lacks many of the interactive features found in other shells (e.g., `bash`) such as filename completion, command-line editing, job control, etc. This is partly because under Plan 9, the terminal of the windowing system `rio` is providing instead those features (see the WINDOWS book [Pad16c]).

## 1.3 Other shells

Here are a few shells that I considered for this book, but which I ultimately discarded:

- The first UNIX shell, originally called `sh`, was written by Ken Thompson, the original author of the UNIX kernel. `sh` started as an assembly program in UNIX V1<sup>3</sup> and finished as a C program in UNIX V6<sup>4</sup>. Ken Thompson's shell introduced the pipe (`'|'`), redirections (`'>'` and `'<'`), as well as wildcard matching<sup>5</sup> (`'*'`). The syntax of those features remained the same in all subsequent shells. Ken Thompson's shell contains only 899 LOC, but it is just a basic command interpreter, not a full scripting language like `rc`.
- The Bourne shell [Bou79], also called `sh`, superseded Ken Thompson's shell (which was renamed `osh`) in UNIX V7<sup>6</sup>. It was written by Stephen Bourne and contains 4145 LOC of C. It is arguably the most famous shell because it defined first the main features that we expect now from a shell: the ability to run commands with pipes and redirections, but also the use of variables and control flow constructs to write scripts. The Bourne shell is both an interactive command interpreter and a full programming language. Most subsequent shells tried to remain backward compatible with the Bourne shell.

The syntax of the Bourne shell was inspired by Algol with pair of keywords (e.g., `begin/end`, `do/done`, `case/esac`) instead of the curly braces of C<sup>7</sup>. The code of `sh` is smaller than the code of `rc`, but it is also harder to understand. as in `rc`, but instead a complex recursive descent parser.

- Bash [Ram94, New95]<sup>8</sup> (for Bourne-Again Shell) is an open-source shell similar to the Bourne shell (hence its name) from the GNU project<sup>9</sup>. It is the default shell in most Linux distributions. It provides many interactive features not found in the Bourne shell such as filename completion, command-line editing,

<sup>2</sup>See <http://plan9.bell-labs.com/magic/man2html/1/rc> for its manual page.

<sup>3</sup><http://minnie.tuhs.org/cgi-bin/utree.pl?file=V1/sh.s>

<sup>4</sup><http://minnie.tuhs.org/cgi-bin/utree.pl?file=V6/usr/source/s2/sh.c>

<sup>5</sup>Also known as globbing.

<sup>6</sup><http://minnie.tuhs.org/cgi-bin/utree.pl?file=V7/usr/src/cmd/sh/>

<sup>7</sup>Stephen Bourne was such a fan of Algol that the C source code of `sh` itself looks like Algol, thanks to macros such as `THEN`, `BEGIN`, `END`, etc.

<sup>8</sup><https://www.gnu.org/software/bash/>

<sup>9</sup><http://www.gnu.org>

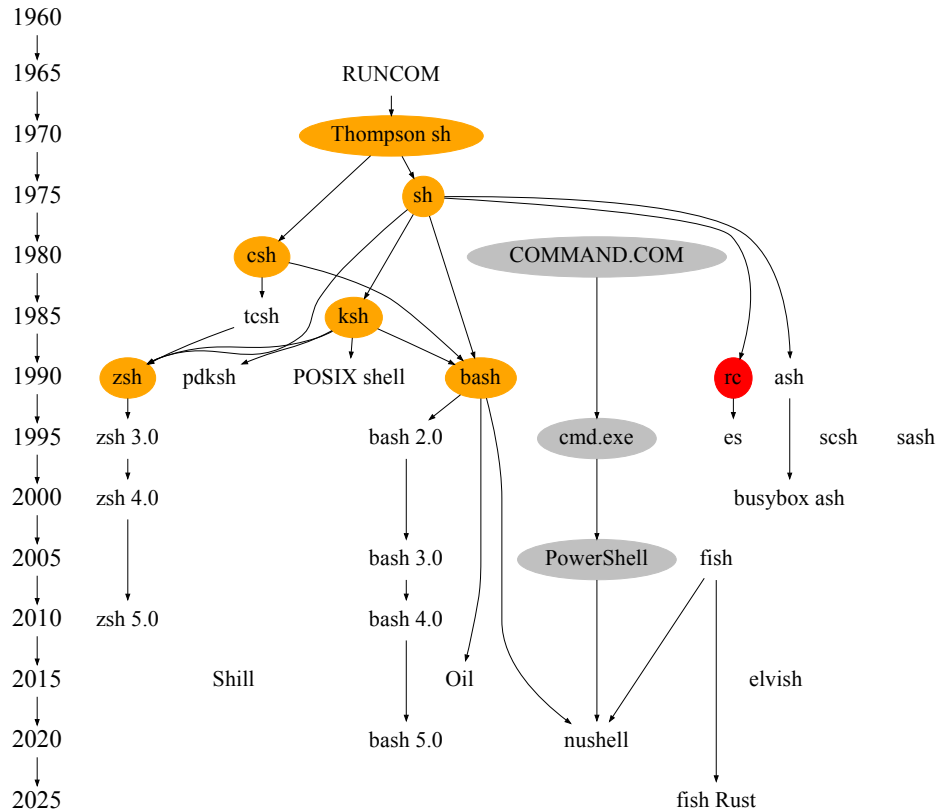


Figure 1.1: Shells timeline

interactive history, job control, etc. Many of those features were partly inspired by the C shell [Joy86], an older shell originating in BSD UNIX using a syntax more similar to C.

Bash relies on the GNU Readline library<sup>10</sup> for many of those interactive features (`rc` relies instead on `rio`'s terminal to provide similar features). However, the codebase of Bash is very large with more than 100 000 LOC (not including the code of the Readline library, which would add another 33 000 LOC), which is more than an order of magnitude more code than in `rc`. Bash's grammar file `parse.y` contains alone 6268 LOC.

The 15× size difference between Bash and `rc` is not just about interactive features. Much of the complexity comes from backward compatibility with the Bourne shell (and POSIX), which forces Bash to support multiple quoting styles, here-documents, arithmetic expansion, brace expansion, parameter expansion with numerous operators (`$x:-default`, `$x##pattern`, etc.), and the context-sensitive grammar inherited from `sh`.

- The Z shell [Kid05]<sup>11</sup> is another open-source shell popular among advanced Linux users. It is extensible through plugins and themes<sup>12</sup>. It contains most of the features found in other shells (e.g., Bash, the C Shell, the Korn Shell [BK89, Kor94]) and introduced many new interactive features such as programmable completion, recursive wildcarding with `**/*` (eliminating the need for the program `find`), and more. However, all those features come at a price: the code of the Z shell contains more than 145 000 LOC (not including the tests).

<sup>10</sup><https://tiswww.case.edu/php/chet/readline/rltop.html>

<sup>11</sup><http://www.zsh.org/>

<sup>12</sup>See <https://github.com/robbyrussell/oh-my-zsh> for a large repository of such contributions.

Figure 1.1 presents a timeline of major UNIX shells (and a few non-UNIX shells). I think `rc` represents the best compromise for this book: it implements the essential features of a shell while still having a small and understandable codebase (6700 LOC).

## 1.4 Getting started

To play with `rc`, you will first need to install the Plan 9 fork used in Principia Softwarica (see <https://www.principia-softwarica.org/getting-started.html>). You can also run `rc` on Linux, macOS, or Windows through `plan9port`<sup>13</sup> or `Goken9cc`<sup>14</sup>, where it is compiled natively using `gcc` or `clang`. Once installed, you do not need to do anything to run `rc` because it is the program started by default by the kernel (see the `KERNEL` book [Pad14]). Once the kernel finished to boot, you should see a percent sign, called the *prompt*, after which you can type any commands as in the following:

```
1  % ls /
2  bin
3  boot
4  ...
5  srv
6  % rc -help
7  Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
8  % rc
9  % prompt='$ '
10 $ ls /
11 bin
12 boot
13 ...
14 srv
15 $ exit
16 %
```

Line 1 runs the program `ls` to list the content of the root directory. Line 6 and Line 8 show that `rc` is a regular program (just like `ls` at Line 1): you can run a shell program under a shell. Line 8 and Line 9 seem to indicate that typing `rc` has no effect, but the percent sign shown at the beginning of Line 9 is in fact displayed by the `rc` process started by Line 8, not the original `rc` process started by the kernel. Line 9 modifies the *special variable* `prompt` (see Section 8.6.3 for a list of those special variables) to better differentiate the two shell processes. Indeed, Line 10 shows that `'$ '` is the new prompt replacing the percent sign. Finally, Line 15 shows the use of the *builtin* `exit` (see Chapter 9 for more information on builtins) to exit from the shell. Doing so goes back to the preceding shell process (the one launched by the kernel) with the original percent prompt at Line 16.

## 1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it. To understand Chapter 7, you will also need to know Yacc [BLM92].

Note that this book is not an introduction to the shell or to shell scripting. I assume you are already familiar with at least one shell, for instance a derivative of `sh` such as `bash`, and so are familiar with concepts such as a

---

<sup>13</sup><https://9fans.github.io/plan9port/>

<sup>14</sup><https://github.com/aryx/goken9cc>

pipe, a redirection, what `#!` means, or what a script is. If not, I suggest you to read either the book introducing the UNIX programming environment [KP84], or the original `sh` tutorial [Bou79], or any more recent books on shell scripting [Mic08, New95, Kid05].

A shell is a thin layer on top of a kernel, and so a shell relies heavily on the services offered by the kernel: system calls (e.g., `rfork()`, `exec()`, `wait()`, `chdir()`), but also device files (e.g., `/dev/cons` to read and write characters on the terminal). Thus, it can be useful to know how the Plan 9 kernel works (see the `KERNEL` book [Pad14]), or at least be familiar with sytem programming under UNIX [Roc04, Ste94], to fully understand some of the code in this book.

A shell is also a full programming language, and as you will see soon, `rc` is internally both a compiler and a bytecode interpreter. I assume you also have a basic understanding of how a compiler works, and for example that you know what a lexer or parser is (see the `COMPILER` book [Pad16b]).

If, while reading this book, you have specific questions on the syntax and interface of `rc`, I suggest you to consult the man page of `rc` at `docs/man/1/rc` in my Plan 9 repository.

Note that the `shells/docs/` directory in my Plan 9 repository contains documents describing either `rc` [Duf00] or `sh` [Bou15]. Those documents are useful to understand some of the design decisions presented in this book, especially how and why `rc` differs from `sh`.

## 1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

## 1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

## 1.8 Acknowledgments

I would like to acknowledge of course the author of `rc`, Tom Duff, who wrote in some sense most of this book.

# Chapter 2

## Overview

Before showing the source code of `rc` in the following chapters, I first give an overview in this chapter of the general features of a shell. I also quickly describe the command-line interface of `rc` and the specific language supported by `rc`. I also define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

### 2.1 Essential shell features

The following sections will explain the essential features of a shell. A shell is a strange beast: it must be both an interactive command interpreter, allowing to run commands easily on one line in a terminal, and a proper programming language, allowing to write complex scripts in a file.

#### 2.1.1 Running commands

The first job of a shell is to allow the user to run commands<sup>1</sup>. Most shells offer a minimalist syntax for running commands, so we can type those commands quickly. Indeed, contrast the shell command `ls /` with the equivalent C program:

```
<lsroot.c 13>≡
#include <u.h>
#include <libc.h>

void main(){
    char* args[] = {"/", nil};
    exec("/bin/ls", args);
}
```

Here are a few notes on the minimalist syntax used by shells:

- *No Parenthesis*: to call a program, you do not need any parenthesis; just type the program name next to its arguments.
- *No quotes*: to pass arguments to the program, you usually do not need any quotes or commas; just type the arguments separated by space. For example, `foo`, `--help`, `42`, `/a/b/c` are all valid arguments<sup>2</sup>. The only time you need to enclose an argument in a quote is when you want to use one of the special character used by the shell (e.g., `'#'`, `'$'`, `'&'`, etc. with `rc`), also known as *meta-characters*.

---

<sup>1</sup>The very first shell, which was written for the CTSS operating system in 1964, was called `RUNCOM`[[Pou00](#)].

<sup>2</sup>You can even sometimes avoid to specify fully all the arguments by using shortcuts, for example, wildcards as explained in Chapter [13](#).

- *No full path:* to call a program, you do not need to specify its path in the filesystem; just type the name of the program and the shell will automatically find its location. Under UNIX, the `PATH` environment variable stores the candidate locations to find programs.
- *No special ending character:* to finish your command, you do not need any special character like a semicolon; just type a newline and the shell will start interpreting your command<sup>3</sup>.
- *No types:* to enter a command, you do not need to specify any types such as `char*` as in C. The arguments are always strings.

Once the command you ran finished, the shell gets back in control and displays another *prompt* to indicate that you can type another command.

To make this concrete, here is a minimal shell in about 20 lines of C. It does almost nothing—just reads a command name, forks a process, and `execs` the program—but it already captures the essence of what a shell is: a loop that reads, forks, execs, and waits.

```
<minishell.c 14>≡
#include <u.h>
#include <libc.h>

void main() {
    char buf[256];
    char *args[2];
    int n;

    for(;;) {
        pwrite(1, "$ ", 2, 0LL);
        n = pread(0, buf, sizeof(buf)-1, 0LL);
        if(n <= 0)
            break;
        buf[n-1] = '\0'; /* strip newline */
        args[0] = buf;
        args[1] = nil;
        if(rfork(RFPROC|RFFDG) == 0) {
            exec(buf, args);
            exits("exec failed");
        }
        await(buf, sizeof(buf));
    }
    exits(0);
}
```

## 2.1.2 Redirections

The second important feature of a shell is to allow to *redirect* the output and input of a program, as in `ls / > listing.txt`.

Many programs (e.g., `ls`, `find`, `rm`, `grep`) live in the command-line world and simply use text for their input and output. By using redirections, you can easily save the output of those programs in a file, or use a previously saved file as input for those programs<sup>4</sup>, without even changing the code of those programs. In fact, because under UNIX and Plan 9 “everything is a file”, including devices, you can use the same program in many different ways. For example, you can print the listing of the root directory by simply typing `ls / > /dev/printer`.

<sup>3</sup>You can also enter commands on multiple lines, which requires to *escape* the newline as explained in Section 5.2.2.

<sup>4</sup>`rc` allows to redirect not just the standard input and output, as explained in Section 14.4.4.

### 2.1.3 Pipes

Because of the universality of plain text, you can easily combine many command-line programs. For example, you can list all the C files in your home directory by combining `find` and `grep` with the two commands `find /home/pad/ > /tmp/list.txt` and `grep '\.c$' < /tmp/list.txt`. In fact, thanks to another great feature of the shell, the *pipe*, you can just use one command: `find /home/pad | grep '\.c$'`. This is not only shorter, but also more efficient, and it gives results more quickly. You can even use multiple pipes in the same command as in `find | grep '\.c$' | xargs cat | grep foo`.

Pipes allow to combine easily full programs, just like you can combine functions in a functional language. Pipes are one of the greatest innovations introduced by UNIX, and one of the first programming construct allowing a form of *component-oriented programming*. Each program can be treated as a component, which can be combined with other components. You do not program at the granularity of functions but at the higher granularity of programs.

To support redirections and pipes, one only needs to manipulate file descriptors in the forked child before calling `exec`. Here is a second mini-shell that handles `>` for output redirection and `|` for a single pipe:

```
<minishell12.c 15>≡
#include <u.h>
#include <libc.h>

void runcmd(char *cmd) {
    char *args[2];
    args[0] = cmd;
    args[1] = nil;
    exec(cmd, args);
    exits("exec failed");
}

void main() {
    char buf[256];
    char *p;
    int n, fd, pfd[2];

    for(;;) {
        pwrite(1, "$ ", 2, 0LL);
        n = pread(0, buf, sizeof(buf)-1, 0LL);
        if(n <= 0)
            break;
        buf[n-1] = '\0';

        if((p = strchr(buf, '>')) != nil) {
            *p++ = '\0';
            while(*p == ' ') p++;
            if(rfork(RFPROC|RFFDG) == 0) {
                fd = create(p, OWRITE, 0666);
                dup(fd, 1);
                close(fd);
                runcmd(buf);
            }
            await(buf, sizeof(buf));
        } else if((p = strchr(buf, '|')) != nil) {
            *p++ = '\0';
            while(*p == ' ') p++;
            pipe(pfd);
            if(rfork(RFPROC|RFFDG) == 0) {
                close(pfd[0]);
                dup(pfd[1], 1);
                close(pfd[1]);
            }
        }
    }
}
```

```

    runcmd(buf);
}
if(rfork(RFPROC|RFFDG) == 0) {
    close(pfd[1]);
    dup(pfd[0], 0);
    close(pfd[0]);
    runcmd(p);
}
close(pfd[0]);
close(pfd[1]);
await(buf, sizeof(buf));
await(buf, sizeof(buf));
} else {
    if(rfork(RFPROC|RFFDG) == 0)
        runcmd(buf);
    await(buf, sizeof(buf));
}
}
exits(0);
}

```

Notice how little code it takes: the key insight is that `fork` gives the child its own copy of the file descriptor table, so we can `dup` a file or pipe end onto stdin or stdout before `exec`—and the executed program never knows the difference. This is precisely the beauty of the `fork/exec` split that the rest of this book will explore in detail.

### 2.1.4 Storing commands in a script

Just like you can type a series of commands separated by newlines in a terminal, you can save those same commands in a file, a *script*, and execute this script with the shell. As the author of the first shell said [Pou00]: “commands should be usable as building blocks for writing more commands, just like subroutine libraries”.

Scripts allow to automate easily repetitive tasks. Moreover, because scripts are commands themselves, they can also be combined, leading to more forms of components-oriented programming.

### 2.1.5 A shell as a scripting language

Once you can write a sequence of commands in a script, you quickly want more, such as the ability to use conditionals or loops. Thus, most shells are also full programming languages, often referred to as *scripting languages*, with *variables* and many *control flow* constructs, and even function definitions. A scripting language acts like a glue, allowing to combine many programs together.

Because a scripting language must also support the basic commands you type on the command-line, it shares many characteristics with those basic commands: a scripting language does not use types and is interpreted.

A scripting language operates mainly on strings. Because the string arguments you type on the command-line do not usually require any quotes, the use of variables requires then a special operator. Most shells use `'$'` as a prefix to differentiate variables from regular string arguments, for example, `find $home`.

The tension between being an interactive command interpreter and a full programming language has shaped many design choices in shells. Ken Thompson’s original V6 shell was purely interactive—it could not be scripted. The Bourne shell added scripting, but the need to keep the minimalist command-line syntax (no quotes, no types, newlines as terminators) forced compromises: variables need the `$` prefix to distinguish them from bare-word arguments, newlines must sometimes be “skipped” after binary operators so that multi-line statements work, and values are always strings.

In `rc`, the basis is even cleaner: all values are lists of strings. This eliminates the `sh` distinction between `*$` (all arguments as one string) and `$@` (all arguments as separate words), and means that `$path` does not need the colon-separated encoding that `$PATH` uses in `sh`.

There are no booleans: control flow is based on the exit status of the command you run. The only comparison operator is `~` (pattern matching on strings), and for numerical tests, the external program `test` is used.

A shell is also a kind of universal read-eval-print loop (REPL): you call programs (like functions), pass arguments (like parameters), use environment variables (like globals), and get results back (via exit status and `stdout`). The core loop is simply: read a line, parse it, fork, exec, wait, repeat.

## 2.2 rc command-line interface

I just described the main features of a shell. I will now focus exclusively on `rc` and give more details about its command-line interface.

It is rare to run `rc` itself on the command-line, like you run `ls`, `cp`, `grep`, etc. After all, if you have a command-line, you already are in a shell. However, as I said in Section 1.4, the shell under UNIX (and Plan 9) is a regular program. You can run a shell under a shell, which can be useful for example if you do not like the default shell when you log-in<sup>5</sup>. To run `rc`, simply type `rc` on the command-line without any arguments, as I did in Section 1.4 Line 8.

You can also pass to `rc` the name of a script as an argument, as well as the arguments of this script, for example, `rc foo.rc arg1 arg2`. However, this is usually not necessary because most scripts use `#!/bin/rc` at the beginning, which is recognized by the kernel (see the KERNEL book [Pad14]).

Here is the full command-line interface of `rc`:

```
% rc -help
Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
```

The `-c` flag allows to execute commands from a string passed as an argument instead of from the content of a script (see Section 14.1 for the code handling `rc -c`).

The `-m` flag allows to specify the initialization script of `rc`. I will explain fully the complex initialization process of `rc` and the code of `rc -m` in Chapter 12.

Finally, `rc` supports a few options to provide advanced features or to help debug `rc` itself. I will present gradually those options in this book.

## 2.3 boot.rc

In this section, I will present an example of an `rc` script showing a few features of `rc`'s scripting language. The goal here is to illustrate the general features of a shell you have seen in Section 2.1 with the concrete syntax of `rc`'s scripting language. This will also help you understand the grammar of `rc` I will present in Chapter 7. Finally, this example will introduce a few concepts specifics to `rc` that are useful to have in mind while I will explain the code of `rc` in the rest of the document.

`boot.rc`, below, is the first program executed by the kernel when running on an ARM machine (see the KERNEL book [Pad14]). The following sections will explain the main features of `rc` used in this script.

```
<kernel/init/user/boot/arm/boot.rc 17>≡
#!/boot/rc -m /boot/rcmain

/boot/echo booooooooooting...

path=(/bin /boot)

# basic devices
bind -c '#e' /env
...
```

---

<sup>5</sup>Under UNIX, you can also use the special program `chsh` to change your login shell.

```
# storage
bind -a '#S' /dev
fdisk -p /dev/sdM0/data >/dev/sdM0/ctl
dossrv
mount -c /srv/dos /root /dev/sdM0/dos
bind -a -c /root /

bind -a /arm/bin /bin
bind -a /rc/bin /bin
...

# to use 5c, 5a, 5l by default in mk
objtype=arm
...

exec /boot/rc -m /boot/rcmain -i
```

### 2.3.1 #!

The first line of `boot.rc` is `#!/boot/rc -m /boot/rcmain`. This is a *shebang* line: when the kernel encounters the `#!` marker at the start of a file being `exec()`-ed, it runs the specified interpreter with the file as argument instead of treating the file as a binary.

This mechanism, introduced in UNIX V8, is what makes scripts indistinguishable from compiled programs: they can be used as filters in pipes, as components in other scripts, or as standalone commands. Before the kernel handled shebangs, only the shell could run scripts—you had to type `rc foo.rc` explicitly, and a C program could not `exec()` a script.

Note that `#!` also works as a comment, since `#` starts a comment in `rc`, so the shebang line is simply ignored when the file is read as a script.

### 2.3.2 Initialization script

The `-m /boot/rcmain` flag specifies the initialization script. By default `rc` sources `/rc/lib/rcmain` at startup, but during boot the root filesystem is not yet mounted, so the boot script must use `-m` to point to a copy stored in the boot partition at `/boot/rcmain`.

Under UNIX, the equivalent mechanism is the cascade of initialization files: `.profile` for the Bourne shell, `.login` for the C shell, `.bashrc` for Bash, etc. In Plan 9, there is a single initialization script `rcmain` shared by all users (see Section D.1 and Chapter 12 for details).

### 2.3.3 Variables

The `boot.rc` script above shows a variable assignment: `path=(/bin /boot)`. In `rc`, variables hold *lists* of strings, and parentheses are used to create lists: `(a b c)` is a three-element list. This is a key design difference from `sh`, where variables hold a single string and lists are simulated by splitting on IFS characters. By treating all values as lists, `rc` avoids many of the quoting pitfalls that plague `sh` scripts.

### 2.3.4 \$path

The variable `$path` is a special variable: it tells `rc` where to look for programs. When you type `ls`, the shell searches each directory in `$path` until it finds an executable named `ls`.

In `sh` and `bash`, the equivalent variable is `$PATH`, which encodes the list as a colon-separated string (e.g., `/bin:/usr/bin`). In `rc`, because variables are already lists, `$path` is simply `(/ /bin)`—no special separator needed.

Under Plan 9, the path list is typically short because the system uses *union directories*: multiple directories are bound together into a single mount point like `/bin`, so there is no need for a long search path. This also means you never need the `/usr/bin/env` trick commonly used in UNIX shebang lines. Under UNIX, different systems install programs in different locations (e.g., `/usr/bin/python` vs. `/usr/local/bin/python`), so script authors write `#!/usr/bin/env python` instead of hardcoding the path: `env` searches `$PATH` and runs the first match. Under Plan 9, with union directories, `/bin/rc` is always the right path regardless of the architecture.

See Section 8.2.5 for the implementation.

### 2.3.5 Comments

Comments in `rc` start with `#` and extend to the end of the line. This is why the `#!` shebang works as described above: the line is simply a comment from `rc`'s perspective.

### 2.3.6 Quoting and escaping

The `boot.rc` script uses single quotes in `bind -c '#e' /env`: the `#` must be quoted, otherwise the shell would treat it as a comment.

In `rc`, single quotes are the only quoting mechanism. To include a literal single quote inside a quoted string, you double it: `'it''s'`. There are no backslash escapes and no double quotes (the `"|` character has a different meaning in `rc`: variable stringification, see Section 6.5).

### 2.3.7 The environment

The `boot.rc` script also assigns `objtype=arm`. This is not just a local shell variable: in Plan 9, all shell variables are automatically exported to the environment—there is no need for an `export` command like in `bash`<sup>6</sup>. When `mk` (the build tool) is later run from this shell, it inherits `$objtype` and uses it to select the correct compiler and assembler (e.g., `5c`, `5a`, `5l` for ARM).

Under Plan 9, the environment is stored as files in the `/env` filesystem: each variable becomes a file `/env/varname` whose content is the variable's value. See Chapter 10 for the implementation.

### 2.3.8 Builtins

The last line of `boot.rc` is `exec /boot/rc -m /boot/rcmain -i`. `exec` is a shell *builtin*: a command that is executed inside the shell process rather than in a child. Builtins exist because some operations must modify the shell's own state. For example, `cd` changes the shell's current directory—if `cd` were a regular program, it would run in a forked child, the child would change *its* directory and exit, and the parent shell would be unaffected.

The `exec` builtin replaces the current shell process with the specified command. Here it replaces the boot shell with an interactive instance of `rc` (the `-i` flag). Without `exec`, the boot shell would fork a child `rc`, wait for it to finish, and then continue—but there is nothing left to do, so the extra process would just waste a process slot until the interactive shell exits. See Chapter 9 for the full list of builtins.

### 2.3.9 Other features

The `boot.rc` script is simple enough that it does not use many of `rc`'s advanced features: no pipes, no control flow, no functions. Those features—pipes, redirections, conditionals, loops, pattern matching, functions, command substitution, and glob patterns—will be presented gradually in the following chapters.

---

<sup>6</sup>If you need environment isolation, `rfork e` creates a private copy of `/env/`; see Section 14.5.3.

## 2.4 Code organization

Table 2.1 presents short descriptions of the source files of `rc`, together with the main entities (e.g., structures, functions, globals) the file defines, and the corresponding chapter in this document in which the code contained in the file is primarily discussed.

| Function                   | Ch. | File                     | Entities                                                                                                                                       | LOC  |
|----------------------------|-----|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------|
| main data structures       | 3   | <code>rc.h</code>        | <code>Tree</code> <sup>27a</sup> <code>Code</code> <sup>29c</sup> <code>Word</code> <sup>31c</sup> <code>Var</code> <sup>33c</sup>             |      |
| execution data structures  | 3   | <code>exec.h</code>      | <code>Thread</code> <sup>30b</sup> <code>List</code> <sup>33b</sup>                                                                            |      |
| AST helpers                | 3   | <code>tree.c</code>      | <code>newtree()</code> <sup>27e</sup> <code>tree1()</code> <sup>28b</sup> <code>tree2()</code> <sup>28c</sup>                                  |      |
| globals                    | 3   | <code>globals.c</code>   | <code>codebuf</code> <sup>29b</sup> <code>runq</code> <sup>30c</sup>                                                                           |      |
| list of strings (words)    | 3   | <code>words.c</code>     | <code>newword()</code> <sup>31d</sup> <code>count()</code> <sup>32a</sup> <code>freewords()</code> <sup>32d</sup>                              |      |
| shell variables            | 3   | <code>var.c</code>       | <code>setvar()</code> <sup>34a</sup> <code>vlook()</code> <sup>34b</sup> <code>gvar</code> <sup>34d</sup> <code>gvlook()</code> <sup>34e</sup> |      |
| function prototypes        | 3   | <code>fns.h</code>       |                                                                                                                                                |      |
| entry point                | 4   | <code>main.c</code>      | <code>main()</code> X                                                                                                                          |      |
| execution utilities        | 4   | <code>executils.c</code> | <code>start()</code> <sup>39a</sup> <code>pushlist()</code> <sup>40a</sup> <code>pushword()</code> <sup>40b</sup>                              |      |
| character input and prompt | 5   | <code>input.c</code>     | <code>getnext()</code> <sup>43</sup> <code>pprompt()</code> <sup>45h</sup> <code>nextc()</code> <sup>46e</sup>                                 |      |
| lexer                      | 6   | <code>lex.c</code>       | <code>yylex()</code> <sup>48a</sup>                                                                                                            |      |
| parser                     | 7   | <code>syn.y</code>       | <code>yyparse()</code> <sup>57b</sup>                                                                                                          |      |
| bytecode generation        | 8   | <code>code.c</code>      | <code>emitf()</code> <sup>64d</sup> <code>compile()</code> <sup>65a</sup> <code>outcode()</code> <sup>65f</sup>                                |      |
| process status             | 8   | <code>status.c</code>    | <code>setstatus()</code> <sup>67c</sup> <code>getstatus()</code> <sup>67d</sup>                                                                |      |
| simple command bytecode    | 8   | <code>simple.c</code>    | <code>Xsimple()</code> <sup>70</sup> <code>execexec()</code> <sup>72b</sup> <code>doredir()</code> <sup>79d</sup>                              |      |
| \$path management          | 8   | <code>path.c</code>      | <code>searchpath()</code> <sup>73a</sup>                                                                                                       |      |
| process-related bytecodes  | 8   | <code>processes.c</code> | <code>execforkexec()</code> <sup>72a</sup> <code>Xpipe()</code> <sup>85b</sup> <code>Xasync()</code> <sup>86d</sup>                            |      |
| other bytecodes            | 8   | <code>exec.c</code>      | <code>Xtrue()</code> <sup>76e</sup> <code>Xjump()</code> <sup>89g</sup> <code>Xfn()</code> <sup>94c</sup>                                      |      |
| shell builtins             | 9   | <code>builtins.c</code>  | <code>execcd()</code> <sup>103c</sup> <code>execexit()</code> <sup>105c</sup> <code>execdot()</code> <sup>105d</sup>                           |      |
| shell environment          | 10  | <code>env.c</code>       | <code>addenv()</code> <sup>111b</sup> <code>Vinit()</code> <sup>111c</sup>                                                                     |      |
| signal management          | 11  | <code>trap.c</code>      | <code>dotrap()</code> <sup>115b</sup> <code>Trapinit()</code> <sup>114d</sup>                                                                  |      |
| initialization (bootstrap) | 12  | <code>main.c</code>      | <code>Rcmain</code> <sup>118a</sup> <code>dotcmds</code> <sup>107b</sup>                                                                       |      |
| wildcard matching          | 13  | <code>glob.c</code>      | <code>glob()</code> <sup>121c</sup> <code>match()</code> <sup>124b</sup>                                                                       |      |
| here documents             | 14  | <code>here.c</code>      | <code>readhere()</code> <sup>132a</sup> <code>heredoc()</code> <sup>133f</sup>                                                                 |      |
| AST dumper                 | A   | <code>pcmd.c</code>      | <code>pcmd()</code> <sup>151e</sup>                                                                                                            |      |
| bytecode dumper            | A   | <code>pfnc.c</code>      | <code>pfnc()</code> <sup>154b</sup>                                                                                                            |      |
| error management           | B   | <code>error.c</code>     | <code>panic()</code> <sup>157c</sup>                                                                                                           |      |
| memory management          | C   | <code>utils.c</code>     | <code>emalloc()</code> <sup>158b</sup> <code>efree()</code> <sup>158d</sup>                                                                    |      |
| command-line flags         | C   | <code>getflags.c</code>  | <code>getflags()</code> <sup>159i</sup> <code>usage()</code> <sup>161</sup>                                                                    |      |
| buffered IO                | C   | <code>io.c</code>        | <code>openfd()</code> <sup>163e</sup> <code>rchr()</code> <sup>164c</sup>                                                                      |      |
| pretty printer             | C   | <code>fnt.c</code>       | <code>pfmt()</code> <sup>166b</sup>                                                                                                            |      |
| Total                      |     |                          |                                                                                                                                                | 6800 |

Table 2.1: Chapters and associated `rc` source files.

Every source file in `rc` includes the same set of headers. `rc.h` defines the main data structures (`Tree`, `Code`, `Word`, `Var`), `exec.h` defines the execution data structures (`Thread`, `List`, `Redir`<sup>78c</sup>), `fns.h` contains all function prototypes, `getflags.h` provides the command-line flag parsing API, and `io.h` defines the buffered IO layer.

```

<includes 21>≡ (213 212 211 210 209 208 207 206 203 202 201 200b 199b)
#include "rc.h"
#include "exec.h"
#include "fns.h"
#include "getflags.h"
#include "io.h"

```

## 2.5 Software architecture

Before describing the internal architecture of `rc` itself, I will first present the broader context in which `rc` operates: the command-line user interface and the other system components (kernel, device drivers, terminal) that `rc` interacts with. Then I will describe `rc`'s own components and trace the execution of a simple command through them.

### 2.5.1 The command-line user interface

The shell is only one component of the command-line user interface (CLI). Figure 2.1 shows the components supporting the CLI under Plan 9 (the components for the CLI under UNIX are very similar).

To run commands, the shell needs first a kernel to create processes. The kernel provides services to applications (including the shell) through its *system call interface* (also known as *syscalls*). For example, under Plan 9, `rfork()` creates a new process in which the shell can then `exec()` a program (see KERNEL book [Pad14]).

A shell needs also device drivers in the kernel handling the terminal with its keyboard and monitor. To access those devices, the kernel provides a *filesystem interface* (called a *namespace* under Plan 9). An application can `open()`, `read()`, `write()`, or `close()` files in this filesystem. Under UNIX and Plan 9, devices are represented as files. For example, `/dev/cons` (for console) is a file representing the terminal. To read characters from the keyboard, an application can simply read characters from `/dev/cons`. To write characters on the monitor, an application can simply write characters in `/dev/cons`.

A key UNIX innovation is that the shell is a regular user program, not part of the kernel. When the kernel boots, it starts an initial process whose `stdin` and `stdout` are connected to `/dev/cons`. Every process forked from it inherits this connection, including `rc`. Because the shell is just a program, you can replace it—or run a different shell inside the current one—without changing the kernel.

### 2.5.2 `rc`'s components

Figure 2.2 describes the main data flow of `rc`, whereas Figure 2.3 describes the main control flow of `rc`. At its core, `rc` is both a compiler and a bytecode interpreter. As shown by Figure 2.2, given a series of characters (coming either from what you typed in the terminal or from the content of a script), `rc` first groups those characters in tokens (with `yylex()`<sup>48a</sup>). Then, `rc` parses those tokens (with `yyparse()`<sup>57b</sup>) and builds an abstract syntax tree (AST) of the program (see `Tree`<sup>27a</sup>). Finally, `rc` transforms this tree in a series of bytecodes (see `Code`<sup>29c</sup>). This is similar to what a compiler such as `5c` does (see COMPILER book [Pad16b]), except a bytecode here is not an instruction from a concrete machine but from a *virtual machine*.

After this compilation, `rc` goes through the series of bytecodes and interprets them. `rc` keeps track of what is currently executing in a `Thread`<sup>30b</sup> data structure and in a queue `runq`<sup>30c</sup>. This data structure is called a “thread” because `rc` must sometimes manage multiple threads of execution. Indeed, some of the bytecodes can create new processes running concurrently (e.g., `Xpipe()`<sup>85b</sup>, the bytecode handling pipes).

In fact, `rc` does not start by compiling, but by interpreting. Indeed, `rc` starts first by interpreting some special bytecodes, called the *bootstrap*, that contains a bytecode (`Xrdcmds()`<sup>41a</sup>) that then triggers the compiler. I will now explain briefly the control flow of `rc`, starting from the top of Figure 2.3.

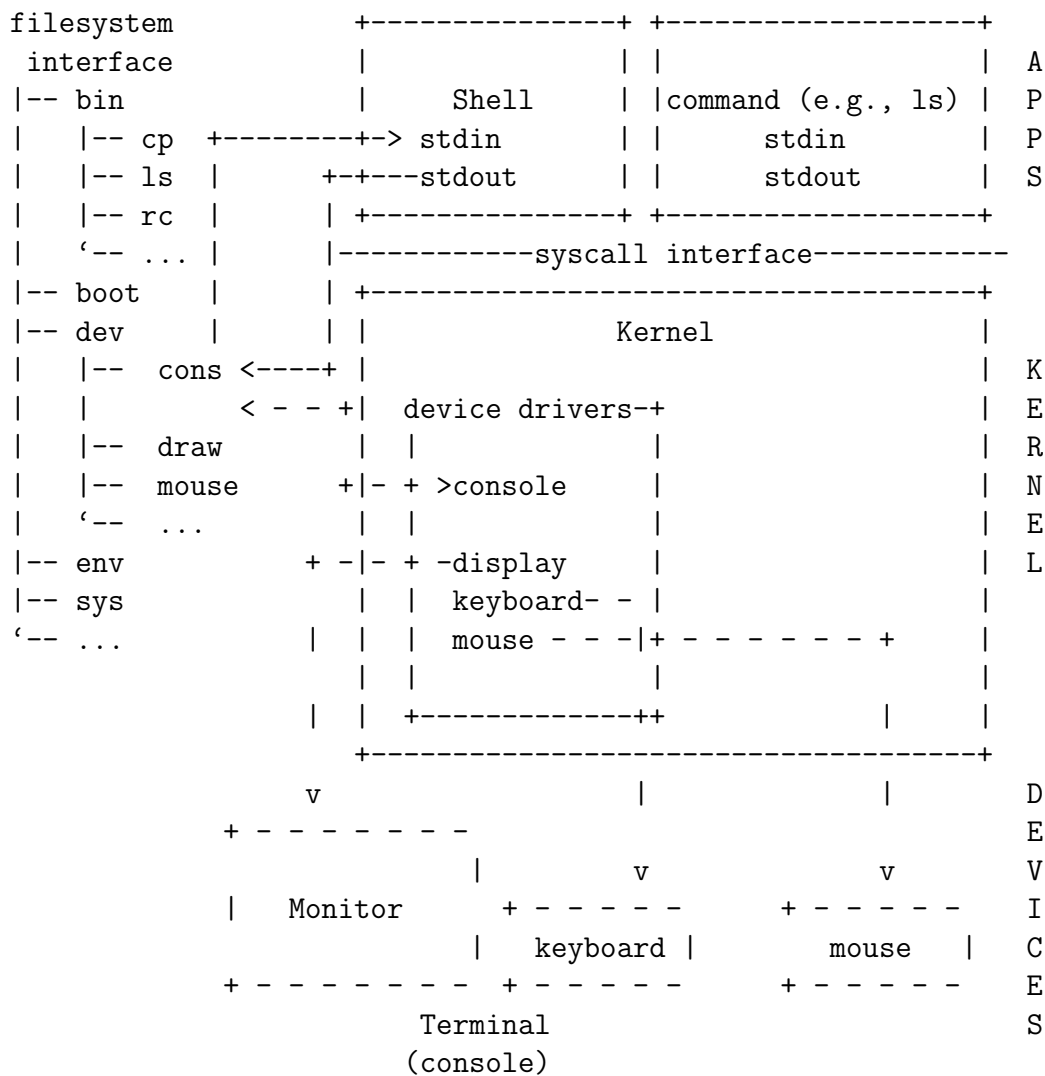


Figure 2.1: Components of the command-line user interface under Plan 9.

Input characters --> tokens --> AST --> bytecodes --> threads --> processes

Figure 2.2: Data flow diagram of rc.

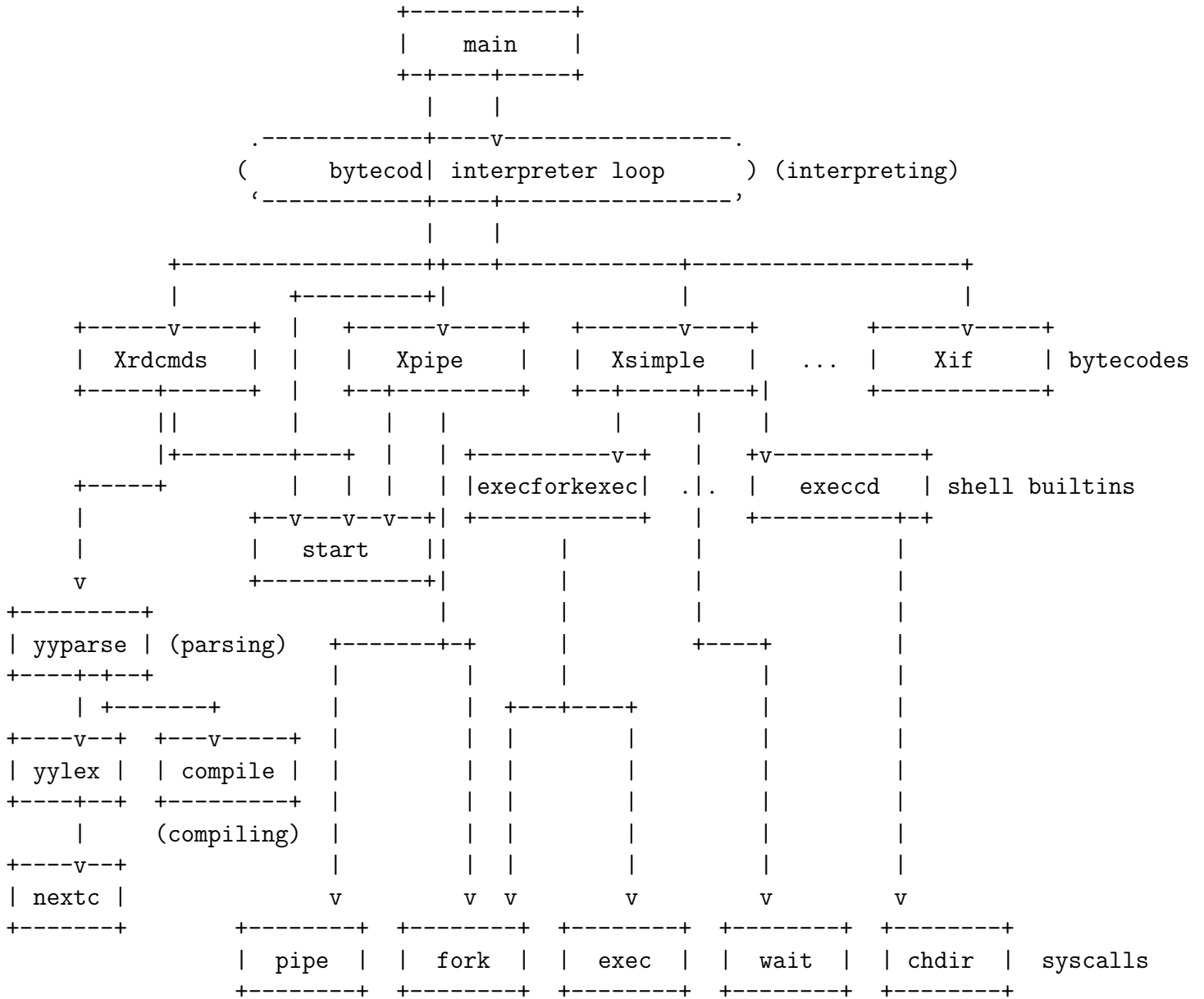


Figure 2.3: Control flow diagram of `rc`.

After some basic initializations, `main()` sets its local `bootstrap`<sup>36a</sup> to contain the initial set of bytecodes to execute. It then calls `start()`<sup>39a</sup> with `bootstrap` as an argument, to start interpreting this series of bytecodes. Internally, `start()` just modifies the global `runq` to point to a newly created `Thread`, and sets the field `Thread.code`<sup>30b</sup> to the content of `bootstrap`. After `start()` returns, `main()` goes in a loop that interprets the bytecodes in `runq->code`.

The most important bytecode in `bootstrap` is `Xrdcmds()`, which reads a command (hence its name) and starts a new `Thread`. `Xrdcmds()` first calls the parser `yyparse()`, which calls the lexer `yylex()`, which calls the input routine `nextc()`<sup>46e</sup> to get the next character. By default, `nextc()` reads characters from the standard input, and so it waits for the characters you type in a terminal. Once you finish entering a command with a newline, `yyparse()` will call `compile()`<sup>65a</sup> with the tree it built during parsing as an argument. Internally, `compile()` modifies the global `codebuf`<sup>29b</sup> to store the bytecodes deriving from the tree. Then, after `yyparse()` returned, `Xrdcmds()` calls `start()` to start a new thread with `codebuf` as a parameter. `start()` then modifies again the global `runq`, and when `Xrdcmds()` returns, the main bytecode interpreter loop will process a new series of bytecodes stored in `runq.code`.

Note that before modifying `runq`, `start()` first links the newly created thread with the old thread (through the `Thread.ret`<sup>30d</sup> field). Moreover, `compile()` adds the bytecode `Xreturn()`<sup>84d</sup> at the end of the series of bytecodes deriving from your command. Thus, after `main()` finished interpreting the bytecodes of your command, it will process `Xreturn()`, which will modify `runq` to point to the old thread, the one containing the bootstrap bytecodes. Then, after `Xreturn()` returns, the main bytecode interpreter loop will process again the bytecodes from the bootstrap, which will read another command through `Xrdcmds()`.

In `rc`, the bytecodes are represented by regular C functions starting by convention with an `X` (e.g., `Xrdcmds()`, `Xpipe()`, `Xif()`<sup>88b</sup>). Thus, the bytecode interpreter is mainly a function dispatcher. Internally those functions perform system calls to the kernel to create a pipe (`pipe()`), to fork a new process (`fork()`), to wait for a child process (`wait()`), or to change directory (`chdir()`). An important bytecode is `Xsimple()`<sup>70</sup>, which `rc` uses to run a “simple” command. `Xsimple()` represents the essence of a shell: with the series of system calls `fork()`, `exec()`, and `wait()`, `rc` can run a command in a new process and wait for its termination (or interruption). `Xsimple()` is also responsible for managing the multiple shell builtins. Indeed, if the name of the “simple” command is a builtin (e.g., `cd`), then `Xsimple()` dispatches the appropriate function (e.g., `execcd`) instead of forking a new process.

### 2.5.3 Trace of a simple command: `ls /`

You can see the bytecodes and the threads created internally by `rc` by running `rc` with the `-r` flag. Here is an example of a trace of the simple command `ls /`:

```
% rc -r
pid 39 cycle 0002D930 1 Xmark ()
...
pid 38 cycle 0001984C 9 Xrdcmds
% ls /
pid 38 cycle 0002CBD0 1 Xmark
pid 38 cycle 0002CBD0 2 Xword ()
pid 38 cycle 0002CBD0 4 Xword (/)
pid 38 cycle 0002CBD0 6 Xsimple (ls /)
bin
boot
...
srv
pid 38 cycle 0002CBD0 7 Xreturn
pid 38 cycle 0001984C 9 Xrdcmds
```

It is not important to fully understand the format of this trace (see Section A.2 for the full explanation and for the code handling `rc -r`), but you can recognize a few of the bytecodes I mentioned before: `Xrdcmds()`<sup>41a</sup>, `Xsimple()`<sup>70</sup>, and `Xreturn()`<sup>84d</sup>. I will explain `Xmark()`<sup>66b</sup> and `Xword()`<sup>66a</sup> later in this document.

## 2.6 Book structure

You now have enough background to understand the source code of `rc`. The rest of the book is organized as follows. I will start by describing the core data structures of `rc` in Chapter 3. Then, I will use a top-down approach, starting with Chapter 4 with the description of `main()`X, the initialization of `rc`, the bytecode interpreter loop, and the function `Xrdcmds()`<sup>41a</sup>. The following chapters will describe the main components of the compiler pipeline: Chapter 5 will present the input routines, Chapter 6 the lexer, Chapter 7 the parser, and finally Chapter 8 the bytecode generator and bytecode interpreter for the main features of `rc`. In Chapter 9, I will present the code of the different shell builtins (e.g., `execd()`<sup>103c</sup> for `cd`). Chapter 10 contains the code to manage the environment of the processes launched from the shell, and Chapter 11 the code to manage the signals sent to the processes (also known as *notes* under Plan 9). Chapter 12 presents the actual code initializing `rc`. Indeed, Chapter 4 presents only a simplified initialization to not introduce too much complexity early-on. Then, I will present advanced features of `rc` that I did not present before to simplify the explanations, for instance, wildcard matching (e.g., `ls *.c`) in Chapter 13, or command substitutions (e.g., `{ls}`) in Chapter 14. Those advanced features tend to crosscut many components of `rc` with extensions to the lexer, the parser, the bytecode generator, and the bytecode interpreter. Finally, Chapter 15 concludes and gives pointers to other books in the Principia Softwarica series.

Some appendices present the code of non-functional properties: code to help debug `rc` itself in Appendix A and code to manage errors in Appendix B. Appendix C contains the code of utility functions used by `rc` but that are not specific to `rc` (e.g., a library to manage string buffers). Finally, Appendix D presents examples of `rc` scripts.

# Chapter 3

## Core Data Structures

*Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.*

*Fred Brooks*

In this chapter, I will present the core data structures of `rc`. Most of those data structures are listed in Figure 2.2, which presented the data flow of `rc`. The first three sections will present the data structures of the compiler part of `rc`: the token, the abstract syntax tree, and the bytecode. The bytecode is a data structure used both by the compiler and interpreter. The following sections will present the remaining data structures of the interpreter part of `rc`: the thread, and the variable, which is itself defined by a list of words.

### 3.1 Token

As mentioned in the previous chapter, the first step of the compilation pipeline is to transform a stream of characters into a stream of tokens (also called lexemes). A token groups one or more characters that logically belong together (e.g., `while` is a single keyword token, not five character tokens). The lexer `yylex()`<sup>48a</sup> (described in Chapter 6) is responsible for this transformation.

The token kinds in `rc` are declared below using the `yacc` syntax `%token` (see COMPILER book [Pad16b]). I have grouped them in categories: the keyword tokens (`FOR`, `WHILE`, `IF`, etc.), the operator tokens (`REDIR`, `PIPE`, `ANDAND`, etc.), and the catch-all `WORD` token for anything else (e.g., command names, arguments, numbers, or file paths). Note that some single-character operators like `'$'`, `'>'`, `'<'`, or `'='` do not have a dedicated token kind; the lexer simply returns their ASCII value directly (see `yylex()` and `pcmd()`<sup>151e</sup>).

```
<token declarations 26>≡ (57a) 27b▷
%token FOR IN WHILE IF NOT SWITCH FN
%token TWIDDLE BANG /** ~ ! */
%token REDIR PIPE /** {>, <, <<, >>} | */
%token ANDAND OROR /** && || */
%token COUNT SUB /** $# ( */
%token WORD /** anything else (e.g. foo, --help, 42, /a/b/c, etc) */
```

### 3.2 Abstract syntax Tree

The second step of the compilation pipeline transforms the stream of tokens into an abstract syntax tree (AST). The parser `yyparse()`<sup>57b</sup> (described in Chapter 7) is responsible for this transformation. `rc` has a *command language*: it has variables, control flow statements (`if`, `while`, `for`, `switch`), functions (`fn`), and operators (pipes, redirections, etc.). All those constructs are represented in the AST using the `Tree` structure below.

The `Tree` structure is a generic tree node with a `type` field indicating what kind of node it is (e.g., a `PIPE`, an `IF`, a `WORD`, or even a single character like `';`' for sequencing), a `str` field to store the string content of the token (e.g., the command name or the variable name), and up to three children. Three children are enough to represent all the constructs of `rc`: for example, an `if` uses `child[0]` for the condition, `child[1]` for the then-body, and leaves `child[2]` unused. A `for` uses all three: `child[0]` for the variable, `child[1]` for the list, and `child[2]` for the body.

```

⟨struct Tree 27a⟩≡ (196c)
struct Tree {

    // either<enum<Token_kind>, char>
    int type;
    // string of the token or AST dump of the whole subtree for certain nodes
    char *str;

    // array<option<ref_own<Tree>>
    tree *child[3];

    ⟨Tree redirection and pipe specific fields 52a⟩
    ⟨Tree word specific fields 54a⟩

    // Extra
    ⟨Tree extra fields 27d⟩
};

```

The parser also introduces a few extra node types that do not correspond to tokens produced by the lexer, but that are useful to structure the AST. For example, `SIMPLE` represents a simple command with its arguments, and `ARGLIST` groups those arguments into a list.

```

⟨token declarations 27b⟩+≡ (57a) <26 63a>
/* not used in syntax */
%token SIMPLE
%token ARGLIST WORDS
%token BRACE PAREN

```

All tree nodes allocated during parsing are linked together in a global list `treenodes` through the `Tree.next` field below. This makes it easy to free all nodes at once (via `freenodes()`<sup>28a</sup>) after the tree has been compiled to bytecodes in `Xrdcmds()`<sup>41a</sup>, instead of having to walk the tree recursively.

```

⟨global treenodes 27c⟩≡ (206b)
// list<ref_own<Tree>> (next = Tree.next)
tree *treenodes;

⟨Tree extra fields 27d⟩≡ (27a)
tree *next;

```

The functions below are used by the parser to build AST nodes. `newtree()`<sup>27e</sup> allocates a fresh node and links it into `treenodes`. The convenience wrappers `tree1()`<sup>28b</sup>, `tree2()`<sup>28c</sup>, and `tree3()`<sup>28d</sup> create a node with a given type and one, two, or three children respectively.

```

⟨function newtree 27e⟩≡ (206b)
/*
 * create and clear a new tree node, and add it
 * to the node list.
 */
tree*
newtree(void)
{
    tree *t = new(tree);
    t->str = nil;
    t->child[0] = t->child[1] = t->child[2] = nil;
}

```

```

// add_list(t, treenodes)
t->next = treenodes;
treenodes = t;

return t;
}

```

Uses treenodes 27c.

```

⟨function freenodes 28a⟩≡ (206b)
void
freenodes(void)
{
    tree *t, *u;
    for(t = treenodes; t; t = u){
        u = t->next;
        if(t->str)
            efree(t->str);
        efree((char *)t);
    }
    treenodes = nil;
}

```

Uses efree() 158d and treenodes 27c.

```

⟨function tree1 28b⟩≡ (206b)
tree*
tree1(int type, tree *c0)
{
    return tree3(type, c0, (tree *)nil, (tree *)nil);
}

```

Uses tree3() 28d.

```

⟨function tree2 28c⟩≡ (206b)
//@Scheck: used by syn.y
tree* tree2(int type, tree *c0, tree *c1)
{
    return tree3(type, c0, c1, (tree *)nil);
}

```

Uses tree3() 28d.

```

⟨function tree3 28d⟩≡ (206b)
tree*
tree3(int type, tree *c0, tree *c1, tree *c2)
{
    tree *t;

    ⟨tree3 if some empty sequence 29a⟩
    // else
    t = newtree();
    t->type = type;
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}

```

Uses newtree() 27e.

The special case below is an optimization for the sequencing operator ‘;’: if one side of a sequence is empty (which happens frequently since each newline produces an empty command), `tree3()` simply returns the other side instead of creating a useless node.

```
⟨tree3 if some empty sequence 29a⟩≡ (28d)
    if(type==';'){
        if(c0==nil)
            return c1;
        if(c1==nil)
            return c0;
    }
```

### 3.3 ByteCode and codebuf

The third step of the compilation pipeline transforms the AST into a series of bytecodes. The function `compile()`<sup>65a</sup> (described in Chapter 8) is responsible for this transformation, and stores the resulting bytecodes in the global `codebuf`<sup>29b</sup> below. Like Java or Python, `rc` does not interpret the AST directly but compiles it first into a lower-level representation that is then interpreted by a small stack-based virtual machine (described in Chapter 4).

```
⟨global codebuf 29b⟩≡ (199b)
// growing_array<ref_own<Code>>
code *codebuf; /* compiler output */
```

A bytecode is represented by the `Code` union below. Each element in the `codebuf` array is either: a function pointer `f` (the bytecode itself, e.g., `Xsimple`<sup>70</sup>, `Xpipe`<sup>85b</sup>), an integer `i` (used for jump offsets or file descriptor numbers), or a string `s` (used for inline string arguments like in `Xword`<sup>66a</sup>). The first element of any code vector is always a reference count integer, managed by `codecopy()`<sup>29d</sup> and `codefree()`<sup>29e</sup> below. This reference counting is needed because bytecode vectors can be shared, for instance when a function is defined with `fn`.

```
⟨struct Code 29c⟩≡ (196c)
/*
 * The first word of any code vector is a reference count.
 * Always create a new reference to a code vector by calling codecopy(.).
 * Always call codefree(.) when deleting a reference.
 */
union Code {
    void (*f)(void); // Xxx() bytecode
    int i;
    char *s;
};
```

```
⟨function codecopy 29d⟩≡ (209)
code*
codecopy(code *cp)
{
    cp[0].i++;
    return cp;
}
```

```
⟨function codefree 29e⟩≡ (209)
void
codefree(code *cp)
{
    code *p;
    // check ref count
    if(--cp[0].i != 0)
        return;
```

```

    for(p = cp+1; p->f; p++){
        <codefree() in loop over code cp, switch bytecode cases 30a>
    }
    efree((char *)cp);
}

```

Uses `efree()` 158d.

```

<codefree() in loop over code cp, switch bytecode cases 30a>≡ (29e) 69f>
    if(p->f==Xfalse || p->f==Xtrue
    || p->f==Xread || p->f==Xwrite || p->f==Xrdwr
    || p->f==Xappend || p->f==Xclose
    || p->f==Xasync || p->f==Xbackq || p->f==Xcase
    || p->f==Xfor || p->f==Xjump
    || p->f==Xsubshell)
        p++;

```

Uses `Xappend()` 82b, `Xasync()` 86d, `Xbackq()` 140, `Xcase()` 92c, `Xclose()` 137i, `Xfalse()` 76f, `Xfor()` 90b, `Xjump()` 89g, `Xrdwr()` 135d, `Xread()` 81c, `Xsubshell()` 130d, `Xtrue()` 76e, and `Xwrite()` 80d.

### 3.4 Thread and runqueue

Once the bytecodes are compiled, `rc` needs a data structure to keep track of the current execution state. This is the role of the `Thread` structure. A `Thread` is similar to a stack frame in a regular program: it contains a pointer to a code vector (`code`) and a program counter (`pc`) indexing the next bytecode to execute. The name “thread” might be confusing since these are not OS-level threads or processes. They are *units of execution* inside the `rc` interpreter. When a bytecode like `Xpipe()`<sup>85b</sup> or `Xasync()`<sup>86d</sup> needs to run commands concurrently, it creates actual OS processes (with `fork()`), but within the interpreter itself, threads are managed as a stack.

The global `runq`<sup>30c</sup> (for “run queue”) points to the top of this stack of threads. Threads are linked through the `ret` field: when the current thread finishes (via `Xreturn()`<sup>84d</sup>), `rc` pops it and resumes the thread pointed to by `ret`. At the bottom of the stack sits the bootstrap thread, which loops forever reading new commands via `Xrdcmds()`<sup>41a</sup>.

```

<struct Thread 30b>≡ (199a)
struct Thread {
    union Code *code; /* code for this thread */
    int pc; /* code[pc] is the next instruction */

    <Thread other fields 31a>

    // Extra
    <Thread extra fields 30d>
};

```

Uses `Code` 29c.

```

<global runq 30c>≡ (199b)
// stack<ref_own<Thread>> (next = Thread.ret)
thread *runq;

```

```

<Thread extra fields 30d>≡ (30b)
thread *ret; /* who continues when this finishes */

```

The most important field of a thread is `argv`, the argument stack. It is a list of lists of words: bytecodes like `Xmark()`<sup>66b</sup> push a new empty list on this stack, and `Xword()`<sup>66a</sup> pushes a word onto the current list. When a

bytecode like `Xsimple()`<sup>70</sup> needs to run a command, it pops the top list from `argv` and uses it as the command and its arguments.

```
<Thread other fields 31a>≡ (30b) 31b▷
// list<list<ref_own<word>>> (next = List.next)
struct List *argv; /* argument stack */
```

Uses List 33b.

Each thread also has a list of local variables. These are created by the `local` builtin or by constructs like `for` that bind a variable for the duration of a block. When looking up a variable with `vlook()`<sup>34b</sup>, `rc` first searches the current thread's locals before falling back to the global variable table.

```
<Thread other fields 31b>+≡ (30b) <31a 41b>
// list<ref_own<Var>> (next = Var.next)
struct Var *local; /* list of local variables */
```

Uses Var 33c.

## 3.5 Words and lists of words

I mentioned the `argv` field in the `Thread` structure above, which is a list of lists of words. Let me now present the `Word` and `List` structures that make up this two-level list.

### 3.5.1 Words

A `Word` is simply a string with a `next` pointer, forming a linked list. This is the fundamental data type in `rc`: a command like `ls -l /tmp` is represented as a list of three words. Variables in `rc` can also hold multiple values (e.g., `x=(a b c)`), which are represented as a list of words.

```
<struct Word 31c>≡ (196c)
/*
 * word lists are in correct order,
 * i.e. word0->word1->word2->word3->nil
 */
struct Word {
    char *word;

    // Extra
    word *next;
};
```

The functions below provide the usual linked-list operations for word lists: `newword()`<sup>31d</sup> allocates a new word and prepends it to a list, `count()`<sup>32a</sup> returns the length, `copywords()`<sup>32c</sup> duplicates a list, and `freewords()`<sup>32d</sup> deallocates it.

```
<function newword 31d>≡ (212b)
word*
newword(char *wd, word *next)
{
    word *p = new(struct Word);
    p->word = strdup(wd);
    p->next = next;
    return p;
}
```

Uses Word 31c.

*<function count 32a>*≡ (212b)

```
int
count(word *w)
{
    int n;
    for(n = 0;w;n++)
        w = w->next;
    return n;
}
```

`copynwords()`<sup>32b</sup> and `copywords()` use the classic C pointer-to-pointer tail linking idiom: `end` starts as `&v` (address of the head pointer), and after each node is created, it advances to `&(*end)->next` (address of the new node's `next` field). This avoids special-casing the first element and builds the list in forward order without reversing at the end.

*<function copynwords 32b>*≡ (212b)

```
word*
copynwords(word *a, word *tail, int n)
{
    word *v = nil;
    word **end = &v;

    while(n-- > 0){
        *end = newword(a->word, 0);
        end = &(*end)->next;
        a = a->next;
    }
    *end = tail;
    return v;
}
```

Uses `newword()` 31d.

*<function copywords 32c>*≡ (212b)

```
/*
 * copy arglist a, adding the copy to the front of tail
 */
word*
copywords(word *a, word *tail)
{
    word *v = nil;
    word **end;

    for(end=&v;a;a = a->next,end=&(*end)->next)
        *end = newword(a->word, nil);
    *end = tail;
    return v;
}
```

Uses `newword()` 31d.

*<function freewords 32d>*≡ (212b)

```
void
freewords(word *w)
{
    word *nw;
    while(w){
        efree(w->word);
        nw = w->next;
        efree((char *)w);
        w = nw;
    }
}
```

```
}
Uses efree() 158d.
```

```
<function freelist 33a>≡ (212b)
void
freelist(word *w)
{
    word *nw;
    while(w){
        nw = w->next;
        efree(w->word);
        efree((char *)w);
        w = nw;
    }
}
```

Uses `efree()` 158d.

### 3.5.2 List of sequence of words

The `List` structure adds one more level of indirection on top of word lists. It is used for `Thread.argv`, the argument stack: each element in this stack is a `List` node containing a word list. For example, when the interpreter processes `ls -l /tmp`, the bytecodes `Xmark()`<sup>66b</sup> and `Xword()`<sup>66a</sup> will first push the words `ls`, `-l`, `/tmp` onto a word list, and this word list sits inside a `List` node on the `argv` stack.

```
<struct List 33b>≡ (199a)
struct List {
    // list<ref_own<Word>> (next = Word.next)
    word *words;

    // Extra
    list *next;
};
```

## 3.6 Variables

I already mentioned local variables in `Thread.local` above. Let me now present the `Var` structure itself and the global variable table. Variables are needed whenever you write something like `x=1` or `path=(/bin /usr/bin)` in the shell. They are also used internally by `rc` for special variables like `status` (the exit status of the last command) or `*` (the script arguments).

A `Var` has a `name` (the variable name as a string), a `val` (the variable value as a list of words, since in `rc` all variables can hold multiple values), and a `next` pointer for chaining in hash buckets or local lists.

```
<struct Var 33c>≡ (196c)
struct Var {
    // key
    char *name; /* ascii name */
    // value
    word *val; /* value */

    <Var other fields 93c>
    // Extra
    <Var extra fields 33d>
};

<Var extra fields 33d>≡ (33c)
var *next; /* next on hash or local list */
```

The main interface to the variable system consists of `setvar()`<sup>34a</sup> to assign a value, and `vlook()`<sup>34b</sup> to look up a variable by name. `vlook()` first searches the local variables of the current thread (stored in `runq->local`), then falls back to the global variable table via `gvlook()`<sup>34e</sup>.

```
<function setvar 34a>≡ (202a)
void
setvar(char *name, word *val)
{
    struct Var *v = vlook(name);
    freewords(v->val);
    v->val = val;
    v->changed = true;
}
```

Uses Var 33c, freewords() 32d, and vlook() 34b.

```
<function vlook 34b>≡ (202a)
var*
vlook(char *name)
{
    var *v;
    if(runq)
        for(v = runq->local; v; v = v->next)
            if(strcmp(v->name, name)==0)
                return v;

    return gvlook(name);
}
```

Uses gvlook() 34e and runq 30c.

Global variables are stored in a simple hash table of 521 buckets, with chaining through `Var.next` for collisions. `gvlook()` looks up a global variable and automatically creates it (with a `nil` value) if it does not exist yet.

```
<constant NVAR 34c>≡ (196c)
#define NVAR 521
```

```
<global gvar 34d>≡ (202a)
// map<string, ref_own<Var>> (next = Var.next in bucket list)
var *gvar[NVAR]; /* hash for globals */
% vs locals
```

Uses NVAR 34c.

```
<function gvlook 34e>≡ (202a)
var*
gvlook(char *name)
{
    int h = hash(name, NVAR);
    var *v;

    for(v = gvar[h]; v; v = v->next)
        if(strcmp(v->name, name)==0)
            return v;
    gvar[h] = newvar(strdup(name), gvar[h]);
    return gvar[h];
}
```

Uses NVAR 34c, gvar 34d, hash() 35a, and newvar() 35b.

$\langle$ function hash 35a $\rangle \equiv$  (202a)

```

unsigned
hash(char *as, int n)
{
    int i = 1;
    unsigned h = 0;
    uchar *s;

    s = (uchar *)as;
    while (*s)
        h += *s++ * i++;
    return h % n;
}
    
```

$\langle$ function newvar 35b $\rangle \equiv$  (202a)

```

var*
newvar(char *name, var *next)
{
    var *v = new(var);
    v->name = name;
    v->val = nil;

    v->fn = nil;
    v->changed = false;
    v->fnchanged = false;

    v->next = next;
    return v;
}
    
```

# Chapter 4

## main()

I now switch from the bottom-up approach of Chapter 3 to a top-down approach: I will describe in the following chapters the main functions of `rc`, starting in this chapter with `main()`X, the entry point of `rc`.

### 4.1 Overview

The `main()`X function of `rc` performs four tasks: it processes the command-line arguments, it initializes the shell (keywords, traps, environment), it sets up the bootstrap bytecodes and the initial thread, and then it enters the bytecode interpreter loop. I will present here a simplified version of the bootstrap; the actual bootstrap code (which sources `rcmain`) is described in Chapter 12.

```
<main() locals 36a>≡ (36b) ??▷
    code bootstrap[17];

<function main (rc/exec.c) 36b>≡ (213a)
    void main(int argc, char *argv[])
    {
        <main() locals 36a>

        <main() argc argv processing, modify flags 37b>

        <main() initialisation 38a>
        <main() initialize bootstrap 117>
        <main() initialize runq with bootstrap code 38d>
        <main() initialize runq->argv 39c>

        <main() interpreter loop 40c>
    }
```

### 4.2 Command-line arguments processing

`rc` accepts many flags (e.g., `-i` for interactive mode, `-l` for login mode, `-e` for exit-on-error). Flags are stored in a global array indexed by the flag character, where a non-nil entry means the flag is set. Some flags like `-c` and `-m` take a string argument, which is why `flag` stores string arrays rather than simple booleans.

```
<global flag 36c>≡ (200a)
    // map<char, option<array<string>>>
    char **flag[NFLAG];

<constant NFLAG 36d>≡ (196a)
    #define NFLAG 128
```

```
⟨global flagset 37a⟩≡ (200a)
char *flagset[] = {"<flag>"};
```

```
⟨main() argc argv processing, modify flags 37b⟩≡ (36b) 37c▷
argc = getflags(argc, argv, "SsrdiIlxepvVc:1m:1[command]", 1);
if(argc==1)
    usage("[file [arg ...]]");
```

Uses getflags() 159i.

getflags() 159i is defined in Appendix C. The format string encodes which flags exist and which ones take an argument (c:1 means -c takes one argument). It populates the global flag<sup>36c</sup> array declared above.

### 4.2.1 Login mode: rc -l

When the kernel starts rc as a login shell, it sets argv[0] to "-rc" (starting with a dash) rather than passing an explicit -l flag. This is a UNIX convention: the code below detects this and sets flag['l'] accordingly, so that rc will later source the user's profile ( \$home/lib/profile).

```
⟨main() argc argv processing, modify flags 37c⟩+≡ (36b) ◁37b 37d▷
if(argv[0][0]=='-')
    flag['l'] = flagset;
```

Uses flag and flagset.

### 4.2.2 Interactive mode: rc -i

Interactive mode determines whether rc will display a prompt and handle signals differently (e.g., not exiting on ^C). The logic below is subtle: rc is interactive by default when no script file is given (argc==1) and standard input is a terminal. The -I flag forces non-interactive mode (useful when piping commands into rc from another program).

```
⟨main() argc argv processing, modify flags 37d⟩+≡ (36b) ◁37c
if(flag['I'])
    flag['i'] = nil;
else
    if(flag['i']==nil && argc==1 && Isatty(STDIN))
        flag['i'] = flagset;
```

Uses flag and flagset.

Under Plan 9, there is no isatty() system call. Instead, Isatty()<sup>37e</sup> checks whether the file descriptor corresponds to /dev/cons (the console device) by inspecting its path with fd2path().

```
⟨function Isatty 37e⟩≡ (213b)
bool
Isatty(fdt fd)
{
    char buf[64];

    if(fd2path(fd, buf, sizeof buf) != 0)
        return false;

    /* might be #c/cons during boot - fixed 22 april 2005, remove this later */
    if(strcmp(buf, "#c/cons") == 0)
        return true;

    /* might be /mnt/term/dev/cons */
    return strlen(buf) >= 9 && strcmp(buf+strlen(buf)-9, "/dev/cons") == 0;
}
```

## 4.3 Initialization

The initialization first sets up `err`, a buffered IO handle for standard error (see Appendix C for `openfd()`<sup>163e</sup>). Then it initializes three subsystems: `kinit()`<sup>55e</sup> registers the shell keywords (`if`, `for`, etc.) so the lexer can recognize them, `Trapinit()`<sup>114d</sup> sets up the signal handler (see Chapter 11), and `Vinit()`<sup>111c</sup> reads the process environment (from `/env`) and populates the global variable table (see Chapter 10).

```
<main() initialisation 38a>≡ (36b) 38b▷
    err = openfd(STDERR);
```

Uses `err` 156b and `openfd()` 163e.

```
<main() initialisation 38b>+≡ (36b) <38a 118c▷
    kinit();    // initialize keywords
    Trapinit(); // notify() function setup
    Vinit();    // read environment variables and add them in gvar
```

Uses `Vinit()` 111c and `kinit()` 55e.

## 4.4 Bootstrapping bytecodes (simplified)

As mentioned earlier, `rc` does not start by compiling source code; it starts by *interpreting* a small series of hand-crafted bytecodes called the bootstrap. In its simplest form, the bootstrap just contains `Xrdcmds`<sup>41a</sup>, the bytecode that reads a command, compiles it, and starts a new thread to execute it. The first element is the reference count (set to 1), and the last element is a zero sentinel marking the end of the code vector. The actual bootstrap is more complex (see Chapter 12): it sources the `rcmain` initialization script, which sets up the default `$path` and runs the user's profile.

```
<main() initialize bootstrap (simplified) 38c>≡
    memset(bootstrap, 0, sizeof bootstrap);

    i = 0;
    bootstrap[i++].i = 1; // reference count
    bootstrap[i++].f = Xrdcmds;
    bootstrap[i].i = 0;
```

## 4.5 Setting runq

`start()`<sup>39a</sup> creates a new thread and pushes it on top of `runq`<sup>30c</sup>. The program counter starts at 1 (not 0) because element 0 of any code vector is reserved for the reference count. Note how `start()` links the new thread to the old `runq` through `p->ret = runq`: this is how the interpreter knows where to resume after the current thread finishes.

```
<main() initialize runq with bootstrap code 38d>≡ (36b) 41c▷
    start(bootstrap, 1, (var *)nil);
```

Uses `start()` 39a.

```

⟨function start 39a⟩≡ (203a)
void
start(code *c, int pc, var *local)
{
    struct Thread *p = new(struct Thread);

    p->code = codecopy(c);
    p->pc = pc;

    p->argv = nil;
    p->local = local;

    p->cmdfile = nil;
    p->cmdfd = nil;
    p->lineno = 1;
    p->eof = false;
    p->iflag = false;

    ⟨start() set redir 78b⟩

    // add_stack(runq, p)
    p->ret = runq;
    runq = p;
}

```

Uses Thread 30b, codecopy() 29d, and runq 30c.

## 4.6 Setting runq->argv

Now that runq<sup>30c</sup> is set, main()X populates its argument stack with the command-line arguments. These arguments are pushed in reverse order because pushword()<sup>40b</sup> prepends to the list, so the first argument ends up at the top. The actual bootstrap code will later assign this list to \\$\$\* (the script arguments variable).

```

⟨global argv0 39b⟩≡ (203a)
/*
 * Start executing the given code at the given pc with the given redirection
 */
char *argv0="rc";

```

Uses argv0 39b.

```

⟨main() initialize runq->argv 39c⟩≡ (36b)
/* prime bootstrap argv */
pushlist();
argv0 = strdup(argv[0]);
for(i = argc-1; i!=0; --i)
    pushword(argv[i]);

```

Uses argv0 39b, pushlist() 40a, and pushword() 40b.

For example, after rc myscript.rc foo bar is initialized, the data structures look like this:

```

runq ----> Thread
    code = bootstrap [1:Xrdcmds, 0]
    pc   = 1
    argv ----> List
                words -> "myscript.rc" -> "foo" -> "bar" -> nil
                next  = nil
    ret   = nil    (bottom of stack)

```

The bootstrap code will then assign this word list to `\$*` and source `rcmain`, which sets up `\$path` and runs the script.

`pushlist()`<sup>40a</sup> pushes a new empty word list onto `runq->argv`, and `pushword()` adds a word to the current top list. These two functions are the main interface for building the argument stack and are used extensively by bytecodes like `Xmark()`<sup>66b</sup> and `Xword()`<sup>66a</sup>.

```
<function pushlist 40a>≡ (203a)
void
pushlist(void)
{
    list *p = new(list);

    // add_list(p, runq->argv)
    p->next = runq->argv;
    p->words = nil;
    runq->argv = p;
}
```

Uses `runq` 30c.

```
<function pushword 40b>≡ (203a)
void
pushword(char *wd)
{
    if(runq->argv==nil)
        panic("pushword but no argv!", 0);
    runq->argv->words = newword(wd, runq->argv->words);
}
```

Uses `newword()` 31d, `panic()` 157c, and `runq` 30c.

## 4.7 Bytecode interpreter loop

The heart of `rc` is this infinite loop. It increments the program counter, then calls the function pointer at the previous position. The increment-then-call-at-minus-one pattern is a common idiom: the `pc` is advanced *before* calling the bytecode function, so that if the bytecode needs to read inline arguments (e.g., a string after `Xword`<sup>66a</sup>), it can read them at `runq->code[runq->pc]` and advance `pc` itself. Between bytecodes, the loop also checks for pending signals (traps).

```
<main() interpreter loop 40c>≡ (36b)
for(;;){
    <main() debug runq in interpreter loop 153b>

    runq->pc++;
    (*runq->code[runq->pc-1].f)();

    <main() handing trap if necessary in interpreter loop 115a>
}
```

Uses `runq` 30c.

## 4.8 Reading commands: `Xrdcmds()`

`Xrdcmds()`<sup>41a</sup> is the bytecode that implements the read-eval loop of `rc`. It calls `yyparse()`<sup>57b</sup> to read and compile one command line, then calls `start()`<sup>39a</sup> to push a new thread executing the compiled bytecodes. The key subtlety is the `--p->pc` before `start()`: this makes the bootstrap thread re-execute `Xrdcmds()` when the command thread finishes (via `Xreturn()`<sup>84d</sup>). This is how `rc` loops forever reading commands without an

explicit loop in the bootstrap bytecodes themselves. Note that `yyparse()` internally calls `compile()`<sup>65a</sup>, which modifies the global `codebuf`<sup>29b</sup>. So when `Xrdcmds()` calls `start(codebuf, ...)`, it starts a thread on the freshly compiled bytecodes.

```

⟨function Xrdcmds 41a⟩≡ (203b)
void
Xrdcmds(void)
{
    struct Thread *p = runq;
    bool error;
    ⟨Xrdcmds() other locals 45e⟩

    ⟨Xrdcmds() flush errors and reset error count 156d⟩
    ⟨Xrdcmds() print status if -s 155a⟩
    ⟨Xrdcmds() set promptstr if interactive mode 45f⟩

    ⟨Xrdcmds() calls Noerror() before yyparse() 116a⟩
    // read one cmd line, compiles it, and modifies codebuf global
    error = yyparse();

    ⟨Xrdcmds() if yyparse() returned an error 41d⟩
    else{
        ⟨Xrdcmds() reset ntrap 116c⟩
        --p->pc; /* re-execute Xrdcmds after codebuf runs */
        // modifies runq, new thread (linked to bootstrap one)
        start(codebuf, 1, runq->local);
    }
    freenodes(); // allocated in yyparse()
}

```

Uses Thread 30b, codebuf 29b, freenodes() 28a, runq 30c, start() 39a, and yyparse().

```

⟨Thread other fields 41b⟩+≡ (30b) <31b 44c>
    struct Io *cmdfd; /* file descriptor for Xrdcmd */
    char *cmdfile; /* file name in Xrdcmd */
    bool iflag; /* interactive? */

```

Uses Io 163d.

Each thread also carries input-related fields: `cmdfd` is the buffered IO handle from which the lexer reads characters, `cmdfile` is the filename (for error messages), and `iflag` determines whether this thread displays a prompt. These are per-thread fields (not globals) because the `.` (dot) builtin and `-c` flag create new threads that read from different sources (a script file, a string), and each thread needs its own input state.

```

⟨main() initialize runq with bootstrap code 41c⟩+≡ (36b) <38d
    runq->cmdfd = openfd(STDIN); // reading from stdin
    runq->cmdfile = "<stdin>";
    runq->iflag = flag['i']? true : false; // interactive mode; will print a prompt

```

Uses flag, openfd() 163e, and runq 30c.

When `yyparse()` returns an error, the behavior depends on the mode. In non-interactive mode (or on a genuine EOF), `rc` closes the input and returns to the parent thread. In interactive mode, it simply prints a newline after an interrupt (`^C`) and goes back to read the next command.

```

⟨Xrdcmds() if yyparse() returned an error 41d⟩≡ (41a)
    if(error){
        if(!p->iflag || p->eof && !Eintr()){
            if(p->cmdfile)
                efree(p->cmdfile);
            closeio(p->cmdfd);

            Xreturn(); /* should this be omitted? */
        }
    }

```

```

}else{
    if(Eintr()){
        pchr(err, '\n');
        p->eof = false;
    }
    --p->pc; /* go back for next command */
}
}

```

Uses Xreturn() 84d, closeio() 165a, efree() 158d, err 156b, and pchr() 164a.

# Chapter 5

## Input

Recall the main flow from Chapter 4: `main()`<sup>X</sup>  $\rightarrow$  `Xrdcmds()`<sup>41a</sup>  $\rightarrow$  `yyparse()`<sup>57b</sup>  $\rightarrow$  `yylex()`<sup>48a</sup>. Before describing the lexer, I will present the input routines it relies on. These routines handle reading characters from the current input source (terminal or script file), managing the prompt, and providing the one-character lookahead that the lexer needs.

### 5.1 Overview

The call chain for reading input is shown below:

```
yylex()
|
+----> nextc(); advance()    one-character lookahead (cached in 'future')
      |
      +----> getnext()       raw character read + prompt + escaped newlines
            |
            +----> rchr(runq->cmdfd)    buffered IO read
                  |
                  +----> read()         system call (terminal or file)
```

`getnext()`<sup>43</sup> reads a raw character and handles escaped newlines (for multi-line commands) and prompt display. `nextc()`<sup>46e</sup> adds a one-character lookahead on top of `getnext()`, and `advance()`<sup>47a</sup> consumes that lookahead.

### 5.2 Reading a character: `getnext()`

`getnext()`<sup>43</sup> reads one character from the current thread's input stream (`runq->cmdfd`). Besides the actual read (via `rchr()`<sup>164c</sup>, the buffered IO read function), it handles three concerns: returning a previously peeked character (`peekc`), displaying the prompt before blocking on input, and converting escaped newlines (`\<newline>`) into spaces to support multi-line commands.

```
<function getnext 43>≡ (201)
/*
 * read a character from the input stream
 */
int
getnext(void)
{
    int c;
    <getnext() other locals 45a>
```

```

⟨getnext() peekc handling 45b⟩
⟨getnext() return if already at EOF 44d⟩
⟨getnext() prompt management before reading the character 45g⟩

c = rchr(runq->cmdfd);

⟨getnext() handle backslash 44e⟩
⟨getnext() prompt management after the character is read 46b⟩
⟨getnext() if character read is EOF 44b⟩
⟨getnext() if not EOF but verbose mode, print character read 155c⟩

return c;
}

```

Uses `rchr()` 164c and `runq` 30c.

## 5.2.1 End-of-file management

When `rchr()` 164c returns EOF, `getnext()` 43 sets the `eof` flag on the current thread. On subsequent calls, `getnext()` returns EOF immediately without trying to read again. This flag is also checked in `Xrdcmds()` 41a to decide whether to close the input and return to the parent thread (see Chapter 4).

```

⟨constant EOF (rc/io.h) 44a⟩≡ (196b)
#define EOF (-1)

```

```

⟨getnext() if character read is EOF 44b⟩≡ (43)
if(c==EOF)
    runq->eof = true;

```

Uses EOF 44a and `runq` 30c.

```

⟨Thread other fields 44c⟩+≡ (30b) <41b 46a>
bool eof; /* is cmdfd at eof? */

```

```

⟨getnext() return if already at EOF 44d⟩≡ (43)
if(runq->eof)
    return EOF;

```

Uses EOF 44a and `runq` 30c.

## 5.2.2 Multiple lines commands and escaped newlines

A backslash before a newline tells the shell that the command continues on the next line. `getnext()` 43 detects this sequence and returns a space instead of the newline, effectively joining the two lines. If the backslash is followed by any other character, it is not consumed: the backslash is returned normally, and the next character is saved in `peekc` for the next call.

```

⟨getnext() handle backslash 44e⟩≡ (43)
if(!inquote && c=='\\'){

    c = rchr(runq->cmdfd);

    if(c=='\n' && !incomm){ /* don't continue a comment */
        ⟨getnext() when backslash and newline, set doprompt 46c⟩
        c=' ';
    }
    else{
        peekc = c;
        c='\\';
    }
}

```

Uses `incomm` 49e, `inquote` 53c, `rchr()` 164c, and `runq` 30c.

*<getnext() other locals 45a>*≡ (43)

```
static int peekc = EOF;
```

Uses EOF 44a.

*<getnext() peekc handling 45b>*≡ (43)

```
if(peekc!=EOF){
    c = peekc;
    peekc = EOF;
    return c;
}
```

Uses EOF 44a.

### 5.2.3 Displaying the prompt

In `rc`, the `$prompt` variable is a two-element list: the first element is the main prompt (e.g., "% "), and the second element is the continuation prompt displayed when a command spans multiple lines (e.g., "\t"). The prompt is displayed just before reading a character, not at the end of executing a command. This is done through the global `doprompt` flag, which is set to `true` after each newline (or EOF) and reset to `false` after the prompt is printed.

*<global doprompt 45c>*≡ (201)

```
bool doprompt = true;
```

Uses `doprompt` 45c.

*<global promptstr 45d>*≡ (199b)

```
char *promptstr;
```

*<Xrdcmds() other locals 45e>*≡ (41a)

```
word *prompt;
```

*<Xrdcmds() set promptstr if interactive mode 45f>*≡ (41a)

```
if(runq->iflag){
    prompt = vlook("prompt")->val;
    if(prompt)
        promptstr = prompt->word;
    else
        promptstr="% ";
}
```

Uses `promptstr` 45d, `runq` 30c, and `vlook()` 34b.

*<getnext() prompt management before reading the character 45g>*≡ (43)

```
if(doprompt)
    // pprompt() internally set doprompt back to false at the end
    pprompt();
```

Uses `doprompt` 45c and `pprompt()` 45h.

The subtlety in `pprompt()` <sup>45h</sup> is that after printing the current prompt string, it switches `promptstr` to the *second* element of `$prompt` (the continuation prompt). This way, if the lexer needs more input for a multi-line command, the next call to `pprompt()` will display the continuation prompt instead of the main one. The main prompt is restored in `Xrdcmds()` <sup>41a</sup> before each new command.

*<function pprompt 45h>*≡ (201)

```
void
pprompt(void)
{
    var *prompt;

    if(runq->iflag){
```

```

    // printing the prompt
    pstr(err, promptstr);
    flush(err);

    // set promptstr for the next pprompt()
    prompt = vlook("prompt");
    if(prompt->val && prompt->val->next)
        promptstr = prompt->val->next->word;
    else
        promptstr="\t";
}
runq->lineno++;
doprompt = false;
}

```

Uses `doprompt` 45c, `err` 156b, `flush()` 163g, `promptstr` 45d, `pstr()` 167a, `runq` 30c, and `vlook()` 34b.

Each thread also tracks the current line number, incremented in `pprompt()` after each newline. This is used for error reporting when running scripts: a syntax error in line 42 of a script is much easier to find than one at an unknown location.

```

⟨Thread other fields 46a⟩+≡ (30b) <44c 79a>
    int lineno; /* line number */

```

```

⟨getnext() prompt management after the character is read 46b⟩≡ (43)
    doprompt = doprompt || c=='\n' || c==EOF;

```

Uses EOF 44a and `doprompt` 45c.

```

⟨getnext() when backslash and newline, set doprompt 46c⟩≡ (44e)
    doprompt = true;

```

Uses `doprompt` 45c.

## 5.3 Looking ahead: `nextc()` and `advance()`

Lexers often need to look one character ahead to decide what token to produce. For example, when the lexer sees `&`, it must peek at the next character to decide whether it is a lone `&` (background operator) or part of `&&` (logical and). `nextc()`<sup>46e</sup> provides this lookahead by caching the next character in the global `future`. Multiple calls to `nextc()` return the same character until `advance()`<sup>47a</sup> is called to consume it. The helper `nextis()`<sup>47b</sup> combines both: it checks whether the next character matches, and consumes it if so.

```

⟨global future 46d⟩≡ (201)
    int future = EOF;

```

Uses EOF 44a and `future` 46d.

```

⟨function nextc 46e⟩≡ (201)
/*
 * Look ahead in the input stream
 */
int
nextc(void)
{
    if(future==EOF)
        future = getnext();
    return future;
}

```

Uses EOF 44a, `future` 46d, and `getnext()` 43.

*⟨function advance 47a⟩*≡ (201)

```
/*
 * Consume the lookahead character.
 */
int
advance(void)
{
    int c = nextc();
    ⟨advance() save future in lastc 157b⟩
    future = EOF;
    return c;
}
```

Uses EOF 44a, future 46d, and nextc() 46e.

*⟨function nextis 47b⟩*≡ (201)

```
bool
nextis(int c)
{
    if(nextc()==c){
        advance();
        return true;
    }
    return false;
}
```

Uses advance() 47a and nextc() 46e.

# Chapter 6

## Lexing

The lexer of `rc` is hand-written (not generated by `lex`), which gives more control over subtle interactions with the rest of the shell, at the cost of some complexity. The main function is `yylex()`<sup>48a</sup>, called by the `yacc`-generated parser whenever it needs the next token.

### 6.1 `yylex()`

The structure of `yylex()`<sup>48a</sup> is a switch on the next character. After skipping whitespace, it dispatches on the character read: operators and special characters are handled as individual cases, and anything else is treated as a word character. There are a few lexing subtleties worth noting: newlines are sometimes skipped (via `skipnl()`<sup>50a</sup>) after certain operators to support multi-line commands; keywords like `if` and `for` are recognized as keywords only in command position (not as arguments); and the `(` character is handled specially depending on whether it follows a `$` (array subscript) or not (grouping/function definition).

```
⟨function yylex 48a⟩≡ (207a)
  @@Scheck: called from yyparse()
  int yylex(void)
  {
    int c;
    ⟨yylex() other locals 51d⟩

    ⟨yylex() hack for SUB 56b⟩
    ⟨yylex() initialisations 51f⟩

    skipwhite();

    switch(c = advance()){
      ⟨yylex() switch c cases 49b⟩
    }
    // else
    ⟨yylex() if c is not a word character 53b⟩
    // else
    ⟨yylex() if c is a word character 54c⟩
  }
```

Uses `advance()` 47a and `skipwhite()` 49f.

```
⟨function wordchr 48b⟩≡ (207a)
  int
  wordchr(int c)
  {
    return !strchr("\n \t#;&|^$='\"{}()<>", c) && c!=EOF;
  }
```

Uses `EOF` 44a.

```

⟨function idchr 49a⟩≡ (207a)
    int
    idchr(int c)
    {
        /*
         * Formerly:
         * return 'a'<=c && c<='z' || 'A'<=c && c<='Z' || '0'<=c && c<='9'
         * || c=='_' || c=='*';
         */
        return c > ' ' && !strchr("!\"#$%&'()+,-./:;<=>?@[\\]^`{|}~", c);
    }

```

```

⟨yylex() switch c cases 49b⟩≡ (48a) 50b▷
    case EOF:
        lastdol = false;
        strcpy(tok, "EOF");
        return EOF;

```

Uses EOF 44a, lastdol 51b, and tok 49c.

```

⟨global tok 49c⟩≡ (207a)
    char tok[NTOK + UTFmax];

```

Uses NTOK-10 49d.

```

⟨constant NTOK 49d⟩≡ (207a)
    #define NTOK 8192 /* maximum bytes in a word (token) */

```

## 6.2 Spaces and comments ('#')

`skipwhite()`<sup>49f</sup> skips spaces, tabs, and comments. In `rc`, comments start with `#` and extend to the end of the line. Note that the `incomm` global is needed by `getnext()`<sup>43</sup>: inside a comment, a backslash-newline should *not* be treated as a line continuation.

```

⟨global incomm 49e⟩≡ (201)
    bool incomm;

```

```

⟨function skipwhite 49f⟩≡ (207a)
    void
    skipwhite(void)
    {
        int c;
        for(;;){
            c = nextc();
            /* Why did this used to be if(!inquote && c=='#') ?? */
            if(c=='#'){
                incomm = true;
                for(;;){
                    c = nextc();
                    if(c=='\n' || c==EOF) {
                        incomm = false;
                        break;
                    }
                    advance();
                }
            }
            if(c==' ' || c=='\t')
                advance();
            else
                return;
        }
    }

```

```

    }
}

```

Uses EOF 44a, advance() 47a, incomm 49e, and nextc() 46e.

## 6.3 Newlines

In `rc`, a newline normally acts as a command terminator (like `;` in C). However, after certain operators like `|`, `&&`, or `||`, a newline should be ignored because the user is clearly continuing the command on the next line. `skipnl()` 50a is called by the lexer after those operators to consume any whitespace and newlines until meaningful input arrives. This is why you can write things like:

```

ls |
wc -l

```

and have it work as expected.

```

⟨function skipnl 50a⟩≡ (207a)
void
skipnl(void)
{
    int c;
    for(;;){
        skipwhite();
        c = nextc();
        if(c!='\n')
            return;
        // consume the newline
        advance();
    }
}

```

Uses advance() 47a, nextc() 46e, and skipwhite() 49f.

## 6.4 Operators ('&', '&&', '|', '||', '<', '>', '\$', ...)

Each operator is handled as a case in the main switch of `yylex()` 48a. For multi-character operators like `&&` or `||`, the lexer uses `nextis()` 47b to check the second character. Note that after binary operators like `&&`, `||`, and `|`, the lexer calls `skipnl()` 50a so that newlines are treated as whitespace, allowing the command to continue on the next line. The `lastdol` flag tracks whether the previous token was `$`, which affects how `(` is lexed (see Section 6.7).

```

⟨yylex() switch c cases 50b⟩+≡ (48a) <49b 51a>
case '&':
    lastdol = false;
    if(nextis('&')){
        skipnl();
        strcpy(tok, "&&");
        return ANDAND;
    }
    strcpy(tok, "&");
    return '&';

```

Uses ANDAND, lastdol 51b, nextis() 47b, skipnl() 50a, and tok 49c.

```

<yylex() switch c cases 51a>+≡ (48a) <50b 51c>
case '$':
    lastdol = true;
    if(nextis('#')){
        strcpy(tok, "$#");
        return COUNT;
    }
    if(nextis('"')){
        strcpy(tok, "$\"");
        return '"';
    }
    strcpy(tok, "$");
    return '$';

```

Uses COUNT, lastdol 51b, nextis() 47b, and tok 49c.

```

<global lastdol 51b>≡ (207a)
bool lastdol; /* was the last token read '$' or '$#' or '"'? */

```

```

<yylex() switch c cases 51c>+≡ (48a) <51a 53f>
case '|':
    lastdol = false;
    if(nextis('|')){
        skipnl();
        strcpy(tok, "||");
        return OROR;
    }
    // FALLTHROUGH
case '<':
case '>':
    lastdol = false;
    <yylex() in switch when redirection character 51g>

```

Uses OROR, lastdol 51b, nextis() 47b, skipnl() 50a, and tok 49c.

Redirections and pipes are the most complex part of the lexer. Unlike simple operators like && or || which map directly to a token, redirections carry extra information: the type (WRITE, READ, APPEND, HERE), and which file descriptors are involved (fd0, fd1). The lexer builds a Tree node right here (rather than in the parser), because the bracket syntax >[2=1] requires parsing digits and = signs that would be awkward to express in the yacc grammar.

```

<yylex() other locals 51d>≡ (48a) 51e>
char *w = tok;

```

Uses tok 49c.

```

<yylex() other locals 51e>+≡ (48a) <51d 56a>
struct Tree *t;

```

Uses Tree.

```

<yylex() initialisations 51f>≡ (48a) 53d>
yylval.tree = nil;

```

Uses yylval.

```

<yylex() in switch when redirection character 51g>≡ (51c)
/*
 * funny redirection tokens:
 * redir: arrow | arrow '[' fd ']'
 * arrow: '<' | '<<' | '>' | '>>' | '|'
 * fd: digit | digit '=' | digit '=' digit
 * digit: '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
 * some possibilities are nonsensical and get a message.

```

```

*/
*w++=c;
t = newtree();
switch(c){
<yyllex() in switch when redirection character, switch c cases 52e>
}
<yyllex() in switch when redirection character, if bracket after 136a>
*w='\0';
yylval.tree = t;
if(t->type==PIPE)
    skipnl();
return t->type;

```

Uses PIPE, newtree() 27e, skipnl() 50a, and yylval.

<Tree redirection and pipe specific fields 52a>≡ (27a)

```

//enum<Redirection_kind>
int rtype;

// For a pipe, fd0 is the left fd of the pipe, and fd1 the right fd.
// For a redirection, fd0 is what we redirect (stdout for >, stdin for <)
// and fd1 is what we possibly redirect to (when DUP).
fdt fd0;
fdt fd1; /* details of REDIR PIPE DUP tokens */

```

<constant WRITE 52b>≡ (196c)

```
#define WRITE 2
```

<constant READ 52c>≡ (196c)

```
#define READ 3
```

<constant APPEND 52d>≡ (196c)

```
#define APPEND 1
```

Each redirection character is handled as a case below. For a pipe, fd0 defaults to 1 (stdout of the left command) and fd1 to 0 (stdin of the right command), which is the standard cmd1 | cmd2 behavior. For >, fd0 defaults to 1 (redirect stdout); for <, fd0 defaults to 0 (redirect stdin). A double >> sets the type to APPEND. These defaults can be overridden with the bracket syntax (e.g., >[2] to redirect stderr); see Section 14.4.4.

<yyllex() in switch when redirection character, switch c cases 52e>≡ (51g) 52f▷

```

case '|':
    t->type = PIPE;
    t->fd0 = 1; // left fd of pipe (stdout of left cmd)
    t->fd1 = 0; // right fd of pipe (stdin of right cmd)
    break;

```

Uses PIPE.

<yyllex() in switch when redirection character, switch c cases 52f>+≡ (51g) <52e 53a>

```

case '>':
    t->type = REDIR;
    if(nextis('>')){
        t->rtype = APPEND;
        *w++=c;
    }
    else
        t->rtype = WRITE;
    t->fd0 = 1;
    break;

```

Uses APPEND 52d, REDIR, WRITE 52b, and nextis() 47b.

*<yylex() in switch when redirection character, switch c cases 53a>+≡ (51g) <52f*

```
case '<':
    t->type = REDIR;
    <yylex() in switch when redirection character, if here document 131b>
    <yylex() in switch when redirection character, if read/write redirect 135b>
    else
        t->rtype = READ;
        t->fd0 = 0;
        break;
```

Uses READ 52c and REDIR.

*<yylex() if c is not a word character 53b>≡ (48a)*

```
if(!wordchr(c)){
    lastdol = false;
    tok[0] = c;
    tok[1]='\0';
    return c;
}
```

Uses lastdol 51b, tok 49c, and wordchr() 48b.

## 6.5 Quoted strings ('...')

In rc, strings are quoted with single quotes. To include a literal single quote inside a quoted string, you double it: 'it's' produces it's. The `inquote` flag is needed by `getnext()`<sup>43</sup> because inside a quoted string, backslash-newline should *not* be treated as a line continuation. The `lastword` flag tracks whether the previous token was a word, which the grammar uses to decide whether adjacent tokens should be concatenated (e.g., a<sup>^</sup>b).

*<global inquote 53c>≡ (201)*

```
bool inquote;
```

*<yylex() initialisations 53d>+≡ (48a) <51f*

```
inquote = false;
```

Uses inquote 53c.

*<global lastword 53e>≡ (207a)*

```
// used also by syn.y
bool lastword; /* was the last token read a word or compound word terminator? */
```

*<yylex() switch c cases 53f>+≡ (48a) <51c*

```
case '\':
    inquote = true;
    lastword = true;
    lastdol = false;
    for(;;){
        c = advance();
        if(c==EOF)
            break;
        if(c=='\'){
            if(nextc()!='\')
                break;
            advance();
        }
        w = addutf(w, c);
    }
    if(w != nil)
        *w='\0';
```

```
t = token(tok, WORD);
t->quoted = true;
```

```
yylval.tree = t;
return t->type;
```

Uses EOF 44a, WORD, addutf() 129b, advance() 47a, inquote 53c, lastdol 51b, lastword 53e, nextc() 46e, tok 49c, token() 54b, and yylval.

```
<Tree word specific fields 54a>≡ (27a)
bool quoted;
```

```
<function token 54b>≡ (201)
tree*
token(char *str, int type)
{
    tree *t = newtree();

    t->type = type;
    t->str = strdup(str);
    return t;
}
```

Uses newtree() 27e.

## 6.6 Keywords and identifiers (if, for, while, switch, fn, ...)

When the lexer encounters a word character (anything not an operator or whitespace), it reads characters until a non-word character appears. After a \$, only identifier characters (`idchr`) are accepted, which excludes slashes and other path characters. Once the word is assembled, `klook()` 55a checks whether it matches a keyword. Keywords are stored in a small hash table initialized by `kinit()` 55e at startup. If the word is a keyword, `klook()` returns the appropriate token type; otherwise it returns WORD.

```
<yylex() if c is a word character 54c>≡ (48a)
for(;;){
    <yylex() when c is a word character, if glob character 119b>
    w = addutf(w, c);

    c = nextc();
    if(lastdol ? !idchr(c) : !wordchr(c))
        break;
    advance();
}

lastword = true;
lastdol = false;
if(w!=nil)
    *w='\0';

t = klook(tok);
if(t->type != WORD)
    lastword = false;

t->quoted = false;

yylval.tree = t;
return t->type;
```

Uses WORD, addutf() 129b, advance() 47a, idchr() 49a, klook() 55a, lastdol 51b, lastword 53e, nextc() 46e, tok 49c, wordchr() 48b, and yylval.

*<function klook 55a>*≡ (201)

```
tree*
klook(char *name)
{
    struct Kw *p;
    tree *t = token(name, WORD);

    for(p = kw[hash(name, NKW)];p;p = p->next)
        if(strcmp(p->name, name)==0){
            t->type = p->type;
            break;
        }
    return t;
}
```

Uses Kw 55b, NKW-8 55d, WORD, hash() 35a, kw 55c, and token() 54b.

*<struct Kw 55b>*≡ (201)

```
struct Kw {
    char *name;
    int type;

    struct Kw *next;
};
```

Uses Kw 55b.

*<global kw 55c>*≡ (201)

```
struct Kw *kw[NKW];
```

Uses Kw 55b and NKW-8 55d.

*<constant NKW 55d>*≡ (201)

```
#define NKW 30
```

*<function kinit 55e>*≡ (201)

```
void
kinit(void)
{
    kenter(FOR, "for");
    kenter(IN, "in");
    kenter(WHILE, "while");
    kenter(IF, "if");
    kenter(NOT, "not");
    kenter(SWITCH, "switch");
    kenter(FN, "fn");

    kenter(TWIDDLE, "~");
    kenter(BANG, "!");
    kenter(SUBSHELL, "@");
}
```

Uses BANG, FN, FOR, IF, IN, NOT, SUBSHELL, SWITCH, TWIDDLE, WHILE, and kenter() 55f.

*<function kenter 55f>*≡ (201)

```
void
kenter(int type, char *name)
{
    int h = hash(name, NKW);
    struct Kw *p = new(struct Kw);
    p->type = type;
    p->name = name;
    p->next = kw[h];
}
```

```

    kw[h] = p;
}

```

Uses Kw 55b, NKW-8 55d, hash() 35a, and kw 55c.

## 6.7 Array subscript (<arr>(<n>))

This is one of the trickiest parts of the lexer. When a word is immediately followed by (, as in \$x(1), it must be treated as an array subscript (SUB) rather than a regular parenthesis. Additionally, when two words are adjacent without whitespace (e.g., -x\$flag), the lexer inserts an implicit ^ (caret, the concatenation operator) between them. Without this, the lexer would produce three separate tokens and there would be no way to distinguish -x\$flag (one argument) from -x \ \$flag (two arguments).

```

<yylex() other locals 56a>+≡ (48a) <51e
    int d = nextc();

```

Uses nextc() 46e.

```

<yylex() hack for SUB 56b>≡ (48a)
/*
 * Embarrassing sneakiness: if the last token read was a quoted or unquoted
 * WORD then we alter the meaning of what follows. If the next character
 * is '(', we return SUB (a subscript paren) and consume the '('. Otherwise,
 * if the next character is the first character of a simple or compound word,
 * we insert a '^' before it.
 */
if(lastword){
    lastword = false;
    if(d=='('){
        advance();
        strcpy(tok, "( [SUB]");
        return SUB;
    }
    if(wordchr(d) || d=='\' ' || d==' ' ' || d=='$' || d=='""){
        strcpy(tok, "^");
        return '^';
    }
}

```

Uses SUB, advance() 47a, lastword 53e, tok 49c, and wordchr() 48b.

# Chapter 7

## Parsing

Now that we have seen how the lexer produces tokens from the input stream, we can describe the parser, which assembles those tokens into an abstract syntax tree.

### 7.1 Overview

The parser of `rc` is written using `yacc` (see COMPILER book [Pad16b] for an introduction to `yacc`). The grammar is quite compact, which reflects the simplicity of the `rc` language. The top-level rule `rc` parses a single line: either an empty line (EOF), or a `line` followed by a newline. When a complete line is parsed, the action calls `compile()`<sup>65a</sup> to transform the AST into bytecodes. Note that newlines have a semantic role in `rc`: they terminate commands (like `;` in C). This is what makes the shell convenient for interactive use — you do not need an explicit semicolon to run a command.

The `.y` file follows the standard `yacc` layout: a prologue section (`%...%`) with C includes, then declarations (`\%union`, `\%token`, `\%left`, `\%type`), and finally the grammar rules after `%%`. The `\%union` declaration tells `yacc` that semantic values (`yylval`, `\$1`, `\$2`, etc.) are `Tree` pointers, since every grammar rule builds or passes along AST nodes.

```
<rc/syn.y 57a>≡
%{
#include "rc.h"
#include "fns.h"
%}

%union {
    struct Tree *tree;
};

<token declarations 26>
<priority and associativity declarations 57c>
<type declarations 58a>

%%
<grammar 58b>

<function yyparse 57b>≡
... generated code from syn.y by yacc ...

<priority and associativity declarations 57c>≡                                     (57a)
/* operator priorities -- lowest first */
%left IF WHILE FOR SWITCH ')' NOT
%left ANDAND OROR
%left BANG SUBSHELL
```

```

%left PIPE
%left '^'
%right '$' COUNT '"'
%left SUB

⟨type declarations 58a⟩≡ (57a) ??▷
%type<tree> line cmd simple word comword

⟨grammar 58b⟩≡ (57a)
rc:
    /*empty*/      { return ERROR_1;}
|   line '\n'      { return !compile($1);}

⟨line rule 58c⟩
⟨cmd rule 58d⟩
⟨simple rule 58e⟩

⟨word rule 59a⟩
⟨comword rule 58g⟩

⟨other rules 58f⟩

```

## 7.2 Simple commands (<cmd> <arg1>...<argn>)

The grammar is layered: a **line** is one or more **cmds** separated by **;** or **&**, a **cmd** is a **simple** command (or a control flow statement), and a **simple** command is a **first** word followed by zero or more **word** arguments. The distinction between **first** and **word** is important: a **first** cannot be a keyword, so that **if** in command position is recognized as a keyword, but **echo if** treats **if** as a regular word argument.

```

⟨line rule 58c⟩≡ (58b)
line:
    cmd
⟨line rule other cases 60a⟩

⟨cmd rule 58d⟩≡ (58b)
cmd:
    /*empty*/      {$$=nil;}
|   simple         {$$=simplemung($1);}
⟨cmd rule other cases 60c⟩

⟨simple rule 58e⟩≡ (58b)
simple:
    first
|   simple word    {$$=tree2(ARGLIST, $1, $2);}
⟨simple rule other cases 61d⟩

⟨other rules 58f⟩≡ (58b) 59b▷
first:
    comword
|   first '^' word {$$=tree2('^', $1, $3);}

⟨comword rule 58g⟩≡ (58b)
comword:
    WORD
⟨comword rule other cases 63g⟩

```

```

⟨word rule 59a⟩≡ (58b)
word:
    comword
|   word `` word    {$$=tree2('`', $1, $3);}
|   keyword          {lastword=true; $1->type=WORD;}

```

```

⟨other rules 59b⟩+≡ (58b) <58f 59c>
keyword: FOR|IN|WHILE|IF|NOT|TWIDDLE|BANG|SUBSHELL|SWITCH|FN

```

```

⟨other rules 59c⟩+≡ (58b) <59b 60b>
words:
    /*empty*/        {$$=(struct Tree*)nil;}
|   words word        {$$=tree2(WORDS, $1, $2);}

```

`simplemung()`<sup>59d</sup> wraps a simple command in a SIMPLE node and also stores a string representation of the whole command in `Tree.str`. This string is used later to store function definitions in the environment (see Chapter 10). It also “percolates” redirections to the root of the subtree, so that the bytecode generator can handle them before executing the command.

```

⟨function simplemung 59d⟩≡ (206b)
/*
 * Add a SIMPLE node at the root of t and percolate all the redirections
 * up to the root.
 */
//@Scheck: used by syn.y
tree* simplemung(tree *t)
{
    tree *u;
    struct Io *s;

    t = tree1(SIMPLE, t);

    s = openstr();
    pfmt(s, "%t", t);
    t->str = strdup((char *)s->strp);
    closeio(s);

    ⟨simplemung() percolate redirections up to the root 59e⟩
    return t;
}

```

Uses Io 163d, SIMPLE, closeio() 165a, openstr() 165b, pfmt() 166b, and tree1() 28b.

```

⟨simplemung() percolate redirections up to the root 59e⟩≡ (59d)
for(u = t->child[0]; u->type==ARGLIST; u = u->child[0]){
    if(u->child[1]->type==REDIR || u->child[1]->type==DUP){
        u->child[1]->child[1] = t;
        t = u->child[1];
        u->child[1] = nil;
    }
}

```

Uses ARGLIST, DUP, and REDIR.

For example, when parsing `grep pattern < input > output`, the parser first builds a flat ARGLIST tree with the redirections mixed among the arguments. Then `simplemung()` percolates the redirections up to the root, producing an AST where redirections wrap the command:

|                                    |                                   |
|------------------------------------|-----------------------------------|
| Before <code>simplemung()</code> : | After <code>simplemung()</code> : |
| SIMPLE                             | > "output" fd0=1                  |

|                  |            |                  |
|------------------|------------|------------------|
|                  |            |                  |
| ARGLIST          |            | < "input" fd0=0  |
| / \              |            |                  |
| ARGLIST          | > "output" | SIMPLE           |
| / \              |            |                  |
| ARGLIST          | < "input"  | ARGLIST          |
| / \              |            | / \              |
| "grep" "pattern" |            | "grep" "pattern" |

This way, the bytecode generator encounters the redirections first and can set them up before executing the command.

## 7.3 Operators

Operators are spread across different grammar levels, which implicitly defines their priorities. Sequencing (; and &) is at the `line` level (lowest priority), logical operators (&&, ||) and pipes (|) are at the `cmd` level, and redirections are at the `simple` level (highest priority, closest to the command words).

### 7.3.1 Sequences (';', ' ', '&')

The ; operator sequences two commands. The & operator runs a command asynchronously (in the background) and is treated as a unary postfix operator: `x & y` is parsed as `(x&); y`.

```

<line rule other cases 60a>≡ (58c)
|   cmdsa line   {$$=tree2(';', $1, $2);}

<other rules 60b>+≡ (58b) <59c 61e>
  cmdsa:
    cmd ' ';
|   cmd '&'      {$$=tree1('&', $1);}

```

### 7.3.2 Logical operators ('&&', '||', '!')

```

<cmd rule other cases 60c>≡ (58d) 60d>
|   cmd ANDAND cmd   {$$=tree2(ANDAND, $1, $3);}
|   cmd OROR  cmd    {$$=tree2(OROR, $1, $3);}

<cmd rule other cases 60d>+≡ (58d) <60c 61a>
|   BANG cmd         {$$=mung1($1, $2);}

<function mung1 60e>≡ (206b)
//@Scheck: used by syn.y
tree* mung1(tree *t, tree *c0)
{
    t->child[0] = c0;
    return t;
}

<function mung3 60f>≡ (206b)
//@Scheck: used by syn.y
tree* mung3(tree *t, tree *c0, tree *c1, tree *c2)
{
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}

```

### 7.3.3 String matching (‘~’)

The ~ (twiddle) operator is the fundamental comparison operator in `rc`. Since all values in `rc` are strings (or lists of strings), there are no arithmetic comparisons; instead, ~ performs pattern matching: ~ \$x foo\* tests whether \$x matches the pattern foo\*<sup>1</sup>. It can also be used for simple equality tests (without wildcards) and even for counting: ~ \$#list 0 tests if a list is empty. This is a major simplification compared to the Bourne shell, which relies on the external `test` program (or [ ]).

```
<cmd rule other cases 61a>+≡ (58d) <60d 61c>
| TWIDDLE word words {$$=mung2($1, $2, $3);}
```

```
<function mung2 61b>≡ (206b)
//@Scheck: used by syn.y
tree* mung2(tree *t, tree *c0, tree *c1)
{
    t->child[0] = c0;
    t->child[1] = c1;
    return t;
}
```

### 7.3.4 Pipe (‘|’)

```
<cmd rule other cases 61c>+≡ (58d) <61a 61f>
| cmd PIPE cmd {$$=mung2($2, $1, $3);}
```

### 7.3.5 Redirections (‘>’, ‘<’)

Redirections are at the `simple` level in the grammar, which means they bind tighter than pipes and logical operators. For example, `ls | grep bi* > foo` is parsed as `ls | (grep bi* > foo)`, not `(ls | grep bi*) > foo`—the redirection applies only to `grep`, not to the whole pipeline. If you want to redirect the output of an entire pipeline, you need braces: `{ls | grep bi*} > foo`. A command can have multiple redirections (e.g., `ls < foo > bar`), and they can appear mixed with arguments. The `simplemung()` function later percolates them to the root of the AST subtree.

```
<simple rule other cases 61d>≡ (58e)
| simple redir {$$=tree2(ARGLIST, $1, $2);}
```

```
<other rules 61e>+≡ (58b) <60b 61g>
redir:
    REDIR word {$$=mung1($1, $1->rtype==HERE ? heredoc($2) : $2);}
<redir rule other cases 137b>
```

```
<cmd rule other cases 61f>+≡ (58d) <61c 62b>
| brace epilog {$$=epimung($1, $2);}
```

```
<other rules 61g>+≡ (58b) <61e 62d>
epilog:
    /*empty*/ {$$=nil;}
| redir epilog {$$=mung2($1, $1->child[0], $2);}
```

---

<sup>1</sup>The \* here is interpreted by the twiddle operator itself using `match()`<sup>124b</sup> (see Chapter 13), not by the shell’s globbing mechanism which expands filenames.

```

⟨function epimung 62a⟩≡ (206b)
  //@Scheck: used by syn.y
  tree* epimung(tree *comp, tree *epi)
  {
    tree *p;
    if(epi==0)
      return comp;
    for(p = epi;p->child[1];p = p->child[1]);
    p->child[1] = comp;
    return epi;
  }

⟨cmd rule other cases 62b⟩+≡ (58d) <61f 62c>
  |   redir cmd  %prec BANG
    { $$=mung2($1, $1->child[0], $2); }

```

## 7.4 Control flow statements (if, if not, while, switch, for)

Note that `rc` uses `if not` instead of `else`. This is because `rc` parses and executes one line at a time. When the user types `if(test) cmd` and presses Enter, `rc` must parse, compile, and execute this line immediately. The problem with `else` is that it would need to appear on the *next* line:

```

if(test) cmd1
else cmd2

```

But `rc` has already fully processed the first line before it even sees `else`. It cannot retroactively turn `if(test) cmd1` into the first half of an `if/else`. The Bourne shell solves this with `if...then...fi`: the explicit `fi` terminator tells the parser when the `if` block ends, so `else` can appear unambiguously between `then` and `fi`. But this requires more keywords. `rc` takes a different approach: `if not` is a completely separate command that checks the exit status left by the previous `if`. No lookahead is needed, no extra keywords, and each line is self-contained. The `for` loop has two forms: `for(x in a b c)` iterates over the explicit list, and `for(x)` iterates over `$*` (the script arguments).

```

⟨cmd rule other cases 62c⟩+≡ (58d) <62b 63d>
  |   IF paren {skipnl();} cmd  { $$=mung2($1, $2, $4); }
  |   IF NOT   {skipnl();} cmd  { $$=mung1($2, $4); }

  |   WHILE paren {skipnl();} cmd  { $$=mung2($1, $2, $4); }
  |   SWITCH word {skipnl();} brace { $$=tree2(SWITCH, $2, $4); }

  /*
   * if ‘‘words’’ is nil, we need a tree element to distinguish between
   * for(i in ) and for(i), the former being a loop over the empty set
   * and the latter being the implicit argument loop. so if $5 is nil
   * (the empty set), we represent it as "()". don't parenthesize non-nil
   * functions, to avoid growing parentheses every time we reread the
   * definition.
   */
  |   FOR '(' word IN words ')' {skipnl();} cmd
    { $$=mung3($1, $3, $5 ? $5 : tree1(PAREN, $5), $8); }

  |   FOR '(' word ')' {skipnl();} cmd
    { $$=mung3($1, $3, (struct Tree *)0, $6); }

⟨other rules 62d⟩+≡ (58b) <61g 63b>
  paren: '(' body ')' { $$=tree1(PCMD, $2); }

```

<token declarations 63a>+≡ (57a) <27b 130a>

```
%token PCMD
```

<other rules 63b>+≡ (58b) <62d 63c>

```
brace: '{' body '}'      {$$=tree1(BRACE, $2);}
```

<other rules 63c>+≡ (58b) <63b 63f>

```
body:
    cmd
|   cmdsan body      {$$=tree2(';', $1, $2);}

cmdsan:
    cmdsa
|   cmd '\n'
```

## 7.5 Functions (fn)

Function definition in `rc` uses `fn name {body}`. The second form, `fn name` without a body, *deletes* the function definition. Note that `words` (not just a single word) is accepted as the function name: this allows defining a function for multiple names at once (e.g., `fn sigint sighup {...}` to handle multiple signals with the same handler).

<cmd rule other cases 63d>+≡ (58d) <62c 63e>

```
|   FN words brace  {$$=tree2(FN, $2, $3);}
|   FN words        {$$=tree1(FN, $2);}
```

## 7.6 Variables (<x> = ...)

Variable assignment in `rc` is written `x=value`. The interesting aspect is that an assignment can be followed by a command: `x=value cmd` makes the assignment local to that command only. When `cmd` is empty, the assignment is global. The assignment appears *before* the command, following the Bourne shell convention (which was itself motivated by programs like `make` that accept `x=y` on the command line). The `%prec BANG` directive tells `yacc` to resolve ambiguities (e.g., `x=1 ls && echo done`) using the priority of `BANG`, giving assignment lower priority than pipes but higher than logical operators.

<cmd rule other cases 63e>+≡ (58d) <63d 130b>

```
|   assign cmd                %prec BANG
      {$$=mung3($1, $1->child[0], $1->child[1], $2);}
```

<other rules 63f>+≡ (58b) <63c>

```
assign: first '=' word      {$$=tree2('=', $1, $3);}
```

<comword rule other cases 63g>≡ (58g) 63h>

```
|   '$' word      {$$=tree1('$', $2);}
|   COUNT word    {$$=tree1(COUNT, $2);}
|   '$' word SUB words ')' {$$=tree2(SUB, $2, $4);}
```

## 7.7 Lists ((...))

In `rc`, parentheses create lists: `(a b c)` is a list of three words. This is how you assign multiple values to a variable: `x=(a b c)`. Lists are first-class values in `rc`, which is one of its main improvements over the Bourne shell.

<comword rule other cases 63h>+≡ (58g) <63g 138c>

```
|   '(' words ')'      {$$=tree1(PAREN, $2);}
```

```
$$
% a bit hard to call it word then ... when it is really a list of words.
```

# Chapter 8

## Bytecode Generation and Interpretation

I merged bytecode generation and interpretation in one chapter because it helps to see the bytecodes generated from a given AST node together with the functions that interpret those bytecodes. We could also group them with the related parsing, but the parser can be understood in isolation.

### 8.1 Overview

Recall the main flow: `main()`<sup>X</sup>  $\rightarrow$  `Xrdcmds()`<sup>41a</sup>  $\rightarrow$  `yyparse()`<sup>57b</sup>  $\rightarrow$  `compile()`<sup>65a</sup>. After `yyparse()` builds the AST, it calls `compile()` which walks the tree and emits bytecodes into `codebuf`<sup>29b</sup>. The bytecodes are then interpreted by the main loop in `main()`<sup>X</sup>.

#### 8.1.1 emitxxx()

The emit functions append elements to `codebuf`<sup>29b</sup>, growing it as needed. There are three variants: `emitf()`<sup>64d</sup> for function pointers (the bytecodes), `emiti()`<sup>64c</sup> for integers (jump offsets, file descriptors), and `emits()`<sup>64e</sup> for strings (inline arguments like the string after `Xword`<sup>66a</sup>). `codep` is the current write position in the buffer, and `ncode` is the allocated capacity.

```
<global codep 64a>≡ (209)
// idx in codebuf
int codep;

<global ncode 64b>≡ (209)
int ncode;

<function emiti 64c>≡ (209)
#define emiti(x) ((codep!=ncode || morecode()), codebuf[codep].i = (x), codep++)

<function emitf 64d>≡ (209)
#define emitf(x) ((codep!=ncode || morecode()), codebuf[codep].f = (x), codep++)

<function emits 64e>≡ (209)
#define emits(x) ((codep!=ncode || morecode()), codebuf[codep].s = (x), codep++)

<function morecode 64f>≡ (209)
//@Scheck: used by the macros above (why marked as dead then??? TODO)
int morecode(void)
{
    ncode+=100;
    codebuf = (code *)realloc((char *)codebuf, ncode*sizeof codebuf[0]);
    if(codebuf==nil)
        panic("Can't realloc %d bytes in morecode!", ncode*sizeof(code));
    return OK_0;
}
```

Uses `codebuf` <sup>29b</sup>, `ncode` <sup>64b</sup>, and `panic()` <sup>157c</sup>.

### 8.1.2 compile()

`compile()`<sup>65a</sup> allocates a fresh code buffer, then calls `outcode()`<sup>65f</sup> to recursively walk the AST and emit bytecodes. It appends `Xreturn`<sup>84d</sup> at the end so that when the bytecodes finish executing, the interpreter returns to the previous thread. Note that `compile()` is called for *each line* entered interactively (or each line of a script). So each line gets its own bytecode sequence ending with `Xreturn`.

```
<function compile 65a>≡ (209)
  //@Scheck: called from syn.y
  error0 compile(tree *t)
  {
    ncode = 100;
    codep = 0;
    codebuf = (code *)emalloc(ncode*sizeof(code));

    emit0(0); /* reference count */
    outcode(t, flag['e'] ? true : false);

    <compile() check nerror 65b>
    <compile() after outcode and error management, read heredoc 131g>

    emitf(Xreturn);
    emitf(nil);

    return OK_1;
  }
```

Uses `Xreturn()` <sup>84d</sup>, `codebuf` <sup>29b</sup>, `codep` <sup>64a</sup>, `emalloc()` <sup>158b</sup>, `emitf`-65 <sup>64d</sup>, `emit0`-66 <sup>64c</sup>, `flag`, and `ncode` <sup>64b</sup>.

```
<compile() check nerror 65b>≡ (65a)
  if(nerror){
    efree((char *)codebuf);
    return ERROR_0;
  }
```

Uses `codebuf` <sup>29b</sup>, `efree()` <sup>158d</sup>, and `nerror` <sup>156c</sup>.

### 8.1.3 outcode()

`outcode()`<sup>65f</sup> is the main recursive function that walks the AST and emits bytecodes. It switches on the node type and emits the appropriate sequence of bytecodes for each construct. The `eflag` parameter propagates the `-e` flag (exit on error): when set, an `Xeflag`<sup>129a</sup> bytecode is emitted after certain commands to check if they failed and exit accordingly.

```
<constant c0 (rc/code.c) 65c>≡ (209)
  #define c0 t->child[0]
```

```
<constant c1 (rc/code.c) 65d>≡ (209)
  #define c1 t->child[1]
```

```
<constant c2 (rc/code.c) 65e>≡ (209)
  #define c2 t->child[2]
```

```
<function outcode 65f>≡ (209)
  void
  outcode(tree *t, bool eflag)
  {
    <outcode() locals 76a>

    if(t==nil)
```

```

    return;

    ⟨outcode() set iflast before switch 89d⟩
    switch(t->type){
    ⟨outcode() cases 69b⟩
    default:
        pfmt(err, "bad type %d in outcode\n", t->type);
        break;
    }
    ⟨outcode() set iflast after switch 89c⟩
}

```

## 8.1.4 argv management

Before describing the compilation of specific AST nodes, let me present the bytecodes used to manage the argument stack. Most compilation schemes follow the same pattern: emit `Xmark`<sup>66b</sup> to start a new word list on `argv`, emit a series of `Xword`<sup>66a</sup> to push words onto it, then emit a bytecode (like `Xsimple`<sup>70</sup>) that consumes the list. For example, `ls -l /tmp` compiles to: `Xmark`, `Xword "ls"`, `Xword "-l"`, `Xword "/tmp"`, `Xsimple`.

```

⟨function Xword 66a⟩≡ (203b)
void
Xword(void)
{
    pushword(runq->code[runq->pc++] .s);
}

```

Uses `pushword()` 40b and `runq` 30c.

```

⟨function Xmark 66b⟩≡ (203b)
void
Xmark(void)
{
    pushlist();
}

```

Uses `pushlist()` 40a.

```

⟨function poplist 66c⟩≡ (203a)
void
poplist(void)
{
    list *p = runq->argv;
    if(p==nil)
        panic("poplist but no argv", 0);
    freelist(p->words);
    runq->argv = p->next;
    efree((char *)p);
}

```

Uses `efree()` 158d, `freelist()` 33a, `panic()` 157c, and `runq` 30c.

```

⟨function popword 66d⟩≡ (203a)
void
popword(void)
{
    word *p;
    ⟨popword() sanity check argv 67a⟩
    p = runq->argv->words;
    ⟨popword() sanity check argv words 67b⟩
    runq->argv->words = p->next;
    efree(p->word);
}

```

```
    efree((char *)p);
}
```

Uses `efree()` 158d and `runq` 30c.

```
<popword() sanity check argv 67a>≡ (66d)
    if(runq->argv==nil)
        panic("popword but no argv!", 0);
```

Uses `panic()` 157c and `runq` 30c.

```
<popword() sanity check argv words 67b>≡ (66d)
    if(p==nil)
        panic("popword but no word!", 0);
```

Uses `panic()` 157c.

## 8.1.5 Process status management

The exit status of the last command is stored in the `$status` variable. `setstatus()` 67c sets it, and `getstatus()` 67d and `truestatus()` 68a read it. In `rc`, a status is a string (not an integer like in the Bourne shell). An empty string or the string `"0"` means success; anything else means failure. This mirrors the Plan 9 system call `exits()` (see *KERNEL* book [Pad14]), which takes a string argument rather than a numeric exit code.

```
<function setstatus 67c>≡ (210c)
    void
    setstatus(char *s)
    {
        setvar("status", newword(s, (word *)nil));
    }
```

Uses `newword()` 31d and `setvar()` 34a.

```
<function getstatus 67d>≡ (210c)
    char*
    getstatus(void)
    {
        var *status = vlook("status");
        return status->val ? status->val->word : "";
    }
```

Uses `vlook()` 34b.

The `|` check in `truestatus()` below deserves explanation. When a pipeline like `ls | wc` finishes, `rc` combines the exit statuses of all commands into a single string separated by `|`, for example `"0|0"` if both succeed or `"error|0"` if the first fails. So `truestatus()` considers a status “true” if it contains only `0` and `|` characters—meaning every stage of the pipeline succeeded. Why not reduce the pipeline status to a single value earlier? Because keeping the full string lets the user inspect `\$status` and see exactly which stage failed (e.g., `"0|error|0"` tells you the middle command had a problem). With numeric exit codes (as in UNIX), reducing is easy: just take the maximum or the last non-zero value. But with string statuses, there is no natural way to merge `"can't open"` and `"0"` into a single meaningful string, so `rc` preserves them all. In `bash`, `\$?` only reports the exit status of the *last* command in the pipeline—a failure in an earlier stage is silently lost. `bash` later added the `PIPESTATUS` array to work around this, but it requires explicit use.

This is another example of `rc` getting the design right from the start, while `bash` accumulates workarounds: pipeline statuses are visible by default in `\$status` (vs. `PIPESTATUS` added later in `bash`), all variables are automatically exported via `/env/` (vs. the error-prone `export` command), `if not` avoids the parsing ambiguity of `else` (vs. the heavyweight `if...then...fi` syntax), and string exit statuses naturally carry diagnostic

information (vs. opaque numeric codes that require `strerror()` or a manual to decode).

```
<function truestatus 68a>≡ (210c)
bool
truestatus(void)
{
    char *s;
    for(s = getstatus();*s;s++)
        if(*s!='|' && *s!='0')
            return false;
    return true;
}
```

Uses `getstatus()` 67d.

## 8.1.6 Subprocesses management

`rc` keeps track of all child processes it has spawned in the `waitpids` array. This is needed for the `wait` builtin (which waits for all children) and for proper cleanup when `rc` exits.

```
<global waitpids 68b>≡ (206a)
// growing_array<pid> (but really a list)
int *waitpids;
```

```
<global nwaitpids 68c>≡ (206a)
int nwaitpids;
```

```
<function addwaitpid 68d>≡ (206a)
void
addwaitpid(int pid)
{
    <addwaitpid() grow waitpids if needed 68e>
    waitpids[nwaitpids++] = pid;
}
```

Uses `nwaitpids` 68c and `waitpids` 68b.

```
<addwaitpid() grow waitpids if needed 68e>≡ (68d)
waitpids = realloc(waitpids, (nwaitpids+1)*sizeof waitpids[0]);
if(waitpids == nil)
    panic("Can't realloc %d waitpids", nwaitpids+1);
```

Uses `nwaitpids` 68c, `panic()` 157c, and `waitpids` 68b.

```
<function delwaitpid 68f>≡ (213b)
void
delwaitpid(int pid)
{
    int r, w;

    for(r=w=0; r<nwaitpids; r++)
        if(waitpids[r] != pid)
            waitpids[w++] = waitpids[r];
    nwaitpids = w;
}
```

Uses `nwaitpids` 68c and `waitpids` 68b.

```
<function clearwaitpids 68g>≡ (206a)
void
clearwaitpids(void)
{
    nwaitpids = 0;
}
```

Uses `nwaitpids` 68c.

```

⟨function havewaitpid 69a⟩≡ (213b)
    bool
    havewaitpid(int pid)
    {
        int i;

        for(i=0; i<nwaitpids; i++)
            if(waitpids[i] == pid)
                return true;
        return false;
    }

```

Uses nwaitpids 68c and waitpids 68b.

## 8.2 Simple commands

Now that we have seen the helper infrastructure—argument stack management, process status, and subprocess tracking—we can describe the bytecode generation and interpretation for simple commands, which is the core of the shell.

### 8.2.1 Bytecode generation

A SIMPLE command compiles to: Xmark<sup>66b</sup> (start a new word list), followed by the bytecodes for the arguments (which push words onto the stack), followed by Xsimple<sup>70</sup> (which pops the word list and executes the command). Note that ARGLIST and WORDS emit their children in reverse order: since Xword<sup>66a</sup> prepends to the word list, emitting the last word first ensures the list ends up in the correct order.

```

⟨outcode() cases 69b⟩≡ (65f) 69c>
    case SIMPLE:
        emitf(Xmark);
        outcode(c0, eflag); // the arguments and argv0
        emitf(Xsimple);
        ⟨outcode() emit Xeflag after Xsimple 128b⟩
        break;

```

```

⟨outcode() cases 69c⟩+≡ (65f) <69b 69d>
    case ARGLIST:
        outcode(c1, eflag);
        outcode(c0, eflag);
        break;

```

```

⟨outcode() cases 69d⟩+≡ (65f) <69c 69e>
    case WORDS:
        outcode(c1, eflag);
        outcode(c0, eflag);
        break;

```

```

⟨outcode() cases 69e⟩+≡ (65f) <69d 75c>
    case WORD:
        emitf(Xword);
        emits(strdup(t->str));
        break;

```

```

⟨codefree() in loop over code cp, switch bytecode cases 69f⟩+≡ (29e) <30a 84c>
    else if(p->f==Xword || p->f==Xdelhere)
        efree(++p->s);

```

Uses Xdelhere() 132c, Xword() 66a, and efree() 158d.

For example, `echo hello world` produces the following AST and bytecodes:

|                 |               |
|-----------------|---------------|
| AST:            | codebuf:      |
| SIMPLE          | Xmark         |
|                 | Xword "world" |
| ARGLIST         | Xword "hello" |
| / \             | Xword "echo"  |
| ARGLIST "world" | Xsimple       |
| / \             |               |
| "echo" "hello"  |               |

Note how the arguments are emitted in reverse order (`ARGLIST` emits `c1` before `c0`), so that after `Xword` prepends each word to the list, `argv` ends up in the correct order: `echo, hello, world`.

### 8.2.2 Xsimple()

`Xsimple()`<sup>70</sup> is the bytecode that executes a simple command. It is the heart of `rc` and handles three cases, in order:

1. If the command name matches a shell *function* (defined with `fn`), it runs the function's bytecodes in a new thread.
2. If the command name matches a *builtin* (e.g., `cd`, `exit`, `.`), it calls the corresponding C function directly (without forking).
3. Otherwise, it forks a child process with `execforkexec()`<sup>72a</sup> and waits for it to finish.

The `while(Waitfor(...) < 0)` loop retries the wait if it was interrupted by a signal (e.g., `^C`): the signal kills the child, but the parent still needs to reap it.

```

⟨function Xsimple 70⟩≡ (208a)
void
Xsimple(void)
{
    word *a;
    thread *p = runq;
    int pid;
    ⟨Xsimple() other locals 95b⟩

    ⟨Xsimple() initializations, globlist() 120c⟩

    a = runq->argv->words;
    ⟨Xsimple() sanity check a 71a⟩

    ⟨Xsimple() if -x 155b⟩

    ⟨Xsimple() if argv0 is a function 95c⟩
    else{
        ⟨Xsimple() if argv0 is a builtin 103a⟩
        ⟨Xsimple() if exitnext() 75a⟩
        else{
            flush(err);
            Updenv(); /* necessary so changes don't go out again */
            if((pid = execforkexec()) < 0){
                Xerror("try again");
                return;
            }
        }
    }
}

```

```

    }

    /* interrupts don't get us out */
    poplist();
    while(Waitfor(pid, true) < 0)
        ;
}
}
}

Uses Xerror() 71b, err 156b, execforkexec() 72a, flush() 163g, poplist() 66c, and runq 30c.

```

```

⟨Xsimple() sanity check a 71a⟩≡ (70)
{
    if(a==nil){
        Xerror1("empty argument list");
        return;
    }
}

```

Uses Xerror1() 71c.

```

⟨function Xerror 71b⟩≡ (203a)
void
Xerror(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s: %r\n", s);
    else
        pfmt(err, "rc (%s): %s: %r\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
}

```

Uses Xreturn() 84d, argv0 39b, err 156b, flush() 163g, pfmt() 166b, runq 30c, and setstatus() 67c.

```

⟨function Xerror1 71c⟩≡ (203a)
void
Xerror1(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s\n", s);
    else
        pfmt(err, "rc (%s): %s\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
}

```

Uses Xreturn() 84d, argv0 39b, err 156b, flush() 163g, pfmt() 166b, runq 30c, and setstatus() 67c.

### 8.2.3 Fork

`execforkexec()`<sup>72a</sup> implements the fundamental `fork+exec+wait` pattern that is the essence of a shell. It forks a child process, and in the child, calls `execexec()`<sup>72b</sup> to replace the child with the actual command. If `exec` fails (e.g., the command does not exist), the child exits with an error; the parent continues normally.

The `pushword("exec")` before `execexec()` is a trick: `execexec()` is also used as the `exec` builtin, where `exec` appears as the first word in `argv`. Here we add it artificially so `execexec()` can `popword()` it uniformly.

```
⟨function execforkexec 72a⟩≡ (206a)
int
execforkexec(void)
{
    int pid;
    int n;
    char buf[ERRMAX];

    // fork()!!
    switch(pid = fork()){
    case -1:
        return -1;
    case 0: // child
        clearwaitpids();
        pushword("exec");
        execexec();

        // should not be reached! unless the command did not exist
        strcpy(buf, "can't exec: ");
        n = strlen(buf);
        errstr(buf+n, ERRMAX-n);
        Exit(buf, __LOC__);
    }
    // parent
    addwaitpid(pid);
    return pid;
}
```

Uses `addwaitpid()` 68d, `clearwaitpids()` 68g, `execexec()` 72b, and `pushword()` 40b.

## 8.2.4 Exec

```
⟨function execexec 72b⟩≡ (208a)
void
execexec(void)
{
    popword(); /* "exec" */
    ⟨execexec() sanity check arguments 72c⟩
    ⟨execexec() perform the redirections 78a⟩

    Execute(runq->argv->words, searchpath(runq->argv->words->word));
    // should not be reached! unless command did not exist
    poplist();
}
```

Uses `Execute()` 73c, `poplist()` 66c, `popword()` 66d, `runq` 30c, and `searchpath()` 73a.

```
⟨execexec() sanity check arguments 72c⟩≡ (72b)
if(runq->argv->words==nil){
    Xerror1("empty argument list");
    return;
}
```

Uses `Xerror1()` 71c and `runq` 30c.

## 8.2.5 \$path management

When the command name starts with `/`, `./`, `../`, or `#` (device path in Plan 9), it is treated as an absolute path and no search is performed. Otherwise, `searchpath()`<sup>73a</sup> returns the list of directories in `$path` to try. `Execute()`<sup>73c</sup> then iterates over these directories, prepending each to the command name and calling `exec()`. If `exec()` fails, it tries the next directory. This is the standard UNIX path search mechanism.

```
<function searchpath 73a>≡ (211)
word*
searchpath(char *w)
{
    word *path;

    if(strncmp(w, "/", 1)==0
    || strncmp(w, "#", 1)==0
    || strncmp(w, "./", 2)==0
    || strncmp(w, "../", 3)==0
    || (path = vlook("path")->val)==nil)
        path=&nullpath;
    return path;
}
```

Uses `nullpath` 73b and `vlook()` 34b.

```
<global nullpath 73b>≡ (211)
struct Word nullpath = { "", nil};
```

Uses `Word` 31c.

```
<function Execute 73c>≡ (206a)
void
Execute(word *args, word *path)
{
    char **argv = mkargv(args);
    char file[1024];
    char errstr[1024];
    int nc;

    Updenv();
    errstr[0] = '\0';

    for(;path;path = path->next){
        nc = strlen(path->word);
        if(nc < sizeof file - 1){ /* 1 for / */
            strcpy(file, path->word);
            if(file[0]){
                strcat(file, "/");
                nc++;
            }
            if(nc + strlen(argv[1]) < sizeof file){
                strcat(file, argv[1]);

                // The actual exec() system call!
                exec(file, argv+1);

                // reached if the file does not exist

                rerrstr(errstr, sizeof errstr);
                /*
                 * if file exists and is executable, exec should
                 * have worked, unless it's a directory or an
                 * executable for another architecture. in
```

```

        * particular, if it failed due to lack of
        * swap/vm (e.g., arg. list too long) or other
        * allocation failure, stop searching and print
        * the reason for failure.
        */
        if (strstr(errstr, " allocat") != nil ||
            strstr(errstr, " full") != nil)
            break;
    }
    else werrstr("command name too long");
}
}
// should not be reached if found an actual binary to exec
pfmt(err, "%s: %s\n", argv[1], errstr);
efree((char *)argv);
}

```

Uses `efree()` 158d, `err` 156b, `mkargv()` 74a, and `pfmt()` 166b.

```

⟨function mkargv 74a⟩≡ (206a)
char **
mkargv(word *a)
{
    char **argv = (char **)emalloc((count(a)+2) * sizeof(char *));
    char **argp = argv+1; /* leave one at front for runcoms */

    for(;a;a = a->next)
        *argp++=a->word;
    *argp = nil;
    return argv;
}

```

Uses `count()` 32a and `emalloc()` 158b.

## 8.2.6 Wait

Back in the parent process, `Waitfor()` <sup>74b</sup> loops calling `wait()` until it reaps the child with the expected `pid`. If `wait()` returns a different child (from a background process started with `&`), its status is recorded but the loop continues. If `wait()` is interrupted by a signal, `Waitfor()` returns `-1`, and the caller (`Xsimple`) retries.

```

⟨function Waitfor 74b⟩≡ (213b)
int
Waitfor(int pid, bool _persist)
{
    thread *p;
    Waitmsg *w;
    char errbuf[ERRMAX];

    if(pid >= 0 && !havewaitpid(pid))
        return 0;

    // wait()!! until we found it
    while((w = wait()) != nil){
        delwaitpid(w->pid);

        if(w->pid==pid){
            setstatus(w->msg);
            free(w);
            return 0;
        }
    }
}

```

```

    ⟨Waitfor() in while loop, if wait returns another pid 86a⟩
}

errstr(errbuf, sizeof errbuf);
if(strcmp(errbuf, "interrupted")==0)
    return -1;
return 0;
}

```

Uses `delwaitpid()` 68f, `havewaitpid()` 69a, and `setstatus()` 67c.

## 8.2.7 Fork optimization

If the next bytecode after `Xsimple`<sup>70</sup> is `Xexit`<sup>87</sup> (possibly preceded by `Xpopredir`<sup>81a</sup> calls), the shell is about to exit anyway, so forking a child and waiting for it is pointless. In that case, `Xsimple` calls `execexec()`<sup>72b</sup> directly, replacing the current process with the command. This is the same optimization that the `exec` builtin provides, applied automatically. This matters in practice for scripts like `rc -c 'ls'`: without this optimization, `rc` would fork, `exec ls` in the child, wait, and then exit—wasting a process.

```

⟨Xsimple() if exitnext() 75a⟩≡ (70)
if(exitnext()){
    /* fork and wait is redundant */
    pushword("exec");
    execexec();
    Xexit();
}

```

Uses `Xexit()` 87, `execexec()` 72b, `exitnext()` 75b, and `pushword()` 40b.

```

⟨function exitnext 75b⟩≡ (208a)
/*
 * Search through the following code to see if we're just going to exit.
 */
int
exitnext(void){
    union Code *c = &runq->code[runq->pc];
    while(c->f==Xpopredir)
        c++;
    return c->f==Xexit;
}

```

Uses `Code` 29c, `Xexit()` 87, `Xpopredir()` 81a, and `runq` 30c.

## 8.3 Operators

### 8.3.1 Basic sequence

The sequencing operator `;` simply emits the bytecodes of both children one after the other.

```

⟨outcode() cases 75c⟩+≡ (65f) <69e 76b>
case ' ';':
    outcode(c0, eflag);
    outcode(c1, eflag);
    break;

```

### 8.3.2 Logical operators

`&&` and `||` use a forward jump technique. For `cmd1 &\ cmd2`, the compiler emits: the bytecodes of `cmd1`, then `Xtrue`<sup>76e</sup> followed by a placeholder jump offset, then the bytecodes of `cmd2`. After emitting `cmd2`, `stuffdot()`<sup>76c</sup> patches the placeholder with the actual address, so `Xtrue` can jump past `cmd2` if the status is false. `Xtrue` and `Xfalse`<sup>76f</sup> are symmetric: `Xtrue` jumps if the status is *false* (skip the second command), and `Xfalse` jumps if the status is *true*.

```
⟨outcode() locals 76a⟩≡ (65f) 84a▷
    int p;
```

```
⟨outcode() cases 76b⟩+≡ (65f) ◁75c 76d▷
    case ANDAND:
        outcode(c0, false);
        emitf(Xtrue);
        p = emitf(0);
        outcode(c1, eflag);
        stuffdot(p);
        break;
```

```
⟨function stuffdot 76c⟩≡ (209)
    void
    stuffdot(int a)
    {
        if(a<0 || codep<=a)
            panic("Bad address %d in stuffdot", a);
        codebuf[a].i = codep;
    }
```

Uses `codebuf` 29b, `codep` 64a, and `panic()` 157c.

```
⟨outcode() cases 76d⟩+≡ (65f) ◁76b 77a▷
    case OROR:
        outcode(c0, false);
        emitf(Xfalse);
        p = emitf(0);
        outcode(c1, eflag);
        stuffdot(p);
        break;
```

```
⟨function Xtrue 76e⟩≡ (203b)
    void
    Xtrue(void)
    {
        if(truestatus()) runq->pc++;
        else runq->pc = runq->code[runq->pc].i;
    }
```

Uses `runq` 30c and `truestatus()` 68a.

```
⟨function Xfalse 76f⟩≡ (203b)
    void
    Xfalse(void)
    {
        if(truestatus()) runq->pc = runq->code[runq->pc].i;
        else runq->pc++;
    }
```

Uses `runq` 30c and `truestatus()` 68a.

```

⟨outcode() cases 77a⟩+≡ (65f) <76d 77c>
case BANG:
    outcode(c0, eflag);
    emitf(Xbang);
    break;

```

```

⟨function Xbang 77b⟩≡ (203b)
void
Xbang(void)
{
    setstatus(truestatus()? "false" : "");
}

```

Uses `setstatus()` 67c and `truestatus()` 68a.

### 8.3.3 String matching

The `~` operator pushes two word lists onto `argv` (the subject and the patterns), then calls `Xmatch`<sup>77d</sup>. Two `Xmark`<sup>66b</sup> bytecodes are needed because both the subject and the patterns can be multi-word (e.g., if they come from variable expansion), and they must be kept separate. `Xmatch()`<sup>77d</sup> converts the subject words to a single string (joining with spaces), then tries each pattern word against it using `match()`<sup>124b</sup> (described in Chapter 13).

```

⟨outcode() cases 77c⟩+≡ (65f) <77a 80b>
case TWIDDLE:
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xmatch);
    if(eflag)
        emitf(Xeflag);
    break;

```

```

⟨function Xmatch 77d⟩≡ (203b)
void
Xmatch(void)
{
    word *p;
    char *subject;

    subject = list2str(runq->argv->words);
    setstatus("no match");
    for(p = runq->argv->next->words;p = p->next)
        if(match(subject, p->word, '\0')){
            setstatus("");
            break;
        }
    efree(subject);
    poplist();
    poplist();
}

```

Uses `efree()` 158d, `list2str()` 77e, `poplist()` 66c, `runq` 30c, and `setstatus()` 67c.

```

⟨function list2str 77e⟩≡ (203b)
char*
list2str(word *words)
{
    char *value, *s, *t;
    int len = 0;

```

```

word *ap;

for(ap = words; ap; ap = ap->next)
    len += 1+strlen(ap->word);
value = emalloc(len+1);

s = value;
for(ap = words; ap; ap = ap->next){
    for(t = ap->word; *t;)
        *s++=*t++;
    *s++=' ';
}
if(s==value)
    *s='\0';
else s[-1]='\0';
return value;
}

```

Uses `emalloc()` 158b.

### 8.3.4 Redirection

Redirections in `rc` are handled in two phases. When the interpreter encounters a redirection bytecode (e.g., `Xwrite`<sup>80d</sup>), it opens the target file and pushes a `Redir` record onto the thread's redirection stack. But it does *not* actually redirect the file descriptors yet. The actual redirection happens later, in `execexec()`<sup>72b</sup> (inside the forked child process), just before `exec()`: `doredir()`<sup>79d</sup> walks the redirection stack and performs the `dup()` and `close()` calls. This two-phase design is necessary because the parent shell must not modify its own file descriptors; only the child (which is about to be replaced by `exec()`) should have its `stdin/stdout/stderr` redirected.

```

⟨execexec() perform the redirections 78a⟩≡ (72b)
    doredir(runq->redir);

```

Uses `doredir()` 79d and `runq` 30c.

```

⟨start() set redir 78b⟩≡ (39a)
    p->redir = p->startredir = runq ? runq->redir : nil;

```

Uses `runq` 30c.

#### Redir

The `Redir` structure records a pending redirection: a `type` (e.g., `ROPEN` for a regular file redirection), a `from` file descriptor (the one that was opened), and a `to` file descriptor (the one that should be replaced, such as 1 for `stdout`). The thread's `redir` field points to a linked list of these records, built up as the interpreter encounters redirection bytecodes. The `startredir` field in `start()`<sup>39a</sup> deserves attention: when a new thread is created, it inherits its parent's redirection stack (so subshells see the parent's redirections), but `startredir` remembers where the inherited portion ends. This way, if an error occurs, the thread can pop only the redirections it added, without disturbing the parent's state.

```

⟨struct Redir 78c⟩≡ (199a)
    struct Redir {
        // enum<redirection_kind_bis>
        char type; /* what to do */

        /* what to do it to */
        short from;
        short to;
    };

```

```

    // Extra
    <Redir extra fields 79b>
};

<Thread other fields 79a>+≡ (30b) <46a 82c>
    // list<ref_own<Redir>> (next = Redir.next)
    struct Redir *redir; /* redirection stack */
Uses Redir 78c.

```

```

<Redir extra fields 79b>≡ (78c)
    struct Redir *next; /* what else to do (reverse order) */
Uses Redir 78c.

```

```

<function pushredir 79c>≡ (203a)
    void
    pushredir(int type, int from, int to)
    {
        redir * rp = new(redir);
        rp->type = type;
        rp->from = from;
        rp->to = to;

        // add_list(runq->redir, rp)
        rp->next = runq->redir;
        runq->redir = rp;
    }
Uses runq 30c.

```

doredir()

Because `pushredir()`<sup>79c</sup> prepends each new record to the front of the list, the list is in reverse order relative to the command line. For `cmd > foo >[2=1]`, the list has `>[2=1]` first, then `> foo`. But the user expects the redirections to be applied left-to-right: first redirect stdout to `foo`, then duplicate fd 2 from fd 1. `doredir()`<sup>79d</sup> solves this by recursing to the end of the list before performing any `dup()` calls, so the deepest (oldest) redirection executes first—restoring the original left-to-right order.

```

<function doredir 79d>≡ (208a)
    void
    doredir(redir *rp)
    {
        if(rp){
            // recurse first, so do them in the reverse order of the list
            doredir(rp->next);

            switch(rp->type){
                <doredir() switch redir type cases 79e>
            }
        }
    }

```

Uses doredir() 79d.

```

<doredir() switch redir type cases 79e>≡ (79d) 138a▷
    case ROPEN:
        if(rp->from != rp->to){
            dup(rp->from, rp->to);
            close(rp->from);
        }
        break;

```

Uses ROPEN 80a.

```

⟨constant ROPEN 80a⟩≡ (199a)
/*
 * redir types
 */
#define ROPEN 1 /* dup2(from, to); close(from); */

```

## Bytecode generation

The bytecode for a REDIR node first evaluates the filename (with globbing), then emits the appropriate redirection bytecode (e.g., Xwrite<sup>80d</sup> for >), followed by the file descriptor number. After the redirected command (c1) finishes, Xpopredir<sup>81a</sup> cleans up by closing the opened file descriptor and removing the Redir record from the stack. This scoping ensures that in cmd1 > foo; cmd2, the redirection applies only to cmd1—once Xpopredir runs, cmd2 sees the original file descriptors.

```

⟨outcode() cases 80b⟩+≡ (65f) <77c 84b>
case REDIR:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xglob);

    switch(t->rtype){
    ⟨outcode() when REDIR case, switch redirection type cases 80c⟩
    }
    emitf(t->fd0);
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;

```

## Xwrite()

Xwrite()<sup>80d</sup> handles the > redirection at runtime. It pops the filename from the argv stack, opens (or creates) the file with Creat()<sup>169g</sup>, and pushes an ROPEN record mapping the newly opened file descriptor to the target fd (typically 1 for stdout, read from the bytecode stream). The actual dup() will happen later in doredir()<sup>79d</sup>.

```

⟨outcode() when REDIR case, switch redirection type cases 80c⟩≡ (80b) 81b>
case WRITE:
    emitf(Xwrite);
    break;

```

```

⟨function Xwrite 80d⟩≡ (203b)
void
Xwrite(void)
{
    char *file;
    fdt f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("> requires singleton\n");
        return;
    case 0:
        Xerror1("> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = Creat(file))<0){
        pfmt(terr, "%s: ", file);
        Xerror("can't open");
    }
}

```

```

        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}

```

Uses `Creat()` 169g, `ROPEN` 80a, `Xerror()` 71b, `Xerror1()` 71c, `count()` 32a, `err` 156b, `pfmt()` 166b, `poplist()` 66c, `pushredir()` 79c, and `runq` 30c.

`Xpopredir()` <sup>81a</sup> is the cleanup counterpart: it pops the top `Redir` record and, if it was an `ROPEN`, closes the file descriptor that was opened for the redirection. This is important because the opened fd (from `Creat()` or `Open()`) is a temporary—after `doredir()` has `dup()`d it onto the target fd in the child process, the parent still holds the original, and it must be closed to avoid leaking file descriptors.

```

⟨function Xpopredir 81a⟩≡ (203a)
void
Xpopredir(void)
{
    struct Redir *rp = runq->redir;

    if(rp==nil)
        panic("turfredir null!", 0);

    // pop_list(runq->redir);
    runq->redir = rp->next;

    if(rp->type==ROPEN)
        close(rp->from);

    efree((char *)rp);
}

```

Uses `ROPEN` 80a, `Redir` 78c, `efree()` 158d, `panic()` 157c, and `runq` 30c.

## Xread()

```

⟨outcode() when REDIR case, switch redirection type cases 81b⟩+≡ (80b) <80c 82a>
case READ:
case HERE:
    emitf(Xread);
    break;

```

```

⟨function Xread 81c⟩≡ (203b)
void
Xread(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("< requires singleton\n");
        return;
    case 0:
        Xerror1("< requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 0))<0){

```

```

    pfmt(err, "%s: ", file);
    Xerror("can't open");
    return;
}
pushredir(ROPEN, f, runq->code[runq->pc].i);
runq->pc++;
poplist();
}

```

Uses ROPEN 80a, Xerror() 71b, Xerror1() 71c, count() 32a, err 156b, pfmt() 166b, poplist() 66c, pushredir() 79c, and runq 30c.

## Xappend()

⟨outcode() when REDIR case, switch redirection type cases 82a⟩+≡ (80b) <81b 135c>

```

case APPEND:
    emitf(Xappend);
    break;

```

⟨function Xappend 82b⟩≡ (203b)

```

void
Xappend(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1(">> requires singleton");
        return;
    case 0:
        Xerror1(">> requires file");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 1))<0 && (f = Creat(file))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    seek(f, 0L, SEEK__END);
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}

```

Uses Creat() 169g, ROPEN 80a, Xerror() 71b, Xerror1() 71c, count() 32a, err 156b, pfmt() 166b, poplist() 66c, pushredir() 79c, and runq 30c.

## Closing redirection opened files

⟨Thread other fields 82c⟩+≡ (30b) <79a 85c>

```

    struct Redir *startredir; /* redir inheritance point */

```

Uses Redir 78c.

*<function turfredir 83a>*≡ (203a)

```
void
turfredir(void)
{
    while(runq->redir != runq->startredir)
        Xpopredir();
}
```

Uses Xpopredir() 81a and runq 30c.

*<Xreturn() pop the redirections from this thread 83b>*≡ (84d)

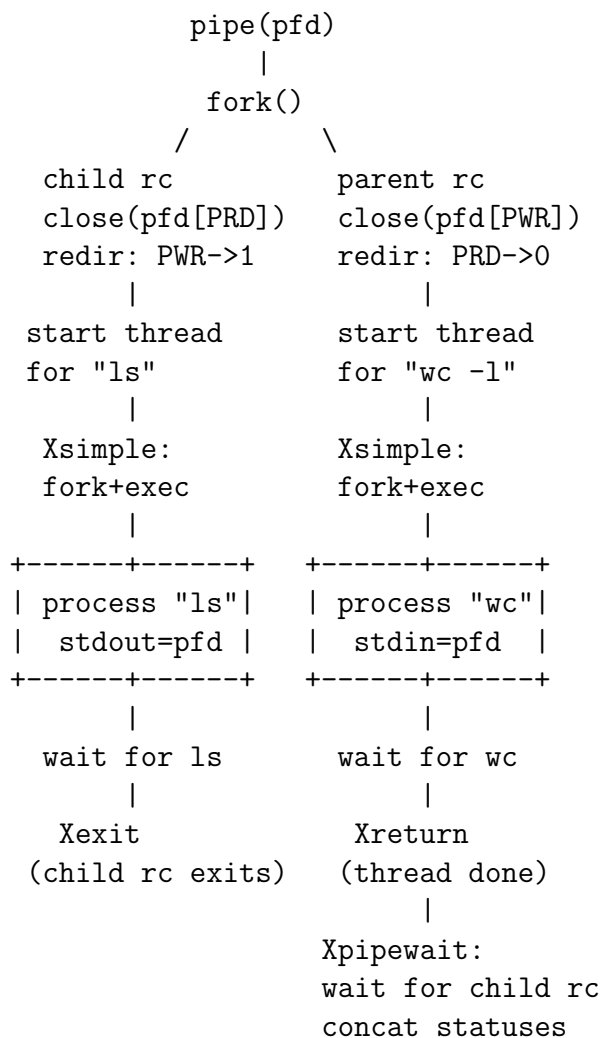
```
turfredir();
```

Uses turfredir() 83a.

### 8.3.5 Pipe

Pipes are arguably the most important operator in a shell. For `cmd1 | cmd2`, four processes end up being involved: the parent `rc`, a child `rc` that interprets `cmd1`'s bytecodes, and then each side fork+exec's the actual command (e.g., `ls` and `wc`). The left side (`cmd1`) runs in a forked child `rc` process. The right side (`cmd2`) runs in the parent `rc`'s new thread. This asymmetry means that the right side can modify the shell's state (e.g., variable assignments), while the left side cannot. The compiled bytecodes contain two jump offsets: one for the right side's code, and one for the parent's Xpipewait<sup>85f</sup>.

Here is the overall picture for `ls | wc -l`:



The child runs `cmd1`'s bytecodes and ends with `Xexit` (it must terminate, not return to the parent's thread stack). The parent creates a new thread for `cmd2` that ends with `Xreturn`, then falls through to `Xpipewait` which waits for the child process and concatenates both exit statuses into a pipe status string like "0|0".

```
<outcode() locals 84a>+≡ (65f) <76a 96a>
    int q;
```

```
<outcode() cases 84b>+≡ (65f) <80b 86c>
    case PIPE:
        emitf(Xpipe);
        emitf(t->fd0); // 1 in the normal case
        emitf(t->fd1); // 0 in the normal case
        p = emitf(0);
        q = emitf(0);

        // for first child
        outcode(c0, eflag);
        emitf(Xexit);

        // for second child
        stuffdot(p);
        outcode(c1, eflag);
        emitf(Xreturn);

        // for parent (rc)
        stuffdot(q);
        emitf(Xpipewait);
        break;
```

```
<codefree() in loop over code cp, switch bytecode cases 84c>+≡ (29e) <69f 94a>
    else if(p->f==Xpipe)
        p+=4;
```

Uses `Xpipe()` 85b.

```
<function Xreturn 84d>≡ (203a)
    void
    Xreturn(void)
    {
        struct Thread *p = runq;

        <Xreturn() pop the redirections from this thread 83b>
        // free p
        while(p->argv)
            poplist();
        codefree(p->code);
        // pop(runq)
        runq = p->ret;
        efree((char *)p);

        if(runq==nil)
            Exit(getstatus(), __LOC__);
    }
```

Uses `Thread` 30b, `codefree()` 29e, `efree()` 158d, `getstatus()` 67d, `poplist()` 66c, and `runq` 30c.

```
<constant PRD 84e>≡ (196c)
/*
 * Which fds are the reading/writing end of a pipe?
 * Unfortunately, this can vary from system to system.
 * 9th edition Unix doesn't care, the following defines
 * work on plan 9.
 */
#define PRD 0
```

⟨constant PWR 85a⟩≡ (196c)

```
#define PWR 1
```

⟨function Xpipe 85b⟩≡ (206a)

```
void
Xpipe(void)
{
    struct Thread *p = runq;
    int pc = p->pc;
    int forkid;
    fdt lfd = p->code[pc++].i;
    fdt rfd = p->code[pc++].i;
    fdt pfd[2];

    if(pipe(pfd)<0){
        Xerror("can't get pipe");
        return;
    }
    switch(forkid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        // pc+2 so jump the jump addresses
        start(p->code, pc+2, runq->local);
        runq->ret = nil;
        close(pfd[PRD]);
        pushredir(ROPEN, pfd[PWR], lfd);
        break;
    default: // parent
        addwaitpid(forkid);
        start(p->code, p->code[pc].i, runq->local);
        close(pfd[PWR]);
        pushredir(ROPEN, pfd[PRD], rfd);
        p->pc = p->code[pc+1].i;
        p->pid = forkid;
        break;
    }
}
```

Uses PRD 84e, PWR 85a, ROPEN 80a, Thread 30b, Xerror() 71b, addwaitpid() 68d, clearwaitpids() 68g, pushredir() 79c, runq 30c, and start() 39a.

⟨Thread other fields 85c⟩+≡ (30b) <82c 85d>

```
// option<int> (None = -1)
int pid; /* process for Xpipewait to wait for */
```

⟨Thread other fields 85d⟩+≡ (30b) <85c 89b>

```
char status[NSTATUS]; /* status for Xpipewait */
```

Uses NSTATUS 85e.

⟨constant NSTATUS 85e⟩≡ (199a)

```
#define NSTATUS ERRMAX /* length of status (from plan 9) */
```

⟨function Xpipewait 85f⟩≡ (203b)

```
void
Xpipewait(void)
{
    char status[NSTATUS+1];
    if(runq->pid== -1)
```

```

        setstatus(concstatus(runq->status, getstatus()));
    else{
        strncpy(status, getstatus(), NSTATUS);
        status[NSTATUS]='\0';
        Waitfor(runq->pid, true);
        runq->pid=-1;
        setstatus(concstatus(getstatus(), status));
    }
}

```

Uses NSTATUS 85e, concstatus() 86b, getstatus() 67d, runq 30c, and setstatus() 67c.

⟨Waitfor() in while loop, if wait returns another pid 86a⟩≡ (74b)

```

// else
for(p = runq->ret;p;p = p->ret)
    if(p->pid==w->pid){
        p->pid=-1;
        strcpy(p->status, w->msg);
    }
free(w);

```

Uses runq 30c.

⟨function concstatus 86b⟩≡ (210c)

```

char*
concstatus(char *s, char *t)
{
    static char v[NSTATUS+1];
    int n = strlen(s);
    strncpy(v, s, NSTATUS);
    if(n<NSTATUS){
        v[n]='|';
        strncpy(v+n+1, t, NSTATUS-n-1);
    }
    v[NSTATUS]='\0';
    return v;
}

```

Uses NSTATUS 85e.

## 8.3.6 Asynchronous execution

The & operator runs a command in the background. The child's stdin is redirected to /dev/null (so it does not compete with the shell for terminal input), and the child is put in a new note group (RFNOTEg) so that interrupts sent to the shell do not kill it. The parent records the child's PID in \$apid (for “asynchronous PID”), allowing the user to wait for it later or send signals to it.

⟨outcode() cases 86c⟩+≡ (65f) <84b 88a>

```

case '&':
    emitf(Xasync);
    p = emitf(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    break;

```

⟨function Xasync 86d⟩≡ (206a)

```

void
Xasync(void)
{
    fdt null = open("/dev/null", 0);

```

```

int pid;
char npid[10];
if(null<0){
    Xerror("Can't open /dev/null\n");
    return;
}
switch(pid = rfork(RFFDG|RFPROC|RFNOTEG)){
case -1:
    close(null);
    Xerror("try again");
    break;
case 0: // child
    clearwaitpids();
    pushredir(ROPEN, null, 0);
    // start a new Thread runq->pc+1 so skip pointer to code after &
    start(runq->code, runq->pc+1, runq->local);
    runq->ret = 0;
    break;
default: // parent
    addwaitpid(pid);
    close(null);
    // jump to code after &
    runq->pc = runq->code[runq->pc].i;
    intoascii(npid, pid);
    setvar("apid", newword(npid, (word *)nil));
    break;
}
}

```

Uses ROPEN 80a, Xerror() 71b, addwaitpid() 68d, clearwaitpids() 68g, intoascii() 169b, newword() 31d, pushredir() 79c, runq 30c, setvar() 34a, and start() 39a.

```

⟨function Xexit 87⟩≡ (203b)
void
Xexit(void)
{
    struct Var *trapreq;
    struct Word *starval;
    static bool beenhere = false;

    if(getpid()==mypid && !beenhere){
        trapreq = vlook("sigexit");
        if(trapreq->fn){
            beenhere = true;
            --runq->pc;
            starval = vlook("*")->val;
            start(trapreq->fn, trapreq->pc, (struct Var *)0);
            runq->local = newvar(strdup("*"), runq->local);
            runq->local->val = copywords(starval, (struct Word *)0);
            runq->local->changed = true;
            runq->redir = runq->startredir = nil;
            return;
        }
    }
    Exit(getstatus(), __LOC__);
}

```

Uses Var 33c, Word 31c, copywords() 32c, getstatus() 67d, mypid 118d, newvar() 35b, runq 30c, start() 39a, and vlook() 34b.

## 8.4 Control flow statements

### 8.4.1 if

The `if` compilation uses the same forward-jump technique as `&&`: emit the condition, then `Xif`<sup>88b</sup> with a jump offset to skip the body if the condition is false. The `Xwastrue`<sup>88d</sup> bytecode at the end resets the `ifnot` flag. This flag is how `rc` connects `if` and `if not`: `Xif` sets `ifnot` to `true`, and `Xifnot`<sup>88f</sup> checks it to decide whether to execute the else-body. If any other command is executed between `if` and `if not`, `Xwastrue` resets the flag, making `if not` a no-op.

```
<outcode() cases 88a>+≡ (65f) <86c 88e>
  case IF:
    outcode(c0, false);
    emitf(Xif);
    p = emitf(0);
    outcode(c1, eflag);
    emitf(Xwastrue);
    stuffdot(p);
    break;
```

```
<function Xif 88b>≡ (203b)
void
Xif(void)
{
    ifnot = true;
    if(truestatus())
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
}
```

Uses `ifnot` 88c, `runq` 30c, and `truestatus()` 68a.

```
<global ifnot 88c>≡ (203b)
bool ifnot; /* dynamic if not flag */
```

```
<function Xwastrue 88d>≡ (203b)
void
Xwastrue(void)
{
    ifnot = false;
}
```

Uses `ifnot` 88c.

```
<outcode() cases 88e>+≡ (65f) <88a 89e>
  case NOT:
    <outcode() when NOT, sanity check last command was an if 89a>
    emitf(Xifnot);
    p = emitf(0);
    outcode(c0, eflag);
    stuffdot(p);
    break;
```

```
<function Xifnot 88f>≡ (203b)
void
Xifnot(void)
{
    if(ifnot)
        runq->pc++;
    else
```

```

        runq->pc = runq->code[runq->pc].i;
    }

```

Uses ifnot 88c and runq 30c.

```

⟨outcode() when NOT, sanity check last command was an if 89a⟩≡ (88e)
    if(!runq->iflast)
        yyerror("'if not' does not follow 'if(...)'");

```

```

⟨Thread other fields 89b⟩+≡ (30b) <85d
    bool iflast; /* static 'if not' checking */

```

```

⟨outcode() set iflast after switch 89c⟩≡ (65f)
    if(t->type!=NOT && t->type!=';')
        runq->iflast = t->type==IF;
    else
        if(c0)
            runq->iflast = c0->type==IF;

```

```

⟨outcode() set iflast before switch 89d⟩≡ (65f)
    if(t->type!=NOT && t->type!=';')
        runq->iflast = false;

```

## 8.4.2 while

The while loop compiles to: the condition bytecodes, Xtrue<sup>76e</sup> (jump out if false), the body bytecodes, Xjump<sup>89g</sup> (jump back to the condition). If the condition is empty (e.g., while()), an Xsettrue<sup>89f</sup> bytecode is emitted to make it loop forever.

```

⟨outcode() cases 89e⟩+≡ (65f) <88e 90a>
    case WHILE:
        q = codep;
        outcode(c0, false);
        if(q==codep)
            emitf(Xsettrue); /* empty condition == while(true) */
        emitf(Xtrue);
        p = emitf(0);
        outcode(c1, eflag);
        emitf(Xjump);
        emitf(q);
        stuffdot(p);
        break;

```

```

⟨function Xsettrue 89f⟩≡ (203b)
    void
    Xsettrue(void)
    {
        setstatus("");
    }

```

Uses setstatus() 67c.

```

⟨function Xjump 89g⟩≡ (203b)
    void
    Xjump(void)
    {
        runq->pc = runq->code[runq->pc].i;
    }

```

Uses runq 30c.

### 8.4.3 for

The **for** loop is more complex. It first evaluates the list to iterate over (or expands `\$*` if no list is given), creates a local variable for the loop variable (via `Xlocal`<sup>97a</sup>), and then loops with `Xfor`<sup>90b</sup>. `Xfor()`<sup>90b</sup> pops the next word from the list, assigns it to the local variable, and executes the body. When the list is exhausted, it jumps past the body and `Xunlocal`<sup>97b</sup> removes the local binding.

```
<outcode() cases 90a>+≡ (65f) <89e 91a>
case FOR:
    emitf(Xmark);
    if(c1){
        outcode(c1, eflag);
        emitf(Xglob);
    }
    else{
        emitf(Xmark);
        emitf(Xword);
        emits(strdup("*"));
        emitf(Xdol);
    }
    emitf(Xmark); /* dummy value for Xlocal */
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xlocal);
    p = emitf(Xfor);
    q = emiti(0);
    outcode(c2, eflag);
    emitf(Xjump);
    emitf(p);
    stuffdot(q);
    emitf(Xunlocal);
    break;
```

```
<function Xfor 90b>≡ (203b)
void
Xfor(void)
{
    if(runq->argv->words==0){
        poplist();
        runq->pc = runq->code[runq->pc].i;
    }
    else{
        freelist(runq->local->val);
        runq->local->val = runq->argv->words;
        runq->local->changed = true;
        runq->argv->words = runq->argv->words->next;
        runq->local->val->next = 0;
        runq->pc++;
    }
}
```

Uses `freelist()` <sup>33a</sup>, `poplist()` <sup>66c</sup>, and `runq` <sup>30c</sup>.

### 8.4.4 switch

The **switch** statement is the most complex control flow construct to compile because it involves multiple jump targets. The generated bytecodes push the switch value onto the stack, then for each **case**, push the case patterns and call `Xcase`<sup>92c</sup>, which uses `match()`<sup>124b</sup> (the same glob matcher used by the `~` operator) to compare.

If the case matches, execution falls through to the case body; otherwise, it jumps to the next case. After a case body executes, it jumps to the end of the switch. The comment in `codeswitch()`<sup>91b</sup> below shows the layout.

```

⟨outcode() cases 91a⟩+≡ (65f) <90a 93a>
  case SWITCH:
    codeswitch(t, eflag);
    break;

⟨function codeswitch 91b⟩≡ (209)
/*
 * switch code looks like this:
 * Xmark
 * (get switch value)
 * Xjump 1f
 * out: Xjump leave
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1:
 * leave:
 * Xpopm
 */
void
codeswitch(tree *t, bool eflag)
{
    int leave; /* patch jump address to leave switch */
    int out; /* jump here to leave switch */
    int nextcase; /* patch jump address to next case */
    tree *tt;

    // c1 is BRACE { ; ; ; }
    if(c1->child[0]==nil
    || c1->child[0]->type != ';' )
    || !iscase(c1->child[0]->child[0])){
        yyerror("case missing in switch");
        return;
    }

    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xjump);

    nextcase = emitf(0);
    out = emitf(Xjump);
    leave = emitf(0);

    stuffdot(nextcase);

    // from now on c0, c1, ... refer to this new t
    t = c1->child[0];
    while(t->type!=';'){
        tt = c1;
        emitf(Xmark);

```

```

    for(t = c0->child[0];t->type==ARGLIST;t = c0)
        outcode(c1, eflag);
    emitf(Xcase);
    nextcase = emitf(0);
    t = tt;
    for(;;){
        if(t->type==';'){
            if(iscase(c0))
                break;
            outcode(c0, eflag);
            t = c1;
        }
        else{
            if(!iscase(t))
                outcode(t, eflag);
            break;
        }
    }
    emitf(Xjump);
    emitf(out);
    stuffdot(nextcase);
}
stuffdot(leave);
emitf(Xpopm);
}

```

Uses ARGLIST, Xcase() 92c, Xjump() 89g, Xmark() 66b, Xpopm() 92a, c0-62 65c, c1-63 65d, emitf-65 64d, emitf-66 64c, iscase() 92b, stuffdot() 76c, and yerror() 156e.

```

⟨function Xpopm 92a⟩≡ (203b)
void
Xpopm(void)
{
    poplist();
}

```

Uses poplist() 66c.

```

⟨function iscase 92b⟩≡ (209)
bool
iscase(tree *t)
{
    if(t->type!=SIMPLE)
        return false;
    do { t = c0; } while(t->type==ARGLIST);
    return t->type==WORD && !t->quoted && strcmp(t->str, "case")==0;
}

```

Uses ARGLIST, SIMPLE, WORD, and c0-62 65c.

```

⟨function Xcase 92c⟩≡ (203b)
void
Xcase(void)
{
    word *p;
    char *s;
    bool ok = false;

    s = list2str(runq->argv->next->words);
    for(p = runq->argv->words;p;p = p->next){
        if(match(s, p->word, '\0')){
            ok = true;

```

```

        break;
    }
}
efree(s);
if(ok)
    runq->pc++;
else
    runq->pc = runq->code[runq->pc].i;
poplist();
}

```

Uses `efree()` 158d, `list2str()` 77e, `poplist()` 66c, and `runq` 30c.

## 8.4.5 Blocks: ‘{...}’

Blocks and parenthesized expressions are transparent to the bytecode generator—they simply recurse into their child node. The distinction between PAREN, PCMD, and BRACE matters to the parser (for grouping word lists vs. command groups), but by the time we generate bytecodes, they all just mean “compile the contents.”

```

⟨outcode() cases 93a⟩+≡ (65f) <91a 93b>
    case PAREN:
        outcode(c0, eflag);
        break;

```

```

⟨outcode() cases 93b⟩+≡ (65f) <93a 93d>
    case PCMD:
    case BRACE:
        outcode(c0, eflag);
        break;

```

## 8.5 Functions

In `rc`, functions are stored in variables: the `Var` structure has an `fn` field pointing to the function’s bytecode vector and a `pc` field indicating where in that vector the function body starts. This design unifies functions and variables in the same namespace and allows functions to be exported to the environment (see Chapter 10).

```

⟨Var other fields 93c⟩≡ (33c) 111a▷
    code *fn; /* pointer to function’s code vector */
    int pc; /* pc of start of function */
    bool fnchanged;

```

### 8.5.1 Function definitions (fn <foo> ... )

When defining a function with `fn foo {body}`, `Xfn`<sup>94c</sup> stores the current code vector (via `codecopy`<sup>29d</sup>) in the variable named `foo`. This is why bytecode vectors are reference-counted: the function’s code vector is shared with the code that defined it. When `fn foo` is used without a body, `Xdelfn`<sup>95a</sup> deletes the function by setting `v->fn` to `nil`.

```

⟨outcode() cases 93d⟩+≡ (65f) <93b 96b>
    case FN:
        emitf(Xmark);
        outcode(c0, eflag);
        if(c1){
            emitf(Xfn);
            p = emitf(0);
            emits(fnstr(c1));

```

```

        outcode(c1, eflag); // body of the function
        emitf(Xunlocal); /* get rid of $* */ //$
        emitf(Xreturn);
        stuffdot(p);
    }
    else
        emitf(Xdelfn);
    break;

```

⟨codefree() in loop over code cp, switch bytecode cases 94a⟩+≡ (29e) <84c 137f>

```

    else if(p->f==Xfn){
        efree(p[2].s);
        p+=2;
    }

```

Uses Xfn() 94c and efree() 158d.

⟨function fnstr 94b⟩≡ (209)

```

char*
fnstr(tree *t)
{
    void *v;
    extern char nl;
    char svnl = nl;
    io *f = openstr();

    nl = ',';
    pfmt(f, "%t", t);
    nl = svnl;
    v = f->strp;
    f->strp = nil;
    closeio(f);
    return v;
}

```

Uses closeio() 165a, nl 151a, openstr() 165b, and pfmt() 166b.

⟨function Xfn 94c⟩≡ (203b)

```

void
Xfn(void)
{
    var *v;
    word *a;
    int end;

    end = runq->code[runq->pc].i;
    globlist();
    for(a = runq->argv->words; a; a = a->next){
        v = gvlook(a->word);
        if(v->fn)
            codefree(v->fn);
        v->fn = codecopy(runq->code);
        v->pc = runq->pc+2;
        v->fnchanged = true;
    }
    runq->pc = end;
    poplist();
}

```

Uses codecopy() 29d, codefree() 29e, globlist() 120d, gvlook() 34e, poplist() 66c, and runq 30c.

```

⟨function Xdelfn 95a⟩≡ (203b)
void
Xdelfn(void)
{
    var *v;
    word *a;

    for(a = runq->argv->words;a;a = a->next){
        v = gvlook(a->word);
        if(v->fn)
            codefree(v->fn);
        v->fn = nil;
        v->fnchanged = true;
    }
    poplist();
}

```

Uses codefree() 29e, gvlook() 34e, poplist() 66c, and runq 30c.

## 8.5.2 Function uses (<foo>(...))

When Xsimple()<sup>70</sup> detects that the command name matches a function, it calls `execfunc()`<sup>95d</sup>. This function pops the command arguments from `argv`, starts a new thread with the function's bytecodes, and binds the arguments to a local `$*` variable. This is how function arguments work in `rc`: inside a function, `$*` contains the arguments passed to the function, shadowing the script's global `$*`.

```

⟨Xsimple() other locals 95b⟩≡ (70) 102c▷
    var *v;

```

```

⟨Xsimple() if argv0 is a function 95c⟩≡ (70)
    v = gvlook(a->word);
    if(v->fn)
        execfunc(v);

```

Uses execfunc() 95d and gvlook() 34e.

```

⟨function execfunc 95d⟩≡ (208a)
void
execfunc(var *func)
{
    word *starval;

    popword();
    starval = runq->argv->words;
    runq->argv->words = nil;
    poplist();
    start(func->fn, func->pc, runq->local);
    runq->local = newvar(strdup("*"), runq->local);
    runq->local->val = starval;
    runq->local->changed = true;
}

```

Uses newvar() 35b, poplist() 66c, popword() 66d, runq 30c, and start() 39a.

## 8.6 Variables

### 8.6.1 Variable definitions (<x>=...)

Variable assignment has two modes: when followed by a command (e.g., `x=1 cmd`), the assignment is local to that command (using `Xlocal`<sup>97a</sup>/`Xunlocal`<sup>97b</sup>); when standalone (e.g., `x=1`), it is global (using `Xassign`<sup>96c</sup>).

The code detects which mode by following the chain of '=' nodes through `child[2]`: if it eventually reaches a non-assignment node, there is a command and the assignments are local. Multiple chained assignments like `A=b C=d cmd` are supported: each creates a local binding, and they are all unlocalized after the command finishes.

```
<outcode() locals 96a>+≡ (65f) <84a
    tree *tt;
```

```
<outcode() cases 96b>+≡ (65f) <93d 97c>
    case '=':
        tt = t;
        for(;t && t->type=='='; t = c2);

        if(t){      /* var=value cmd */
            for(t = tt;t->type=='=';t = c2){
                emitf(Xmark);
                outcode(c1, eflag);
                emitf(Xmark);
                outcode(c0, eflag);
                emitf(Xlocal); /* push var for cmd */
            }
            outcode(t, eflag); /* gen. code for cmd */
            for(t = tt; t->type == '='; t = c2)
                emitf(Xunlocal); /* pop var */
        }
        else{      /* var=value */
            for(t = tt;t;t = c2){
                emitf(Xmark);
                outcode(c1, eflag);
                emitf(Xmark);
                outcode(c0, eflag);
                emitf(Xassign); /* set var permanently */
            }
        }
        t = tt; /* so tests below will work */
        break;
```

```
<function Xassign 96c>≡ (203b)
    void
    Xassign(void)
    {
        var *v;
        if(count(runq->argv->words)!=1){
            Xerror1("variable name not singleton!");
            return;
        }
        deglob(runq->argv->words->word); // remove the special \001 mark
        v = vlook(runq->argv->words->word);
        poplist();

        globlist();
        freewords(v->val);
        v->val = runq->argv->words;
        v->changed = true;
        runq->argv->words = nil;
        poplist();
    }
```

Uses `Xerror1()` 71c, `count()` 32a, `deglob()` 122a, `freewords()` 32d, `globlist()` 120d, `poplist()` 66c, `runq` 30c, and `vlook()` 34b.

```

⟨function Xlocal 97a⟩≡ (203b)
void
Xlocal(void)
{
    if(count(runq->argv->words)!=1){
        Xerror1("variable name must be singleton\n");
        return;
    }
    deglob(runq->argv->words->word);
    runq->local = newvar(strdup(runq->argv->words->word), runq->local);
    poplist();
    globlist();
    runq->local->val = runq->argv->words;
    runq->local->changed = true;
    // change ownership
    runq->argv->words = nil;
    poplist();
}

```

Uses Xerror1() 71c, count() 32a, deglob() 122a, globlist() 120d, newvar() 35b, poplist() 66c, and runq 30c.

```

⟨function Xunlocal 97b⟩≡ (203b)
void
Xunlocal(void)
{
    var *v = runq->local, *hid;
    if(v==0)
        panic("Xunlocal: no locals!", 0);
    runq->local = v->next;
    hid = vlook(v->name);
    hid->changed = true;
    efree(v->name);
    freewords(v->val);
    efree((char *)v);
}

```

Uses efree() 158d, freewords() 32d, panic() 157c, runq 30c, and vlook() 34b.

## 8.6.2 Variable uses (\$<x>)

Xdol() <sup>97d</sup> (dollar) handles variable expansion. It looks up the variable name and pushes its value (a list of words) onto the argument stack. A subtle feature: if the variable name is a number (e.g., \$1, \$2), Xdol() treats it as a positional parameter, indexing into \$\* instead of looking up a named variable.

```

⟨outcode() cases 97c⟩+≡ (65f) <96b 98a>
case '$': //$
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xdol);
    break;

```

```

⟨function Xdol 97d⟩≡ (203b)
void
Xdol(void)
{
    word *a, *star;
    char *s, *t;
    int n;

    if(count(runq->argv->words)!=1){

```

```

        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);

    n = 0;
    for(t = s; '0' <= *t && *t <= '9'; t++)
        n = n*10 + *t - '0';

    a = runq->argv->next->words;
    if(n==0 || *t)
        a = copywords(vlook(s)->val, a);
    else{
        star = vlook("*")->val;
        if(star && 1 <= n && n <= count(star)){
            while(--n)
                star = star->next;
            a = newword(star->word, a);
        }
    }
    poplist();
    runq->argv->words = a;
}

```

⟨outcode() cases 98a⟩+≡ (65f) <97c 99a>

```

case COUNT:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xcount);
    break;

```

⟨function Xcount 98b⟩≡ (203b)

```

void
Xcount(void)
{
    word *a;
    char *s, *t;
    int n;
    char num[12];

    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);

    // n = int_of_string(s)
    n = 0;
    for(t = s; '0' <= *t && *t <= '9'; t++)
        n = n*10 + *t - '0';
    if(n==0 || *t){
        a = vlook(s)->val;
        inttoascii(num, count(a));
    }
    else{
        a = vlook("*")->val;
        inttoascii(num, (a && 1<=n && n<=count(a)) ? 1 : 0);
    }
}

```

```

    poplist();
    pushword(num);
}

```

Subscripting (`$x(2)` or `$x(2-5)`) extracts elements from a list variable by index or range. `subwords()`<sup>99c</sup> does the heavy lifting: it parses each subscript (which can be a single number or a `n-m` range, where a bare `n-` means “from `n` to the end”), walks `n-1` links into the variable’s value list, and calls `copywords()`<sup>32b</sup> to extract the slice. The recursion on `sub->next` handles multiple subscripts like `$x(1 3 5)`, accumulating results in reverse so the final list comes out in the right order.

```

⟨outcode() cases 99a⟩+≡ (65f) <98a 100a>

```

```

case SUB:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xsub);
    break;

```

```

⟨function Xsub 99b⟩≡ (203b)

```

```

void
Xsub(void)
{
    word *a, *v;
    char *s;
    if(count(runq->argv->next->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->next->words->word;
    deglob(s);
    a = runq->argv->next->next->words;
    v = vlook(s)->val;
    a = subwords(v, count(v), runq->argv->words, a);
    poplist();
    poplist();
    runq->argv->words = a;
}

```

Uses `Xerror1()` 71c, `count()` 32a, `deglob()` 122a, `poplist()` 66c, `runq` 30c, and `vlook()` 34b.

```

⟨function subwords 99c⟩≡ (203b)

```

```

word*
subwords(word *val, int len, word *sub, word *a)
{
    int n, m;
    char *s;
    if(!sub)
        return a;
    a = subwords(val, len, sub->next, a);
    s = sub->word;
    deglob(s);
    m = 0;
    n = 0;
    while('0'<=*s && *s<='9')
        n = n*10+ *s++ -'0';
    if(*s == '-'){
        if(++s == 0)
            m = len - n;
        else{
            while('0'<=*s && *s<='9')

```

```

        m = m*10+ *s++ -'0';
    m -= n;
}
}
if(n<1 || n>len || m<0)
    return a;
if(n+m>len)
    m = len-n;
while(--n > 0)
    val = val->next;
return copynwords(val, a, m+1);
}

```

⟨outcode() cases 100a⟩≡ (65f) <99a 130c>

```

case '^':
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xconc);
    break;

```

⟨function Xconc 100b⟩≡ (203b)

```

void
Xconc(void)
{
    word *lp = runq->argv->words;
    word *rp = runq->argv->next->words;
    word *vp = runq->argv->next->next->words;
    int lc = count(lp), rc = count(rp);
    if(lc!=0 || rc!=0){
        if(lc==0 || rc==0){
            Xerror1("null list in concatenation");
            return;
        }
        if(lc!=1 && rc!=1 && lc!=rc){
            Xerror1("mismatched list lengths in concatenation");
            return;
        }
        vp = conclist(lp, rp, vp);
    }
    poplist();
    poplist();
    runq->argv->words = vp;
}

```

Uses Xerror1() 71c, conclist() 100c, count() 32a, poplist() 66c, and runq 30c.

⟨function conclist 100c⟩≡ (203b)

```

word*
conclist(word *lp, word *rp, word *tail)
{
    char *buf;
    word *v;
    if(lp->next || rp->next)
        tail = conclist(lp->next==0? lp: lp->next,
            rp->next==0? rp: rp->next, tail);
    buf = emalloc(strlen(lp->word)+strlen((char *)rp->word)+1);
    strcpy(buf, lp->word);
    strcat(buf, rp->word);
    v = newword(buf, tail);
}

```

```
    efree(buf);  
    return v;  
}
```

Uses `conclist()` 100c, `efree()` 158d, `emalloc()` 158b, and `newword()` 31d.

### 8.6.3 Special variables

# Chapter 9

## Builtins

Shell builtins are commands that must be executed inside the shell process itself, rather than in a forked child. The most obvious example is `cd`: if it were an external program, the `chdir()` would happen in the child process and the parent (the shell) would remain in the old directory.

### 9.1 Overview

Builtins are stored in a simple array of name/function pairs. When `Xsimple()`<sup>70</sup> detects that the command name matches a builtin, it calls the corresponding function directly. The `builtin` keyword allows bypassing a function that shadows a builtin: `builtin cd` always calls the builtin `cd`, even if a function named `cd` exists.

```
<struct Builtin 102a>≡ (199a)
    struct Builtin {
        char *name;
        void (*fnc)(void);
    };

```

```
<global builtins 102b>≡ (210d)
    builtin builtins[] = {
        "cd"      ,      execcd,
        "exit"    ,      execexit,
        "."       ,      execdot,
        "eval"    ,      execeval,

        "whatis"  ,      execwhatis,

        "exec"    ,      execexec, /* but with popword first */
        "rfork"   ,      execnewprg,
        "wait"    ,      execwait,

        "shift"   ,      execshift,
        "finit"   ,      execfinit,
        "flag"    ,      execflag,
        //TODO: unix-specific "umask", execumask,
        0
    };

```

Uses `execcd()` 103c, `execeval()` 107e, `execexec()` 72b, `execexit()` 105c, `execfinit()` 147a, `execflag()` 145, `execnewprg()` 144b, and `execwait()` 108c.

```
<Xsimple() other locals 102c>+≡ (70) <95b
    struct Builtin *bp;

```

Uses `Builtin` 102a.

```

⟨Xsimple() if argv0 is a builtin 103a⟩≡ (70)
⟨Xsimple() if argv0 is the builtin keyword 103b⟩
for(bp = builtins;bp->name;bp++)
    if(strcmp(a->word, bp->name)==0){
        (*bp->fnc)();
        return;
    }

```

Uses builtins 102b.

```

⟨Xsimple() if argv0 is the builtin keyword 103b⟩≡ (103a)
if(strcmp(a->word, "builtin")==0){
    if(count(a)==1){
        pfmt(err, "builtin: empty argument list\n");
        setstatus("empty arg list");
        poplist();
        return;
    }
    a = a->next;
    popword();
}

```

Uses count() 32a, err 156b, pfmt() 166b, poplist() 66c, popword() 66d, and setstatus() 67c.

## 9.2 \$ cd

cd with no argument changes to \$home. With one argument, it searches the \$cdpath directories (similar to how \$path works for commands). When the directory is found via \$cdpath, cd prints the resolved path so the user knows where they ended up. Under rio, cd also writes the new directory to /dev/wdir so the window manager can track it.

```

⟨function execcd 103c⟩≡ (210d)
void
execcd(void)
{
    word *a = runq->argv->words;
    word *cdpath;
    char *dir;

    setstatus("can't cd");
    cdpath = vlook("cdpath")->val;

    switch(count(a)){
    case 1:
        a = vlook("home")->val;
        if(count(a)>=1){
            if(dochdir(a->word)>=0)
                setstatus("");
            else
                pfmt(err, "Can't cd %s: %r\n", a->word);
        }
        else
            pfmt(err, "Can't cd -- $home empty\n"); //$
        break;
    case 2:
        if(a->next->word[0]=='/' || cdpath==nil)
            cdpath = &nullpath;
        for(; cdpath; cdpath = cdpath->next){
            if(cdpath->word[0] != '\0')

```

```

        dir = appfile(cdpath->word, a->next->word);
    else
        dir = strdup(a->next->word);

    if(dochdir(dir) >= 0){
        if(cdpath->word[0] != '\0' &&
            strcmp(cdpath->word, ".") != 0)
            pfmt(err, "%s\n", dir);
        free(dir);
        setstatus("");
        break;
    }
    free(dir);
}
if(cdpath==nil)
    pfmt(err, "Can't cd %s: %r\n", a->next->word);
break;
default:
    pfmt(err, "Usage: cd [directory]\n");
    break;
}
poplist();
}

```

Uses `appfile()` 104a, `count()` 32a, `dochdir()` 104b, `err` 156b, `nullpath` 73b, `pfmt()` 166b, `poplist()` 66c, `runq` 30c, `setstatus()` 67c, and `vlook()` 34b.

```

⟨function appfile 104a⟩≡ (210d)
static char *
appfile(char *dir, char *comp)
{
    int dirlen, complen;
    char *s, *p;

    dirlen = strlen(dir);
    complen = strlen(comp);
    s = emalloc(dirlen + 1 + complen + 1);
    memmove(s, dir, dirlen);
    p = s + dirlen;
    *p++ = '/';
    memmove(p, comp, complen);
    p[complen] = '\0';
    return s;
}

```

Uses `emalloc()` 158b.

```

⟨function dochdir 104b⟩≡ (210d)
    errorneg1
    dochdir(char *word)
    {
        ⟨dochdir() locals 105a⟩

        // the actual syscall
        if(chdir(word)<0)
            return ERROR_NEG1;

        ⟨dochdir() adjust /dev/wdir if run under rio 105b⟩
        return OK_1;
    }

```

In Plan 9, the window manager `rio` (see WINDOWS book [Pad16c]) tracks the current working directory of each window via the `/dev/wdir` file. When `cd` succeeds in an interactive shell, it writes the new directory to this file so that `rio` can display it in the window title or use it when spawning new windows. The file descriptor is cached in a static variable and opened only once (the `-2` sentinel means “not yet tried”).

```
<dochdir() locals 105a>≡ (104b)
/* report to /dev/wdir if it exists and we're interactive */
static fdt wdirfd = -2;
```

```
<dochdir() adjust /dev/wdir if run under rio 105b>≡ (104b)
if(flag['i']!=nil){
    if(wdirfd== -2) /* try only once */
        wdirfd = open("/dev/wdir", OWRITE); // TODO: |OEXEC but plan9 specific?
    if(wdirfd>=0) {
        //fcntl(wdirfd, F_SETFD, FD_CLOEXEC);
        write(wdirfd, word, strlen(word));
    }
}
```

Uses `flag`.

## 9.3 \$ exit

```
<function execexit 105c>≡ (210d)
void
execexit(void)
{
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: exit [status]\nExiting anyway\n");
        // FALLTHROUGH
    case 2:
        setstatus(runq->argv->words->next->word);
        // FALLTHROUGH
    case 1:
        Xexit();
    }
}
```

Uses `Xexit()` 87, `count()` 32a, `err` 156b, `pfmt()` 166b, `runq` 30c, and `setstatus()` 67c.

## 9.4 \$ .

The `.` (dot) builtin sources a script: it reads and executes the commands from a file in the current shell (without forking a new process). This is how `rc` loads initialization files like `rcmain` and the user’s profile. It creates a new thread whose `cmdfd` reads from the sourced file, so `Xrdcmds()`<sup>41a</sup> will read commands from that file instead of the terminal.

```
<function execdot 105d>≡ (210d)
void
execdot(void)
{
    thread *p = runq;
    bool iflag = false;
    fdt fd;
    list *av;
    char *zero; // new stdin
```

```

char *file;
word *path;
⟨execdot() other locals 107c⟩

⟨execdot() if first 107d⟩
⟨execdot() if not first execution 128d⟩

popword(); // "."
if(p->argv->words && strcmp(p->argv->words->word, "-i")==0){
    iflag = true;
    popword();
}

/* get input file */
if(p->argv->words==nil){
    Xerror1("Usage: . [-i] file [arg ...]");
    return;
}
zero = strdup(p->argv->words->word);
popword();

fd = -1;
for(path = searchpath(zero); path; path = path->next){
    if(path->word[0] != '\0')
        file = appfile(path->word, zero);
    else
        file = strdup(zero);

    fd = open(file, 0);
    free(file);
    if(fd >= 0)
        break;
}
if(fd<0){
    pfmt(err, "%s: ", zero);
    setstatus("can't open"); // what for? it is reseted by Xerror anyway
    Xerror(".: can't open");
    return;
}

/* set up for a new command loop */
start(dotcmds, 1, (struct Var *)nil);

pushredir(RCLOSE, fd, 0);
runq->cmdfile = zero;
runq->cmdfd = openfd(fd);
runq->iflag = iflag;

runq->iflast = false;

/* push $* value */
pushlist();
runq->argv->words = p->argv->words;

/* free caller's copy of $* */
av = p->argv;
p->argv = av->next;
efree((char *)av);

/* push $0 value */

```

```

pushlist();
pushword(zero);

```

```

ndot++;
}

```

Uses RCLOSE 137h, Var 33c, Xerror() 71b, Xerror1() 71c, appfile() 104a, dotcmds 107b, efree() 158d, err 156b, ndot 107a, openfd() 163e, pfmt() 166b, popword() 66d, pushlist() 40a, pushredir() 79c, pushword() 40b, runq 30c, searchpath() 73a, setstatus() 67c, and start() 39a.

*<global ndot 107a>*≡ (199b)

```

/*
 * How many dot commands have we executed?
 * Used to ensure that -v flag doesn't print rcmain.
 */
int ndot;

```

*<global dotcmds 107b>*≡ (210d)

```

union Code dotcmds[14];

```

Uses Code 29c.

*<execdot() other locals 107c>*≡ (105d)

```

static bool first = true;

```

*<execdot() if first 107d>*≡ (105d)

```

if(first){
    dotcmds[0].i = 1;
    dotcmds[1].f = Xmark;
    dotcmds[2].f = Xword;
    dotcmds[3].s = "0";
    dotcmds[4].f = Xlocal; // will pop_list twice

    dotcmds[5].f = Xmark;
    dotcmds[6].f = Xword;
    dotcmds[7].s="*";
    dotcmds[8].f = Xlocal; // will pop_list twice

    dotcmds[9].f = Xrdcmds; // =~ a REPL

    dotcmds[10].f = Xunlocal;
    dotcmds[11].f = Xunlocal;
    dotcmds[12].f = Xreturn;

    first = false;
}

```

Uses Xlocal() 97a, Xmark() 66b, Xrdcmds() 41a, Xreturn() 84d, Xunlocal() 97b, Xword() 66a, and dotcmds 107b.

## 9.5 \$ eval

**eval** takes a string and executes it as a command. This is useful when the command to execute is constructed dynamically (e.g., from a variable). It joins the arguments into a single string, then creates a new thread reading from that string (via **opencore()** <sup>165c</sup>, which creates a buffered IO from memory).

*<function execeval 107e>*≡ (210d)

```

void
execeval(void)
{
    char *cmdline, *s, *t;

```

```

int len = 0;
word *ap;

if(count(runq->argv->words)<=1){
    Xerror1("Usage: eval cmd ...");
    return;
}
eflagok = true;
for(ap = runq->argv->words->next; ap; ap = ap->next)
    len+=1+strlen(ap->word);

cmdline = emalloc(len);
s = cmdline;
for(ap = runq->argv->words->next; ap; ap = ap->next){
    for(t = ap->word; *t;) *s++=*t++;
    *s++=' ';
}
s[-1]='\n';
poplist();

execcmds(opencore(cmdline, len));
efree(cmdline);
}

```

Uses Xerror1() 71c, count() 32a, eflagok 128c, efree() 158d, emalloc() 158b, execcmds() 108b, opencore() 165c, poplist() 66c, and runq 30c.

*<global rdcmds 108a>*≡ (208a)  
 union Code rdcmds[4];

Uses Code 29c.

*<function execcmds 108b>*≡ (208a)  
 void  
 execcmds(io \*f)  
 {  
 static bool first = true;  
 if(first){  
 rdcmds[0].i = 1;  
 rdcmds[1].f = Xrdcmd;  
 rdcmds[2].f = Xreturn;  
 first = false;  
 }  
  
 start(rdcmds, 1, runq->local);  
 runq->cmdfd = f;  
 runq->iflast = false;  
 }

Uses Xrdcmd() 41a, Xreturn() 84d, rdcmds 108a, runq 30c, and start() 39a.

## 9.6 \$ wait

*<function execwait 108c>*≡ (210d)  
 void  
 execwait(void)  
 {  
 switch(count(runq->argv->words)){  
 default:  
 Xerror1("Usage: wait [pid]");  
 }

```

        return;
    case 2:
        Waitfor(atoi(runq->argv->words->next->word), false);
        break;
    case 1:
        Waitfor(-1, false);
        break;
}
poplist();
}

```

Uses Xerror1() 71c, count() 32a, poplist() 66c, and runq 30c.

# Chapter 10

## Environment

Under Plan 9, the environment is not passed via the `envp` parameter of `main()` as in UNIX. Instead, each environment variable is stored as a file under `/env/`: the variable `$path` is stored in `/env/path`. This is a more elegant design because child processes automatically inherit the environment through the namespace, and changes are visible to all processes sharing the same namespace. `rc` must synchronize its internal variable table with these files in both directions: `Updenv()`<sup>110a</sup> writes changed variables to `/env/`, and `Vinit()`<sup>111c</sup> reads variables from `/env/` at startup.

### 10.1 Updenv()

`Updenv()`<sup>110a</sup> iterates over all global and local variables and writes any that have changed (with `v->changed == true`) to the `/env/` filesystem. The `changed` flag avoids redundant writes.

```
<function Updenv 110a>≡ (213b)
void
Updenv(void)
{
    var *v, **h;

    for(h = gvar; h != &gvar[NVAR]; h++)
        for(v = *h; v; v = v->next)
            addenv(v);
    if(runq)
        updenvlocal(runq->local);
}
```

Uses `NVAR` 34c, `addenv()` 111b, `gvar` 34d, `runq` 30c, and `updenvlocal()` 110b.

```
<function updenvlocal 110b>≡ (213b)
void
updenvlocal(var *v)
{
    if(v){
        updenvlocal(v->next);
        addenv(v);
    }
}
```

Uses `addenv()` 111b and `updenvlocal()` 110b.

```
<enum MiscPlan9 110c>≡ (213b)
enum {
    Maxenvname = 256, /* undocumented limit */
};
```

```

⟨Var other fields 111a⟩ +=
    bool changed;                                     (33c) <93c

⟨function addenv 111b⟩ ≡                               (213b)
void
addenv(var *v)
{
    char envname[Maxenvname];
    word *w;
    int f;
    io *fd;

    if(v->changed){
        v->changed = false;
        snprintf(envname, sizeof envname, "/env/%s", v->name);
        if((f = Creat(envname)) < 0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            for(w = v->val; w; w = w->next)
                write(f, w->word, strlen(w->word)+1L);
            close(f);
        }
    }
    if(v->fnchanged){
        v->fnchanged = false;
        snprintf(envname, sizeof envname, "/env/fn#%s", v->name);
        if((f = Creat(envname)) < 0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            if(v->fn){
                fd = openfd(f);
                pfmt(fd, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
                closeio(fd);
            }
            close(f);
        }
    }
}

```

Uses `Creat()` 169g, `Maxenvname-2` 110c, `closeio()` 165a, `err` 156b, `openfd()` 163e, and `pfmt()` 166b.

## 10.2 Vinit()

`Vinit()` <sup>111c</sup> reads the `/env/` directory at startup and populates the global variable table. Each environment file contains the variable's words separated by null bytes. `Vinit()` reads the file, splits on nulls, and calls `setvar()` <sup>34a</sup> for each variable. It then resets the `changed` flag so that `Updenv()` <sup>110a</sup> does not write them back immediately. The null byte separator is the only safe choice: unlike spaces or newlines, null bytes cannot appear in filenames or variable values, so the encoding is unambiguous. This is the same insight behind UNIX tools like `find -print0` and `xargs -0`.

```

⟨function Vinit 111c⟩ ≡                               (213b)
void
Vinit(void)
{
    int dir, f, len, i, n, nent;
    char *buf, *s;
    char envname[Maxenvname];
    word *val;

```

```

Dir *ent;

dir = open("/env", OREAD);
if(dir<0){
    pfmt(err, "rc: can't open /env: %r\n");
    return;
}
ent = nil;
for(;;){
    nent = dirread(dir, &ent);
    if(nent <= 0)
        break;
    for(i = 0; i<nent; i++){
        len = ent[i].length;
        if(len && strcmp(ent[i].name, "fn#", 3)!=0){
            snprintf(envname, sizeof envname, "/env/%s", ent[i].name);
            if((f = open(envname, 0))>=0){
                buf = emalloc(len+1);
                n = readn(f, buf, len);
                if (n <= 0)
                    buf[0] = '\0';
                else
                    buf[n] = '\0';
                val = 0;
                /* Charitably add a 0 at the end if need be */
                if(buf[len-1])
                    buf[len++]='\0';
                s = buf+len-1;
                for(;;){
                    while(s!=buf && s[-1]!='\0') --s;
                    val = newword(s, val);
                    if(s==buf)
                        break;
                    --s;
                }
                setvar(ent[i].name, val);
                vlook(ent[i].name)->changed = false;
                close(f);
                efree(buf);
            }
        }
    }
    free(ent);
}
close(dir);
}

```

Uses Maxenvname-2 110c, efree() 158d, emalloc() 158b, err 156b, newword() 31d, pfmt() 166b, setvar() 34a, and vlook() 34b.

$\langle \text{global envdir } 112 \rangle \equiv$  (203b)  
 fdt envdir;

# Chapter 11

## Signals

Signal handling is critical for a shell because the shell sits between the user and every running program. When the user presses `^C`, the interrupt signal is sent to all processes in the foreground group—including the shell itself. If the shell simply died along with the interrupted command, the user would lose their terminal session. Instead, the shell must catch the signal, cancel the current command, and return to its prompt, ready for the next input. Beyond `^C`, a shell must also handle hangup signals (so it can clean up or notify background jobs when a terminal disconnects), and let users define their own trap handlers for scripting (e.g., cleaning up temporary files on exit). See *KERNEL* book [Pad14] for how the kernel delivers notes to processes.

### 11.1 Overview

Under Plan 9, signals are called *notes*. When a note is received (e.g., an interrupt from `^C`), the kernel calls the handler registered by `Trapinit()`<sup>114d</sup>. The handler does not execute the trap function immediately (which would be unsafe in the middle of interpreting bytecodes). Instead, it increments a counter (`ntrap`) and the main interpreter loop checks this counter after each bytecode (see the `handling trap if necessary` chunk in `main()X`). Users can define signal handlers with `fn sigint {...}`, `fn sighup {...\}`, etc. If no handler is defined, the default behavior applies (usually terminating the process).

```
<constant NSIG 113a>≡ (196c)
#define NSIG 32
```

```
<constant SIGINT 113b>≡ (196c)
#define SIGINT 2
```

```
<constant SIGQUIT 113c>≡ (196c)
#define SIGQUIT 3
```

```
<global syssigname 113d>≡ (213b)
char *syssigname[] = {
    "exit", /* can't happen */
    "hangup",
    "interrupt",
    "quit", /* can't happen */
    "alarm",
    "kill",
    "sys: fp: ",
    "term",
    0
};
```

```

⟨global signame 114a⟩≡ (213b)
char *signame[] = {
    "sigexit",
    "sighup",
    "sigint",
    "sigquit",
    "sigalrm",
    "sigkill",
    "sigfpe",
    "sigterm",
    0
};

```

```

⟨global ntrap 114b⟩≡ (207b)
int ntrap; /* number of outstanding traps */

```

```

⟨global trap 114c⟩≡ (207b)
int trap[NSIG]; /* number of outstanding traps per type */

```

## 11.2 Registering handlers: Trapinit()

```

⟨function Trapinit 114d⟩≡ (213b)
void
Trapinit(void)
{
    notify(notifyf);
}

```

Uses notifyf() 114e.

```

⟨function notifyf 114e⟩≡ (213b)
void
notifyf(void*, char *s)
{
    int i;
    for(i = 0; syssigname[i]; i++)
        if(strncmp(s, syssigname[i], strlen(syssigname[i]))==0){
            if(strncmp(s, "sys: ", 5)!=0)
                interrupted = true;
            goto Out;
        }
    pfmt(err, "rc: note: %s\n", s);
    noted(NDFLT);
    return;
Out:
    if(strcmp(s, "interrupt")!=0 || trap[i]==0){
        trap[i]++;
        ntrap++;
    }
    if(ntrap>=32){ /* rc is probably in a trap loop */
        pfmt(err, "rc: Too many traps (trap %s), aborting\n", s);
        abort();
    }
    noted(NCONT);
}

```

Uses err 156b, interrupted 115c, ntrap 114b, pfmt() 166b, syssigname 113d, and trap 114c.

## 11.3 Trap dispatch: dotrap()

```
⟨main() handing trap if necessary in interpreter loop 115a⟩≡ (40c)
    if(ntrap)
        dotrap();
```

Uses dotrap() 115b and ntrap 114b.

```
⟨function dotrap 115b⟩≡ (207b)
void
dotrap(void)
{
    int i;
    struct Var *trapreq;
    struct Word *starval;

    starval = vlook("*")->val;
    while(ntrap)
        for(i = 0; i!=NSIG; i++)
            while(trap[i]){
                --trap[i];
                --ntrap;
                if(getpid()!=mypid)
                    Exit(getstatus(), __LOC__);
                trapreq = vlook(signame[i]);
                if(trapreq->fn){
                    start(trapreq->fn, trapreq->pc, (struct Var *)nil);
                    runq->local = newvar(strdup("*"), runq->local);
                    runq->local->val = copywords(starval, (struct Word *)nil);
                    runq->local->changed = true;
                    runq->redir = runq->startredir = nil;
                }
            }
        else if(i==SIGINT || i==SIGQUIT){
            /*
             * run the stack down until we uncover the
             * command reading loop. Xreturn will exit
             * if there is none (i.e. if this is not
             * an interactive rc.)
             */
            while(!runq->iflag)
                Xreturn();
        }
        else
            Exit(getstatus(), __LOC__);
    }
}
```

Uses Var 33c, Word 31c, Xreturn() 84d, copywords() 32c, getstatus() 67d, mypid 118d, newvar() 35b, ntrap 114b, runq 30c, start() 39a, trap 114c, and vlook() 34b.

```
⟨global interrupted 115c⟩≡ (213b)
    bool interrupted = false;
```

Uses interrupted 115c.

```
⟨function Eintr 115d⟩≡ (213b)
bool
Eintr(void)
{
    return interrupted;
}
```

Uses interrupted 115c.

$\langle \text{Xrdcmds()} \text{ calls } \text{Noerror()} \text{ before } \text{yyparse()} \text{ 116a} \rangle \equiv$  (41a)  
    `Noerror();`

$\langle \text{function Noerror 116b} \rangle \equiv$  (213b)  
    `void`  
    `Noerror(void)`  
    `{`  
        `interrupted = false;`  
    `}`

Uses `interrupted` 115c.

$\langle \text{Xrdcmds()} \text{ reset } \text{ntrap} \text{ 116c} \rangle \equiv$  (41a)  
    `ntrap = 0; /* avoid double-interrupts during blocked writes */`

Uses `ntrap` 114b.

# Chapter 12

## Initialization

In Chapter 4, we showed a simplified version of `main()` to introduce the interpreter loop. This chapter reveals the real initialization sequence: how `rc` fabricates its own bootstrap bytecodes, sources the `rcmain` script to set up the default environment, and transitions into the read-eval-print loop that makes an interactive shell usable.

### 12.1 Actual bootstrapping code

The bootstrap performs two operations encoded as hand-crafted bytecodes: first, it assigns the command-line arguments to `$*` (with `*(argv)`), then it executes `. rcmain $*` to source the initialization script. The bootstrap is essentially the bytecode equivalent of:

```
*(argv)
. /rc/lib/rcmain $*
```

The two-line script above is encoded as a sequence of bytecodes filling the `bootstrap` array. The first group (`Xmark`, `Xword` `"*"`, `Xassign`) assigns the command-line arguments to `$*`. The second group builds the argument list for the `.` builtin: it pushes `$*` (via `Xdol`), the path to `rcmain`, and the `.` command itself, then calls `Xsimple`.

`<main() initialize bootstrap 117>≡` (36b)

```
memset(bootstrap, 0, sizeof bootstrap);

i = 0;
bootstrap[i++].i=1;
// runq->argv is populated with the arguments to rc
// we just need to add '*(argv)'
bootstrap[i++].f = Xmark;
    bootstrap[i++].f = Xword;
    bootstrap[i++].s="*";
bootstrap[i++].f = Xassign; // will pop_list() x2

bootstrap[i++].f = Xmark;
    bootstrap[i++].f = Xmark;
    bootstrap[i++].f = Xword;
    bootstrap[i++].s="*";
bootstrap[i++].f = Xdol; // will pop_list()
bootstrap[i++].f = Xword;
bootstrap[i++].s = rcmain;
bootstrap[i++].f = Xword;
bootstrap[i++].s=".";
bootstrap[i++].f = Xsimple; // will pop_list()

bootstrap[i++].f = Xexit;
```

```
bootstrap[i].i = 0;
```

Uses Xassign() 96c, Xexit() 87, Xmark() 66b, Xsimple() 70, and Xword() 66a.

## 12.2 Initialization script and rc -m /path/to/rcmain

The default initialization script is /rc/lib/rcmain, which sets up \$path and, for login shells, sources the user's \$home/lib/profile. The -m flag allows overriding this path, which is useful during system bootstrap when the standard rcmain may not be available yet.

```
<global Rcmain 118a>≡ (213b)
char *Rcmain = "/rc/lib/rcmain";
```

```
<main() locals 118b>+≡ (36b) <?? 118e>
char *rcmain;
```

```
<main() initialisation 118c>+≡ (36b) <38b 118f>
rcmain = flag['m'] ? flag['m'][0] : Rcmain;
```

Uses flag.

## 12.3 Actual environment

Before entering the interpreter loop, rc also sets up a few built-in variables: \$pid (the shell's own process id, useful for creating unique temporary filenames) and \$rcname (the name rc was invoked as, i.e., argv[0]).

```
<global mypid 118d>≡ (199b)
int mypid;
```

```
<main() locals 118e>+≡ (36b) <118b
char num[12];
```

```
<main() initialisation 118f>+≡ (36b) <118c 118g>
mypid = getpid();
intoascii(num, mypid);
setvar("pid", newword(num, (word *)nil));
```

Uses intoascii() 169b, mypid 118d, newword() 31d, and setvar() 34a.

```
<main() initialisation 118g>+≡ (36b) <118f 128a>
setvar("rcname", newword(argv[0], (word *)nil));
```

Uses newword() 31d and setvar() 34a.

# Chapter 13

## Globbering

Globbering (also called wildcard expansion or pathname expansion) is one of the key features of a shell: when you write `ls *.c`, the shell expands `*.c` into the list of matching filenames *before* passing them to `ls`. This is a deliberate design choice: the expansion is done by the shell, not by the individual programs, which factorizes the functionality. In the original Unix shell, globbing was actually handled by a separate external program called `glob`. In MS-DOS, the opposite approach was taken: wildcards are passed unexpanded to each program, which must interpret them itself. This is generally worse (every program must reimplement the logic), but it does allow commands like `ren *.txt *.bak` to have a meaning—the program sees both patterns and can pair them up, something that is impossible when the shell expands both arguments independently.

The glob patterns `*`, `?`, and `[]` are different from regular expressions: `*` matches any string (not “any number of the preceding character”), and `?` matches any single character. This is more practical for filenames, where `*.c` is more natural than the regex `.*\.`.

Globbering touches several phases of the interpreter. The lexer marks glob characters with a special escape byte so they can be distinguished from literal characters. The bytecode generator emits `Xglob`<sup>121a</sup> instructions at the right points. And at runtime, the actual matching is done by `globlist()`<sup>120d</sup>, which walks directories and filters filenames against the pattern using `matchfn()`<sup>124a</sup>.

### 13.1 Lexing globbering characters

Glob characters (`*`, `?`, `[]`) are marked during lexing by inserting a GLOB byte (`\001`) before them. This way, the rest of the shell can distinguish a literal `*` (inside quotes, no GLOB prefix) from a wildcard `*` (with a GLOB prefix). The `deglob()`<sup>122a</sup> function removes these markers when they are no longer needed.

```
<constant GLOB 119a>≡ (196c)
/*
 * Glob character escape in strings:
 * In a string, GLOB must be followed by *?[ or GLOB.
 * GLOB* matches any string
 * GLOB? matches any single character
 * GLOB[...] matches anything in the brackets
 * GLOBGLOB matches GLOB
 */
#define GLOB '\001'
```

```
<yylex() when c is a word character, if glob character 119b>≡ (54c)
    if(c=='*' || c=='[' || c=='?' || c==GLOB)
        w = addtok(w, GLOB);
```

Uses GLOB 119a and addtok() 120a.

```

⟨function addtok 120a⟩≡ (207a)
char*
addtok(char *p, int val)
{
    if(p==nil)
        return nil;
    if(p >= &tok[NTOK]){
        *p = '\0';
        yyerror("token buffer too short");
        return nil;
    }
    *p++=val;
    return p;
}

```

Uses NTOK-10 49d, tok 49c, and yyerror() 156e.

## 13.2 Expanding globbing characters

The expansion happens at runtime, not at compile time. The Xglob<sup>121a</sup> bytecode (emitted after evaluating command arguments) scans the word list in `argv` and replaces any word containing GLOB markers with the list of matching filenames from the filesystem.

```

⟨global globv 120b⟩≡ (202c)
struct Word *globv;

```

Uses Word 31c.

```

⟨Xsimple() initializations, globlist() 120c⟩≡ (70)
globlist();

```

Uses globlist() 120d.

```

⟨function globlist 120d⟩≡ (202c)
void
globlist(void)
{
    word *a;
    globv = nil;

    globlist1(runq->argv->words);
    poplist();
    pushlist();

    if(globv){
        for(a = globv;a->next;a = a->next)
            ;
        a->next = runq->argv->words;
        runq->argv->words = globv;
    }
}

```

Uses globlist1() 120e, globv 120b, poplist() 66c, pushlist() 40a, and runq 30c.

```

⟨function globlist1 120e⟩≡ (202c)
void
globlist1(word *gl)
{
    if(gl){
        globlist1(gl->next);
        glob(gl->word);
    }
}

```

```

    }
}

```

Uses `glob()` 121c and `globlist1()` 120e.

```

⟨function Xglob 121a⟩≡ (203b)
void
Xglob(void)
{
    globlist();
}

```

Uses `globlist()` 120d.

## 13.3 glob()

`glob()` 121c takes a pattern string and pushes all matching filenames onto `globv`. If no files match, the original pattern (with GLOB markers removed) is kept as-is. `globdir()` 122d is the recursive workhorse: it scans the pattern for the first component containing a metacharacter, reads the corresponding directory, and recurses for each matching entry. The results are sorted alphabetically by `globsort()` 122c.

```

⟨global globname 121b⟩≡ (202c)
char *globname;

```

```

⟨function glob 121c⟩≡ (202c)
/*
 * Push all file names matched by p on the current thread's stack.
 * If there are no matches, the list consists of p.
 */
void
glob(void *ap)
{
    uchar *p = ap;
    word *svglobv = globv;
    int globlen = Globsize(ap);

    if(!globlen){
        deglob(p);
        globv = newword((char *)p, globv);
        return;
    }
    globname = emalloc(globlen);
    globname[0]='\0';
    globdir(p, (uchar *)globname);
    efree(globname);
    if(svglobv==globv){
        deglob(p);
        globv = newword((char *)p, globv);
    }
    else
        globsort(globv, svglobv);
}

```

Uses `deglob()` 122a, `efree()` 158d, `emalloc()` 158b, `globdir()` 122d, `globname` 121b, `globsort()` 122c, `globv` 120b, and `newword()` 31d.

*<function deglob 122a>*≡ (202c)

```
/*
 * delete all the GLOB marks from s, in place
 */
void
deglob(void *as)
{
    char *s = as;
    char *t = s;
    do{
        if(*t==GLOB)
            t++;
        *s++=*t;
    }while(*t++);
}
```

Uses GLOB 119a.

*<function globcmp 122b>*≡ (202c)

```
int
globcmp(const void *s, const void *t)
{
    return strcmp(*(char**)s, *(char**)t);
}
```

*<function globsort 122c>*≡ (202c)

```
void
globsort(word *left, word *right)
{
    char **list;
    word *a;
    int n = 0;
    for(a = left; a!=right; a = a->next) n++;
    list = (char **)emalloc(n*sizeof(char *));
    for(a = left, n = 0; a!=right; a = a->next, n++) list[n] = a->word;
    qsort((void *)list, n, sizeof(void *), globcmp);
    for(a = left, n = 0; a!=right; a = a->next, n++) a->word = list[n];
    efree((char *)list);
}
```

Uses efree() 158d, emalloc() 158b, and globcmp() 122b.

*<function globdir 122d>*≡ (202c)

```
/*
 * Push names prefixed by globname and suffixed by a match of p onto the astack.
 * namep points to the end of the prefix in globname.
 */
void
globdir(uchar *p, uchar *namep)
{
    uchar *t, *newp;
    int f;
    /* scan the pattern looking for a component with a metacharacter in it */
    if(*p=='\0'){
        globv = newword(globname, globv);
        return;
    }
    t = namep;
    newp = p;
    while(*newp){
        if(*newp==GLOB)
            break;
    }
}
```

```

        *t=*newp++;
        if(*t++=='/'){
            namep = t;
            p = newp;
        }
    }
    /* If we ran out of pattern, append the name if accessible */
    if(*newp=='\0'){
        *t='\0';
        if(access(globname, 0)==0)
            globv = newword(globname, globv);
        return;
    }
    /* read the directory and recur for any entry that matches */
    *namep='\0';
    if((f = opendir(globname[0]?globname:"."))<0) return;
    while(*newp!='/' && *newp!='\0') newp++;
    while(Readdir(f, namep, *newp=='/')){
        if(matchfn(namep, p)){
            for(t = namep;*t;t++){
                globdir(newp, t);
            }
        }
    }
    Closedir(f);
}

```

Uses GLOB 119a, `opendir()` 126d, `globdir()` 122d, `globname` 121b, `globv` 120b, `matchfn()` 124a, and `newword()` 31d.

*<function equtf 123a>*≡ (202c)

```

/*
 * Do p and q point at equal utf codes
 */
bool
equtf(uchar *p, uchar *q)
{
    Rune pr, qr;
    if(*p!=*q)
        return false;

    chartorune(&pr, (char*)p);
    chartorune(&qr, (char*)q);
    return pr == qr;
}

```

*<function nextutf 123b>*≡ (202c)

```

/*
 * Return a pointer to the next utf code in the string,
 * not jumping past nuls in broken utf codes!
 */
uchar*
nextutf(uchar *p)
{
    Rune dummy;
    return p + chartorune(&dummy, (char*)p);
}

```

*<function unicode 123c>*≡ (202c)

```

/*
 * Convert the utf code at *p to a unicode value
 */
int

```

```

unicode(uchar *p)
{
    Rune r;

    chartorune(&r, (char*)p);
    return r;
}

```

## 13.4 match()

`matchfn()`<sup>124a</sup> is the entry point called from `glob()`<sup>121c</sup>. It enforces one subtle rule: the dot-files `.` and `..` are only matched if the pattern explicitly starts with a dot. This prevents `*` from matching `.` and `..`, which would cause recursive globbing to loop or produce confusing results.

The actual matching is done by `match()`<sup>124b</sup>, which walks the pattern and the string in lockstep. Non-GLOB bytes must match literally. GLOB bytes introduce the special characters: `*` tries every possible suffix via recursion (backtracking), `?` matches exactly one UTF-8 character, and `[...]` matches a character class with optional ranges (`a-z`) and complement (`~`). Note that `rc` uses `~` instead of `^` for complement inside character classes, since `^` is already the concatenation operator.

The `stop` parameter is `/`: a single glob pattern is matched one path component at a time by `globdir()`<sup>122d</sup>, so `match()` stops at slashes rather than consuming them.

*⟨function matchfn 124a⟩*≡ (202c)

```

/*
 * Does the string s match the pattern p
 * . and .. are only matched by patterns starting with .
 * * matches any sequence of characters
 * ? matches any single character
 * [...] matches the enclosed list of characters
 */
bool
matchfn(void *as, void *ap)
{
    uchar *s = as, *p = ap;

    if(s[0]=='.' && (s[1]!='\0' || s[1]=='.' && s[2]!='\0') && p[0]!='.')
        return false;
    return match(s, p, '/');
}

```

*⟨function match 124b⟩*≡ (202c)

```

bool
match(void *as, void *ap, int stop)
{
    int compl, hit, lo, hi, t, c;
    uchar *s = as, *p = ap;

    for(; *p!=stop && *p!='\0'; s = nextutf(s), p = nextutf(p)){
        if(*p!=GLOB){
            if(!equtf(p, s)) return false;
        }
        else switch(*++p){
            case GLOB:
                if(*s!=GLOB)
                    return false;
                break;
            case '*':
                for(;;){

```

```

        if(match(s, nextutf(p), stop)) return 1;
        if(!*s)
            break;
        s = nextutf(s);
    }
    return false;
case '?':
    if(*s=='\0')
        return false;
    break;
case '[':
    if(*s=='\0')
        return false;
    c = unicode(s);
    p++;
    compl=*p=='~';
    if(compl)
        p++;
    hit = 0;
    while(*p!=']'){
        if(*p=='\0')
            return false; /* syntax error */
        lo = unicode(p);
        p = nextutf(p);
        if(*p!='-')
            hi = lo;
        else{
            p++;
            if(*p=='\0')
                return false; /* syntax error */
            hi = unicode(p);
            p = nextutf(p);
            if(hi<lo){ t = lo; lo = hi; hi = t; }
        }
        if(lo<=c && c<=hi)
            hit = 1;
    }
    if(compl)
        hit=!hit;
    if(!hit)
        return false;
    break;
}
}
return *s=='\0';
}

```

⟨constant NDIR 125a⟩≡ (213b)  
 #define NDIR 256 /\* should be a better way \*/

⟨function Globsize 125b⟩≡ (213b)  
 int  
 Globsize(char \*p)  
 {  
 int isglob = 0, globlen = NDIR+1;  
 for(;\*p;p++){  
 if(\*p==GLOB){  
 p++;  
 if(\*p!=GLOB)  
 isglob++;  
 }  
 }  
 }

```

        globlen+=*p=='?'?NDIR:1;
    }
    else
        globlen++;
}
return isglob?globlen:0;
}

```

Uses GLOB 119a and NDIR-3 125a.

```

⟨constant NFD 126a⟩≡ (213b)
#define NFD 50

```

```

⟨struct DirEntryWrapper 126b⟩≡ (213b)
struct DirEntryWrapper {
    Dir *dbuf;
    int i;
    int n;
};

```

```

⟨global dir 126c⟩≡ (213b)
struct DirEntryWrapper dir[NFD];

```

Uses DirEntryWrapper 126b and NFD-4 126a.

```

⟨function Opendir 126d⟩≡ (213b)
int
Opendir(char *name)
{
    Dir *db;
    int f;
    f = open(name, 0);
    if(f==-1)
        return f;
    db = dirfstat(f);
    if(db!=nil && (db->mode&DMDIR)){
        if(f<NFD){
            dir[f].i = 0;
            dir[f].n = 0;
        }
        free(db);
        return f;
    }
    free(db);
    close(f);
    return -1;
}

```

Uses NFD-4 126a and dir 126c.

```

⟨function trimdirs 126e⟩≡ (213b)
static int
trimdirs(Dir *d, int nd)
{
    int r, w;

    for(r=w=0; r<nd; r++)
        if(d[r].mode&DMDIR)
            d[w++] = d[r];
    return w;
}

```

```

⟨function Readdir 127a⟩≡ (213b)
/*
 * onlydirs is advisory -- it means you only
 * need to return the directories.  it's okay to
 * return files too (e.g., on unix where you can't
 * tell during the readdir), but that just makes
 * the globber work harder.
 */
int
Readdir(int f, void *p, int onlydirs)
{
    int n;

    if(f<0 || f>=NFD)
        return 0;
Again:
    if(dir[f].i==dir[f].n){ /* read */
        free(dir[f].dbuf);
        dir[f].dbuf = 0;
        n = dirread(f, &dir[f].dbuf);
        if(n>0){
            if(onlydirs){
                n = trimdirs(dir[f].dbuf, n);
                if(n == 0)
                    goto Again;
            }
            dir[f].n = n;
        }else
            dir[f].n = 0;
        dir[f].i = 0;
    }
    if(dir[f].i == dir[f].n)
        return 0;
    strcpy(p, dir[f].dbuf[dir[f].i].name);
    dir[f].i++;
    return 1;
}

```

Uses NFD-4 126a, dir 126c, and trimdirs() 126e.

```

⟨function Closedir 127b⟩≡ (213b)
void
Closedir(int f)
{
    if(f>=0 && f<NFD){
        free(dir[f].dbuf);
        dir[f].i = 0;
        dir[f].n = 0;
        dir[f].dbuf = 0;
    }
    close(f);
}

```

Uses NFD-4 126a and dir 126c.

# Chapter 14

## Advanced Topics

In this chapter I present features that are not essential for understanding the core architecture of `rc`, but that are important in practice. Many of these features—here documents, command substitution, process substitution, general redirections—are what make a shell truly useful for scripting.

Note that in Plan 9, many shell-like features that other systems implement inside the shell are instead provided by the window manager `rio`: command-line editing, history, filename completion, and job control. This keeps `rc` simple.

### 14.1 Reading commands from a string: `rc -c`

The `-c` flag lets you pass a command string directly on the command line, as in `rc -c 'echo hello'`. This is how other programs (like `mk`, see the Build book) invoke the shell to execute a recipe: they build a command string and pass it via `-c` rather than writing it to a temporary file.

```
<main() initialisation 128a>+≡ (36b) <118g
    setvar("cflag", flag['c']? newword(flag['c'][0], (word *)nil) : (word *)nil);
```

Uses `flag`, `newword()` 31d, and `setvar()` 34a.

### 14.2 Failing fast: `rc -e`

The `-e` flag makes `rc` exit immediately when a simple command fails (returns non-null status). This is extremely important for build systems: `mk` (see the Build book) invokes `rc -e` for recipes, so that a failure in any command—especially inside a loop iterating over subdirectories—stops the entire build rather than silently continuing.

Note that `-e` is a command-line flag, not a `set` option, so you cannot enable it from within a script. The check is only inserted after simple commands, not after control flow constructs like `if` or `||`, since a false condition in `if(test ...)` should not abort the script.

```
<outcode() emit Xeflag after Xsimple 128b>≡ (69b)
    if(eflag)
        emitf(Xeflag);
```

```
<global eflagok 128c>≡ (199b)
    bool eflagok; /* kludge flag so that -e doesn't exit in startup */
```

```
<execdot() if not first execution 128d>≡ (105d)
    else
        eflagok = true;
```

Uses `eflagok` 128c.

```

⟨function Xeflag 129a⟩≡ (203b)
void
Xeflag(void)
{
    if(eflagok && !truestatus())
        Xexit();
}

```

Uses `Xexit()` 87, `eflagok` 128c, and `truestatus()` 68a.

The `eflagok` guard prevents `-e` from triggering during startup (while sourcing `rcmain`). It is set to true only after the first `dot` command finishes, so initialization errors do not cause an immediate exit.

## 14.3 Unicode

Plan 9 was the birthplace of UTF-8, and `rc` handles Unicode transparently. The key insight is that UTF-8 is self-synchronizing: the lexer can consume multi-byte runes by reading the first byte (which encodes the length) and then pulling the appropriate number of continuation bytes. `addutf()` 129b does exactly this during lexing: after `addtok` stores the first byte, it reads continuation bytes by checking the leading-bit pattern.

```

⟨function addutf 129b⟩≡ (207a)
char*
addutf(char *p, int c)
{
    uchar b, m;
    int i;

    p = addtok(p, c); /* 1-byte UTF runes are special */
    if(c < Runeself)
        return p;

    m = 0xc0;
    b = 0x80;
    for(i=1; i < UTFmax; i++){
        if((c&m) == b)
            break;
        p = addtok(p, advance());
        b = m;
        m = (m >> 1)|0x80;
    }
    return p;
}

```

Uses `addtok()` 120a and `advance()` 47a.

## 14.4 Advanced constructs

### 14.4.1 Subshell: @ <cmd>

The `@` operator runs a command in a child process, so that side effects like `cd` or variable assignments do not affect the parent. Braces `{...}` group commands but execute them in the current process—so `{cd /tmp; ls}` would change the parent's directory. With `@{cd /tmp; ls}`, the directory change is confined to the subshell.

This is particularly useful in `mkfile` recipes that iterate over subdirectories:

```

for(i in $DIRS) @{
    cd $i
    mk $MKFLAGS $stem
}

```

```
}
```

The implementation is straightforward: fork, run the body in the child (which will `Xexit`), and `Waitfor` in the parent. The forward-jump past the body (via `stuffdot`) lets the parent skip the child's code.

```
<token declarations 130a>+≡ (57a) <63a 137a>
%token SUBSHELL /** @ */
```

```
<cmd rule other cases 130b>+≡ (58d) <63e
| SUBSHELL cmd {$$=mung1($1, $2);}
```

```
<outcode() cases 130c>+≡ (65f) <100a 137d>
case SUBSHELL:
    emitf(Xsubshell);
    p = emitf(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    if(eflag)
        emitf(Xeflag);
    break;
```

```
<function Xsubshell 130d>≡ (206a)
void
Xsubshell(void)
{
    int pid;
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(runq->code, runq->pc+1, runq->local);
        runq->ret = nil;
        break;
    default: // parent
        addwaitpid(pid);
        Waitfor(pid, true);
        runq->pc = runq->code[runq->pc].i;
        break;
    }
}
```

Uses `Xerror()` 71b, `addwaitpid()` 68d, `clearwaitpids()` 68g, `runq` 30c, and `start()` 39a.

## 14.4.2 Here documents: << <HERE>

Here documents let you embed multi-line *data* directly in a script, avoiding external file dependencies. The syntax <<TAG reads input until a line matching TAG appears. For example, a phone-lookup script can carry its own database inline:

```
grep $1 <<EOF
Alice 555-1234
Bob 555-5678
EOF
```

The program and its data live in the same file, so the script is self-contained and easy to move around. A variable would not work as well here: shell variables are word lists, so multi-line structure (newlines, whitespace, special characters) would be lost. A here document feeds the data as a raw byte stream to stdin, which is exactly what programs like `grep` or `sed` expect.

The implementation is clever: during parsing, `heredoc()` creates a temporary file (in `/tmp`) and records the tag and filename in a linked list. After the entire command line is parsed and compiled, `readhere()` goes back and reads the here-document bodies from the input, writing them to those temporary files. This two-phase approach is necessary because a command line can have multiple here documents (e.g., `cmd <<A <<B`), and the bodies appear after the full command line.

If the here-tag was not quoted, variable substitution is performed via `psubst()`<sup>134a</sup>; otherwise the text is copied literally. The temporary files are cleaned up by emitting `Xdelhere` bytecodes that `remove()` them after use.

```
<constant HERE 131a>≡ (196c)
#define HERE 4
```

```
<yylex() in switch when redirection character, if here document 131b>≡ (53a)
if(nextis(c)){
    t->rtype = HERE;
    *w++=c;
}
```

Uses HERE 131a and nextis() 47b.

```
<struct Here 131c>≡ (196c)
struct Here {
    tree *tag;
    char *name;
    struct Here *next;
};
```

Uses Here 131c.

```
<global here 131d>≡ (208c)
struct Here *here;
```

Uses Here 131c.

```
<global ehere 131e>≡ (208c)
struct Here **ehere;
```

Uses Here 131c.

```
<constant NLINE 131f>≡ (208c)
/*
 * bug: lines longer than NLINE get split -- this can cause spurious
 * missubstitution, or a misrecognized EOF marker.
 */
#define NLINE 4096
```

```
<compile() after outcode and error management, read heredoc 131g>≡ (65a)
readhere();
```

Uses readhere() 132a.

*<function readhere 132a>*≡ (208c)

```
void
readhere(void)
{
    int c, subst;
    char *s, *tag;
    char line[NLINE+1];
    io *f;
    struct Here *h, *nexth;

    for(h = here; h; h = nexth){
        subst = !h->tag->quoted;
        tag = h->tag->str;
        c = Creat(h->name);
        if(c < 0)
            yyerror("can't create here document");
        f = openfd(c);
        s = line;
        pprompt();
        while((c = rchr(runq->cmdfd)) != EOF){
            if(c == '\n' || s == &line[NLINE]){
                *s = '\0';
                if(tag && strcmp(line, tag) == 0)
                    break;
                if(subst)
                    psubst(f, (uchar *)line);
                else
                    pstr(f, line);
                s = line;
                if(c == '\n'){
                    pprompt();
                    pchr(f, c);
                }else
                    *s++ = c;
            }else
                *s++ = c;
        }
        flush(f);
        closeio(f);
        cleanhere(h->name);
        nexth = h->next;
        efree((char *)h);
    }
    here = nil;
    doprompt = true;
}
```

Uses Creat() 169g, EOF 44a, Here 131c, NLINE-9 131f, cleanhere() 132b, closeio() 165a, doprompt 45c, efree() 158d, flush() 163g, here 131d, openfd() 163e, pchr() 164a, pprompt() 45h, pstr() 167a, rchr() 164c, runq 30c, and yyerror() 156e.

*<function cleanhere 132b>*≡ (209)

```
void
cleanhere(char *f)
{
    emitf(Xdelhere);
    emits(strdup(f));
}
```

Uses Xdelhere() 132c, emitf-65 64d, and emits-67 64e.

*<function Xdelhere 132c>*≡ (203b)

```

void
Xdelhere(void)
{
    remove(runq->code[runq->pc++].s);
}

```

Uses runq 30c.

*<function Unlink 133a>*≡ (210b)

*<global ser 133b>*≡ (208c)  
 int ser = 0;

Uses ser 133b.

*<global tmp (rc/here.c) 133c>*≡ (208c)  
 char tmp[] = "/tmp/here0000.0000";

Uses tmp 133c.

*<global hex 133d>*≡ (208c)  
 char hex[] = "0123456789abcdef";

Uses hex 133d.

*<function hexnum 133e>*≡ (208c)  
 void  
 hexnum(char \*p, int n)  
 {  
 \*p++ = hex[(n>>12)&0xF];  
 \*p++ = hex[(n>>8)&0xF];  
 \*p++ = hex[(n>>4)&0xF];  
 \*p = hex[n&0xF];  
 }

Uses hex 133d.

*<function heredoc 133f>*≡ (208c)  
 //@Scheck: used by syn.y  
 tree\* heredoc(tree \*tag)  
 {  
 struct Here \*h = new(struct Here);  
  
 if(tag->type != WORD)  
 yyerror("Bad here tag");  
 h->next = 0;  
 if(here)  
 \*ehere = h;  
 else  
 here = h;  
 ehere = &h->next;  
 h->tag = tag;  
 hexnum(&tmp[9], getpid());  
 hexnum(&tmp[14], ser++);  
 h->name = strdup(tmp);  
 return token(tmp, WORD);  
 }

Uses Here 131c, WORD, ehere 131e, here 131d, hexnum() 133e, ser 133b, tmp 133c, token() 54b, and yyerror() 156e.

*<function psubst 134a>*≡ (208c)

```

void
psubst(io *f, uchar *s)
{
    int savec, n;
    uchar *t, *u;
    Rune r;
    word *star;

    while(*s){
        if(*s != '$'){ /* copy plain text rune */
            if(*s < Runeself)
                pchr(f, *s++);
            else{
                n = chartorune(&r, (char *)s);
                while(n-- > 0)
                    pchr(f, *s++);
            }
        }else{ /* $something -- perform substitution */
            t = ++s;
            if(*t == '$')
                pchr(f, *t++);
            else{
                while(*t && idchr(*t))
                    t++;
                savec = *t;
                *t = '\0';
                n = 0;
                for(u = s; *u && '0' <= *u && *u <= '9'; u++)
                    n = n*10 + *u - '0';
                if(n && *u == '\0'){
                    star = vlook("*")->val;
                    if(star && 1 <= n && n <= count(star)){
                        while(--n)
                            star = star->next;
                        pstr(f, star->word);
                    }
                }else
                    pstrs(f, vlook((char *)s->val));
                *t = savec;
                if(savec == '^')
                    t++;
            }
            s = t;
        }
    }
}

```

*<function pstrs 134b>*≡ (208c)

```

void
pstrs(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
            pstr(f, a->word);
            pchr(f, ' ');
            a = a->next;
        }
        pstr(f, a->word);
    }
}

```

```
}
```

Uses `pchr()` 164a and `pstr()` 167a.

### 14.4.3 Read-write redirections: `<>` `<file>`

The `<>` operator opens a file for both reading and writing (`ORDWR`), useful for in-place modification of files.

*<constant RDWR 135a>*≡ (196c)

```
#define RDWR 7
```

*<yyllex() in switch when redirection character, if read/write redirect 135b>*≡ (53a)

```
else if (nextis('>>')){
    t->rtype = RDWR;
    *w++=c;
}
```

Uses `RDWR` 135a and `nextis()` 47b.

*<outcode() when REDIR case, switch redirection type cases 135c>*+≡ (80b) <82a

```
case RDWR:
    emitf(Xrdwr);
    break;
```

*<function Xrdwr 135d>*≡ (203b)

```
void
Xrdwr(void)
{
    char *file;
    int f;

    switch(count(runq->argv->words)){
    default:
        Xerror1("<> requires singleton\n");
        return;
    case 0:
        Xerror1("<> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, ORDWR))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses `ROPEN` 80a, `Xerror()` 71b, `Xerror1()` 71c, `count()` 32a, `err` 156b, `pfmt()` 166b, `poplist()` 66c, `pushredir()` 79c, and `runq` 30c.

### 14.4.4 General redirections: `>[2]` `<file>`

The basic redirections `>` and `<` always use file descriptors 1 and 0 respectively. The bracket syntax `>[2]` or `<[3]` lets you redirect any file descriptor. For example, `echo error >[2] /tmp/log` redirects `stderr`.

The lexer handles the bracket syntax inline: after seeing > or <, if a [ follows, it reads the file descriptor number. If it then sees =, the redirection becomes a DUP (file descriptor duplication or closing) rather than a file redirection.

*<yylex() in switch when redirection character, if bracket after 136a>≡ (51g)*

```

if(nextis('[')){
    *w++='[';
    c = advance();
    *w++=c;
    if(c<'0' || '9'<c){
RedirErr:
        *w = 0;
        yyerror(t->type==PIPE?"pipe syntax"
                : "redirection syntax");
        return EOF;
    }
    t->fd0 = 0;
    do{
        t->fd0 = t->fd0*10 + c-'0';
        *w++=c;
        c = advance();
    }while('0'<=c && c<='9');

    if(c=='='){
        *w++='=';
        if(t->type==REDIR)
            // change the token type
            t->type = DUP;
        c = advance();
        if('0'<=c && c<='9'){
            t->rtype = DUPFD;
            t->fd1 = t->fd0;
            t->fd0 = 0;
            do{
                t->fd0 = t->fd0*10+c-'0';
                *w++=c;
                c = advance();
            }while('0'<=c && c<='9');
        }
        else{
            if(t->type==PIPE)
                goto RedirErr;
            t->rtype = CLOSE;
        }
    }
    if(c!=']')
        || t->type==DUP && (t->rtype==HERE || t->rtype==APPEND))
        goto RedirErr;
    *w++=']';
}

```

Uses APPEND 52d, CLOSE 136b, DUP, DUPFD 137c, EOF 44a, HERE 131a, PIPE, REDIR, advance() 47a, nextis() 47b, and yyerror() 156e.

*<constant CLOSE 136b>≡ (196c)*

```

#define CLOSE 6

```

### 14.4.5 Advanced dup: >[<fd0>=<fd1> ], <>[<fd0>=<fd1>] , <[<fd0>=<fd1>]

The DUP construct performs `dup2` between file descriptors. `>[1=2]` makes fd 1 point to the same destination as fd 2 (so stdout goes to stderr). `>[1=]` (with nothing after the equals) closes fd 1. This is cleaner than the Bourne shell's `2>&1` syntax.

Like other redirections, the implementation uses the two-phase approach: `Xdup` pushes a `RDUP` record onto the `redir` stack (undone by `Xpopredir` afterward), and the actual `dup2` happens in `doredir()` when a child process is forked.

```
<token declarations 137a>+≡ (57a) <130a 138e>
%token DUP
```

```
<redir rule other cases 137b>≡ (61e)
|    DUP
```

```
<constant DUPFD 137c>≡ (196c)
#define DUPFD 5
```

```
<outcode() cases 137d>+≡ (65f) <130c 138f>
case DUP:
    if(t->rtype==DUPFD){
        emitf(Xdup);
        emitf(t->fd0);
        emitf(t->fd1);
    }
    else{
        emitf(Xclose);
        emitf(t->fd0);
    }
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;
```

```
<function Xdup 137e>≡ (203b)
void
Xdup(void)
{
    pushredir(RDUP, runq->code[runq->pc].i, runq->code[runq->pc+1].i);
    runq->pc+=2;
}
```

Uses `RDUP` 137g, `pushredir()` 79c, and `runq` 30c.

```
<codefree() in loop over code cp, switch bytecode cases 137f>+≡ (29e) <94a
    else if(p->f==Xdup || p->f==Xpipefd)
        p+=2;
```

Uses `Xdup()` 137e and `Xpipefd()` 138h.

```
<constant RDUP 137g>≡ (199a)
#define RDUP 2 /* dup2(from, to); */
```

```
<constant RCLOSE 137h>≡ (199a)
#define RCLOSE 3 /* close(from); */
```

```
<function Xclose 137i>≡ (203b)
void
Xclose(void)
{
    pushredir(RCLOSE, runq->code[runq->pc].i, 0);
    runq->pc++;
}
```

Uses `RCLOSE` 137h, `pushredir()` 79c, and `runq` 30c.

```

<doredir() switch redir type cases 138a>+≡ (79d) <79e 138b>
    case RDUP:
        dup(rp->from, rp->to);
        break;
Uses RDUP 137g.

```

```

<doredir() switch redir type cases 138b>+≡ (79d) <138a
    case RCLOSE:
        close(rp->from);
        break;
Uses RCLOSE 137h.

```

## 14.4.6 Advanced pipes: |[<fd>] , |[<fd0>=<fd1>]

Just as general redirections extend > and < with bracket syntax, pipes can also specify which file descriptors to connect. `cmd1 |[2] cmd2` pipes fd 2 of `cmd1` (stderr) to `cmd2`'s stdin.

## 14.4.7 Command output as a file: '<{<cmd>}'

```

<comword rule other cases 138c>+≡ (58g) <63h 138d>
    | REDIR brace    {$$=mung1($1, $2); $$->type=PIPEFD;}

```

Process substitution (called “command output as a file” in Plan 9) lets you use a command's output as if it were a file. `cmp <{old} <{new}` runs the two commands and passes their outputs as filenames to `cmp`.

This appears in the `comword` rule because it really resolves to a filename (`/dev/fd/N`) connected to the command's output via a pipe. The implementation forks a child that runs the command with its stdout (or stdin for >) connected to one end of a pipe, while the parent pushes the filename `/fd/N` (the other end) as a word onto the argv stack.

```

<comword rule other cases 138d>+≡ (58g) <138c 139a>
    | REDIR brace    {$$=mung1($1, $2); $$->type=PIPEFD;}

```

```

<token declarations 138e>+≡ (57a) <137a
    %token PIPEFD

```

```

<outcode() cases 138f>+≡ (65f) <137d 139b>
    case PIPEFD:
        emitf(Xpipefd);
        emitf(t->rtype);
        p = emitf(0);
        outcode(c0, eflag);
        emitf(Xexit);
        stuffdot(p);
        break;

```

```

<global Fdprefix 138g>≡ (213b)
    char *Fdprefix = "/fd/";

```

```

<function Xpipefd 138h>≡ (206a)
    void
    Xpipefd(void)
    {
        struct Thread *p = runq;
        int pc = p->pc, pid;
        char name[40];
        int pfd[2];
        int sidefd, mainfd;
        if(pipe(pfd)<0){

```

```

        Xerror("can't get pipe");
        return;
    }
    if(p->code[pc].i==READ){
        sidefd = pfd[PWR];
        mainfd = pfd[PRD];
    }
    else{
        sidefd = pfd[PRD];
        mainfd = pfd[PWR];
    }
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(p->code, pc+2, runq->local);
        close(mainfd);
        pushredir(ROPEN, sidefd, p->code[pc].i==READ?1:0);
        runq->ret = 0;
        break;
    default: // parent
        addwaitpid(pid);
        close(sidefd);
        pushredir(ROPEN, mainfd, mainfd); /* isn't this a noop? */
        strcpy(name, Fdprefix);
        intoascii(name+strlen(name), mainfd);
        pushword(name);
        p->pc = p->code[pc+1].i;
        break;
    }
}

```

Uses PRD 84e, PWR 85a, READ 52c, ROPE 80a, Thread 30b, Xerror() 71b, addwaitpid() 68d, clearwaitpids() 68g, intoascii() 169b, pushredir() 79c, pushword() 40b, runq 30c, and start() 39a.

#### 14.4.8 Command substitution: ‘{<cmd>}’

Command substitution captures a command’s stdout as a list of words. The Bourne shell uses backticks ‘cmd’, which cannot nest; rc uses ‘{cmd}’ with braces, which nests naturally.

The output is split into words using \$ifs (inter-field separators, defaulting to space, tab, newline). The implementation forks a child connected via a pipe; the parent reads the output, splits on \$ifs characters, and pushes the resulting word list onto the argv stack.

<comword rule other cases 139a>+≡ (58g) <138d 141a>

```

|   ‘‘ brace      {$$=tree1(‘‘, $2);}

```

<outcode() cases 139b>+≡ (65f) <138f 141b>

```

case ‘‘:
    emitf(Xbackq);
    p = emit(0);
    outcode(c0, false);
    emitf(Xexit);
    stuffdot(p);
    break;

```

*<function Xbackq 140>*≡

(206a)

```
/*
 * Who should wait for the exit from the fork?
 */
void
Xbackq(void)
{
    int n, pid;
    int pfd[2];
    char *stop;
    char utf[UTFmax+1];
    struct Io *f;
    var *ifs = vlook("ifs");
    word *v, *nextv;
    Rune r;
    //String *word;

    stop = ifs->val? ifs->val->word: "";
    if(pipe(pfd)<0){
        Xerror("can't make pipe");
        return;
    }
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        close(pfd[PRD]);
        close(pfd[PWR]);
        return;
    case 0: // child
        clearwaitpids();
        close(pfd[PRD]);
        start(runq->code, runq->pc+1, runq->local);
        pushredir(ROPEN, pfd[PWR], 1);
        return;
    default: // parent
        Exit("TODO", __LOC__);
        //pad: commented because use the string.h s_xxx library
        // that I didn't want to port to goken
        //addwaitpid(pid);
        //close(pfd[PWR]);
        //f = openfd(pfd[PRD]);
        //word = s_new();
        //v = nil;
        ///* rutf requires at least UTFmax+1 bytes in utf */
        //while((n = rutf(f, utf, &r)) != EOF){
        //    utf[n] = '\0';
        //    if(utfutf(stop, utf) == nil)
        //        s_nappend(word, utf, n);
        //    else
        //        /*
        //         * utf/r is an ifs rune (e.g., \t, \n), thus
        //         * ends the current word, if any.
        //         */
        //        if(s_len(word) > 0){
        //            v = newword(s_to_c(word), v);
        //            s_reset(word);
        //        }
        //    }
        //if(s_len(word) > 0)
        //    v = newword(s_to_c(word), v);
```

```

    //s_free(word);
    //closeio(f);
    //Waitfor(pid, false);
    /** v points to reversed arglist -- reverse it onto argv */
    //while(v){
    //    nextv = v->next;
    //    v->next = runq->argv->words;
    //    runq->argv->words = v;
    //    v = nextv;
    //}
    //runq->pc = runq->code[runq->pc].i;
    //return;
}
}

```

Uses Io 163d, PRD 84e, PWR 85a, ROPEEN 80a, Xerror() 71b, clearwaitpids() 68g, pushredir() 79c, runq 30c, start() 39a, and vlook() 34b.

#### 14.4.9 Stringification of variables: \$"<foo>

The \$" operator converts a list variable into a single string with spaces between elements. If \$x is (a b c), then \$"x is a b c as a single word. This is the inverse of \$ifs-splitting in command substitution: command substitution splits strings into lists, while \$" joins lists back into strings.

```

<comword rule other cases 141a>+≡ (58g) <139a
|   '"" word      {$$=tree1('""', $2);}

```

```

<outcode() cases 141b>+≡ (65f) <139b
case '""':
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xqdol);
    break;

```

```

<function Xqdol 141c>≡ (203b)
void
Xqdol(void)
{
    word *a, *p;
    char *s;
    int n;
    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);
    a = vlook(s)->val;
    poplist();
    n = count(a);
    if(n==0){
        pushword("");
        return;
    }
    for(p = a;p;p = p->next) n+=strlen(p->word);
    s = emalloc(n);
    if(a){
        strcpy(s, a->word);
        for(p = a->next;p;p = p->next){
            strcat(s, " ");

```

```

        strcat(s, p->word);
    }
}
else
    s[0]='\0';
pushword(s);
efree(s);
}

```

Uses `Xerror1()` 71c, `count()` 32a, `deglob()` 122a, `efree()` 158d, `emalloc()` 158b, `poplist()` 66c, `pushword()` 40b, `runq` 30c, and `vlook()` 34b.

## 14.5 Advanced builtins

These builtins could in principle be external programs (like `echo` and `test` are), but they need access to shell internals: `exec` replaces the shell process, `whatism` inspects variables and functions, `rforke` manipulates namespaces, `flag` queries command-line flags, and `shift` modifies `$*`.

### 14.5.1 \$ exec

The `exec` builtin replaces the current shell process with the given command (no fork). It is called from `Xsimple()`<sup>70</sup> when the first word of the command is `exec`.

### 14.5.2 \$ whatism

`execwhatism()`<sup>142</sup> is the shell's introspection tool. Given a name, it checks (in order): is it a variable? a function? a builtin? an executable on `$path`? It prints the definition in a form that can be fed back to `rc`—variables as `x=value`, functions as `fn name {body}`, builtins as `builtin name`.

The subtle `mapfd()` helper is needed because `whatism` writes to fd 1, but fd 1 might have been redirected; `mapfd` walks the `redir` stack to find where fd 1 actually points.

```

⟨function execwhatism 142⟩≡ (210d)
void
execwhatism(void){ /* mildly wrong -- should fork before writing */
    word *a, *b, *path;
    var *v;
    struct Builtin *bp;
    char *file;
    struct Io out[1];
    int found, sep;
    a = runq->argv->words->next;
    if(a==0){
        Xerror1("Usage: whatism name ...");
        return;
    }
    setstatus("");
    memset(out, 0, sizeof out);
    out->fd = mapfd(1);
    out->bufp = out->buf;
    out->ebuf = &out->buf[NBUF];
    out->strp = nil;
    for(;a;a = a->next){
        v = vlook(a->word);
        if(v->val){
            pfmt(out, "%s=", a->word);
            if(v->val->next==0)

```

```

        pfmt(out, "%q\n", v->val->word);
    else{
        sep='(';
        for(b = v->val;b && b->word;b = b->next){
            pfmt(out, "%c%q", sep, b->word);
            sep=' ';
        }
        pfmt(out, ")\n");
    }
    found = 1;
}
else
    found = 0;
v = gvlook(a->word);
if(v->fn)
    pfmt(out, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
else{
    for(bp = builtins;bp->name;bp++){
        if(strcmp(a->word, bp->name)==0){
            pfmt(out, "builtin %s\n", a->word);
            break;
        }
    }
    if(!bp->name){
        for(path = searchpath(a->word); path;
            path = path->next){
            if(path->word[0] != '\0')
                file = appfile(path->word, a->word);
            else
                file = strdup(a->word);
            if(Executable(file)){
                pfmt(out, "%s\n", file);
                free(file);
                break;
            }
            free(file);
        }
        if(!path && !found){
            pfmt(err, "%s: not found\n", a->word);
            setstatus("not found");
        }
    }
}
}
}
poplist();
flush(err);
}

```

*<function mapfd 143>*≡

(210d)

```

int
mapfd(int fd)
{
    redir *rp;
    for(rp = runq->redir;rp;rp = rp->next){
        switch(rp->type){
            case RCLOSE:
                if(rp->from==fd)
                    fd=-1;
                break;
            case RDUP:
            case ROPEN:

```

```

        if(rp->to==fd)
            fd = rp->from;
        break;
    }
}
return fd;
}

```

Uses `RCLOSE` 137h, `RDUP` 137g, `ROPEN` 80a, and `runq` 30c.

```

⟨function Executable 144a⟩≡ (210d)
bool
Executable(char *file)
{
    Dir *statbuf;
    bool ret;

    statbuf = dirstat(file);
    if(statbuf == nil)
        return false;
    ret = ((statbuf->mode&0111)!=0 && (statbuf->mode&DMDIR)==0);
    free(statbuf);
    return ret;
}

```

### 14.5.3 \$ rfork

The `rfork` builtin exposes Plan 9's `rfork(2)` system call, which selectively shares or unshares resources between parent and child. The flags control which namespaces to copy or clear: `n/N` for the mount namespace, `e/E` for environment, `s` for note group, `f/F` for file descriptors. This is the Plan 9 equivalent of Linux containers and namespaces—but from 1992.

The environment flag is particularly interesting. As mentioned in Chapter 10, all `rc` variables are automatically exported to `/env/`—there is no `export` command like in `bash`. But this raises the question: how do you *prevent* a variable from being visible to children? The answer is `rfork`: `rfork e` creates a *copy* of the current `/env/`, so subsequent changes in either the parent or the child are invisible to the other; `rfork E` creates an entirely *empty* `/env/`, starting with a clean slate. Note that the lowercase flags (`n`, `e`, `f`) copy the resource, while the uppercase flags (`N`, `E`, `F`) clear it entirely. When called with no arguments, `rfork` defaults to `RFENVG|RFNAMEG|RFNOTEg`, creating new environment, mount namespace, and note groups all at once.

```

⟨function execnewpgrp 144b⟩≡ (210d)
void
execnewpgrp(void)
{
    int arg;
    char *s;
    switch(count(runq->argv->words)){
    case 1:
        arg = RFENVG|RFNAMEG|RFNOTEg;
        break;
    case 2:
        arg = 0;
        for(s = runq->argv->words->next->word;*s;s++) switch(*s){
        default:
            goto Usage;
        case 'n':
            arg|=RFNAMEG; break;
        case 'N':
            arg|=RFCNAMEG;

```

```

        break;
    //pad: commented for goken
    //case 'm':
    //    arg|=RFNOMNT; break;
    case 'e':
        arg|=RFENVG; break;
    case 'E':
        arg|=RFCENVG; break;
    case 's':
        arg|=RFNOTEg; break;
    case 'f':
        arg|=RFFDG; break;
    case 'F':
        arg|=RFCFDG; break;
    }
    break;
default:
Usage:
    pfmt(err, "Usage: %s [fnesFNEm]\n", runq->argv->words->word);
    setstatus("rfork usage");
    poplist();
    return;
}
if(rfork(arg)==-1){
    pfmt(err, "rc: %s failed\n", runq->argv->words->word);
    setstatus("rfork failed");
}
else
    setstatus("");
poplist();
}

```

Uses `count()` 32a, `err` 156b, `pfmt()` 166b, `poplist()` 66c, `runq` 30c, and `setstatus()` 67c.

## 14.5.4 \$ flag

The `flag` builtin queries and sets `rc`'s own command-line flags at runtime. `flag e +` enables `-e` (exit on error), `flag e -` disables it, and `flag e` alone tests whether it is set (via the exit status).

```

⟨function execflag 145⟩≡ (210d)
void
execflag(void)
{
    char *letter, *val;

    switch(count(runq->argv->words)){
    case 2:
        setstatus(flag[(uchar)runq->argv->words->next->word[0]]?"":"flag not set");
        break;
    case 3:
        letter = runq->argv->words->next->word;
        val = runq->argv->words->next->next->word;
        if(strlen(letter)==1){
            if(strcmp(val, "+")==0){
                flag[(uchar)letter[0]] = flagset;
                break;
            }
            if(strcmp(val, "-")==0){
                flag[(uchar)letter[0]] = 0;
                break;
            }
        }
    }
}

```

```

    }
}
default:
    Xerror1("Usage: flag [letter] [--]");
    return;
}
poplist();
}

```

Uses `Xerror1()` 71c, `count()` 32a, `flag`, `flagset`, `poplist()` 66c, `runq` 30c, and `setstatus()` 67c.

### 14.5.5 \$ shift

`shift` removes the first `n` elements (default 1) from `$*`, the positional parameters. It modifies the variable in place by walking the linked list and freeing the removed nodes.

```

⟨function execshift 146a⟩≡ (210d)
void
execshift(void)
{
    int n;
    word *a;
    var *star;
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: shift [n]\n");
        setstatus("shift usage");
        poplist();
        return;
    case 2:
        n = atoi(runq->argv->words->next->word);
        break;
    case 1:
        n = 1;
        break;
    }
    star = vlook("*");
    for(;n && star->val;--n){
        a = star->val->next;
        efree(star->val->word);
        efree((char *)star->val);
        star->val = a;
        star->changed = true;
    }
    setstatus("");
    poplist();
}

```

### 14.5.6 \$ finit

The `finit` builtin re-reads function definitions from the `/env` filesystem. This is Plan 9-specific: when another process (e.g., a parent shell) defines a function, it appears as a file `/env/fn#name`. `finit` scans `/env` for these `fn#` files and executes them to import the function definitions into the current shell. It uses a small hand-crafted bytecode loop (`rdfns`) similar to the bootstrap code, with `Xrdfn` reading one function file per iteration.

```

⟨global rdfns 146b⟩≡ (210d)
    union Code rdfns[4];

```

Uses Code 29c.

*<function execfinit 147a>*≡ (210d)

```
void
execfinit(void)
{
    //TODO: commented for goken for now because Linux does not have a /env
    // probably need to move to Plan9.c
    //static bool first = true;
    //if(first){
    //    rdfns[0].i = 1;
    //    rdfns[1].f = Xrdfn;
    //    rdfns[2].f = Xjump;
    //    rdfns[3].i = 1;
    //    first = false;
    //}
    //Xpopm(); // pop_list()
    //envdir = open("/env", 0);
    //if(envdir<0){
    //    pfmt(err, "rc: can't open /env: %r\n");
    //    return;
    //}
    //start(rdfns, 1, runq->local);
}
```

*<function Xrdfn 147b>*≡ (213b)

```
void
Xrdfn(void)
{
    int f, len;
    Dir *e;
    char envname[Maxenvname];
    static Dir *ent, *allocent;
    static int nent;

    for(;;){
        if(nent == 0){
            free(allocent);
            nent = dirread(envdir, &allocent);
            ent = allocent;
        }
        if(nent <= 0)
            break;
        while(nent){
            e = ent++;
            nent--;
            len = e->length;
            if(len && strncmp(e->name, "fn#", 3)==0){
                snprintf(envname, sizeof envname, "/env/%s", e->name);
                if((f = open(envname, 0))>=0){
                    execcmds(openfd(f));
                    return;
                }
            }
        }
    }
    close(envdir);
    Xreturn();
}
```

Uses Maxenvname-2 110c, Xreturn() 84d, envdir 112, execcmds() 108b, and openfd() 163e.

# Chapter 15

## Conclusion

You now know how the Plan 9 shell `rc` works—from the lexer that splits input into tokens, through the Yacc parser that builds an AST, the bytecode compiler that emits function pointers, to the interpreter that executes them via the `fork/exec/wait` system call pattern—and more generally how many shells work.

Despite being only about 6700 lines of C (plus a small Yacc grammar), `rc` implements a complete programming language: a hand-written lexer, a Yacc parser, a bytecode compiler, and a bytecode interpreter—the same architecture found in much larger language implementations like Python or Java. Along the way, you have seen how `rc` uses only a handful of system calls—`rfork()`, `exec()`, `wait()`, `pipe()`, `open()`, `close()`, `dup()`—to implement pipes, redirections, subshells, and background jobs. The beauty of the UNIX process model is that these few primitives are enough to build a surprisingly powerful command-line environment.

### 15.1 Patterns and techniques

These techniques apply far beyond shells:

- *Compilation pipeline*: the lexer  $\rightarrow$  parser  $\rightarrow$  bytecode compiler  $\rightarrow$  interpreter architecture is the same one used by Python, Lua, and most scripting languages. Any time you process a structured input (config files, query languages, templates), this four-stage pipeline is the standard approach.
- *Bytecode dispatch*: representing each operation as a function pointer and looping with `(*pc++)()` is how CPython's `ceval.c` and the JVM work. It decouples the description of an action from its execution—useful in command queues, event systems, and undo/redo frameworks.
- *Transparent interposition*: I/O redirection via `dup()` lets a program's input and output be rewired without its knowledge. The same idea—inserting a layer between producer and consumer transparently—is the basis of middleware, proxy servers, and the decorator pattern.
- *Uniform data representation*: every value in `rc` is a list of strings. Lisp does the same with S-expressions, Tcl with strings. A single universal type eliminates conversion boilerplate and makes composition trivial.

### 15.2 Connections to other books

A shell sits at the intersection of many topics covered in other books of Principia Softwarica:

- The `KERNEL` book [Pad14] explains what lies behind the system calls you have seen throughout this book: `rfork()`, `exec()`, `wait()`, `chdir()`, and the `/env` and `/dev/cons` device files. Understanding the kernel will deepen your understanding of how pipes are implemented at the OS level, how the `#!` shebang mechanism works, and how signals (notes) are delivered.

- The COMPILER book [Pad16b] uses the same compilation pipeline as `rc`—lexing, parsing, bytecode generation—but for the C language and targeting real machine code rather than a bytecode interpreter. Having seen `rc`’s simpler compiler first should make the C compiler easier to follow.
- The GENERATORS book [?] (Lex and Yacc) explains what the `yacc`-generated parser in `rc` actually does behind the scenes: the shift-reduce algorithm, the parse tables, and the interaction between `yylex()` and `yyparse()`.
- The BUILDER book [Pad16a] describes `mk`, the Plan 9 build system, which relies heavily on `rc` for executing recipes. Features like `-e` (exit on error) and `-c` (command from string) exist largely because `mk` needs them.

## 15.3 Missing features

`rc` is deliberately minimal. Many features that other shells provide are instead handled by the Plan 9 window manager `rio`:

- *Command-line editing and history*: Under `rio`, you can scroll back in the terminal window and copy-paste previous commands. There is no need for a readline library.
- *Filename completion*: Provided by `rio`’s terminal, not by `rc` itself.
- *Job control* (`bg`, `fg`, `C-z`): Less necessary when you have multiple windows. The `&` operator and `wait` `$apid` cover most use cases.
- *Aliases*: Not needed because `fn` (functions) serve the same purpose and are stored persistently in the environment.

This separation of concerns—keeping the shell simple and letting the terminal handle interactive features—is a characteristic Plan 9 design decision. It contrasts with shells like `bash` or `zsh`, which include tens of thousands of lines of code for features that `rc` delegates elsewhere.

## 15.4 Beyond the Plan 9 shell

The UNIX shell world has evolved considerably since `rc` was written. Here are some of the features found in modern shells:

- *Rich data types*: `rc` has one data type: lists of strings. `bash` (version 4+) adds indexed arrays, associative arrays, and arithmetic expressions. PowerShell takes a radically different approach, passing structured .NET objects through pipelines instead of text. Nushell follows a similar philosophy with typed, tabular data—an interesting departure from the UNIX tradition of plain-text pipelines.
- *String manipulation*: `bash` and `zsh` provide parameter expansion operators for substring extraction (`${var:offs}`), pattern replacement (`${var//pat/rep}`), and case conversion. `rc`’s position is that these operations belong in dedicated tools like `sed` and `awk`, not in the shell itself—keeping the shell language small and letting each tool do one thing well.
- *Process substitution*: `bash`’s `diff <(cmd1) <(cmd2)` uses `/dev/fd` to present command output as file arguments. `rc` has the backquote operator `{cmd}` for command substitution into strings, and in Plan 9, the `<cmd` syntax provides equivalent functionality through the `/fd` file system.

- *Interactive features*: beyond the readline and history that `rio` provides, modern shells add programmable completion (context-aware suggestions for flags, filenames, git branches), syntax highlighting as you type (`fish`, `zsh` with plugins), and autosuggestions from history (`fish`). `zsh` has entire plugin ecosystems (`oh-my-zsh`).
- *Error handling*: `rc`'s error model is straightforward: `$status` and `if not`. `bash` adds `set -e` (exit on error) and `set -o pipefail` (propagate pipe failures). The Oil shell (Oils) goes further with more principled error handling designed to avoid the well-known pitfalls of `set -e`.
- *POSIX compliance*: `rc` deliberately ignores the POSIX shell standard, which gives it a cleaner syntax but means scripts are not portable. Most Linux distributions require `/bin/sh` scripts to work with any POSIX-compliant shell (`dash`, `bash --posix`), and POSIX compliance remains important for portable system scripts.

At the end of the day, the core model remains the one that `rc` shares with the Bourne shell: `fork`, `exec`, `wait`, pipes, and file descriptors. `rc` captures this essential engine in 6700 lines of C. `bash`'s 170 000+ lines add convenience, compatibility, and interactive polish—but the fundamental architecture is the same one you have studied in this book.

# Appendix A

## Debugging

`rc` has several built-in debugging aids. The `-x` flag traces each simple command before execution (printing the command and its arguments). The `-v` flag echoes each input line before parsing. This appendix presents the code behind these flags, as well as the AST pretty-printer used to serialize function bodies into the `/env/` filesystem.

### A.1 AST dumper

$\langle \text{global nl } 151a \rangle \equiv$  (208b)  
`char nl='\n'; /* change to semicolon for bourne-proofing */`

Uses `nl 151a`.

$\langle \text{constant c0 } 151b \rangle \equiv$  (208b)  
`#define c0 t->child[0]`

$\langle \text{constant c1 } 151c \rangle \equiv$  (208b)  
`#define c1 t->child[1]`

$\langle \text{constant c2 } 151d \rangle \equiv$  (208b)  
`#define c2 t->child[2]`

$\langle \text{function pcmd } 151e \rangle \equiv$  (208b)  
`void  
pcmd(io *f, tree *t)  
{  
 if(t==nil)  
 return;  
 assert(f != nil);  
  
 switch(t->type){  
 case '$': pfmt(f, "$%t", c0); break;  
 case '"': pfmt(f, "$\"%t\"", c0); break;  
 case '&': pfmt(f, "%t&", c0); break;  
 case '^': pfmt(f, "%t^%t", c0, c1); break;  
 case '`': pfmt(f, "`%t", c0); break;  
 case ANDAND: pfmt(f, "%t && %t", c0, c1); break;  
 case OROR: pfmt(f, "%t || %t", c0, c1); break;  
 case BANG: pfmt(f, "! %t", c0); break;  
 case BRACE: pfmt(f, "{%t}", c0); break;  
 case COUNT: pfmt(f, "$#%t", c0); break;  
 case FN: pfmt(f, "fn %t %t", c0, c1); break;  
 case IF: pfmt(f, "if%t%t", c0, c1); break;  
 case NOT: pfmt(f, "if not %t", c0); break;`

```

case PCMD:
case PAREN: pfmt(f, "(%t)", c0); break;
case SUB: pfmt(f, "$%t(%t)", c0, c1); break;
case SIMPLE: pfmt(f, "%t", c0); break;
case SUBSHELL: pfmt(f, "@ %t", c0); break;
case SWITCH: pfmt(f, "switch %t %t", c0, c1); break;
case TWIDDLE: pfmt(f, "~ %t %t", c0, c1); break;
case WHILE: pfmt(f, "while %t%t", c0, c1); break;

case ARGLIST:
    if(c0==nil)
        pfmt(f, "%t", c1);
    else if(c1==nil)
        pfmt(f, "%t", c0);
    else
        pfmt(f, "%t %t", c0, c1);
    break;
case ';;':
    if(c0){
        if(c1)
            pfmt(f, "%t%c%t", c0, nl, c1);
        else pfmt(f, "%t", c0);
    }
    else pfmt(f, "%t", c1);
    break;
case WORDS:
    if(c0)
        pfmt(f, "%t ", c0);
    pfmt(f, "%t", c1);
    break;
case FOR:
    pfmt(f, "for(%t", c0);
    if(c1)
        pfmt(f, " in %t", c1);
    pfmt(f, ")%t", c2);
    break;
case WORD:
    if(t->quoted)
        pfmt(f, "%Q", t->str);
    else pdeglob(f, t->str);
    break;
case DUP:
    if(t->rtype==DUPFD)
        pfmt(f, ">[%d=%d]", t->fd1, t->fd0); /* yes, fd1, then fd0; read lex.c */
    else
        pfmt(f, ">[%d=]", t->fd0);
    pfmt(f, "%t", c1);
    break;
case PIPEFD:
case REDIR:
    switch(t->rtype){
    case HERE:
        pchr(f, '<');
    case READ:
    case RDWR:
        pchr(f, '>');
        if(t->rtype==RDWR)
            pchr(f, '>');
        if(t->fd0!=0)
            pfmt(f, "[%d]", t->fd0);
    }

```

```

        break;
    case APPEND:
        pchr(f, '>');
    case WRITE:
        pchr(f, '>');
        if(t->fd0!=1)
            pfmt(f, "[%d]", t->fd0);
        break;
    }
    pfmt(f, "%t", c0);
    if(c1)
        pfmt(f, " %t", c1);
    break;
case '=':
    pfmt(f, "%t=%t", c0, c1);
    if(c2)
        pfmt(f, " %t", c2);
    break;
case PIPE:
    pfmt(f, "%t|", c0);
    if(t->fd1==0){
        if(t->fd0!=1)
            pfmt(f, "[%d]", t->fd0);
    }
    else pfmt(f, "[%d=%d]", t->fd0, t->fd1);
    pfmt(f, "%t", c1);
    break;

default:
    pfmt(f, "bad cmd %d %p %p %p", t->type, c0, c1, c2);
    break;
}
}

```

Uses ANDAND, APPEND 52d, ARGVLIST, BANG, BRACE, COUNT, DUP, DUPFD 137c, FN, FOR, HERE 131a, IF, NOT, OROR, PAREN, PCMD, PIPE, PIPEFD, RDWR 135a, READ 52c, REDIR, SIMPLE, SUB, SUBSHELL, SWITCH, TWIDDLE, WHILE, WORD, WORDS, WRITE 52b, c0-5 151b, c1-6 151c, c2-7 151d, nl 151a, pchr() 164a, pdeglob() 153a, and pfmt() 166b.

```

⟨function pdeglob 153a⟩≡ (208b)
void
pdeglob(io *f, char *s)
{
    while(*s){
        if(*s==GLOB)
            s++;
        pchr(f, *s++);
    }
}

```

Uses GLOB 119a and pchr() 164a.

## A.2 Bytecode generator trace: rc -r

```

⟨main() debug runq in interpreter loop 153b⟩≡ (40c)
    if(flag['r'])
        pfnc(err, runq);

```

Uses err 156b, flag, pfnc() 154b, and runq 30c.

*<global fname 154a>*≡ (210a)

```
struct{
    void (*f)(void);
    char *name;
} fname[] = {
    Xappend, "Xappend",
    Xassign, "Xassign",
    Xasync, "Xasync",
    Xbackq, "Xbackq",
    Xbang, "Xbang",
    Xcase, "Xcase",
    Xclose, "Xclose",
    Xconc, "Xconc",
    Xcount, "Xcount",
    Xdelfn, "Xdelfn",
    Xdelhere, "Xdelhere",
    Xdol, "Xdol",
    Xdup, "Xdup",
    Xeflag, "Xeflag",
    (void (*)(void))Xerror, "Xerror",
    Xexit, "Xexit",
    Xfalse, "Xfalse",
    Xfn, "Xfn",
    Xfor, "Xfor",
    Xglob, "Xglob",
    Xif, "Xif",
    Xifnot, "Xifnot",
    Xjump, "Xjump",
    Xlocal, "Xlocal",
    Xmark, "Xmark",
    Xmatch, "Xmatch",
    Xpipe, "Xpipe",
    Xpipefd, "Xpipefd",
    Xpipewait, "Xpipewait",
    Xpopm, "Xpopm",
    Xpopredir, "Xpopredir",
    Xqdol, "Xqdol",
    Xrdcmds, "Xrdcmds",
    Xrdfn, "Xrdfn",
    Xrdwr, "Xrdwr",
    Xread, "Xread",
    Xreturn, "Xreturn",
    Xsimple, "Xsimple",
    Xsub, "Xsub",
    Xsubshell, "Xsubshell",
    Xtrue, "Xtrue",
    Xunlocal, "Xunlocal",
    Xwastrue, "Xwastrue",
    Xword, "Xword",
    Xwrite, "Xwrite",
    0
};
```

Uses Xappend() 82b, Xassign() 96c, Xasync() 86d, Xbackq() 140, Xbang() 77b, Xcase() 92c, Xclose() 137i, Xconc() 100b, Xdelfn() 95a, Xdelhere() 132c, Xdup() 137e, Xeflag() 129a, Xerror() 71b, Xexit() 87, Xfalse() 76f, Xfn() 94c, Xfor() 90b, Xglob() 121a, Xif() 88b, Xifnot() 88f, Xjump() 89g, Xlocal() 97a, Xmark() 66b, Xmatch() 77d, Xpipe() 85b, Xpipefd() 138h, Xpipewait() 85f, Xpopm() 92a, Xpopredir() 81a, Xqdol() 141c, Xrdcmds() 41a, Xrdfn() 147b, Xrdwr() 135d, Xread() 81c, Xreturn() 84d, Xsimple() 70, Xsub() 99b, Xsubshell() 130d, Xtrue() 76e, Xunlocal() 97b, Xwastrue() 88d, Xword() 66a, Xwrite() 80d, and \_\_anon\_struct\_4 154a.

*<function pfnc 154b>*≡ (210a)

```

void
pfnc(io *fd, thread *t)
{
    int i;
    void (*fn)(void) = t->code[t->pc].f;
    list *a;

    pfmt(fd, "pid %d cycle %p %d ", getpid(), t->code, t->pc);
    for (i = 0; fname[i].f; i++)
        if (fname[i].f == fn) {
            pstr(fd, fname[i].name);
            break;
        }
    if (!fname[i].f)
        pfmt(fd, "%p", fn);
    for (a = t->argv; a; a = a->next)
        pfmt(fd, " (%v)", a->words);
    pchr(fd, '\n');
    flush(fd);
}

```

Uses flush() 163g, fname 154a, pchr() 164a, pfmt() 166b, and pstr() 167a.

### A.3 Printing subprocesses status: rc -s

```

⟨Xrdcmds() print status if -s 155a⟩≡ (41a)
    if(flag['s'] && !truestatus())
        pfmt(err, "status=%v\n", vlook("status")->val);

```

Uses err 156b, flag, pfmt() 166b, truestatus() 68a, and vlook() 34b.

### A.4 Printing commands: rc -x

```

⟨Xsimple() if -x 155b⟩≡ (70)
    if(flag['x'])
        pfmt(err, "%v\n", p->argv->words); /* wrong, should do redirs */

```

Uses err 156b, flag, and pfmt() 166b.

### A.5 Printing characters: rc -v

```

⟨getnext() if not EOF but verbose mode, print character read 155c⟩≡ (43)
    else
        if(flag['V'] || ndot>=2 && flag['v'])
            pchr(err, c);

```

Uses err 156b, flag, ndot 107a, and pchr() 164a.

### A.6 Printing all characters: rc -V

# Appendix B

## Error Management

`rc` has three levels of error response. Syntax and parse errors (reported by `yyerror()`) are recoverable: `rc` prints the error, discards the current input line, and returns to the prompt. Runtime errors (reported by `Xerror()`) set the exit status but continue execution. Fatal errors (reported by `panic()`) print a message and call `abort()`—these indicate bugs in `rc` itself, not in the user’s script.

```
<constant ERRMAX 156a>≡ (196c)
#define ERRMAX 128
```

```
<global err 156b>≡ (212c)
io *err;
```

```
<global nerror 156c>≡ (212c)
int nerror; /* number of errors encountered during compilation */
```

```
<Xrdcmds() flush errors and reset error count 156d>≡ (41a)
flush(err);
nerror = 0;
```

Uses `err 156b`, `flush()` 163g, and `nerror 156c`.

```
<function yyerror 156e>≡ (207a)
void
yyerror(char *m)
{
    pfmt(err, "rc: ");
    if(runq->cmdfile && !runq->iflag)
        pfmt(err, "%s:%d: ", runq->cmdfile, runq->lineno);
    else if(runq->cmdfile)
        pfmt(err, "%s: ", runq->cmdfile);
    else if(!runq->iflag)
        pfmt(err, "line %d: ", runq->lineno);

    if(tok[0] && tok[0]!='\n')
        pfmt(err, "token %q: ", tok);
    pfmt(err, "%s\n", m);
    flush(err);
    lastword = false;
    lastdol = false;

    while(lastc!='\n' && lastc!=EOF)
        advance();

    nerror++;
    setvar("status", newword(m, (word *)nil));
}
```

Uses `EOF 44a`, `advance()` 47a, `err 156b`, `flush()` 163g, `lastc 157a`, `lastdol 51b`, `lastword 53e`, `nerror 156c`, `newword()` 31d, `pfmt()` 166b, `runq 30c`, `setvar()` 34a, and `tok 49c`.

*<global lastc 157a>*≡ (201)  
 int lastc;

*<advance() save future in lastc 157b>*≡ (47a)  
 lastc = future;

Uses future 46d and lastc 157a.

*<function panic 157c>*≡ (212c)  
 void  
 panic(char \*s, int n)  
 {  
     pfmt(err, "rc: ");  
     pfmt(err, s, n);  
     pchr(err, '\n');  
     flush(err);  
     Abort();  
 }

Uses Abort() 157d, err 156b, flush() 163g, pchr() 164a, and pfmt() 166b.

*<function Abort 157d>*≡ (212c)  
 void  
 Abort(void)  
 {  
     pfmt(err, "aborting\n");  
     flush(err);  
     Exit("aborting", \_\_LOC\_\_);  
 }

Uses err 156b, flush() 163g, and pfmt() 166b.

*<function Exit 157e>*≡ (213b)  
 void  
 Exit(char \*stat, char\* \_LOC)  
 {  
     USED(\_LOC);  
     Updenv();  
     setstatus(stat);  
     exits(truestatus() ? "" : getstatus());  
 }

Uses getstatus() 67d, setstatus() 67c, and truestatus() 68a.

# Appendix C

## Utilities

This appendix collects the utility code that `rc` uses throughout: memory allocation wrappers (`emalloc`, `erealloc`), the buffered I/O system (the `io` structure and its read/write functions), command-line flag parsing (`getflags`), and string conversion helpers.

```
<function _efgfmt 158a>≡ (213b)
/* avoid loading any floating-point library code */
/*@Scheck: weird, probably linker trick
int _efgfmt(Fmt *)
{
    return -1;
}
```

### C.1 Memory management

```
<function emalloc 158b>≡ (210b)
void *
emalloc(long n)
{
    void *p = Malloc(n);

    if(p==nil)
        panic("Can't malloc %d bytes", n);
    return p;
}
```

Uses `panic()` 157c.

```
<function Malloc 158c>≡ (213b)
void*
Malloc(ulong n)
{
    return mallocz(n, 1);
}
```

```
<function efree 158d>≡ (210b)
void
efree(void *p)
{
    if(p)
        free(p);
    else pfmt(err, "free 0\n");
}
```

Uses `err` 156b and `pfmt()` 166b.

## C.2 Command-line arguments

*<global cmdname 159a>*≡ (200a)  
char \*cmdname;

*<global flagarg 159b>*≡ (200a)  
static char \*flagarg="";

*<global reason 159c>*≡ (200a)  
static int reason;

*<constant RESET 159d>*≡ (200a)  
#define RESET 1

*<constant FEWARGS 159e>*≡ (200a)  
#define FEWARGS 2

*<constant FLAGSYN 159f>*≡ (200a)  
#define FLAGSYN 3

*<constant BADFLAG 159g>*≡ (200a)  
#define BADFLAG 4

*<global badflag 159h>*≡ (200a)  
static int badflag;

*<function getflags 159i>*≡ (200a)  
int  
getflags(int argc, char \*argv[], char \*flags, int stop)  
{  
 char \*s;  
 int i, j, c, count;  
 flagarg = flags;  
 if(cmdname==0)  
 cmdname = argv[0];  
  
 i = 1;  
 while(i!=argc){  
 if(argv[i][0] != '-' || argv[i][1] == '\0'){  
 if(stop) /\* always true in rc \*/  
 return argc;  
 i++;  
 continue;  
 }  
 s = argv[i]+1;  
 while(\*s){  
 c=\*s++;  
 count = scanflag(c, flags);  
 if(count==-1)  
 return -1;  
 if(flag[c]){ reason = RESET; badflag = c; return -1; }  
 if(count==0){  
 flag[c] = flagset;  
 if(\*s=='\0'){  
 for(j = i+1;j<=argc;j++)  
 argv[j-1] = argv[j];  
 --argc;  
 }  
 }  
 }  
 else{

```

        if(*s=='\0'){
            for(j = i+1;j<=argc;j++)
                argv[j-1] = argv[j];
            --argc;
            s = argv[i];
        }
        if(argc-i<count){
            reason = FEWARGS;
            badflag = c;
            return -1;
        }
        reverse(argv+i, argv+argc);
        reverse(argv+i, argv+argc-count);
        reverse(argv+argc-count+1, argv+argc);
        argc-=count;
        flag[c] = argv+argc+1;
        flag[c][0] = s;
        s="";
    }
}
return argc;
}

```

Uses FEWARGS-20, RESET-19, badflag-23 200a, cmdname, flag, flagarg-17, flagset, reason-18 200a, and reverse() 160a.

```

⟨function reverse 160a⟩≡ (200a)
static void
reverse(char **p, char **q)
{
    char *t;
    for(;p<q;p++,--q){ t=*p; *p=*q; *q = t; }
}

```

```

⟨function scanflag 160b⟩≡ (200a)
static int
scanflag(int c, char *f)
{
    int fc, count;
    if(0<=c && c<NFLAG)
        while(*f){
            if(*f==' '){
                f++;
                continue;
            }
            fc=*f++;
            if(*f==':'){
                f++;
                if(*f<'0' || '9'<*f){ reason = FLAGSYN; return -1; }
                count = 0;
                while('0'<=*f && *f<='9') count = count*10+*f++-'0';
            }
            else
                count = 0;
            if(*f=='['){
                do{
                    f++;
                    if(*f=='\0'){ reason = FLAGSYN; return -1; }
                }while(*f!=']');
                f++;
            }
        }
}

```

```

        if(c==fc)
            return count;
    }
    reason = BADFLAG;
    badflag = c;
    return -1;
}

```

⟨function usage 161⟩≡ (200a)

```

void
usage(char *tail)
{
    char *s, *t, c;
    int count, nflag = 0;
    switch(reason){
    case RESET:
        errs("Flag -");
        errc(badflag);
        errs(": set twice\n");
        break;
    case FEWARGS:
        errs("Flag -");
        errc(badflag);
        errs(": too few arguments\n");
        break;
    case FLAGSYN:
        errs("Bad argument to getflags!\n");
        break;
    case BADFLAG:
        errs("Illegal flag -");
        errc(badflag);
        errc('\n');
        break;
    }
    errs("Usage: ");
    errs(cmdname);
    for(s = flagarg;*s;){
        c=*s;
        if(*s++==' ')
            continue;
        if(*s==':'){
            s++;
            count = 0;
            while('0'<=*s && *s<='9') count = count*10+*s++-'0';
        }
        else count = 0;
        if(count==0){
            if(nflag==0)
                errs(" [-");
            nflag++;
            errc(c);
        }
        if(*s=='['){
            s++;
            while(*s!=']' && *s!='\0') s++;
            if(*s==']')
                s++;
        }
    }
    if(nflag)

```

```

    errs("]");
for(s = flagarg;*s;){
    c=*s;
    if(*s++==' ')
        continue;
    if(*s==':') {
        s++;
        count = 0;
        while('0'<=*s && *s<='9') count = count*10+*s++-'0';
    }
    else count = 0;
    if(count!=0){
        errs(" [-");
        errc(c);
        if(*s=='['){
            s++;
            t = s;
            while(*s!=']' && *s!='\0') s++;
            errs(" ");
            errn(t, s-t);
            if(*s==']')
                s++;
        }
        else
            while(count-->0) errs(" arg");
        errs("]");
    }
    else if(*s==','){
        s++;
        while(*s!=']' && *s!='\0') s++;
        if(*s==']')
            s++;
    }
}
if(tail){
    errs(" ");
    errs(tail);
}
errs("\n");
Exit("bad flags", "getflags.c");
}

```

```

⟨function errn 162a⟩≡ (200a)
static void
errn(char *s, int count)
{
    while(count){ errc(*s++); --count; }
}

```

```

⟨function errs 162b⟩≡ (200a)
static void
errs(char *s)
{
    while(*s) errc(*s++);
}

```

```

⟨constant NBUF (rc/getflags.c) 162c⟩≡ (200a)
#define NBUF 80

```

```

⟨global buf 162d⟩≡ (200a)
static char buf[NBUF];

```

*<global bufp 163a>*≡ (200a)

```
static char *bufp = buf;
```

*<function errc 163b>*≡ (200a)

```
static void
errc(int c)
{
    *bufp++=c;
    if(bufp==&buf[NBUF] || c=='\n'){
        write(2, buf, bufp-buf);
        bufp = buf;
    }
}
```

Uses NBUF-24, buf-25, and bufp-26 200a.

## C.3 Buffered IO

*<constant NBUF 163c>*≡ (196b)

```
#define NBUF 512
```

*<struct Io 163d>*≡ (196b)

```
struct Io {
    fdt fd;
    byte *bufp, *ebuf, *strp;
    byte buf[NBUF];
};
```

Uses NBUF 163c.

*<function openfd 163e>*≡ (200b)

```
io*
openfd(fdt fd)
{
    io *f = new(struct Io);
    f->fd = fd;
    f->bufp = f->ebuf = f->buf;
    f->strp = nil;
    return f;
}
```

Uses Io 163d.

*<enum MiscConstants 163f>*≡ (200b)

```
enum { Stralloc = 100, };
```

*<function flush 163g>*≡ (200b)

```
void
flush(io *f)
{
    int n;

    if(f->strp){
        n = f->ebuf - f->strp;
        f->strp = realloc(f->strp, n+Stralloc+1);
        if(f->strp==0)
            panic("Can't realloc %d bytes in flush!", n+Stralloc+1);
        f->bufp = f->strp + n;
        f->ebuf = f->bufp + Stralloc;
        memset(f->bufp, '\0', Stralloc+1);
    }
}
```

```

else{
    n = f->bufp-f->buf;
    if(n && write(f->fd, f->buf, n) != n){
        write(2, "Write error\n", 12);
        if(ntrap)
            dotrap();
    }
    f->bufp = f->buf;
    f->ebuf = f->buf+NBUF;
}
}

```

Uses NBUF 163c, Stralloc-1 163f, dotrap() 115b, ntrap 114b, and panic() 157c.

```

⟨function pchr 164a⟩≡ (212a)
void
pchr(io *b, int c)
{
    if(b->bufp==b->ebuf)
        fullbuf(b, c);
    else *b->bufp++=c;
}

```

Uses fullbuf() 164b.

```

⟨function fullbuf 164b⟩≡ (200b)
int
fullbuf(io *f, int c)
{
    flush(f);
    return *f->bufp++=c;
}

```

Uses flush() 163g.

```

⟨function rchr 164c⟩≡ (200b)
int
rchr(io *b)
{
    if(b->bufp==b->ebuf)
        return emptybuf(b);
    return *b->bufp++;
}

```

Uses emptybuf() 164d.

```

⟨function emptybuf 164d⟩≡ (200b)
int
emptybuf(io *f)
{
    int n;
    if(f->fd==-1 || (n = read(f->fd, f->buf, NBUF))<=0) return EOF;
    f->bufp = f->buf;
    f->ebuf = f->buf + n;
    return *f->bufp++;
}

```

Uses EOF 44a and NBUF 163c.

```

⟨function rutf 164e⟩≡ (200b)
int
rutf(io *b, char *buf, Rune *r)
{
    int n, i, c;

```

```

c = rchr(b);
if(c == EOF)
    return EOF;
*buf = c;
if(c < Runesync){
    *r = c;
    return 1;
}
for(i = 1; (c = rchr(b)) != EOF; ){
    buf[i++] = c;
    buf[i] = 0;
    if(fullrune(buf, i)){
        n = chartorune(r, buf);
        b->bufp -= i - n; /* push back unconsumed bytes */
        assert(b->fd == -1 || b->bufp > b->buf);
        return n;
    }
}
/* at eof */
b->bufp -= i - 1; /* consume 1 byte */
*r = Runeerror;
return runetochar(buf, r);
}

```

Uses EOF 44a and rchr() 164c.

```

⟨function closeio 165a⟩≡ (200b)
void
closeio(io *io)
{
    if(io->fd>=0)
        close(io->fd);
    if(io->strp)
        efree(io->strp);
    efree(io);
}

```

Uses efree() 158d.

```

⟨function openstr 165b⟩≡ (200b)
io*
openstr(void)
{
    io *f = new(struct Io);

    f->fd = -1;
    f->bufp = f->strp = emalloc(Stralloc+1);
    f->ebuf = f->bufp + Stralloc;
    memset(f->bufp, '\0', Stralloc+1);
    return f;
}

```

Uses Io 163d, Stralloc-1 163f, and emalloc() 158b.

```

⟨function opencore 165c⟩≡ (200b)
/*
 * Open a corebuffer to read. EOF occurs after reading len
 * characters from buf.
 */

io*

```

```

opencore(char *s, int len)
{
    io *f = new(struct Io);
    uchar *buf = emalloc(len);

    f->fd = -1 /*open("/dev/null", 0)*/;
    f->bufp = f->strp = buf;
    f->ebuf = buf+len;
    Memcpy(buf, s, len);
    return f;
}

```

Uses Io 163d, Memcpy() 169i, and emalloc() 158b.

## C.4 Format

*<global pfmtnest 166a>*≡ (212a)  
 int pfmtnest = 0;

Uses pfmtnest 166a.

*<function pfmt 166b>*≡ (212a)  
 void  
 pfmt(io \*f, char \*fmt, ...)  
 {  
 va\_list ap;  
 char err[ERRMAX];  
  
 va\_start(ap, fmt);  
 pfmtnest++;  
 for(;\*fmt;fmt++) {  
 if(\*fmt!='%') {  
 pchr(f, \*fmt);  
 continue;  
 }  
 if(++fmt == '\0') /\* "blah%"? \*/  
 break;  
 switch(\*fmt){  
 case 'c':  
 pchr(f, va\_arg(ap, int));  
 break;  
 case 'd':  
 pdec(f, va\_arg(ap, int));  
 break;  
 case 'o':  
 poct(f, va\_arg(ap, unsigned int));  
 break;  
 case 'p':  
 pptr(f, va\_arg(ap, void\*));  
 break;  
 case 'Q':  
 pquo(f, va\_arg(ap, char \*));  
 break;  
 case 'q':  
 pwrdf(f, va\_arg(ap, char \*));  
 break;  
 case 's':  
 pstr(f, va\_arg(ap, char \*));  
 break;

```

    case 'r':
        errstr(err, sizeof err); pstr(f, err);
        break;

    // rc specific, TODO LP split here
    case 't':
        pcmd(f, va_arg(ap, struct Tree *));
        break;
    case 'v':
        pval(f, va_arg(ap, struct Word *));
        break;

    default:
        pchr(f, *fmt);
        break;
}
}
va_end(ap);
if(--pfmtnest==0)
    flush(f);
}

```

Uses err 156b, flush() 163g, pchr() 164a, pcmd() 151e, pdec() 167b, pfmtnest 166a, poct() 168a, pptr() 168d, pquo() 168b, pstr() 167a, pval() 168e, and pwrdr() 168c.

```

⟨function pstr 167a⟩≡ (212a)
void
pstr(io *f, char *s)
{
    if(s==0)
        s="(null)";
    while(*s) pchr(f, *s++);
}

```

Uses pchr() 164a.

```

⟨function pdec 167b⟩≡ (212a)
void
pdec(io *f, int n)
{
    if(n<0){
        n=-n;
        if(n>=0){
            pchr(f, '-');
            pdec(f, n);
            return;
        }
        /* n is two's complement minimum integer */
        n = 1-n;
        pchr(f, '-');
        pdec(f, n/10);
        pchr(f, n%10+'1');
        return;
    }
    if(n>9)
        pdec(f, n/10);
    pchr(f, n%10+'0');
}

```

Uses pchr() 164a and pdec() 167b.

```

⟨function poct 168a⟩≡ (212a)
void
poct(io *f, unsigned int n)
{
    if(n>7)
        poct(f, n>>3);
    pchr(f, (n&7)+'0');
}

```

Uses pchr() 164a and poct() 168a.

```

⟨function pquo 168b⟩≡ (212a)
void
pquo(io *f, char *s)
{
    pchr(f, '\\');
    for(;*s;s++)
        if(*s=='\\')
            pfmt(f, "\\");
        else pchr(f, *s);
    pchr(f, '\\');
}

```

Uses pchr() 164a and pfmt() 166b.

```

⟨function pwrđ 168c⟩≡ (212a)
void
pwrđ(io *f, char *s)
{
    char *t;
    for(t = s;*t;t++) if(*t >= 0 && needsrcquote(*t)) break;
    if(t==s || *t)
        pquo(f, s);
    else pstr(f, s);
}

```

Uses needsrcquote() 214, pquo() 168b, and pstr() 167a.

```

⟨function pptr 168d⟩≡ (212a)
void
pptr(io *f, void *v)
{
    int n;
    uintptr p;

    p = (uintptr)v;
    if(sizeof(uintptr) == sizeof(uvlong) && p>>32)
        for(n = 60;n>=32;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);

    for(n = 28;n>=0;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);
}

```

Uses pchr() 164a.

```

⟨function pval 168e⟩≡ (212a)
void
pval(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
            pwrđ(f, (char *)a->word);
            pchr(f, ' ');
            a = a->next;
        }
    }
}

```

```

    }
    pwrdr(f, (char *)a->word);
}
}

```

Uses `pchr()` 164a and `pwrdr()` 168c.

## C.5 String conversions

*<global bp 169a>*≡ (210b)  
 char \*bp;

*<function intoascii 169b>*≡ (210b)  
 void  
 intoascii(char \*s, long n)  
 {  
     bp = s;  
     iacvt(n);  
     \*bp='\0';  
 }

Uses `bp` 169a and `iacvt()` 169c.

*<function iacvt 169c>*≡ (210b)  
 static void  
 iacvt(int n)  
 {  
     if(n<0){  
         \*bp++='-';  
         n=-n; /\* doesn't work for n==-inf \*/  
     }  
     if(n/10)  
         iacvt(n/10);  
     \*bp++=n%10+'0';  
 }

Uses `bp` 169a and `iacvt()` 169c.

## C.6 Misc

*<function Write 169d>*≡ (210b)

*<function Read 169e>*≡ (210b)

*<function Seek 169f>*≡ (210b)

*<function Creat 169g>*≡ (210b)

```

int
Creat(char *file)
{
    return create(file, 1, 0666L);
}

```

*<function Dup 169h>*≡ (210b)

*<function Malloc 169i>*≡ (210b)

```

void
Malloc(void *a, void *b, long n)
{
    memmove(a, b, n);
}

```

# Appendix D

## Examples of rc scripts

### D.1 /rc/lib/rcmain

This is the initialization script sourced by the bootstrap bytecodes (see Chapter 12). It sets up default values for `$home`, `$ifs`, `$prompt`, and `$path`, calls `finit` to import shell functions from the environment, and then dispatches based on how `rc` was invoked: with `-c` (run a command string), with `-i` (interactive, source the user's profile and read from the console), or with a script file argument.

```
# rcmain: Plan 9 version
if(~ $#home 0) home=/
if(~ $#ifs 0) ifs='
'

switch($#prompt){
case 0
  prompt=( '% ' ' ' )
case 1
  prompt=( $prompt ' ' )
}

if(~ $rcname ?.out) prompt=( 'broken! ' ' ' )

if(flag p) path=/bin
if not{
  finit
  if(~ $#path 0) path=(. /bin)
}

fn sigexit

if(! ~ $#cflag 0){
  if(flag l && /bin/test -r $home/lib/profile) . $home/lib/profile
  status=''
  eval $cflag
}
if not if(flag i){
  if(flag l && /bin/test -r $home/lib/profile) . $home/lib/profile
  status=''
}
```

```

if(! ~ $#* 0) . $*
. -i '#d/0'
}
if not if(~ $#* 0) . '#d/0'
if not{
    status=''
    . $*
}
exit $status

```

## D.2 /usr/glenda/lib/profile

This is Glenda's (the default Plan 9 user) login profile, sourced by `rcmain` when `rc` is invoked with the `-l` (login shell) flag (the `flag 1` test in `rcmain` above). It uses `bind` to overlay the user's personal `bin` directories onto `/bin` (so user scripts and binaries are found automatically), sets up the font and mail filesystem (`upas/fs`), and then dispatches based on `$service`: on a terminal, it starts the window manager `rio`; on a CPU server, it connects to the terminal's devices via `/mnt/term`.

```

bind -a $home/bin/rc /bin
bind -a $home/bin/$cputype /bin
bind -c tmp /tmp
if(! syscall create /tmp/xxx 1 0666 >[2]/dev/null)
    ramfs # in case we're running off a cd
font = /lib/font/bit/pelm/euro.9.font
upas/fs
fn cd { builtin cd $* && awd } # for acme
switch($service){
case terminal
    plumber
    echo -n accelerated > '#m/mousectl'
    echo -n 'res 3' > '#m/mousectl'
    prompt=('term% ' ' ')
    fn term%{ $* }
    exec rio -i riostart
case cpu
    if (test -e /mnt/term/mnt/wsys) { # rio already running
        bind -a /mnt/term/mnt/wsys /dev
        if(test -w /dev/label)
            echo -n $sysname > /dev/label
    }
    bind /mnt/term/dev/cons /dev/cons
    bind /mnt/term/dev/consctl /dev/consctl
    bind -a /mnt/term/dev /dev
    prompt=('cpu% ' ' ')
    fn cpu%{ $* }
    news
    if (! test -e /mnt/term/mnt/wsys) { # cpu call from drawterm
        font=/lib/font/bit/pelm/latin1.8.font
        exec rio
    }
}

```

```
}
case con
  prompt=('cpu% ' ' ')
  news
}
```

# Appendix E

## Shell Companion Utilities

This appendix presents the source code of a few small programs that are closely tied to the shell. They are too simple to deserve their own book, but they are used so frequently in shell scripts and interactive sessions that understanding their internals is worthwhile. For more general Plan 9 utilities, see UTILITIES book [\[Pad25\]](#).

### E.1 echo

`echo` concatenates its arguments separated by spaces, followed by a newline. The `-n` flag suppresses the trailing newline. Note that it builds the entire output in a single buffer and writes it with one `write()` call, so the output is atomic—important when multiple processes write to the same file descriptor.

```
⟨function main(echo) 173)≡ (191d)
    void
    main(int argc, char *argv[])
    {
        int nflag;
        int i, len;
        char *buf, *p;

        nflag = 0;
        if(argc > 1 && strcmp(argv[1], "-n") == 0)
            nflag = 1;

        len = 1;
        for(i = 1+nflag; i < argc; i++){
            len += strlen(argv[i])+1;

        buf = malloc(len);
        if(buf == 0)
            exits("no memory");

        p = buf;
        for(i = 1+nflag; i < argc; i++){
            strcpy(p, argv[i]);
            p += strlen(p);
            if(i < argc-1)
                *p++ = ' ';
        }

        if(!nflag)
            *p++ = '\n';

        if(write(1, buf, p-buf) < 0){
            fprintf(2, "echo: write error: %r\n");
```

```

        exits("write error");
    }

    exits((char *)nil);
}

```

## E.2 pwd

`pwd` simply calls `getwd()` and prints the result. In Plan 9, the current directory is a per-process kernel state, not an environment variable like `\$PWD` in some UNIX shells.

```

⟨function main (misc/pwd.c) 174a⟩≡ (191e)
/*
 * Print working (current) directory
 */
void
main(int argc, char *argv[])
{
    char pathname[512];

    USED(argc, argv);
    if(getwd(pathname, sizeof(pathname)) == 0) {
        fprintf(2, "pwd: %r\n");
        exits("getwd");
    }
    print("%s\n", pathname);
    exits(nil);
}

```

## E.3 test

`test` evaluates conditional expressions, used heavily in shell scripts (e.g., `if(test -f foo) ...`). It is an external program, not a builtin—unlike in `bash` where `test` and `[` are built in. The implementation is a small recursive-descent parser with three precedence levels: `-o` (or), `-a` (and), and `!` (not), followed by the primary tests (`-f`, `-d`, `-r`, `-eq`, etc.).

```

⟨function EQ 174b⟩≡ (191f)
#define EQ(a,b) ((tmp=a)==0?0:(strcmp(tmp,b)==0))

⟨global ap 174c⟩≡ (191f)
int ap;

⟨global ac 174d⟩≡ (191f)
int ac;

⟨global av 174e⟩≡ (191f)
char **av;

⟨global tmp 174f⟩≡ (191f)
static char *tmp;

```

*<function main (misc/test.c) 175a>*≡ (191f)

```
void
main(int argc, char *argv[])
{
    int r;
    char *c;

    ac = argc; av = argv; ap = 1;
    if(EQ(argv[0], "[")) {
        if(!EQ(argv[--ac], "]"))
            synbad("] missing", "");
    }
    argv[ac] = 0;
    if (ac<=1)
        exits("usage");
    r = e();
    /*
     * nice idea but short-circuit -o and -a operators may have
     * not consumed their right-hand sides.
     */
    if(false && (c = nxtarg(1)) != nil)
        synbad("unexpected operator/operand: ", c);
    exits(r? nil : "false");
}
```

*<function nxtarg 175b>*≡ (191f)

```
char *
nxtarg(int mt)
{
    if(ap>=ac){
        if(mt){
            ap++;
            return(0);
        }
        synbad("argument expected", "");
    }
    return(av[ap++]);
}
```

*<function nxtintarg 175c>*≡ (191f)

```
bool
nxtintarg(int *pans)
{
    if(ap<ac && isint(av[ap], pans)){
        ap++;
        return true;
    }
    return false;
}
```

*<function e 175d>*≡ (191f)

```
int
e(void)
{
    int p1;

    p1 = e1();
    if (EQ(nxtarg(1), "-o"))
        return(p1 || e());
    ap--;
```

```

        return(p1);
    }

⟨function e1 176a⟩≡ (191f)
    int
    e1(void)
    {
        int p1;

        p1 = e2();
        if (EQ(nxtarg(1), "-a"))
            return (p1 && e1());
        ap--;
        return(p1);
    }

⟨function e2 176b⟩≡ (191f)
    int
    e2(void)
    {
        if (EQ(nxtarg(0), "!"))
            return(!e2());
        ap--;
        return(e3());
    }

⟨function e3 176c⟩≡ (191f)
    bool
    e3(void)
    {
        int p1, int1, int2;
        char *a, *p2;

        a = nxtarg(0);
        if(EQ(a, "(")) {
            p1 = e();
            if(!EQ(nxtarg(0), ")"))
                synbad(" expected", "");
            return(p1);
        }

        if(EQ(a, "-A"))
            return(hasmode(nxtarg(0), DMAPPEND));

        if(EQ(a, "-L"))
            return(hasmode(nxtarg(0), DMEXCL));

        if(EQ(a, "-T"))
            return(hasmode(nxtarg(0), DMTMP));

        if(EQ(a, "-f"))
            return(isreg(nxtarg(0)));

        if(EQ(a, "-d"))
            return(isdir(nxtarg(0)));

        if(EQ(a, "-r"))
            return(tio(nxtarg(0), 4));

        if(EQ(a, "-w"))

```

```

    return(tio(nxtarg(0), 2));

if(EQ(a, "-x"))
    return(tio(nxtarg(0), 1));

if(EQ(a, "-e"))
    return(tio(nxtarg(0), 0));

if(EQ(a, "-c"))
    return false;

if(EQ(a, "-b"))
    return false;

if(EQ(a, "-u"))
    return false;

if(EQ(a, "-g"))
    return false;

if(EQ(a, "-s"))
    return(fsizep(nxtarg(0)));

if(EQ(a, "-t"))
    if(ap>=ac)
        return(isatty(1));
    else if(nxtintarg(&int1))
        return(isatty(int1));
    else
        synbad("not a valid file descriptor number ", "");

if(EQ(a, "-n"))
    return(!EQ(nxtarg(0), ""));
if(EQ(a, "-z"))
    return(EQ(nxtarg(0), ""));

p2 = nxtarg(1);
if (p2==0)
    return(!EQ(a, ""));
if(EQ(p2, "="))
    return(EQ(nxtarg(0), a));

if(EQ(p2, "!="))
    return(!EQ(nxtarg(0), a));

if(EQ(p2, "-older"))
    return(isolder(nxtarg(0), a));

if(EQ(p2, "-ot"))
    return(isolderthan(nxtarg(0), a));

if(EQ(p2, "-nt"))
    return(isnewerthan(nxtarg(0), a));

if(!isint(a, &int1))
    synbad("unexpected operator/operand: ", p2);

if(nxtintarg(&int2)){
    if(EQ(p2, "-eq"))
        return(int1==int2);

```

```

        if(EQ(p2, "-ne"))
            return(int1!=int2);
        if(EQ(p2, "-gt"))
            return(int1>int2);
        if(EQ(p2, "-lt"))
            return(int1<int2);
        if(EQ(p2, "-ge"))
            return(int1>=int2);
        if(EQ(p2, "-le"))
            return(int1<=int2);
    }

    synbad("unknown operator ",p2);
    return false; /* to shut ken up */
}

```

```

⟨function tio 178a⟩≡ (191f)
bool
tio(char *a, int f)
{
    return access (a, f) >= 0;
}

```

```

⟨function hasmode 178b⟩≡ (191f)
/*
 * note that the name strings pointed to by Dir members are
 * allocated with the Dir itself (by the same call to malloc),
 * but are not included in sizeof(Dir), so copying a Dir won't
 * copy the strings it points to.
 */
bool
hasmode(char *f, ulong m)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;
    r = (dir->mode & m) != 0;
    free(dir);
    return r;
}

```

```

⟨function isdir 178c⟩≡ (191f)
bool
isdir(char *f)
{
    return hasmode(f, DMDIR);
}

```

```

⟨function isreg 178d⟩≡ (191f)
bool
isreg(char *f)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)

```

```

        return false;
    r = (dir->mode & DMDIR) == 0;
    free(dir);
    return r;
}

```

*<function isatty 179a>*≡ (191f)

```

int
isatty(int fd)
{
    int r;
    Dir *d1, *d2;

    d1 = dirfstat(fd);
    d2 = dirstat("/dev/cons");
    if (d1 == nil || d2 == nil)
        r = 0;
    else
        r = d1->type == d2->type && d1->dev == d2->dev &&
            d1->qid.path == d2->qid.path;
    free(d1);
    free(d2);
    return r;
}

```

*<function fsizep 179b>*≡ (191f)

```

bool
fsizep(char *f)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;
    r = dir->length > 0;
    free(dir);
    return r;
}

```

*<function synbad 179c>*≡ (191f)

```

void
synbad(char *s1, char *s2)
{
    int len;

    write(2, "test: ", 6);
    if ((len = strlen(s1)) != 0)
        write(2, s1, len);
    if ((len = strlen(s2)) != 0)
        write(2, s2, len);
    write(2, "\n", 1);
    exits("bad syntax");
}

```

*<function isint 179d>*≡ (191f)

```

int
isint(char *s, int *pans)
{
    char *ep;

```

```

    *pans = strtol(s, &ep, 0);
    return (*ep == 0);
}

```

*<function isolder 180>*≡ (191f)

```

bool
isolder(char *pin, char *f)
{
    int r, rel;
    ulong n, m;
    char *p = pin;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;

    /* parse time */
    n = 0;
    rel = 0;
    while(*p){
        m = strtoul(p, &p, 0);
        switch(*p){
            case 0:
                n = m;
                break;
            case 'y':
                m *= 12;
                /* fall through */
            case 'M':
                m *= 30;
                /* fall through */
            case 'd':
                m *= 24;
                /* fall through */
            case 'h':
                m *= 60;
                /* fall through */
            case 'm':
                m *= 60;
                /* fall through */
            case 's':
                n += m;
                p++;
                rel = 1;
                break;
            default:
                synbad("bad time syntax, ", pin);
        }
    }
    if (!rel)
        m = n;
    else{
        m = time(0);
        if (n > m) /* before epoch? */
            m = 0;
        else
            m -= n;
    }
}

```

```

    r = dir->mtime < m;
    free(dir);
    return r;
}

⟨function isolderthan 181a⟩≡ (191f)
int
isolderthan(char *a, char *b)
{
    int r;
    Dir *ad, *bd;

    ad = dirstat(a);
    bd = dirstat(b);
    if (ad == nil || bd == nil)
        r = 0;
    else
        r = ad->mtime > bd->mtime;
    free(ad);
    free(bd);
    return r;
}

⟨function isnewerthan 181b⟩≡ (191f)
int
isnewerthan(char *a, char *b)
{
    int r;
    Dir *ad, *bd;

    ad = dirstat(a);
    bd = dirstat(b);
    if (ad == nil || bd == nil)
        r = 0;
    else
        r = ad->mtime < bd->mtime;
    free(ad);
    free(bd);
    return r;
}

```

## E.4 sh, the simple shell

This is a minimal shell, useful during early stages of porting Plan 9 to a new architecture when `rc` is not yet available. It supports basic command execution, pipes, redirections, and backgrounding, but has no variables, functions, or control flow. Comparing it to `rc` helps appreciate how much complexity the full shell adds.

```

⟨constant MAXLINE 181c⟩≡ (192)
#define MAXLINE 200 /* maximum line length */

⟨constant WORD 181d⟩≡ (192)
#define WORD 256 /* token code for words */

⟨constant EOF 181e⟩≡ (192)
#define EOF -1 /* token code for end of file */

⟨function ispunct 181f⟩≡ (192)
#define ispunct(c) (c=='|' || c=='&' || c==',' || c=='<' || \
    c=='>' || c=='(' || c==')' || c=='\n')

```

```

⟨function isspace 182a⟩≡ (192)
    #define isspace(c) (c==' ' || c=='\t')

⟨function execute 182b⟩≡ (192)
    #define execute(np) (ignored = (np? (*(np)->op)(np) : 0))

⟨struct Node 182c⟩≡ (192)
    struct Node { /* parse tree node */
        int (*op)(Node *); /* operator function */
        Node *args[2]; /* argument nodes */
        char *argv[100]; /* argument pointers */
        char *io[3]; /* i/o redirection */
    };

⟨global nodes 182d⟩≡ (192)
    Node nodes[25]; /* node pool */

⟨global nfree 182e⟩≡ (192)
    Node *nfree; /* next available node */

⟨global strspace 182f⟩≡ (192)
    char strspace[10*MAXLINE]; /* string storage */

⟨global sfree 182g⟩≡ (192)
    char *sfree; /* next free character in strspace */

⟨global t 182h⟩≡ (192)
    int t; /* current token code */

⟨global token 182i⟩≡ (192)
    char *token; /* current token text (in strspace) */

⟨global putback 182j⟩≡ (192)
    int putback = 0; /* lookahead */

⟨global status 182k⟩≡ (192)
    char status[256]; /* exit status of most recent command */

⟨global cflag 182l⟩≡ (192)
    int cflag = 0; /* command is argument to sh */

⟨global tflag 182m⟩≡ (192)
    int tflag = 0; /* read only one line */

⟨global interactive 182n⟩≡ (192)
    bool interactive = false; /* prompt */

⟨global cflagp 182o⟩≡ (192)
    char *cflagp; /* command line for cflag */

⟨global path 182p⟩≡ (192)
    char *path[] = {"/bin", 0};

⟨global ignored 182q⟩≡ (192)
    int ignored;

```

*<function main (sh/sh.c) 183a>*≡ (192)

```
void
main(int argc, char *argv[])
{
    Node *np;

    if(argc>1 && strcmp(argv[1], "-t")==0)
        tflag++;
    else if(argc>2 && strcmp(argv[1], "-c")==0){
        cflag++;
        cflagp = argv[2];
    }else if(argc>1){
        close(0);
        if(open(argv[1], 0) != 0){
            error(": can't open", argv[1]);
            exits("argument");
        }
    }else
        interactive = true;
    for(;;){
        if(interactive)
            fprintf(2, "%d$ ", getpid());
        nfree = nodes;
        sfree = strspace;
        if((t=gettoken()) == EOF)
            break;
        if(t != '\n')
            if(np = list())
                execute(np);
            else
                error("syntax error", "");
        while(t!=EOF && t!='\n') /* flush syntax errors */
            t = gettoken();
    }
    exits(status);
}
```

*<function alloc 183b>*≡ (192)

```
/* alloc - allocate for op and return a node */
Node*
alloc(int (*op)(Node *))
{
    if(nfree < nodes+sizeof(nodes)){
        nfree->op = op;
        nfree->args[0] = nfree->args[1] = 0;
        nfree->argv[0] = nfree->argv[1] = 0;
        nfree->io[0] = nfree->io[1] = nfree->io[2] = 0;
        return nfree++;
    }
    error("node storage overflow", "");
    exits("node storage overflow");
    return nil;
}
```

*<function builtin 183c>*≡ (192)

```
/* builtin - check np for builtin command and, if found, execute it */
bool
builtin(Node *np)
{
    int n = 0;
```

```

char name[MAXLINE];
Waitmsg *wmsg;

if(np->argv[1])
    n = strtoul(np->argv[1], 0, 0);
if(strcmp(np->argv[0], "cd") == 0){
    if(chdir(np->argv[1]? np->argv[1] : "/") == -1)
        error(": bad directory", np->argv[0]);
    return true;
}else if(strcmp(np->argv[0], "exit") == 0)
    exits(np->argv[1]? np->argv[1] : status);
else if(strcmp(np->argv[0], "bind") == 0){
    if(np->argv[1]==0 || np->argv[2]==0)
        error("usage: bind new old", "");
    else if(bind(np->argv[1], np->argv[2], 0)==-1)
        error("bind failed", "");
    return true;
}

#ifdef asdf
// }else if(strcmp(np->argv[0], "unmount") == 0){
//     if(np->argv[1] == 0)
//         error("usage: unmount [new] old", "");
//     else if(np->argv[2] == 0){
//         if(unmount((char *)0, np->argv[1]) == -1)
//             error("unmount:", "");
//     }else if(unmount(np->argv[1], np->argv[2]) == -1)
//         error("unmount", "");
//     return true;
// }
#endif

}else if(strcmp(np->argv[0], "wait") == 0){
    while((wmsg = wait()) != nil){
        strncpy(status, wmsg->msg, sizeof(status)-1);
        if(n && wmsg->pid==n){
            n = 0;
            free(wmsg);
            break;
        }
        free(wmsg);
    }
    if(n)
        error("wait error", "");
    return true;
}else if(strcmp(np->argv[0], "rfork") == 0){
    char *p;
    int mask;

    p = np->argv[1];
    if(p == 0 || *p == 0)
        p = "ens";
    mask = 0;

    while(*p)
        switch(*p++){
            case 'n': mask |= RFNAMEG; break;
            case 'N': mask |= RFCNAMEG; break;
            case 'e': mask |= RFENVG; break;
            case 'E': mask |= RFCENVG; break;
            case 's': mask |= RFNOTEG; break;
            case 'f': mask |= RFFDGD; break;
            case 'F': mask |= RFCFDGD; break;
            case 'm': mask |= RFNOMNT; break;

```

```

        default: error(np->argv[1], "bad rfork flag");
    }
    rfork(mask);

    return OK_1;
}else if(strcmp(np->argv[0], "exec") == 0){
    redirect(np);
    if(np->argv[1] == (char *) 0)
        return 1;
    exec(np->argv[1], &np->argv[1]);
    n = np->argv[1][0];
    if(n!='/' && n!='#' && (n!='.' || np->argv[1][1]!='/'))
        for(n = 0; path[n]; n++){
            sprintf(name, "%s/%s", path[n], np->argv[1]);
            exec(name, &np->argv[1]);
        }
    error(": not found", np->argv[1]);
    return true;
}
// no builtin found
return false;
}

```

⟨function command 185a⟩≡ (192)

```

/* command - ( list ) [ ( < | > | >> ) word ]* | simple */
Node*
command(void)
{
    Node *np;

    if(t != '(')
        return simple();
    np = alloc(xsubshell);
    t = gettoken();
    if((np->args[0]=list())==0 || t!='(')
        return nil;
    while((t=gettoken())=='<' || t=='>')
        if(!setio(np))
            return nil;
    return np;
}

```

⟨function getch 185b⟩≡ (192)

```

/* getch - get next, possibly pushed back, input character */
int
getch(void)
{
    unsigned char c;
    static int done=0;

    if(putback){
        c = putback;
        putback = 0;
    }else if(tflag){
        if(done || read(0, &c, 1)!=1){
            done = 1;
            return EOF;
        }
        if(c == '\n')
            done = 1;
    }
}

```

```

}else if(cflag){
    if(done)
        return EOF;
    if((c=*cflagp++) == 0){
        done = 1;
        c = '\n';
    }
}
}else if(read(0, &c, 1) != 1)
    return EOF;
return c;
}

```

⟨function gettoken 186a⟩≡ (192)

```

/* gettoken - get next token into string space, return token code */
int
gettoken(void)
{
    int c;

    while((c = getch()) != EOF)
        if(!isspace(c))
            break;
    if(c==EOF || ispunct(c))
        return c;
    token = sfree;
    do{
        if(sfree >= strspace+sizeof(strspace) - 1){
            error("string storage overflow", "");
            exits("string storage overflow");
        }
        *sfree++ = c;
    }while((c=getch()) != EOF && !ispunct(c) && !isspace(c));
    *sfree++ = 0;
    putback = c;
    return WORD;
}

```

⟨function list 186b⟩≡ (192)

```

/* list - pipeline ( ( ; | & ) pipeline ) * [ ; | & ] (not LL(1), but ok) */
Node*
list(void)
{
    Node *np, *np1;

    np = alloc(0);
    if((np->args[1]=pipeline()) == 0)
        return nil;
    while(t==';' || t=='&'){
        np->op = (t==';')? xwait : xnowait;
        t = gettoken();
        if(t==';' || t=='\n') /* tests ~first(pipeline) */
            break;
        np1 = alloc(0);
        np1->args[0] = np;
        if((np1->args[1]=pipeline()) == 0)
            return nil;
        np = np1;
    }
    if(np->op == 0)
        np->op = xwait;
}

```

```

    return np;
}

⟨function error 187a⟩≡ (192)
/* error - print error message s, prefixed by t */
void
error(char *s, char *t)
{
    char buf[256];

    fprintf(2, "%s%s", t, s);
    errstr(buf, sizeof buf);
    fprintf(2, ": %s\n", buf);
}

⟨function pipeline 187b⟩≡ (192)
/* pipeline - command ( | command )* */
Node*
pipeline(void)
{
    Node *np, *np1;

    if((np=command()) == 0)
        return nil;
    while(t == '|'){
        np1 = alloc(xpipeline);
        np1->args[0] = np;
        t = gettoken();
        if((np1->args[1]=command()) == 0)
            return nil;
        np = np1;
    }
    return np;
}

⟨function redirect 187c⟩≡ (192)
/* redirect - redirect i/o according to np->io[] values */
void
redirect(Node *np)
{
    int fd;

    if(np->io[0]){
        if((fd = open(np->io[0], 0)) < 0){
            error(": can't open", np->io[0]);
            exits("open");
        }
        dup(fd, 0);
        close(fd);
    }
    if(np->io[1]){
        if((fd = create(np->io[1], 1, 0666L)) < 0){
            error(": can't create", np->io[1]);
            exits("create");
        }
        dup(fd, 1);
        close(fd);
    }
    if(np->io[2]){
        if((fd = open(np->io[2], 1)) < 0 && (fd = create(np->io[2], 1, 0666L)) < 0){

```

```

        error(": can't write", np->io[2]);
        exits("write");
    }
    dup(fd, 1);
    close(fd);
    seek(1, 0, 2);
}
}

⟨function setio 188a⟩≡ (192)
/* setio - ( < | > | >> ) word; fill in np->io[] */
error0
setio(Node *np)
{
    if(t == '<'){
        t = gettoken();
        np->io[0] = token;
    }else if(t == '>'){
        t = gettoken();
        if(t == '>'){
            t = gettoken();
            np->io[2] = token;
        }else
            np->io[1] = token;
    }else
        return ERROR_0;
    if(t != WORD)
        return ERROR_0;
    return OK_1;
}

⟨function simple 188b⟩≡ (192)
/* simple - word ( [ < | > | >> ] word ) * */
Node*
simple(void)
{
    Node *np;
    int n = 1;

    if(t != WORD)
        return nil;
    np = alloc(xsimple);
    np->argv[0] = token;
    while((t = gettoken())==WORD || t=='<' || t=='>')
        if(t == WORD)
            np->argv[n++] = token;
        else if(!setio(np))
            return nil;
    np->argv[n] = 0;
    return np;
}

⟨function xpipeline 188c⟩≡ (192)
/* xpipeline - execute cmd | cmd */
int
xpipeline(Node *np)
{
    int pid, fd[2];

    if(pipe(fd) < 0){

```

```

        error("can't create pipe", "");
        return 0;
    }
    if((pid=fork()) == 0){ /* left side; redirect stdout */
        dup(fd[1], 1);
        close(fd[0]);
        close(fd[1]);
        execute(np->args[0]);
        exits(status);
    }else if(pid == -1){
        error("can't create process", "");
        return 0;
    }
    if((pid=fork()) == 0){ /* right side; redirect stdin */
        dup(fd[0], 0);
        close(fd[0]);
        close(fd[1]);
        pid = execute(np->args[1]); /*BUG: this is wrong sometimes*/
        if(pid > 0)
            while(waitpid()!=pid)
                ;
        exits(nil);
    }else if(pid == -1){
        error("can't create process", "");
        return 0;
    }
    close(fd[0]); /* avoid using up fd's */
    close(fd[1]);
    return pid;
}

```

*<function xsimple 189>*≡ (192)

```

/* xsimple - execute a simple command */
int
xsimple(Node *np)
{
    char name[MAXLINE];
    int pid, i;

    if(builtin(np))
        return 0;

    if(pid = fork()){
        if(pid == -1)
            error(": can't create process", np->argv[0]);
        return pid;
    }
    redirect(np); /* child process */
    exec(np->argv[0], &np->argv[0]);
    i = np->argv[0][0];

    if(i!='/' && i!='#' && (i!='.' || np->argv[0][1]!='/'))
        for(i = 0; path[i]; i++){
            sprintf(name, "%s/%s", path[i], np->argv[0]);
            exec(name, &np->argv[0]);
        }
    error(": not found", np->argv[0]);
    exits("not found");
    return -1; // suppress compiler warnings
}

```

```

⟨function xsubshell 190a⟩≡ (192)
/* xsubshell - execute (cmd) */
int
xsubshell(Node *np)
{
    int pid;

    if(pid = fork()){
        if(pid == -1)
            error("can't create process", "");
        return pid;
    }
    redirect(np); /* child process */
    execute(np->args[0]);
    exits(status);
    return -1; // suppress compiler warnings
}

```

```

⟨function xnowait 190b⟩≡ (192)
/* xnowait - execute cmd & */
int
xnowait(Node *np)
{
    int pid;

    execute(np->args[0]);
    pid = execute(np->args[1]);
    if(interactive)
        fprintf(2, "%d\n", pid);
    return 0;
}

```

```

⟨function xwait 190c⟩≡ (192)
/* xwait - execute cmd ; */
int xwait(Node *np)
{
    int pid;
    Waitmsg *wmsg;

    execute(np->args[0]);
    pid = execute(np->args[1]);
    if(pid > 0){
        while((wmsg = wait()) != nil){
            if(wmsg->pid == pid)
                break;
            free(wmsg);
        }
        if(wmsg == nil)
            error("wait error", "");
        else {
            strncpy(status, wmsg->msg, sizeof(status)-1);
            free(wmsg);
        }
    }
    return 0;
}

```

# Appendix F

## Extra Code

### F.1 misc/

*<rc/tests/ls\_root.c 191a>*≡

*<rc/y.tab.h 191b>*≡

*<rc/y.tab.c 191c>*≡

#### F.1.1 misc/echo.c

*<misc/echo.c 191d>*≡

`#include <u.h>`

`#include <libc.h>`

*<function main(echo) 173>*

#### F.1.2 misc/pwd.c

*<misc/pwd.c 191e>*≡

`#include <u.h>`

`#include <libc.h>`

*<function main (misc/pwd.c) 174a>*

#### F.1.3 misc/test.c

*<misc/test.c 191f>*≡

`/*`

`* POSIX standard`

`* test expression`

`* [ expression ]`

`*`

`* Plan 9 additions:`

`* -A file exists and is append-only`

`* -L file exists and is exclusive-use`

`* -T file exists and is temporary`

`*/`

`#include <u.h>`

`#include <libc.h>`

*<function EQ 174b>*

*<global ap 174c>*

*<global ac 174d>*

*<global av 174e>*

*<global tmp 174f>*

```
void synbad(char *, char *);
int fsizep(char *);
int isdir(char *);
int isreg(char *);
int isatty(int);
int isint(char *, int *);
int isolder(char *, char *);
int isolderthan(char *, char *);
int isnewerthan(char *, char *);
int hasmode(char *, ulong);
int tio(char *, int);
int e(void), e1(void), e2(void), e3(void);
char *nxtarg(int);
```

*<function main (misc/test.c) 175a>*

*<function nxtarg 175b>*

*<function nxtintarg 175c>*

*<function e 175d>*

*<function e1 176a>*

*<function e2 176b>*

*<function e3 176c>*

*<function tio 178a>*

*<function hasmode 178b>*

*<function isdir 178c>*

*<function isreg 178d>*

*<function isatty 179a>*

*<function fsizep 179b>*

*<function synbad 179c>*

*<function isint 179d>*

*<function isolder 180>*

*<function isolderthan 181a>*

*<function isnewerthan 181b>*

## F.2 sh/

### F.2.1 sh/sh.c

*<sh/sh.c 192>*≡

```
/* sh - simple shell - great for early stages of porting */
```

```

#include <u.h>
#include <libc.h>

<constant MAXLINE 181c>
<constant WORD 181d>
<constant EOF 181e>
<function ispunct 181f>
<function isspace 182a>
<function execute 182b>

typedef struct Node Node;
<struct Node 182c>

<global nodes 182d>
<global nfree 182e>
<global strspace 182f>
<global sfree 182g>
<global t 182h>
<global token 182i>
<global putback 182j>
<global status 182k>
<global cflag 182l>
<global tflag 182m>
<global interactive 182n>
<global cflagp 182o>
<global path 182p>
<global ignored 182q>

// lexer
int gettoken(void);
int getch(void);
// parser (recursive descent)
Node *list(void);
Node *command(void);
Node *simple(void);
Node *pipeline(void);
int setio(Node *np);
// interpreter
// see execute macro above
int builtin(Node *np);

void redirect(Node *np);

int xpipeline(Node *np);
int xsimple(Node *np);
int xsubshell(Node *np);
int xnowait(Node *np);
int xwait(Node *np);

// error management
void error(char *s, char *t);
// memory management
Node *alloc(int (*op)(Node *));

<function main (sh/sh.c) 183a>

<function alloc 183b>

<function builtin 183c>

```

<function command 185a>  
 <function getch 185b>  
 <function gettoken 186a>  
 <function list 186b>  
 <function error 187a>  
 <function pipeline 187b>  
 <function redirect 187c>  
 <function setio 188a>  
 <function simple 188b>  
 <function xpipeline 188c>  
 <function xsimple 189>  
 <function xsubshell 190a>  
 <function xnowait 190b>  
 <function xwait 190c>

## F.3 rc/

### F.3.1 rc/fns.h

<rc/fns.h 194>≡

```

// for var.c, words.c, tree.c: see rc.h
// see also io.h and getflags.h

// input.c
int getnext(void);
int advance(void);
int nextc(void);
bool nextis(int c);
void pprompt(void);
void kinit(void);
tree *token(char*, int);
tree *klook(char*);

// lex.c
int yylex(void);
/*@Scheck: used in syn.y
void skipnl(void);
int idchr(int);

// syn.y
/*@Scheck: defined in syn.y and y.tab.c
int yyparse(void);

```

```

// executils.c
void start(code*, int, var*);
void pushlist(void);
void poplist(void);
void pushword(char*);
void popword(void);
void pushredir(int, int, int);

// status.c
char *getstatus(void);
void setstatus(char*);
char* concstatus(char *s, char *t);
bool truestatus(void);

// path.c
word* searchpath(char*);

// env.c
void Updenv(void);
void Vinit(void);

// processes.c
int Waitfor(int, bool);
void Execute(word*, word*);

// code.c
error0 compile(tree*);
code *codecopy(code*);
void codefree(code*);
void cleanhere(char*);

// trap.c
void dotrap(void);
void Trapinit(void);
bool Eintr(void);
void Noerror(void);

// here.c
void readhere(void);
tree *heredoc(tree*);

// glob.c
void deglob(void*);
void globlist(void);
bool match(void*, void*, int);

// utils.c
#define new(type) ((type *)emalloc(sizeof(type)))
void *emalloc(long);
void efree(void *);
void Memcpy(void*, void*, long);
int Creat(char*);
int Opendir(char*);
int Readdir(int, void*, int);
void Closedir(int);
void intoascii(char*, long);

// error.c
void panic(char*, int);
void yyerror(char*);

```

```
void Exit(char*, char*);
```

### F.3.2 rc/getflags.h

```
<rc/getflags.h 196a>≡  
  <constant NFLAG 36d>  
  
extern char **flag[NFLAG];  
extern char *flagset[];  
  
int getflags(int, char*[], char*, int);  
void usage(char*);
```

### F.3.3 rc/io.h

```
<rc/io.h 196b>≡  
  <constant EOF (rc/io.h) 44a>  
  <constant NBUF 163c>  
  
  <struct Io 163d>  
  
extern io *err;  
  
// io.c  
io *openfd(fdt);  
io *openstr(void);  
io *opencore(char *, int);  
void closeio(io*);  
void flush(io*);  
  
int rchr(io*);  
int rutf(io*, char*, Rune*);  
  
// fmt.c  
void pchr(io*, int);  
void pstr(io*, char*);  
void pfmt(io*, char*, ...);  
  
// pcmd.c  
void pcmd(io*, tree*);  
// pfnc.c  
void pfnc(io*, thread*);
```

### F.3.4 rc/rc.h

```
<rc/rc.h 196c>≡  
/*  
 * Assume plan 9 by default; if Unix is defined, assume unix.  
 * Please don't litter the code with ifdefs. The five below should be enough.  
 */  
#ifndef Unix  
/* plan 9 */  
#include <u.h>  
#include <libc.h>  
  
// could be in trap.c  
<constant NSIG 113a>
```

```

<constant SIGINT 113b>
<constant SIGQUIT 113c>

//???
//#define fcntl(fd, op, arg) /* unix compatibility */
//#define F_SETFD
//#define FD_CLOEXEC

//TODO:
#define __LOC__ "NO__LOC__INFO"

#else
#include "unix.h"

// magic incantation for cpp (found by chatGPT)
#define STR2(x) #x
#define STR(x) STR2(x)
#define __LOC__ __FILE__ ":" STR(__LINE__)

#endif

//#ifndef YYPREFIX
//#ifndef PAREN
//#include "x.tab.h"
//pad: better like that, otherwise get some "redefined FOR macro" error
//#endif
//#endif

//#ifndef Unix
//#pragma incomplete word
//#pragma incomplete io
//#endif

// MiscPlan9 is back in plan9.c

#ifndef ERRMAX
<constant ERRMAX 156a>
#endif

<constant YYMAXDEPTH ??>

// forward decls
typedef struct Tree tree;
typedef struct Word word;
typedef struct Io io;
typedef union Code code;
typedef struct Var var;
typedef struct List list;
typedef struct Redir redir;
typedef struct Thread thread;
typedef struct Builtin builtin;

<struct Tree 27a>

// tree.c
tree *newtree(void);
//@Scheck: useful, for syn.y, and not just for tree.c
tree *tree1(int, tree*);
tree *tree2(int, tree*, tree*);

```

```

//@Scheck: useful, for syn.y, and not just for tree.c
tree *tree3(int, tree*, tree*, tree*);
tree *mung1(tree*, tree*);
tree *mung2(tree*, tree*, tree*);
tree *mung3(tree*, tree*, tree*, tree*);
tree *epimung(tree*, tree*);
tree *simplemung(tree*);
void freenodes(void);

⟨struct Code 29c⟩

⟨constant APPEND 52d⟩
⟨constant WRITE 52b⟩
⟨constant READ 52c⟩
⟨constant HERE 131a⟩
⟨constant DUPFD 137c⟩
⟨constant CLOSE 136b⟩
⟨constant RDWR 135a⟩

⟨struct Word 31c⟩

// words.c
word *newword(char *, word *);
word *copywords(word *, word *);
word* copynwords(word *a, word *tail, int n);
void freelist(word *w);
void freewords(word*);
int count(word*);

⟨struct Var 33c⟩

// var.c
var *vlook(char*);
var *gvlook(char*);
var *newvar(char*, var*);
void setvar(char*, word*);

⟨constant NVAR 34c⟩
extern var *gvar[NVAR]; /* hash for globals */

⟨struct Here 131c⟩

⟨constant GLOB 119a⟩

⟨constant PRD 84e⟩
⟨constant PWR 85a⟩

// globals.c
extern int mypid;
extern char *promptstr;
extern int ndot;
// input.c
extern bool doprompt;
extern bool inquote;
extern bool incomm;
extern int lastc;
// lex.c
extern bool lastword;
// error.c
extern int nerror; /* number of errors encountered during compilation */

```

Uses Builtin 102a, Code 29c, Io 163d, List 33b, Redir 78c, Thread 30b, Tree, Var 33c, and Word 31c.

### F.3.5 rc/exec.h

```
<rc/exec.h 199a>≡
/*
 * Definitions used in the interpreter
 */
extern void Xappend(void), Xasync(void), Xbackq(void), Xbang(void), Xclose(void);
extern void Xconc(void), Xcount(void), Xdelfn(void), Xdol(void), Xqdol(void), Xdup(void);
extern void Xexit(void), Xfalse(void), Xfn(void), Xfor(void), Xglob(void);
extern void Xjump(void), Xmark(void), Xmatch(void), Xpipe(void), Xread(void);
extern void Xrdwr(void);
extern void Xrdfn(void), Xreturn(void), Xsubshell(void);
extern void Xtrue(void), Xword(void), Xwrite(void), Xpipefd(void), Xcase(void);
extern void Xlocal(void), Xunlocal(void), Xassign(void), Xsimple(void), Xpopm(void);
extern void Xrdcmds(void), Xwastrue(void), Xif(void), Xifnot(void), Xpipewait(void);
extern void Xdelhere(void), Xpopredir(void), Xsub(void), Xeflag(void), Xsettrue(void);
extern void Xerror(char*);
extern void Xerror1(char*);

<struct Redir 78c>

<constant NSTATUS 85e>
<constant ROPEN 80a>
<constant RDUP 137g>
<constant RCLOSE 137h>

<struct List 33b>

<struct Thread 30b>

<struct Builtin 102a>

// globals.c
extern thread *runq;
extern code *codebuf; /* compiler output */
extern bool eflagok; /* kludge flag so that -e doesn't exit in startup */
// path.c
extern word nullpath;
// trap.c
extern int ntrap; /* number of outstanding traps */
// builtins.c
extern struct Builtin builtins[];

// simple.c
void execexec(void);
void execcmds(io *);
// processes.c
int execforkexec(void);
```

### F.3.6 rc/globals.c

```
<rc/globals.c 199b>≡
<includes 21>

// was in rc.h
<global mypid 118d>
```

```

⟨global promptstr 45d⟩
⟨global ndot 107a⟩

// was in exec.h
⟨global runq 30c⟩
⟨global codebuf 29b⟩
⟨global eflagok 128c⟩

```

### F.3.7 rc/getflags.c

```

⟨rc/getflags.c 200a⟩≡
#include "getflags.h"

extern void Exit(char*, char*);

static void reverse(char**, char**);
static int scanflag(int, char*);
static void errn(char*, int);
static void errs(char*);
static void errc(int);

⟨global flagset 37a⟩
⟨global flag 36c⟩
⟨global cmdname 159a⟩
⟨global flagarg 159b⟩
⟨global reason 159c⟩

⟨constant RESET 159d⟩
⟨constant FEWARGS 159e⟩
⟨constant FLAGSYN 159f⟩
⟨constant BADFLAG 159g⟩

⟨global badflag 159h⟩

⟨function getflags 159i⟩

⟨function reverse 160a⟩

⟨function scanflag 160b⟩

⟨function usage 161⟩

⟨function errn 162a⟩

⟨function errs 162b⟩
⟨constant NBUF (rc/getflags.c) 162c⟩
⟨global buf 162d⟩
⟨global bufp 163a⟩

⟨function errc 163b⟩
Uses buf-25 and bufp-26 200a.

```

### F.3.8 rc/io.c

```

⟨rc/io.c 200b⟩≡
⟨includes 21⟩

```

*<enum MiscConstants 163f>*

```
// forward decls
int emptybuf(io*);
int fullbuf(io*, int);
```

*<function rchr 164c>*

*<function rutf 164e>*

*<function fullbuf 164b>*

*<function flush 163g>*

*<function openfd 163e>*

*<function openstr 165b>*

*<function opencore 165c>*

*<function closeio 165a>*

*<function emptybuf 164d>*

### F.3.9 rc/input.c

*<rc/input.c 201>*≡

*<includes 21>*

```
#include "y.tab.h"
```

```
// was in lex.c
```

*<global future 46d>*

*<global doprompt 45c>*

*<global inquote 53c>*

*<global incomm 49e>*

```
// was in rc.h
```

*<global lastc 157a>*

```
// forward decl
```

```
int getnext(void);
```

*<function nextc 46e>*

*<function advance 47a>*

*<function getnext 43>*

*<function pprompt 45h>*

*<function nextis 47b>*

```
extern unsigned hash(char *as, int n);
```

```
// was in tree.c
```

*<function token 54b>*

```
// was in var.c

⟨constant NKW 55d⟩
⟨struct Kw 55b⟩
⟨global kw 55c⟩

⟨function kenter 55f⟩

⟨function kinit 55e⟩

⟨function klook 55a⟩
```

### F.3.10 rc/var.c

```
⟨rc/var.c 202a⟩≡
⟨includes 21⟩

// was in rc.h
⟨global gvar 34d⟩

⟨function hash 35a⟩

⟨function gvlook 34e⟩

⟨function vlook 34b⟩

⟨function setvar 34a⟩

⟨function newvar 35b⟩
```

### F.3.11 rc/env.c

```
⟨rc/env.c 202b⟩≡
⟨includes 21⟩

// the Vxxx are back in plan9.c
```

### F.3.12 rc/glob.c

```
⟨rc/glob.c 202c⟩≡
⟨includes 21⟩

bool matchfn(void*, void*);

// NDIR and Globsize are back in plan9.c
extern int Globsize(char *p);

⟨global globname 121b⟩
⟨global globv 120b⟩
⟨function deglob 122a⟩

⟨function globcmp 122b⟩

⟨function globsort 122c⟩
⟨function globdir 122d⟩
⟨function glob 121c⟩
```

<function equtf 123a>  
 <function nextutf 123b>  
 <function unicode 123c>  
 <function matchfn 124a>  
 <function match 124b>  
 <function globlist1 120e>  
 <function globlist 120d>

### F.3.13 rc/executils.c

<rc/executils.c 203a>≡  
 <includes 21>  
  
 <global argv0 39b>  
  
 <function start 39a>  
  
  
 <function pushword 40b>  
 <function popword 66d>  
 <function pushlist 40a>  
 <function poplist 66c>  
  
  
 <function pushredir 79c>  
  
  
 <function Xerror 71b>  
 <function Xerror1 71c>  
  
  
 <function turfredir 83a>  
 <function Xpopredir 81a>  
 <function Xreturn 84d>

### F.3.14 rc/exec.c

<rc/exec.c 203b>≡  
 <includes 21>  
  
 /\*  
 \* Opcode routines  
 \* Arguments on stack (...)  
 \* Arguments in line [...]  
 \* Code in line with jump around {...}

```

*
* Xappend(file)[fd]    open file to append
* Xassign(name, val)   assign val to name
* Xasync{... Xexit}    make thread for {}, no wait
* Xbackq{... Xreturn}  make thread for {}, push stdout
* Xbang    complement condition
* Xcase(pat, value){...} exec code on match, leave (value) on
*      stack
* Xclose[i]    close file descriptor
* Xconc(left, right) concatenate, push results
* Xcount(name) push var count
* Xdelfn(name) delete function definition
* Xdelhere
* Xdol(name)   get variable value
* Xdup[i j]    dup file descriptor
* Xeflag
* Xerror
* Xexit    rc exits with status
* Xfalse{...} execute {} if false
* Xfn(name){... Xreturn} define function
* Xfor(var, list){... Xreturn} for loop
* Xglob
* Xif
* Xifnot
* Xjump[addr] goto
* Xlocal(name, val) create local variable, assign value
* Xmark    mark stack
* Xmatch(pat, str) match pattern, set status
* Xpipe[i j]{... Xreturn}{... Xreturn} construct a pipe between 2 new threads,
*      wait for both
* Xpipefd[type]{... Xreturn} connect {} to pipe (input or output,
*      depending on type), push /dev/fd/??
* Xpipewait
* Xpopm(value) pop value from stack
* Xpopredir
* Xrdcmds
* Xrdfs
* Xrdwr(file)[fd] open file for reading and writing
* Xread(file)[fd] open file to read
* Xqdol(name) concatenate variable components
* Xreturn    kill thread
* Xsimple(args) run command and wait
* Xsub
* Xsubshell{... Xexit} execute {} in a subshell and wait
* Xtrue{...} execute {} if true
* Xunlocal delete local variable
* Xwastrue
* Xword[string] push string
* Xwrite(file)[fd] open file to write
*/

```

*<function Xappend 82b>*

*<function Xsettrue 89f>*

*<function Xbang 77b>*

*<function Xclose 137i>*

*<function Xdup 137e>*

*<function Xeflag 129a>*  
*<function Xexit 87>*  
*<function Xfalse 76f>*  
*<global ifnot 88c>*  
*<function Xifnot 88f>*  
*<function Xjump 89g>*  
*<function Xmark 66b>*  
*<function Xpopm 92a>*  
*<function Xread 81c>*  
*<function Xrdwr 135d>*  
*<function Xtrue 76e>*  
*<function Xif 88b>*  
*<function Xwastrue 88d>*  
*<function Xword 66a>*  
*<function Xwrite 80d>*  
*<function list2str 77e>*  
*<function Xmatch 77d>*  
*<function Xcase 92c>*  
*<function conclist 100c>*  
*<function Xconc 100b>*  
*<function Xassign 96c>*  
*<function Xdol 97d>*  
*<function Xqdol 141c>*  
*<function subwords 99c>*  
*<function Xsub 99b>*  
*<function Xcount 98b>*  
*<function Xlocal 97a>*  
*<function Xunlocal 97b>*  
*<function Xfn 94c>*  
*<function Xdelfn 95a>*

```

⟨function Xpipewait 85f⟩
⟨function Xrdcmds 41a⟩
⟨function Xdelhere 132c⟩
⟨function Xfor 90b⟩
⟨function Xglob 121a⟩
⟨global envdir 112⟩

// Xrdfs is back in plan9.c

```

### F.3.15 rc/processes.c

```

⟨rc/processes.c 206a⟩≡
  ⟨includes 21⟩
  // #include <string.h>

  // Fdprefix, delwaitpid(), havewaitpid() Waitfor() are back in plan9.c
  extern char *Fdprefix;

  ⟨global waitpids 68b⟩
  ⟨global nwaitpids 68c⟩
  ⟨function addwaitpid 68d⟩
  ⟨function clearwaitpids 68g⟩

  ⟨function mkargv 74a⟩
  ⟨function Execute 73c⟩

  // was in havefork.c

  ⟨function Xasync 86d⟩
  ⟨function Xpipe 85b⟩
  ⟨function Xbackq 140⟩
  ⟨function Xpipefd 138h⟩
  ⟨function Xsubshell 130d⟩
  ⟨function execforkexec 72a⟩

```

### F.3.16 rc/tree.c

```

⟨rc/tree.c 206b⟩≡
  ⟨includes 21⟩
  #include "y.tab.h"

  ⟨global treenodes 27c⟩
  ⟨function newtree 27e⟩

  ⟨function freenodes 28a⟩

```

<function tree1 28b>  
 <function tree2 28c>  
 <function tree3 28d>  
 <function mung1 60e>  
 <function mung2 61b>  
 <function mung3 60f>  
 <function epimung 62a>  
 <function simplemung 59d>

### F.3.17 rc/lex.c

<rc/lex.c 207a>≡  
 <includes 21>  
 #include "y.tab.h"  
  
 <constant NTOK 49d>  
  
 // was used by subr.c  
 <global lastdol 51b>  
 <global lastword 53e>  
 // was in rc.h  
 <global tok 49c>  
  
 <function wordchr 48b>  
  
 <function idchr 49a>  
  
  
 <function yyerror 156e>  
  
  
 <function skipwhite 49f>  
  
 <function skipnl 50a>  
  
 <function addtok 120a>  
  
 <function addutf 129b>  
  
 <function yylex 48a>

### F.3.18 rc/trap.c

<rc/trap.c 207b>≡  
 <includes 21>  
  
 // NSIG, SIGINT, SIGQUIT are back in rc.h (under an ifdef for plan9)  
  
 // was in exec.h  
 <global ntrap 114b>  
 <global trap 114c>

```
// signame, syssigname, interrputed are back in plan9.c
// notifyf(), Trapinit(), Eintr(), Noerror() are back in plan9.c
extern char *signame[];

// generic part independent of plan9
⟨function dotrap 115b⟩
```

### F.3.19 rc/simple.c

```
⟨rc/simple.c 208a⟩≡
/*
 * Maybe 'simple' is a misnomer.
 */
⟨includes 21⟩

// forward decls
void execfunc(var*);

⟨function exitnext 75b⟩

⟨function Xsimple 70⟩

⟨function doredir 79d⟩

⟨function execexec 72b⟩

⟨function execfunc 95d⟩

⟨global rdcmds 108a⟩

⟨function execcmds 108b⟩
```

### F.3.20 rc/pcmd.c

```
⟨rc/pcmd.c 208b⟩≡
⟨includes 21⟩
#include "y.tab.h"

⟨global nl 151a⟩
⟨constant c0 151b⟩
⟨constant c1 151c⟩
⟨constant c2 151d⟩

⟨function pdeglob 153a⟩

⟨function pcmd 151e⟩
```

### F.3.21 rc/here.c

```
⟨rc/here.c 208c⟩≡
⟨includes 21⟩
#include "y.tab.h"

⟨global here 131d⟩
⟨global ehere 131e⟩
⟨global ser 133b⟩
⟨global tmp (rc/here.c) 133c⟩
```

*<global hex 133d>*

```
void psubst(io*, uchar*);  
void pstrs(io*, word*);
```

*<function hexnum 133e>*

*<function heredoc 133f>*

*<constant NLINE 131f>*

*<function readhere 132a>*

*<function psubst 134a>*

*<function pstrs 134b>*

### F.3.22 rc/code.c

*<rc/code.c 209>*≡

*<includes 21>*

```
#include "y.tab.h"
```

*<constant c0 (rc/code.c) 65c>*

*<constant c1 (rc/code.c) 65d>*

*<constant c2 (rc/code.c) 65e>*

*<global codep 64a>*

*<global ncode 64b>*

*<function emitf 64d>*

*<function emitl 64c>*

*<function emits 64e>*

```
// forward decls  
void outcode(tree*, bool);  
void codeswitch(tree*, bool);  
bool iscase(tree*);
```

*<function morecode 64f>*

*<function stuffdot 76c>*

*<function compile 65a>*

*<function cleanhere 132b>*

*<function fnstr 94b>*

*<function outcode 65f>*

*<function codeswitch 91b>*

*<function iscase 92b>*

*<function codecopy 29d>*

*<function codefree 29e>*

### F.3.23 rc/pfnc.c

```
<rc/pfnc.c 210a>≡  
  <includes 21>  
  
  <global fname 154a>  
  
  <function pfnc 154b>
```

### F.3.24 rc/utills.c

```
<rc/utills.c 210b>≡  
  <includes 21>  
  
  // Opendir, Closedir and so on are back in plan9.c  
  
  <function Unlink 133a>  
  <function Write 169d>  
  <function Read 169e>  
  <function Seek 169f>  
  
  <function Creat 169g>  
  
  <function Dup 169h>  
  
  <function Malloc 169i>  
  
  // back in plan9.c  
  extern void* Malloc(ulong n);  
  <function emalloc 158b>  
  
  <function efree 158d>  
  
  <global bp 169a>  
  
  <function iacvt 169c>  
  
  <function intoascii 169b>
```

### F.3.25 rc/status.c

```
<rc/status.c 210c>≡  
  <includes 21>  
  
  <function setstatus 67c>  
  
  <function getstatus 67d>  
  
  <function truesstatus 68a>  
  
  <function concstatus 86b>
```

### F.3.26 rc/builtins.c

```
<rc/builtins.c 210d>≡
```

$\langle$ *includes* 21 $\rangle$   
 $\langle$ *function* dochdir 104b $\rangle$   
 $\langle$ *function* appfile 104a $\rangle$   
 $\langle$ *function* mapfd 143 $\rangle$   
  
 $\langle$ *function* execcd 103c $\rangle$   
 $\langle$ *function* execexit 105c $\rangle$   
 $\langle$ *function* execeval 107e $\rangle$   
 $\langle$ *function* execflag 145 $\rangle$   
  
// was in plan9.c  
 $\langle$ *function* Executable 144a $\rangle$   
 $\langle$ *function* execwhatis 142 $\rangle$   
  
 $\langle$ *function* execshift 146a $\rangle$   
  
 $\langle$ *global* dotcmds 107b $\rangle$   
 $\langle$ *function* execdots 105d $\rangle$   
  
 $\langle$ *function* execwait 108c $\rangle$   
  
 $\langle$ *function* execnewpgrp 144b $\rangle$   
  
 $\langle$ *global* rdfs 146b $\rangle$   
extern fdt envdir;  
 $\langle$ *function* execfinit 147a $\rangle$   
  
 $\langle$ *global* builtins 102b $\rangle$

### F.3.27 rc/path.c

$\langle$ rc/path.c 211 $\rangle \equiv$   
 $\langle$ *includes* 21 $\rangle$   
  
 $\langle$ *global* nullpath 73b $\rangle$   
  
 $\langle$ *function* searchpath 73a $\rangle$

### F.3.28 rc/fmt.c

```
<rc/fmt.c 212a>≡  
<includes 21>  
  
<global pfmtnest 166a>  
  
// forward decls  
void pdec(io*, int);  
void poct(io*, unsigned int);  
void pptr(io*, void*);  
void pval(io*, word*);  
void pquo(io*, char*);  
void pwrđ(io*, char*);  
  
// in io.c  
int fullbuf(io*, int);  
  
<function pfmt 166b>  
  
<function pchr 164a>  
  
<function pquo 168b>  
  
<function pwrđ 168c>  
  
<function pptr 168d>  
  
<function pstr 167a>  
  
<function pdec 167b>  
  
<function poct 168a>  
  
<function pval 168e>  
  
// _efgfmt is back in plan9.c
```

### F.3.29 rc/words.c

```
<rc/words.c 212b>≡  
<includes 21>  
  
<function newword 31d>  
  
<function freewords 32d>  
  
<function count 32a>  
  
<function copynwords 32b>  
  
<function copywords 32c>  
  
<function freelist 33a>
```

### F.3.30 rc/error.c

```
<rc/error.c 212c>≡
```

```

<includes 21>

// was in rc.h
<global nerror 156c>
<global err 156b>

// forward decls
void Abort(void);

<function panic 157c>

// Exit is back in plan9.c

<function Abort 157d>

```

### F.3.31 rc/main.c

```

<rc/main.c 213a>≡
<includes 21>

// Rmain and Isatty are back in plan9.c
extern char* Rmain;
extern bool Isatty(fdt fd);

/*
 * get command line flags.
 * initialize keywords & traps.
 * get values from environment.
 * set $pid, $cflag, $*
 * fabricate bootstrap code and start it (*(argv);. /usr/lib/rcmain $*)
 * start interpreting code
 */
<function main (rc/exec.c) 36b>

```

### F.3.32 rc/plan9.c

```

<rc/plan9.c 213b>≡
/*
 * Plan 9 versions of system-specific functions
 * By convention, exported routines herein have names beginning with an
 * upper case letter.
 */
<includes 21>

// could be in main.c
<global Rmain 118a>
<function Isatty 37e>

// could be in rc.h
<enum MiscPlan9 110c>

// defined in trap.c
extern int trap[NSIG];
// defined in exec.c
extern fdt envdir;

// could be in env.c
<function Vinit 111c>

```

```

⟨function addenv 111b⟩
⟨function updenvironment 110b⟩
⟨function Updenvironment 110a⟩

// could be in glob.c
⟨constant NDIR 125a⟩
⟨function Globsize 125b⟩

// could be in processes.c
⟨global Fdprefix 138g⟩

// in processes.c
extern int *waitpids;
extern int nwaitpids;

⟨function delwaitpid 68f⟩
⟨function havewaitpid 69a⟩
⟨function Waitfor 74b⟩

// could be in trap.c
⟨global signame 114a⟩
⟨global syssigname 113d⟩
⟨global interrupted 115c⟩
⟨function notifyf 114e⟩
⟨function Trapinit 114d⟩
⟨function Eintr 115d⟩
⟨function Noerror 116b⟩

// could be in fmt.c
⟨function _efgfmt 158a⟩

// could be in error.c
⟨function Exit 157e⟩

// could be in exec.c
⟨function Xrdfs 147b⟩

// could be in utils.c
⟨function Malloc 158c⟩

// could be in utils.c
⟨constant NFD 126a⟩
⟨struct DirEntryWrapper 126b⟩
⟨global dir 126c⟩
⟨function trimdirs 126e⟩
⟨function Readdir 127a⟩
⟨function Opendir 126d⟩
⟨function Closedir 127b⟩

```

### F.3.33 rc/unix.c

```

⟨rc/unix.c 214⟩≡
/*
 * Unix versions of system-specific functions
 * By convention, exported routines herein have names beginning with an
 * upper case letter.
 */
// to avoid conflict for wait(), waitpid() signatures
#define NOPLAN9DEFINES

```

```

#include "rc.h"
#include "io.h"
#include "exec.h"
#include "getflags.h"

//TODO? use plan9port errstr instead?
#include <errno.h>

// system-specific globals defined here but used in other files
char *Rcmain = "/usr/lib/rcmain";
char *Fdprefix = "/dev/fd/";

//*****
// Environment
//*****

#define SEP '\\1'

word*
enval(char *s)
{
    char *t, c;
    word *v;
    for(t = s; *t && *t!=SEP; t++);
    c=*t;
    *t='\\0';
    v = newword(s, c=='\\0'?(word *)0:enval(t+1));
    *t = c;
    return v;
}

// set in Vinit()
char **environp;

void
Vinit(void)
{
    // see Unix environ(7) man page
    extern char **environ;
    char *s;
    char **env = environ;
    word* wd;
    environp = env;
    for(; *env; env++){
        for(s=*env; *s && *s!='(' && *s!='='; s++);
        switch(*s){
            case '\\0':
                pfmt(err, "environment %q?\\n", *env);
                break;
            case '=':
                *s='\\0';
                wd = enval(s+1);
                setvar(*env, wd);
                //TODO: should generalize this in main.c so it also applies for Plan9
                if(strcmp(*env, "RCMAIN") == 0) {
                    Rcmain = strdup(wd->word);
                }
                *s='=';
                break;
            case '(': /* ignore functions for now */

```

```

        break;
    }
}

//TODO: should be set in execfinit
char **envp = nil;

void
Xrdfn(void)
{
    char *s;
    int len;
    for(;*envp;envp++){
        for(s=*envp;*s && *s!='(' && *s!='=';s++){
            switch(*s){
                case '\\0':
                    pfmt(err, "environment %q?\n", *envp);
                    break;
                case '=': /* ignore variables */
                    break;
                case '(': /* Bourne again */
                    s=*envp+3;
                    envp++;
                    len = strlen(s);
                    s[len]='\\n';
                    execcmds(opencore(s, len+1));
                    s[len]='\\0';
                    return;
            }
        }
    }
    Xreturn();
}

//
//union code rdfns[4];
//
//void
//execfinit(void)
//{
// static int first = 1;
// if(first){
//     rdfns[0].i = 1;
//     rdfns[1].f = Xrdfn;
//     rdfns[2].f = Xjump;
//     rdfns[3].i = 1;
//     first = 0;
// }
// Xpopm();
// envp = environp;
// start(rdfns, 1, runq->local);
//}
//
//int
//cmpenv(const void *aa, const void *ab)
//{
// char **a = aa, **b = ab;
//
// return strcmp(*a, *b);
//}
//

```

```

//char **
//mkenv(void)
//{
// char **env, **ep, *p, *q;
// struct var **h, *v;
// struct word *a;
// int nvar = 0, nchr = 0, sep;
//
// /*
//  * Slightly kludgy loops look at locals then globals.
//  * locals no longer exist - geoff
//  */
// for(h = gvar-1; h != &gvar[NVAR]; h++)
// for(v = h >= gvar? *h: runq->local; v ;v = v->next){
//   if((v==vlook(v->name)) && v->val){
//     nvar++;
//     nchr+=strlen(v->name)+1;
//     for(a = v->val;a;a = a->next)
//       nchr+=strlen(a->word)+1;
//   }
//   if(v->fn){
//     nvar++;
//     nchr+=strlen(v->name)+strlen(v->fn[v->pc-1].s)+8;
//   }
// }
// env = (char **)emalloc((nvar+1)*sizeof(char *)+nchr);
// ep = env;
// p = (char *)&env[nvar+1];
// for(h = gvar-1; h != &gvar[NVAR]; h++)
// for(v = h >= gvar? *h: runq->local;v;v = v->next){
//   if((v==vlook(v->name)) && v->val){
//     *ep++=p;
//     q = v->name;
//     while(*q) *p++=*q++;
//     sep='';
//     for(a = v->val;a;a = a->next){
//       *p++=sep;
//       sep = SEP;
//       q = a->word;
//       while(*q) *p++=*q++;
//     }
//     *p++='\0';
//   }
//   if(v->fn){
//     *ep++=p;
//     *p++='#'; *p++='('; *p++=')'; /* to fool Bourne */
//     *p++='f'; *p++='n'; *p++=' ';
//     q = v->name;
//     while(*q) *p++=*q++;
//     *p++=' ';
//     q = v->fn[v->pc-1].s;
//     while(*q) *p++=*q++;
//     *p++='\0';
//   }
// }
// }
// *ep = 0;
// qsort((void *)env, nvar, sizeof ep[0], cmpenv);
// return env;
//}

```

```

void
Updenv(void)
{
}

//*****
// Signals/notes and Waitfor()
//*****

char *sigmsg[] = {
/* 0 normal */ 0,
/* 1 SIGHUP */ "Hangup",
/* 2 SIGINT */ 0,
/* 3 SIGQUIT */ "Quit",
/* 4 SIGILL */ "Illegal instruction",
/* 5 SIGTRAP */ "Trace/BPT trap",
/* 6 SIGIOT */ "abort",
/* 7 SIGEMT */ "EMT trap",
/* 8 SIGFPE */ "Floating exception",
/* 9 SIGKILL */ "Killed",
/* 10 SIGBUS */ "Bus error",
/* 11 SIGSEGV */ "Memory fault",
/* 12 SIGSYS */ "Bad system call",
/* 13 SIGPIPE */ 0,
/* 14 SIGALRM */ "Alarm call",
/* 15 SIGTERM */ "Terminated",
/* 16 unused */ "signal 16",
/* 17 SIGSTOP */ "Process stopped",
/* 18 unused */ "signal 18",
/* 19 SIGCONT */ "Process continued",
/* 20 SIGCHLD */ "Child death",
};

//pad: Note that plan9 Waitfor(), that was in processes.c (now in plan9.c), was
// compiling correctly also under Unix and I originally used it and rc
// was partially working. However, when called from mk (via MKSHELL), I had
// weird bugs like the $status was containing weird strings and so
// failing mk even if the command was run correctly.
// Anyway, simpler to uncomment and use the Waitfor() below.
void
Waitfor(int pid, bool persist)
{
    int wpid, sig;
    thread *p;
    int wstat;
    char wstatstr[12];

    for(;;){
        errno = 0;
        wpid = wait(&wstat);
        if(errno==EINTR && persist)
            continue;
        if(wpid==-1)
            break;
        sig = wstat&0177;
        if(sig==0177){
            pfmt(err, "trace: ");
            sig = (wstat>>8)&0177;
        }
        if(sig>(sizeof sigmsg/sizeof sigmsg[0]) || sigmsg[sig]){

```

```

    if(pid!=wpid)
        pfmt(err, "%d: ", wpid);
    if(sig<=(sizeof sigmsg/sizeof sigmsg[0]))
        pfmt(err, "%s", sigmsg[sig]);
    else if(sig==0177) pfmt(err, "stopped by ptrace");
    else pfmt(err, "signal %d", sig);
    if(wstat&0200)pfmt(err, " -- core dumped");
    pfmt(err, "\n");
}
wstat = sig?sig+1000:(wstat>>8)&0xFF;
if(wpid==pid){
    intoascii(wstatstr, wstat);
    setstatus(wstatstr);
    break;
}
else{
    for(p = runq->ret;p = p->ret)
        if(p->pid==wpid){
            p->pid=-1;
            intoascii(p->status, wstat);
            break;
        }
}
}
}
}

char *signame[] = {
    "sigexit", "sighup", "sigint", "sigquit",
    "sigill", "sigtrap", "sigiot", "sigemt",
    "sigfpe", "sigkill", "sigbus", "sigsegv",
    "sigsys", "sigpipe", "sigalrm", "sigterm",
    "sigl6", "sigstop", "sigtstp", "sigcont",
    "sigchld", "sigttin", "sigttou", "sigtint",
    "sigxcpu", "sigxfsz", "sig26", "sig27",
    "sig28", "sig29", "sig30", "sig31",
    0,
};

// defined in trap.c
extern int trap[NSIG];

void
gettrap(int sig)
{
    signal(sig, gettrap);
    trap[sig]++;
    ntrap++;
    if(ntrap>=NSIG){
        pfmt(err, "rc: Too many traps (trap %d), dumping core\n", sig);
        signal(SIGABRT, (void (*)())0);
        kill(getpid(), SIGABRT);
    }
}

void
Trapinit(void)
{
    int i;
    void (*sig)();

```

```

    if(1 || flag['d']){ /* wrong!!! */
        sig = signal(SIGINT, gettrap);
        if(sig==SIG_IGN)
            signal(SIGINT, SIG_IGN);
    }
    else{
        for(i = 1;i<=NSIG;i++) if(i!=SIGCHLD){
sig = signal(i, gettrap);
if(sig==SIG_IGN)
    signal(i, SIG_IGN);
        }
    }
}

//*****
// Errno
//*****

bool
Eintr(void){
    return errno==EINTR;
}

void
Noerror(void)
{
    errno = 0;
}

//*****
// Directories
//*****

#define NDIR 14 /* should get this from param.h */
int
Globsize(char *p)
{
    int isglob = 0, globlen = NDIR+1;
    for(;*p;p++){
        if(*p==GLOB){
            p++;
            if(*p!=GLOB)
isglob++;
            globlen+=*p=='?'*NDIR:1;
        }
        else
            globlen++;
    }
    return isglob?globlen:0;
}

#include <sys/types.h>
#include <dirent.h>

#define NDIRLIST 50

DIR *dirlist[NDIRLIST];

Opendir(name)
    char *name;

```

```

{
    DIR **dp;
    for(dp = dirlist; dp != &dirlist[NDIRLIST]; dp++)
        if(*dp == 0){
            *dp = opendir(name);
            return *dp ? dp - dirlist : -1;
        }
    return -1;
}

int
Readdir(int f, char *p, int onlydirs)
{
    struct dirent *dp = readdir(dirlist[f]);

    if(dp == 0)
        return 0;
    strcpy(p, dp->d_name);
    return 1;
}

void
Closedir(int f)
{
    closedir(dirlist[f]);
    dirlist[f] = 0;
}

//*****
// Misc
//*****

//char **
//mkargv(a)
//register struct word *a;
//{
// char **argv = (char **)emalloc((count(a)+2)*sizeof(char *));
// char **argp = argv+1; /* leave one at front for runcoms */
//
// for(; a; a = a->next)
//     *argp++ = a->word;
// *argp = 0;
// return argv;
//}
//

//void
//Execute(struct word *args, struct word *path)
//{
// char *msg = "not found";
// int txtbusy = 0;
// char **env = mkenv();
// char **argv = mkargv(args);
// char file[512];
//
// for(; path; path = path->next){
//     strcpy(file, path->word);
//     if(file[0])
//         strcat(file, "/");
//     strcat(file, argv[1]);

```

```

//ReExec:
//  execve(file, argv+1, env);
//  switch(errno){
//  case ENOEXEC:
//    pfmt(err, "%s: Bourne again\n", argv[1]);
//    argv[0]="sh";
//    argv[1] = strdup(file);
//    execve("/bin/sh", argv, env);
//    goto Bad;
//  case ETXTBSY:
//    if(++txtbusy!=5){
//      sleep(txtbusy);
//      goto ReExec;
//    }
//    msg="text busy"; goto Bad;
//  case EACCES:
//    msg="no access";
//    break;
//  case ENOMEM:
//    msg="not enough memory"; goto Bad;
//  case E2BIG:
//    msg="too big"; goto Bad;
//  }
// }
//Bad:
// pfmt(err, "%s: %s\n", argv[1], msg);
// efree((char *)env);
// efree((char *)argv);
//}

/*
 * Wrong:  should go through components of a|b|c and return the maximum.
 */
void
Exit(char *stat, char* loc)
{
    int n = 0;
    USED(loc);
    //if(flag['s'])
    // fprintf(STDERR, "Exit from %s: %s\n", loc, stat);

    while(*stat){
        if(*stat!='|'){
            if(*stat<'0' || '9'<*stat)
exit(1);
            else n = n*10+*stat-'0';
        }
        stat++;
    }
    exit(n);
}

bool Isatty(fdt fd){
    return isatty(fd);
}

//void
//execumask(void) /* wrong -- should fork before writing */
//{

```

```

// int m;
// struct io out[1];
// switch(count(runq->argv->words)){
// default:
//   pfmt(err, "Usage: umask [umask]\n");
//   setstatus("umask usage");
//   poplist();
//   return;
// case 2:
//   umask(octal(runq->argv->words->next->word));
//   break;
// case 1:
//   umask(m = umask(0));
//   out->fd = mapfd(1);
//   out->bufp = out->buf;
//   out->ebuf=&out->buf[NBUF];
//   out->strp = 0;
//   pfmt(out, "%o\n", m);
//   break;
// }
// setstatus("");
// poplist();
//}

```

```

void*
Malloc(unsigned long n)
{
    return (void *)malloc(n);
}

```

```

int
needsrcquote(int c)
{
    if(c <= ' ')
        return 1;
    if(strchr("`^#*[]=|\\\"?${}()<>&;", c))
        return 1;
    return 0;
}

```

```

//void
//delwaitpid(int pid)
//{
// int r, w;
//
// for(r=w=0; r<nwaitpids; r++)
//   if(waitpids[r] != pid)
//     waitpids[w++] = waitpids[r];
// nwaitpids = w;
//}

```

```

//int
//havewaitpid(int pid)
//{
// int i;
//
// for(i=0; i<nwaitpids; i++)
//   if(waitpids[i] == pid)
//     return 1;

```

```
// return 0;  
//  
//}
```

Uses GLOB 119a, NDIR-15 214, NDIRLIST-16 214, dirlist 214, envp 214, err 156b, flag, gettrap() 214, intoascii() 169b, ntrap 114b, pfmt() 166b, runq 30c, setstatus() 67c, sigmsg 214, and trap 114c.

# Glossary

LOC = Lines Of Code

CLI = Command-Line Interface

GUI = Graphical User Interface

IDE = Integrated Development Environment

# Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Abort(): [157c](#), [157d](#)  
addenv(): [110a](#), [110b](#), [111b](#)  
addtok(): [119b](#), [120a](#), [129b](#)  
addutf(): [53f](#), [54c](#), [129b](#)  
addwaitpid(): [68d](#), [72a](#), [85b](#), [86d](#), [130d](#), [138h](#)  
advance(): [47a](#), [47b](#), [48a](#), [49f](#), [50a](#), [53f](#), [54c](#), [56b](#), [129b](#), [136a](#), [156e](#)  
ANDAND: [50b](#), [151e](#)  
APPEND: [52d](#), [52f](#), [136a](#), [151e](#)  
appfile(): [103c](#), [104a](#), [105d](#)  
ARGLIST: [59e](#), [91b](#), [92b](#), [151e](#)  
argv0: [39b](#), [39b](#), [39c](#), [71b](#), [71c](#)  
BADFLAG-22: [200a](#)  
badflag-23: [159i](#), [200a](#)  
BANG: [55e](#), [151e](#)  
bp: [169a](#), [169b](#), [169c](#)  
BRACE: [151e](#)  
buf-25: [163b](#), [200a](#)  
bufp-26: [163b](#), [200a](#), [200a](#)  
Builtin: [102a](#), [102c](#), [196c](#)  
Builtin.fnc: [102a](#)  
Builtin.name: [102a](#)  
builtin (typedef): [196c](#)  
builtins: [102b](#), [103a](#)  
c0-5: [151b](#), [151e](#)  
c0-62: [65c](#), [91b](#), [92b](#)  
c1-6: [151c](#), [151e](#)  
c1-63: [65d](#), [91b](#)  
c2-64: [65e](#)  
c2-7: [151d](#), [151e](#)  
cleanhere(): [132a](#), [132b](#)  
clearwaitpids(): [68g](#), [72a](#), [85b](#), [86d](#), [130d](#), [138h](#), [140](#)  
CLOSE: [136a](#), [136b](#)  
Closedir(): [127b](#)  
closeio(): [41d](#), [59d](#), [94b](#), [111b](#), [132a](#), [165a](#)  
cmdname: [159i](#)  
Code: [29c](#), [30b](#), [75b](#), [107b](#), [108a](#), [146b](#), [196c](#)  
Code.f: [29c](#)  
Code.i: [29c](#)

Code.s: [29c](#)  
 code (typedef): [196c](#)  
 codebuf: [29b](#), [41a](#), [64f](#), [65a](#), [65b](#), [76c](#)  
 codecop(): [29d](#), [39a](#), [94c](#)  
 codefree(): [29e](#), [84d](#), [94c](#), [95a](#)  
 codep: [64a](#), [65a](#), [76c](#)  
 codeswitch(): [91b](#)  
 compile(): [65a](#)  
 conclist(): [100b](#), [100c](#), [100c](#)  
 concstatus(): [85f](#), [86b](#)  
 copynwords(): [32b](#)  
 copywords(): [32c](#), [87](#), [115b](#)  
 COUNT: [51a](#), [151e](#)  
 count(): [32a](#), [74a](#), [80d](#), [81c](#), [82b](#), [96c](#), [97a](#), [99b](#), [100b](#), [103b](#), [103c](#), [105c](#), [107e](#), [108c](#), [135d](#), [141c](#), [144b](#), [145](#)  
 Creat(): [80d](#), [82b](#), [111b](#), [132a](#), [169g](#)  
 deglob(): [96c](#), [97a](#), [99b](#), [121c](#), [122a](#), [141c](#)  
 delwaitpid(): [68f](#), [74b](#)  
 dir: [126c](#), [126d](#), [127a](#), [127b](#)  
 DirEntryWrapper: [126b](#), [126c](#)  
 DirEntryWrapper.dbuf: [126b](#)  
 DirEntryWrapper.i: [126b](#)  
 DirEntryWrapper.n: [126b](#)  
 dirlist: [214](#), [214](#)  
 dochdir(): [103c](#), [104b](#)  
 doprompt: [45c](#), [45c](#), [45g](#), [45h](#), [46b](#), [46c](#), [132a](#)  
 doredir(): [78a](#), [79d](#), [79d](#)  
 dotcmds: [105d](#), [107b](#), [107d](#)  
 dotrap(): [115a](#), [115b](#), [163g](#)  
 DUP: [59e](#), [136a](#), [151e](#)  
 DUPFD: [136a](#), [137c](#), [151e](#)  
 eflagok: [107e](#), [128c](#), [128d](#), [129a](#)  
 efree(): [28a](#), [29e](#), [32d](#), [33a](#), [41d](#), [65b](#), [66c](#), [66d](#), [69f](#), [73c](#), [77d](#), [81a](#), [84d](#), [92c](#), [94a](#), [97b](#), [100c](#), [105d](#), [107e](#), [111c](#), [121c](#), [122c](#), [132a](#), [141c](#), [158d](#), [165a](#)  
 ehere: [131e](#), [133f](#)  
 Eintr(): [115d](#)  
 emalloc(): [65a](#), [74a](#), [77e](#), [100c](#), [104a](#), [107e](#), [111c](#), [121c](#), [122c](#), [141c](#), [158b](#), [165b](#), [165c](#)  
 emitf-65: [64d](#), [65a](#), [91b](#), [132b](#)  
 emitf-66: [64c](#), [65a](#), [91b](#)  
 emits-67: [64e](#), [132b](#)  
 emptybuf(): [164c](#), [164d](#)  
 envdir: [112](#), [147b](#)  
 environp: [214](#)  
 envp: [214](#), [214](#)  
 EOF: [44a](#), [44b](#), [44d](#), [45a](#), [45b](#), [46b](#), [46d](#), [46e](#), [47a](#), [48b](#), [49b](#), [49f](#), [53f](#), [132a](#), [136a](#), [156e](#), [164d](#), [164e](#)  
 epimung(): [62a](#)  
 equdf(): [123a](#)  
 err: [38a](#), [41d](#), [45h](#), [70](#), [71b](#), [71c](#), [73c](#), [80d](#), [81c](#), [82b](#), [103b](#), [103c](#), [105c](#), [105d](#), [111b](#), [111c](#), [114e](#), [135d](#), [144b](#), [153b](#), [155a](#), [155b](#), [155c](#), [156b](#), [156d](#), [156e](#), [157c](#), [157d](#), [158d](#), [166b](#), [214](#)  
 errc(): [163b](#)

errn(): [162a](#)  
errs(): [162b](#)  
execcd(): [102b](#), [103c](#)  
execcmds(): [107e](#), [108b](#), [147b](#)  
execdot(): [102b](#), [105d](#)  
execeval(): [102b](#), [107e](#)  
execexec(): [72a](#), [72b](#), [75a](#), [102b](#)  
execexit(): [102b](#), [105c](#)  
execfinit(): [102b](#), [147a](#)  
execflag(): [102b](#), [145](#)  
execforkexec(): [70](#), [72a](#)  
execfunc(): [95c](#), [95d](#)  
execnewpgrp(): [102b](#), [144b](#)  
Executable(): [144a](#)  
Execute(): [72b](#), [73c](#)  
execwait(): [102b](#), [108c](#)  
Exit(): [157e](#)  
exitnext(): [75a](#), [75b](#)  
Fdprefix: [138g](#)  
FEWARGS-20: [159i](#)  
flag: [37c](#), [37d](#), [41c](#), [65a](#), [105b](#), [118c](#), [128a](#), [145](#), [153b](#), [155a](#), [155b](#), [155c](#), [159i](#), [214](#)  
flagarg-17: [159i](#)  
flagset: [37c](#), [37d](#), [145](#), [159i](#)  
flush(): [45h](#), [70](#), [71b](#), [71c](#), [132a](#), [154b](#), [156d](#), [156e](#), [157c](#), [157d](#), [163g](#), [164b](#), [166b](#)  
FN: [55e](#), [151e](#)  
fname: [154a](#), [154b](#)  
fnstr(): [94b](#)  
FOR: [55e](#), [151e](#)  
freelist(): [33a](#), [66c](#), [90b](#)  
freenodes(): [28a](#), [41a](#)  
freewords(): [32d](#), [34a](#), [96c](#), [97b](#)  
fullbuf(): [164a](#), [164b](#)  
future: [46d](#), [46d](#), [46e](#), [47a](#), [157b](#)  
getflags(): [37b](#), [159i](#)  
getnext(): [43](#), [46e](#)  
getstatus(): [67d](#), [68a](#), [84d](#), [85f](#), [87](#), [115b](#), [157e](#)  
gettrap(): [214](#), [214](#)  
GLOB: [119a](#), [119b](#), [122a](#), [122d](#), [125b](#), [153a](#), [214](#)  
glob(): [120e](#), [121c](#)  
globcmp(): [122b](#), [122c](#)  
globdir(): [121c](#), [122d](#), [122d](#)  
globlist(): [94c](#), [96c](#), [97a](#), [120c](#), [120d](#), [121a](#)  
globlist1(): [120d](#), [120e](#), [120e](#)  
globname: [121b](#), [121c](#), [122d](#)  
Globsize(): [125b](#)  
globsort(): [121c](#), [122c](#)  
globv: [120b](#), [120d](#), [121c](#), [122d](#)  
gvar: [34d](#), [34e](#), [110a](#)  
gvlook(): [34b](#), [34e](#), [94c](#), [95a](#), [95c](#)

hash(): [34e](#), [35a](#), [55a](#), [55f](#)  
havewaitpid(): [69a](#), [74b](#)  
HERE: [131a](#), [131b](#), [136a](#), [151e](#)  
Here: [131c](#), [131c](#), [131d](#), [131e](#), [132a](#), [133f](#)  
here: [131d](#), [132a](#), [133f](#)  
Here.name: [131c](#)  
Here.next: [131c](#)  
Here.tag: [131c](#)  
heredoc(): [133f](#)  
hex: [133d](#), [133d](#), [133e](#)  
hexnum(): [133e](#), [133f](#)  
iacvt(): [169b](#), [169c](#), [169c](#)  
idchr(): [49a](#), [54c](#)  
IF: [55e](#), [151e](#)  
ifnot: [88b](#), [88c](#), [88d](#), [88f](#)  
IN: [55e](#)  
incomm: [44e](#), [49e](#), [49f](#)  
inquote: [44e](#), [53c](#), [53d](#), [53f](#)  
interrupted: [114e](#), [115c](#), [115c](#), [115d](#), [116b](#)  
intoascii(): [86d](#), [118f](#), [138h](#), [169b](#), [214](#)  
Io: [41b](#), [59d](#), [140](#), [163d](#), [163e](#), [165b](#), [165c](#), [196c](#)  
Io.buf: [163d](#)  
Io.bufp: [163d](#)  
Io.ebuf: [163d](#)  
Io.fd: [163d](#)  
Io.strp: [163d](#)  
io (typedef): [196c](#)  
Isatty(): [37e](#)  
iscase(): [91b](#), [92b](#)  
kenter(): [55e](#), [55f](#)  
kinit(): [38b](#), [55e](#)  
klook(): [54c](#), [55a](#)  
Kw: [55a](#), [55b](#), [55b](#), [55c](#), [55f](#)  
kw: [55a](#), [55c](#), [55f](#)  
Kw.name: [55b](#)  
Kw.next: [55b](#)  
Kw.type: [55b](#)  
lastc: [156e](#), [157a](#), [157b](#)  
lastdol: [49b](#), [50b](#), [51a](#), [51b](#), [51c](#), [53b](#), [53f](#), [54c](#), [156e](#)  
lastword: [53e](#), [53f](#), [54c](#), [56b](#), [156e](#)  
List: [31a](#), [33b](#), [196c](#)  
List.next: [33b](#)  
List.words: [33b](#)  
list2str(): [77d](#), [77e](#), [92c](#)  
list (typedef): [196c](#)  
main-11(): [36b](#)  
Malloc(): [158c](#)  
mapfd(): [143](#)  
matchfn(): [122d](#), [124a](#)

Maxenvname-2: [110c](#), [111b](#), [111c](#), [147b](#)  
 Malloc(): [165c](#), [169i](#)  
 mkargv(): [73c](#), [74a](#)  
 morecode(): [64f](#)  
 mung1(): [60e](#)  
 mung2(): [61b](#)  
 mung3(): [60f](#)  
 mypid: [87](#), [115b](#), [118d](#), [118f](#)  
 NBUF: [163c](#), [163d](#), [163g](#), [164d](#)  
 NBUF-24: [163b](#)  
 ncode: [64b](#), [64f](#), [65a](#)  
 NDIR-15: [214](#), [214](#)  
 NDIR-3: [125a](#), [125b](#)  
 NDIRLIST-16: [214](#), [214](#)  
 ndot: [105d](#), [107a](#), [155c](#)  
 needsrcquote(): [168c](#), [214](#)  
 nerror: [65b](#), [156c](#), [156d](#), [156e](#)  
 newtree(): [27e](#), [28d](#), [51g](#), [54b](#)  
 newvar(): [34e](#), [35b](#), [87](#), [95d](#), [97a](#), [115b](#)  
 newword(): [31d](#), [32b](#), [32c](#), [40b](#), [67c](#), [86d](#), [100c](#), [111c](#), [118f](#), [118g](#), [121c](#), [122d](#), [128a](#), [156e](#)  
 nextc(): [46e](#), [47a](#), [47b](#), [49f](#), [50a](#), [53f](#), [54c](#), [56a](#)  
 nextis(): [47b](#), [50b](#), [51a](#), [51c](#), [52f](#), [131b](#), [135b](#), [136a](#)  
 nextutf(): [123b](#)  
 NFD-4: [126a](#), [126c](#), [126d](#), [127a](#), [127b](#)  
 NFLAG: [36d](#)  
 NKW-8: [55a](#), [55c](#), [55d](#), [55f](#)  
 nl: [94b](#), [151a](#), [151a](#), [151e](#)  
 NLINE-9: [131f](#), [132a](#)  
 Noerror(): [116b](#)  
 NOPLAN9DEFINES-13: [214](#)  
 NOT: [55e](#), [151e](#)  
 notifyf(): [114d](#), [114e](#)  
 NSTATUS: [85d](#), [85e](#), [85f](#), [86b](#)  
 NTOK-10: [49c](#), [49d](#), [120a](#)  
 ntrap: [114b](#), [114e](#), [115a](#), [115b](#), [116c](#), [163g](#), [214](#)  
 nullpath: [73a](#), [73b](#), [103c](#)  
 NVAR: [34c](#), [34d](#), [34e](#), [110a](#)  
 nwaitpids: [68c](#), [68d](#), [68e](#), [68f](#), [68g](#), [69a](#)  
 opencore(): [107e](#), [165c](#)  
 Opendir(): [122d](#), [126d](#)  
 openfd(): [38a](#), [41c](#), [105d](#), [111b](#), [132a](#), [147b](#), [163e](#)  
 openstr(): [59d](#), [94b](#), [165b](#)  
 OROR: [51c](#), [151e](#)  
 panic(): [40b](#), [64f](#), [66c](#), [67a](#), [67b](#), [68e](#), [76c](#), [81a](#), [97b](#), [157c](#), [158b](#), [163g](#)  
 PAREN: [151e](#)  
 pchr(): [41d](#), [132a](#), [134b](#), [151e](#), [153a](#), [154b](#), [155c](#), [157c](#), [164a](#), [166b](#), [167a](#), [167b](#), [168a](#), [168b](#), [168d](#), [168e](#)  
 PCMD: [151e](#)  
 pcmd(): [151e](#), [166b](#)  
 pdec(): [166b](#), [167b](#), [167b](#)

pdeglob(): [151e](#), [153a](#)  
 pfmt(): [59d](#), [71b](#), [71c](#), [73c](#), [80d](#), [81c](#), [82b](#), [94b](#), [103b](#), [103c](#), [105c](#), [105d](#), [111b](#), [111c](#), [114e](#), [135d](#), [144b](#), [151e](#),  
[154b](#), [155a](#), [155b](#), [156e](#), [157c](#), [157d](#), [158d](#), [166b](#), [168b](#), [214](#)  
 pfmtnest: [166a](#), [166a](#), [166b](#)  
 pfnc(): [153b](#), [154b](#)  
 PIPE: [51g](#), [52e](#), [136a](#), [151e](#)  
 PIPEFD: [151e](#)  
 poct(): [166b](#), [168a](#), [168a](#)  
 poplist(): [66c](#), [70](#), [72b](#), [77d](#), [80d](#), [81c](#), [82b](#), [84d](#), [90b](#), [92a](#), [92c](#), [94c](#), [95a](#), [95d](#), [96c](#), [97a](#), [99b](#), [100b](#), [103b](#), [103c](#),  
[107e](#), [108c](#), [120d](#), [135d](#), [141c](#), [144b](#), [145](#)  
 popword(): [66d](#), [72b](#), [95d](#), [103b](#), [105d](#)  
 pprompt(): [45g](#), [45h](#), [132a](#)  
 pptr(): [166b](#), [168d](#)  
 pquo(): [166b](#), [168b](#), [168c](#)  
 PRD: [84e](#), [85b](#), [138h](#), [140](#)  
 promptstr: [45d](#), [45f](#), [45h](#)  
 pstr(): [45h](#), [132a](#), [134b](#), [154b](#), [166b](#), [167a](#), [168c](#)  
 pstrs(): [134b](#)  
 pushlist(): [39c](#), [40a](#), [66b](#), [105d](#), [120d](#)  
 pushredir(): [79c](#), [80d](#), [81c](#), [82b](#), [85b](#), [86d](#), [105d](#), [135d](#), [137e](#), [137i](#), [138h](#), [140](#)  
 pushword(): [39c](#), [40b](#), [66a](#), [72a](#), [75a](#), [105d](#), [138h](#), [141c](#)  
 pval(): [166b](#), [168e](#)  
 PWR: [85a](#), [85b](#), [138h](#), [140](#)  
 pwr(): [166b](#), [168c](#), [168e](#)  
 rchr(): [43](#), [44e](#), [132a](#), [164c](#), [164e](#)  
 RCLOSE: [105d](#), [137h](#), [137i](#), [138b](#), [143](#)  
 Rcmain: [118a](#)  
 rdcmds: [108a](#), [108b](#)  
 rdfns: [146b](#)  
 RDUP: [137e](#), [137g](#), [138a](#), [143](#)  
 RDWR: [135a](#), [135b](#), [151e](#)  
 READ: [52c](#), [53a](#), [138h](#), [151e](#)  
 Readdir(): [127a](#)  
 readhere(): [131g](#), [132a](#)  
 reason-18: [159i](#), [200a](#)  
 REDIR: [52f](#), [53a](#), [59e](#), [136a](#), [151e](#)  
 Redir: [78c](#), [79a](#), [79b](#), [81a](#), [82c](#), [196c](#)  
 Redir.from: [78c](#)  
 Redir.next: [79b](#)  
 Redir.to: [78c](#)  
 Redir.type: [78c](#)  
 redir (typedef): [196c](#)  
 RESET-19: [159i](#)  
 reverse(): [159i](#), [160a](#)  
 ROPEN: [79e](#), [80a](#), [80d](#), [81a](#), [81c](#), [82b](#), [85b](#), [86d](#), [135d](#), [138h](#), [140](#), [143](#)  
 runq: [30c](#), [34b](#), [39a](#), [40a](#), [40b](#), [40c](#), [41a](#), [41c](#), [43](#), [44b](#), [44d](#), [44e](#), [45f](#), [45h](#), [66a](#), [66c](#), [66d](#), [67a](#), [70](#), [71b](#), [71c](#), [72b](#),  
[72c](#), [75b](#), [76e](#), [76f](#), [77d](#), [78a](#), [78b](#), [79c](#), [80d](#), [81a](#), [81c](#), [82b](#), [83a](#), [84d](#), [85b](#), [85f](#), [86a](#), [86d](#), [87](#), [88b](#), [88f](#), [89g](#), [90b](#),  
[92c](#), [94c](#), [95a](#), [95d](#), [96c](#), [97a](#), [97b](#), [99b](#), [100b](#), [103c](#), [105c](#), [105d](#), [107e](#), [108b](#), [108c](#), [110a](#), [115b](#), [120d](#), [130d](#), [132a](#),  
[132c](#), [135d](#), [137e](#), [137i](#), [138h](#), [140](#), [141c](#), [143](#), [144b](#), [145](#), [153b](#), [156e](#), [214](#)

rutf(): [164e](#)  
searchpath(): [72b](#), [73a](#), [105d](#)  
SEP-14: [214](#)  
ser: [133b](#), [133b](#), [133f](#)  
setstatus(): [67c](#), [71b](#), [71c](#), [74b](#), [77b](#), [77d](#), [85f](#), [89f](#), [103b](#), [103c](#), [105c](#), [105d](#), [144b](#), [145](#), [157e](#), [214](#)  
setvar(): [34a](#), [67c](#), [86d](#), [111c](#), [118f](#), [118g](#), [128a](#), [156e](#)  
sigmsg: [214](#), [214](#)  
signame: [114a](#)  
SIMPLE: [59d](#), [92b](#), [151e](#)  
simplemung(): [59d](#)  
skipnl(): [50a](#), [50b](#), [51c](#), [51g](#)  
skipwhite(): [48a](#), [49f](#), [50a](#)  
start(): [38d](#), [39a](#), [41a](#), [85b](#), [86d](#), [87](#), [95d](#), [105d](#), [108b](#), [115b](#), [130d](#), [138h](#), [140](#)  
STR: [196c](#)  
STR2: [196c](#)  
Stralloc-1: [163f](#), [163g](#), [165b](#)  
stuffdot(): [76c](#), [91b](#)  
SUB: [56b](#), [151e](#)  
SUBSHELL: [55e](#), [151e](#)  
SWITCH: [55e](#), [151e](#)  
syssigname: [113d](#), [114e](#)  
Thread: [30b](#), [39a](#), [41a](#), [84d](#), [85b](#), [138h](#), [196c](#)  
Thread.argv: [31a](#)  
Thread.cmdfd: [41b](#)  
Thread.cmdfile: [41b](#)  
Thread.code: [30b](#)  
Thread.eof: [44c](#)  
Thread.iflag: [41b](#)  
Thread.iflast: [89b](#)  
Thread.lineno: [46a](#)  
Thread.local: [31b](#)  
Thread.pc: [30b](#)  
Thread.pid: [85c](#)  
Thread.redir: [79a](#)  
Thread.ret: [30d](#)  
Thread.startredir: [82c](#)  
Thread.status: [85d](#)  
thread (typedef): [196c](#)  
tmp: [133c](#), [133c](#), [133f](#)  
tok: [49b](#), [49c](#), [50b](#), [51a](#), [51c](#), [51d](#), [53b](#), [53f](#), [54c](#), [56b](#), [120a](#), [156e](#)  
token(): [53f](#), [54b](#), [55a](#), [133f](#)  
trap: [114c](#), [114e](#), [115b](#), [214](#)  
Trapinit(): [114d](#)  
Tree: [51e](#), [196c](#)  
tree1(): [28b](#), [59d](#)  
tree2(): [28c](#)  
tree3(): [28b](#), [28c](#), [28d](#)  
tree (typedef): [196c](#)  
treenodes: [27c](#), [27e](#), [28a](#)

trimdirs(): [126e](#), [127a](#)  
truestatus(): [68a](#), [76e](#), [76f](#), [77b](#), [88b](#), [129a](#), [155a](#), [157e](#)  
turfredir(): [83a](#), [83b](#)  
TWIDDLE: [55e](#), [151e](#)  
unicode(): [123c](#)  
Updenv(): [110a](#)  
updenvlocal(): [110a](#), [110b](#), [110b](#)  
Var: [31b](#), [33c](#), [34a](#), [87](#), [105d](#), [115b](#), [196c](#)  
Var.changed: [111a](#)  
Var.fn: [93c](#)  
Var.fnchanged: [93c](#)  
Var.name: [33c](#)  
Var.next: [33d](#)  
Var.pc: [93c](#)  
Var.val: [33c](#)  
var (typedef): [196c](#)  
Vinit(): [38b](#), [111c](#)  
vlook(): [34a](#), [34b](#), [45f](#), [45h](#), [67d](#), [73a](#), [87](#), [96c](#), [97b](#), [99b](#), [103c](#), [111c](#), [115b](#), [140](#), [141c](#), [155a](#)  
Waitfor(): [74b](#)  
waitpids: [68b](#), [68d](#), [68e](#), [68f](#), [69a](#)  
WHILE: [55e](#), [151e](#)  
WORD: [53f](#), [54c](#), [55a](#), [92b](#), [133f](#), [151e](#)  
Word: [31c](#), [31d](#), [73b](#), [87](#), [115b](#), [120b](#), [196c](#)  
Word.next: [31c](#)  
Word.word: [31c](#)  
word (typedef): [196c](#)  
wordchr(): [48b](#), [53b](#), [54c](#), [56b](#)  
WORDS: [151e](#)  
WRITE: [52b](#), [52f](#), [151e](#)  
Xappend(): [30a](#), [82b](#), [154a](#)  
Xassign(): [96c](#), [117](#), [154a](#)  
Xasync(): [30a](#), [86d](#), [154a](#)  
Xbackq(): [30a](#), [140](#), [154a](#)  
Xbang(): [77b](#), [154a](#)  
Xcase(): [30a](#), [91b](#), [92c](#), [154a](#)  
Xclose(): [30a](#), [137i](#), [154a](#)  
Xconc(): [100b](#), [154a](#)  
Xdelfn(): [95a](#), [154a](#)  
Xdelhere(): [69f](#), [132b](#), [132c](#), [154a](#)  
Xdup(): [137e](#), [137f](#), [154a](#)  
Xeflag(): [129a](#), [154a](#)  
Xerror(): [70](#), [71b](#), [80d](#), [81c](#), [82b](#), [85b](#), [86d](#), [105d](#), [130d](#), [135d](#), [138h](#), [140](#), [154a](#)  
Xerror1(): [71a](#), [71c](#), [72c](#), [80d](#), [81c](#), [82b](#), [96c](#), [97a](#), [99b](#), [100b](#), [105d](#), [107e](#), [108c](#), [135d](#), [141c](#), [145](#)  
Xexit(): [75a](#), [75b](#), [87](#), [105c](#), [117](#), [129a](#), [154a](#)  
Xfalse(): [30a](#), [76f](#), [154a](#)  
Xfn(): [94a](#), [94c](#), [154a](#)  
Xfor(): [30a](#), [90b](#), [154a](#)  
Xglob(): [121a](#), [154a](#)  
Xif(): [88b](#), [154a](#)

Xifnot(): [88f](#), [154a](#)  
Xjump(): [30a](#), [89g](#), [91b](#), [154a](#)  
Xlocal(): [97a](#), [107d](#), [154a](#)  
Xmark(): [66b](#), [91b](#), [107d](#), [117](#), [154a](#)  
Xmatch(): [77d](#), [154a](#)  
Xpipe(): [84c](#), [85b](#), [154a](#)  
Xpipefd(): [137f](#), [138h](#), [154a](#)  
Xpipewait(): [85f](#), [154a](#)  
Xpopm(): [91b](#), [92a](#), [154a](#)  
Xpopredir(): [75b](#), [81a](#), [83a](#), [154a](#)  
Xqdol(): [141c](#), [154a](#)  
Xrdcmds(): [41a](#), [107d](#), [108b](#), [154a](#)  
Xrdfs(): [147b](#), [154a](#)  
Xrdwr(): [30a](#), [135d](#), [154a](#)  
Xread(): [30a](#), [81c](#), [154a](#)  
Xreturn(): [41d](#), [65a](#), [71b](#), [71c](#), [84d](#), [107d](#), [108b](#), [115b](#), [147b](#), [154a](#)  
Xsettrue(): [89f](#)  
Xsimple(): [70](#), [117](#), [154a](#)  
Xsub(): [99b](#), [154a](#)  
Xsubshell(): [30a](#), [130d](#), [154a](#)  
Xtrue(): [30a](#), [76e](#), [154a](#)  
Xunlocal(): [97b](#), [107d](#), [154a](#)  
Xwastrue(): [88d](#), [154a](#)  
Xword(): [66a](#), [69f](#), [107d](#), [117](#), [154a](#)  
Xwrite(): [30a](#), [80d](#), [154a](#)  
yyerror(): [91b](#), [120a](#), [132a](#), [133f](#), [136a](#), [156e](#)  
yylex(): [48a](#)  
yylval: [51f](#), [51g](#), [53f](#), [54c](#)  
YYMAXDEPTH:  
yyparse(): [41a](#)  
\_efgfmt(): [158a](#)  
\_\_anon\_enum\_1: [163f](#)  
\_\_anon\_enum\_2: [110c](#)  
\_\_anon\_struct\_4.f: [154a](#)  
\_\_anon\_struct\_4.name: [154a](#)  
\_\_anon\_struct\_4: [154a](#), [154a](#)

# Bibliography

- [BK89] Morris Bolsky and David Korn. *The KornShell Command and Programming Language*. Prentice Hall, 1989. cited page(s) 10
- [BKM86] Jon Bentley, Donald Knuth, and Doug McIlroy. Programming pearls: A literate program. *Communication of the ACM*, 29(6):471–483, June 1986. The end of the article contains a one-line shell script solving a problem that required Donald Knuth a hundred lines of Pascal code. cited page(s) 8
- [BLM92] Doug Brown, John Levine, and Tony Mason. *Lex and Yacc*. O’Reilly, 1992. cited page(s) 11
- [Bou79] Stephen R. Bourne. An introduction to the unix shell. In *Unix Programmer’s Manual Vol 2a*, 1979. Also available at [shell/docs/sh.pdf](#). cited page(s) 9, 12
- [Bou15] Stephen R. Bourne. Early days of unix and design of sh. In *BSDCan - The BSD Conference*, 2015. [https://www.bsdcan.org/2015/schedule/attachments/306\\_srbBSDCan2015.pdf](https://www.bsdcan.org/2015/schedule/attachments/306_srbBSDCan2015.pdf), also in [shells/docs/early-days-unix-sh-bourne.pdf](#). cited page(s) 12
- [Duf90] Tom Duff. Rc – a shell for plan 9 and unix. In *Unix Research System: Papers (Vol 2) 10th ed.* Saunders College, 1990. cited page(s) 9
- [Duf00] Tom Duff. Rc – the plan 9 shell. Technical report, Bell Labs, 2000. Also available at [shells/docs/rc.pdf](#). cited page(s) 12
- [Joh79] Stephen C. Johnson. Yacc: Yet another compiler-compiler. In *Unix Programmer’s Manual Vol 2b*, 1979. Also available at [generators/docs/yacc.pdf](#). cited page(s) 9
- [Joy86] William Joy. An introduction to the c shell. In *UNIX Programmer’s Supplementary Documents 1*, 1986. Available at <https://docs.freebsd.org/44doc/usd/04.csh/paper.pdf>. cited page(s) 10
- [Kid05] Oliver Kiddle. *From Bash to Z Shell*. Apress, 2005. cited page(s) 10, 12
- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 12
- [Kor94] David G. Korn. ksh - an extensible high level language. In *Proceedings of the USENIX 1994 Very High Level Languages Symposium*, 1994. cited page(s) 10
- [KP84] Brian W. Kernighan and Rob Pike. *The UNIX Programming Environment*. Prentice Hall, 1984. cited page(s) 12
- [KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 11
- [Mic08] Randal K. Michael. *Master UNIX Shell Scripting*. Wiley, 2008. cited page(s) 12
- [New95] Cameron Newham. *Learning the bash Shell*. O’Reilly, 1995. cited page(s) 9, 12

- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s) 12
- [Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 11, 12, 17, 21, 67, 113, 148
- [Pad16a] Yoann Padioleau. *Principia Softwarica: The Build System mk*. 2016. cited page(s) 149
- [Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 C Compiler 5c*. 2016. cited page(s) 12, 21, 26, 57, 149
- [Pad16c] Yoann Padioleau. *Principia Softwarica: The Plan 9 Windowing System rio*. 2016. cited page(s) 9, 105
- [Pad25] Yoann Padioleau. *Principia Softwarica: The Plan 9 Utilities*. 2025. cited page(s) 173
- [Pou00] Louis Pouzin. The origin of the shell, 2000. <http://multicians.org/shell.html>. cited page(s) 13, 16
- [Ram94] Chet Ramey. Bash, the bourne-again shell, 1994. Available in [bash/doc/rose94.pdf](#). cited page(s) 9
- [Roc04] Marc J. Rochkind. *Advanced UNIX Programming*. Addison-Wesley, 2004. cited page(s) 8, 12
- [Ste94] Richard W. Stevens. *Advanced Programming in the UNIX Environment*. Addison-Wesley, 1994. cited page(s) 12